

0.24 Les deux axes à la suite difficulté 0/10

0.24 Les deux axes à la suite — de base, ailleurs difficulté 0/10

0.24 Les deux axes à la suite — doublé, deux positions difficulté 0/10

0.25 Le carré, côté par côté difficulté 0/10

0.25 Le carré, côté par côté — de base, ailleurs difficulté 0/10

0.25 Le carré, côté par côté — doublé, deux positions difficulté 0/10

0.26 En diagonale : deplace difficulté 0/10

0.26 En diagonale : deplace — de base, ailleurs difficulté 0/10

0.26 En diagonale : deplace — doublé, deux positions difficulté 0/10

0.27 Le carré avec deplace difficulté 0/10

0.27 Le carré avec deplace — de base, ailleurs difficulté 0/10

0.27 Le carré avec deplace — doublé, deux positions difficulté 0/10

0.28 Aller à une case : va_a difficulté 0/10

0.28 Aller à une case : va_a — de base, ailleurs difficulté 0/10

0.28 Aller à une case : va_a — doublé, deux positions difficulté 0/10

0.29 Revenir au départ avec va_a difficulté 0/10

0.29 Revenir au départ avec va_a — de base, ailleurs difficulté 0/10

0.29 Revenir au départ avec va_a — doublé, deux positions difficulté 0/10

0.30 Un pas sans attendre : un_pas difficulté 0/10

0.30 Un pas sans attendre : un_pas — de base, ailleurs difficulté 0/10

0.30 Un pas sans attendre : un_pas — doublé, deux positions difficulté 0/10

0.31 Deux lettres à la fois difficulté 0/10

0.31 Deux lettres à la fois — de base, ailleurs difficulté 0/10

0.31 Deux lettres à la fois — doublé, deux positions difficulté 0/10

0.32 Le trajet dans un tableau difficulté 0/10

0.32 Le trajet dans un tableau — de base, ailleurs difficulté 0/10

0.32 Le trajet dans un tableau — doublé, deux positions difficulté 0/10

0.33 Écrire soi-même sa fonction difficulté 0/10

0.33 Écrire soi-même sa fonction — de base, ailleurs difficulté 0/10

0.33 Écrire soi-même sa fonction — doublé, deux positions difficulté 0/10

0.34 Un carré autour d'une lettre, avec carre difficulté 0/10

0.34 Un carré autour d'une lettre, avec carre — de base, ailleurs difficulté 0/10

0.34 Un carré autour d'une lettre, avec carre — doublé, deux positions difficulté 0/10

0.35 Taille 2 : un carré de 5 × 5 difficulté 0/10

0.35 Taille 2 : un carré de 5 × 5 — de base, ailleurs difficulté 0/10

0.35 Taille 2 : un carré de 5 × 5 — doublé, deux positions difficulté 0/10

0.36 Taille 3 : un carré de 7 × 7 difficulté 0/10

0.36 Taille 3 : un carré de 7 × 7 — de base, ailleurs difficulté 0/10

0.36 Taille 3 : un carré de 7 × 7 — doublé, deux positions difficulté 0/10

0.37 Taille 4 : un carré de 9 × 9 difficulté 0/10

0.37 Taille 4 : un carré de 9 × 9 — de base, ailleurs difficulté 0/10

0.37 Taille 4 : un carré de 9 × 9 — doublé, deux positions difficulté 0/10

0.38 Taille 5 : un carré de 11 × 11 difficulté 0/10

0.38 Taille 5 : un carré de 11 × 11 — de base, ailleurs difficulté 0/10

0.38 Taille 5 : un carré de 11 × 11 — doublé, deux positions difficulté 0/10

0.39 Taille 6 : un carré de 13 × 13 difficulté 0/10

0.39 Taille 6 : un carré de 13 × 13 — de base, ailleurs difficulté 0/10

0.39 Taille 6 : un carré de 13 × 13 — doublé, deux positions difficulté 0/10

0.40 Taille 7 : un carré de 15 × 15 difficulté 0/10

0.40 Taille 7 : un carré de 15 × 15 — de base, ailleurs difficulté 0/10

0.40 Taille 7 : un carré de 15 × 15 — doublé, deux positions difficulté 0/10

0.41 Taille 8 : un carré de 17 × 17 difficulté 0/10

0.41 Taille 8 : un carré de 17 × 17 — de base, ailleurs difficulté 0/10

0.41 Taille 8 : un carré de 17 × 17 — doublé, deux positions difficulté 0/10

0.42 Dans l'autre sens : sens -1 difficulté 0/10

0.42 Dans l'autre sens : sens -1 — de base, ailleurs difficulté 0/10

0.65 Une lettre qui avance — en simple difficulté 0/10

0.66 La lettre bouge avec la croix difficulté 0/10

0.66 La lettre bouge avec la croix — de base, ailleurs difficulté 0/10

0.66 La lettre bouge avec la croix — doublé, deux positions difficulté 0/10

0.66 La lettre bouge avec la croix — en simple difficulté 0/10

0.67 Les variables de la lettre dans leur propre fichier difficulté 0/10

0.67 Les variables de la lettre dans leur propre fichier — de base, ailleurs difficulté 0/10

0.67 Les variables de la lettre dans leur propre fichier — doublé, deux positions difficulté 0/10

0.67 Les variables de la lettre dans leur propre fichier — en simple difficulté 0/10

0.68 Voir la position du A en direct difficulté 0/10

0.68 Voir la position du A en direct — de base, ailleurs difficulté 0/10

0.68 Voir la position du A en direct — doublé, deux positions difficulté 0/10

0.68 Voir la position du A en direct — en simple difficulté 0/10

0.69 Chercher le A sur l'écran : lire difficulté 0/10

0.69 Chercher le A sur l'écran : lire — de base, ailleurs difficulté 0/10

0.69 Chercher le A sur l'écran : lire — doublé, deux positions difficulté 0/10

0.69 Chercher le A sur l'écran : lire — en simple difficulté 0/10

0.70 Plus fluide : la lettre au pixel près difficulté 0/10

0.70 Plus fluide : la lettre au pixel près — de base, ailleurs difficulté 0/10

0.70 Plus fluide : la lettre au pixel près — doublé, deux positions difficulté 0/10

0.70 Plus fluide : la lettre au pixel près — en simple difficulté 0/10

0.71 La croix en une ligne : deplace_croix difficulté 0/10

0.71 La croix en une ligne : deplace_croix — de base, ailleurs difficulté 0/10

0.71 La croix en une ligne : deplace_croix — doublé, deux positions difficulté 0/10

0.72 Plus vite sur la grille : deplace_croix à 100 difficulté 0/10

0.72 Plus vite sur la grille : deplace_croix à 100 — de base, ailleurs difficulté 0/10

0.72 Plus vite sur la grille : deplace_croix à 100 — doublé, deux positions difficulté 0/10

0.73 Glisser en une ligne : glisse_croix difficulté 0/10

0.73 Glisser en une ligne : glisse_croix — de base, ailleurs difficulté 0/10

0.73 Glisser en une ligne : glisse_croix — doublé, deux positions difficulté 0/10

0.74 Glisser plus vite : glisse_croix à 3 difficulté 0/10

0.74 Glisser plus vite : glisse_croix à 3 — de base, ailleurs difficulté 0/10

0.74 Glisser plus vite : glisse_croix à 3 — doublé, deux positions difficulté 0/10

0.75 Arrivé en (0, 0), le A devient B difficulté 0/10

0.75 Arrivé en (0, 0), le A devient B — de base, ailleurs difficulté 0/10

0.75 Arrivé en (0, 0), le A devient B — doublé, deux positions difficulté 0/10

0.76 Un déclencheur en (0, 0) : actif difficulté 0/10

0.76 Un déclencheur en (0, 0) : le B apparaît au milieu difficulté 0/10

0.76 Un déclencheur en (0, 0) : le B tourne en carré difficulté 0/10

0.76 Un déclencheur en (0, 0) : le carré plus rapide difficulté 0/10

0.76 Un déclencheur en (0, 0) : deux B difficulté 0/10

0.76 Un déclencheur en (0, 0) : le C file à gauche difficulté 0/10

0.76 Un déclencheur en (0, 0) : le D file à droite difficulté 0/10

0.76 Un déclencheur en (0, 0) : actif — en simple difficulté 0/10

0.76 Un déclencheur en (0, 0) : le B apparaît au milieu — en simple difficulté 0/10

0.76 Un déclencheur en (0, 0) : le B tourne en carré — en simple difficulté 0/10

0.76 Un déclencheur en (0, 0) : le carré plus rapide — en simple difficulté 0/10

0.76 Un déclencheur en (0, 0) : deux B — en simple difficulté 0/10

0.76 Un déclencheur en (0, 0) : le C file à gauche — en simple difficulté 0/10

0.76 Un déclencheur en (0, 0) : le D file à droite — en simple difficulté 0/10

0.77 Plusieurs rythmes sans compteur : chaque difficulté 0/10

0.78 Le rebond : une vitesse qui change de signe difficulté 0/10

0.78 Le rebond : une vitesse qui change de signe — en diagonale difficulté 0/10

0.79 Suivre une autre lettre difficulté 0/10

0.79 Suivre une autre lettre — de plus en plus vite difficulté 0/10

0.0. Le programme qui ne fait rien

Le squelette de tout programme : `int main()`, et la boucle `while`.

`int main()` est le point de départ. Quand la console s'allume, c'est là qu'elle arrive, et nulle part ailleurs. Tout ce que fait le programme s'écrit entre ses accolades `{ }`.

`while (true)` veut dire « tant que vrai » : ce qui est entre ses accolades recommence sans jamais s'arrêter. C'est la boucle de jeu. Un jeu ne se termine pas tout seul :

il attend le joueur. Une traverse pas : le M difficulté 0/10

`image()` attend le prochain état du labyrinthe en 60 par seconde : la boucle fait donc un tour toutes les six centièmes de seconde.

Un son quand on ramasse : `note` difficulté 0/10

Ce programme ne montre rien, mais il tourne. Tous les suivants partent de lui : on ajoutera des choses **avant** la boucle (ce qui se fait une fois) et **dedans** (ce qui se fait à

chaque image) quand on ramasse : `note` — et un bruit contre le mur difficulté 0/10

Des vies et une fin de partie difficulté 0/10

```
int main() {
    while (true) {
        image();
    }
}
```

Effacer sans compter difficulté 1/10

Ranger le mot sous un nom difficulté 1/10

Effacer le mot rangé sous un nom difficulté 1/10

La place du mot dans deux variables difficulté 1/10

Effacer à la place rangée difficulté 1/10

Le mot et sa place sous un seul nom difficulté 1/10

Effacer un Mot difficulté 1/10

Un Mot fait de variables créées d'avance difficulté 1/10

Effacer un Mot fait de variables difficulté 1/10

Les variables dans leur propre fichier difficulté 1/10

Effacer quand on appuie sur A difficulté 1/10

Changer de message avec A et B difficulté 1/10

Effacer ce qu'on a écrit difficulté 1/10

Les messages dans leur propre fichier difficulté 1/10

Un carré de 8 × 8 difficulté 1/10

Écrire à plusieurs endroits difficulté 1/10

Une constante plutôt qu'un nombre difficulté 1/10

Réécrire au bout d'une seconde difficulté 1/10

Un nombre à l'écran difficulté 1/10

Effacer sans compter les lettres difficulté 1/10

NIVEAU 2 — RETENIR, ET RÉAGIR

Une variable, un octet difficulté 2/10

Lire un bouton difficulté 2/10

Un compteur qui déborde à 255 difficulté 2/10

Un dé, et le hasard du matériel difficulté 2/10

Deux mots de même longueur difficulté 2/10

Se souvenir de l'état d'avant difficulté 2/10

Un choix, puis plusieurs difficulté 2/10

Deux touches, deux effets difficulté 2/10

NIVEAU 3 — DESSINER

Dessiner sa propre tuile difficulté 3/10

Le dessin écrit là où on le pose difficulté 3/10

Intégrer une carte dans son projet difficulté 3/10

Les couleurs : palettes, variétés et thèmes difficulté 3/10

Poser tes tuiles à la souris difficulté 3/10

Poser une tuile difficulté 3/10

Quatre nuances, quatre signes difficulté 3/10

Une rangée, puis un mur difficulté 3/10

Un damier difficulté 3/10

Un lutin, hors de la grille difficulté 3/10

NIVEAU 4 — LE MOUVEMENT

Le front, ou pourquoi un appui compte trois fois difficulté 4/10

Un lutin, au pixel près difficulté 4/10

Un objet qui tombe difficulté 4/10

Un personnage de seize difficulté 4/10

Le garder dans l'écran difficulté 4/10

Un tour de toutes les six centièmes de seconde.

Seules sans s'arrêter difficulté 4/10

defiler(), et l'octet qui boucle difficulté 4/10

NIVEAU 5 — LE SON

Une petite melodie à la main difficulté 5/10

Un bruit difficulté 5/10

Une partition qui avance seule difficulté 5/10

Une musique qui reprend au bout difficulté 5/10

NIVEAU 6 — RANGER CE QU'ON MANIPULE

Un tableau, et une boucle « for » difficulté 6/10

Le panneau, et les nuances difficulté 6/10

Remplir un tableau, puis le dessiner difficulté 6/10

Une table gravée difficulté 6/10

Une grille difficulté 6/10

Chercher dans un tableau difficulté 6/10

Un texte qui porte un nom difficulté 6/10

NIVEAU 7 — DÉCOUPER SON PROGRAMME

Une fonction qui prend des arguments difficulté 7/10

Découper son programme difficulté 7/10

Nommer un geste qu'on refait difficulté 7/10

Une fonction qui rend une valeur difficulté 7/10

Un argument par défaut difficulté 7/10

Pourquoi une fonction ne s'appelle pas elle-même difficulté 7/10

#include, et les onglets de la page difficulté 7/10

NIVEAU 8 — CE QUI VA ENSEMBLE

Une « struct » : ce qui va ensemble difficulté 8/10

Le décor qui défile difficulté 8/10

La caméra : c'est le monde qui bouge difficulté 8/10

Trois variables qui parlent de la même chose difficulté 8/10

Un tableau de « struct » difficulté 8/10

Une méthode : la fonction qui connaît son objet difficulté 8/10

Les trois façons de faire la même chose difficulté 8/10

Une méthode qui en appelle une autre difficulté 8/10

NIVEAU 9 — UN VRAI JEU

Passer d'un écran à l'autre difficulté 9/10

La pesanteur et le saut difficulté 9/10

Le verre qui se brise difficulté 9/10

Un état de jeu difficulté 9/10

Le score, et le panneau difficulté 9/10

Des ennemis qui vivent et qui meurent difficulté 9/10

Les collisions difficulté 9/10

Ralentir sans compter les images à la main difficulté 9/10

Passer à la salle suivante, sans bouton difficulté 9/10

NIVEAU 10 — ALLER AU BOUT

Se souvenir d'une partie à l'autre difficulté 10/10

CE QU'ON DOIT VOIR

Rien : l'écran reste vide, mais la console tourne.

l'écran, au bout de vingt images

76

Le monde glisse, le héros ne bouge pas

difficulté 10/10

77

Un record qui survit à l'extinction

difficulté 10/10

0.1. Afficher la lettre A

Une seule lettre, dans la toute première case de l'écran.

Les lettres sont **déjà dessinées** dans le programme : c'est la police, chargée au démarrage. Il suffit de dire laquelle afficher, et où.

`texte(0, 0, "A")` écrit **A** dans la case de la colonne **0**, ligne **0** : celle en haut à gauche. L'écran fait 20 colonnes (0 à 19) sur 18 lignes (0 à 17).

La ligne est écrite **avant** la boucle : on n'écrit le A qu'une fois, et il reste affiché.

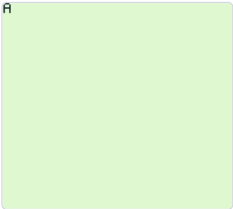
LE PROGRAMME — 1285 OCTETS

```
int main() {
    texte(0, 0, "A"); // colonne 0, ligne 0 : la case en haut à gauche

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un A, tout en haut à gauche de l'écran.



l'écran, au bout de vingt images

0.1.1. Afficher la lettre A — de base, ailleurs

Le même A qu'au 0.1, mais au milieu de l'écran : `texte(10, 8, "A")`.

C'est le 0.1, avec un seul changement : la **place** du A. Il n'est plus en (0, 0), en haut à gauche, mais en **(10, 8)**, au milieu de l'écran.

Les deux premiers nombres de `texte` sont la colonne (de 0 à 19, de gauche à droite) et la ligne (de 0 à 17, de haut en bas). Le milieu de l'écran est vers la colonne 10 et la ligne 8.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.1.2 met les deux ensemble.

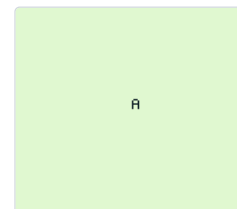
LE PROGRAMME — 1285 OCTETS

```
int main() {
    // Le changement : la place. (0, 0) → (10, 8).
    //
    //  texte(10, 8, "A");
    //
    //      |  |
    //      |  +-- ligne 8   : vers le milieu, de haut en bas (0 à 17)
    //      +----- colonne 10 : vers le milieu, de gauche à droite (0 à 19)
    texte(10, 8, "A");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un A au milieu de l'écran.



l'écran, au bout de vingt images

0.1.2. Afficher la lettre A — doublé, deux positions

Deux A : un en haut à gauche (0, 0), un au milieu (10, 8). Deux lignes texte, deux places.

C'est le 0.1 et le 0.1.1 réunis : deux lignes `texte`, une par place.

Chaque appel écrit sa lettre à sa place, et les deux restent à l'écran : elles ne sont pas au même endroit, donc aucune n'écrase l'autre (le 0.3 montre ce qui arrive quand elles le sont).

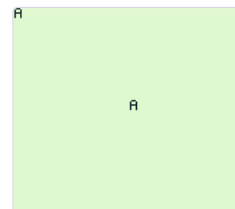
LE PROGRAMME — 1297 OCTETS

```
int main() {
    texte(0, 0, "A");    // 1. en haut à gauche (le 0.1)
    texte(10, 8, "A");   // 2. au milieu (le 0.1.1)

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Deux A : un en haut à gauche, un au milieu de l'écran.



l'écran, au bout de vingt images

0.2. A, pris dans ALPHABET à l'indice 0

L'alphabet de la console est un tableau : sa première case est l'indice 0.

`ALPHABET` est le tableau des 26 lettres de la console. Il existe déjà : on ne le crée pas, on s'en sert.

En C++, un tableau commence à l'indice **0**. `ALPHABET[0]` est donc la **première** lettre, A ; `ALPHABET[25]` est la dernière, Z.

`poser(0, 0, ALPHABET[0])` pose cette lettre dans la case (0, 0).

Pourquoi `poser()` et pas `texte()` ? `ALPHABET[0]` n'est pas un texte : c'est un **nombre**, le numéro de la tuile du A (1). `texte()` n'accepte que des lettres entre guillemets ; pour mettre une tuile à partir de son numéro, c'est `poser()`. Écrire `texte(0, 0, ALPHABET[0])` est une erreur : le compilateur la refuse.

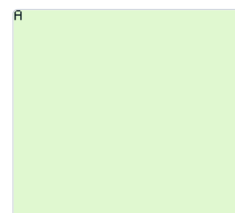
LE PROGRAMME — 1298 OCTETS

```
int main() {
    poser(0, 0, ALPHABET[0]); // l'indice 0 : la première lettre, A
    // (ne pas faire cette erreur : texte(0, 0, ALPHABET[0]) est refusé,
    // car ALPHABET[0] est un numéro de tuile, pas un texte)

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un A, tout en haut à gauche de l'écran.



l'écran, au bout de vingt images

0.2.1. A, pris dans ALPHABET à l'indice 0 — de base, ailleurs

Le même `poser(..., ALPHABET[0])` qu'au 0.2, au milieu de l'écran.

C'est le 0.2, avec un seul changement : la **place** : (10, 8), le milieu de l'écran, au lieu de (0, 0).

`poser` prend la même place que `texte` : la colonne d'abord (0 à 19), puis la ligne (0 à 17).

C'est la version de base : une seule chose, à la deuxième place. Le 0.2.2 met les deux ensemble.

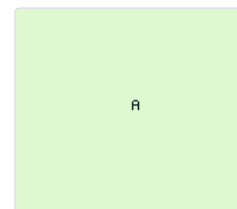
LE PROGRAMME — 1300 OCTETS

```
int main() {
    // Le changement : la place. (0, 0) → (10, 8).
    poser(10, 8, ALPHABET[0]); // l'indice 0 : A, au milieu de l'écran

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un A au milieu de l'écran.



l'écran, au bout de vingt images

0.2.2. A, pris dans ALPHABET à l'indice 0 — doublé, deux positions

Deux poser, deux places : ALPHABET[0] en haut à gauche et au milieu.

C'est le 0.2 et le 0.2.1 réunis : deux lignes `poser`, une par place.

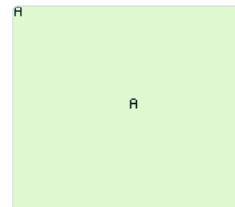
LE PROGRAMME — 1325 OCTETS

```
int main() {
    poser(0, 0, ALPHABET[0]); // 1. en haut à gauche (le 0.2)
    poser(10, 8, ALPHABET[0]); // 2. au milieu (le 0.2.1)

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Deux A : un en haut à gauche, un au milieu.



l'écran, au bout de vingt images

0.3. Écrire au même endroit écrase

La même position, exprès : la nouvelle lettre remplace l'ancienne.

Les deux lignes visent **exprès la même case**, (0, 0). Une case ne contient qu'**une seule lettre** : écrire le B au même endroit **efface** le A et prend sa place.

Les lignes s'exécutent **de haut en bas** : le A d'abord, puis le B. C'est donc le B qui reste.

Le A a bien été écrit, mais il est remplacé avant même la première image : on ne le voit jamais.

LE PROGRAMME — 1297 OCTETS

```
int main() {
    texte(0, 0, "A"); // colonne 0, ligne 0 : la case en haut à gauche
    texte(0, 0, "B"); // même case : le B écrase le A

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Seulement un B, en haut à gauche : le A a été écrasé.

B

l'écran, au bout de vingt images

0.3.1. Écrire au même endroit écrase — de base, ailleurs

Le 0.3 au milieu de l'écran : le B écrase encore le A, en (10, 8).

C'est le 0.3, avec un seul changement : la **place** : (10, 8) au lieu de (0, 0), pour les deux lignes.

La règle ne dépend pas de la place : deux écritures dans la **même** case, la seconde efface la première. Ici, les deux sont en (10, 8) : on ne voit que le B.

C'est la version de base : une seule chose, à la deuxième place. Le 0.3.2 met les deux ensemble.

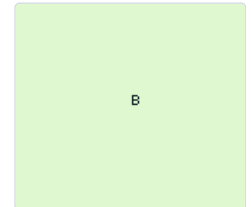
LE PROGRAMME — 1297 OCTETS

```
int main() {
    texte(10, 8, "A"); // au milieu...
    texte(10, 8, "B"); // ..même case : le B écrase le A

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B au milieu de l'écran ;
le A a été écrasé.



l'écran, au bout de vingt images

0.3.2. Écrire au même endroit écrase — doublé, deux positions

L'écrasement à deux places : en haut à gauche ET au milieu, le B écrase le A.

C'est le 0.3 et le 0.3.1 réunis : les deux écrasements, chacun à sa place.

Chaque case est indépendante : ce qui s'écrit en (0, 0) ne touche pas (10, 8). Aux deux endroits, le B écrase le A.

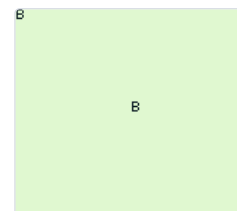
LE PROGRAMME — 1321 OCTETS

```
int main() {
    texte(0, 0, "A"); // en haut à gauche (le 0.3)...
    texte(0, 0, "B"); // ..écrasé par le B
    texte(10, 8, "A"); // au milieu (le 0.3.1)...
    texte(10, 8, "B"); // ..écrasé par le B

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Deux B : en haut à gauche et au milieu. Aucun A.



l'écran, au bout de vingt images

0.4. L'alphabet à la main, jusqu'au bout de la ligne

Une lettre par case : au bout de 20, la ligne est pleine.

Chaque lettre prend **une case de 8 pixels** de large. L'écran en fait 160 : **20 lettres** remplissent donc toute la ligne, de la colonne 0 à la colonne **19**.

On s'arrête à **T**, la 20e lettre. Il n'y a **pas de colonne 20** : `texte(20, 0, "U")` est refusé par le compilateur, qui répond que l'écran fait 20 colonnes.

Vingt lignes presque identiques : c'est long à écrire, et c'est exactement ce qu'une boucle saura faire à notre place.

LE PROGRAMME — 1513 OCTETS

```
int main() {
    texte(0, 0, "A");
    texte(1, 0, "B");
    texte(2, 0, "C");
    texte(3, 0, "D");
    texte(4, 0, "E");
    texte(5, 0, "F");
    texte(6, 0, "G");
    texte(7, 0, "H");
    texte(8, 0, "I");
    texte(9, 0, "J");
    texte(10, 0, "K");
    texte(11, 0, "L");
    texte(12, 0, "M");
    texte(13, 0, "N");
    texte(14, 0, "O");
    texte(15, 0, "P");
    texte(16, 0, "Q");
    texte(17, 0, "R");
    texte(18, 0, "S");
    texte(19, 0, "T"); // colonne 19 : la dernière de la ligne
    // On s'arrête ici : l'écran fait 20 colonnes (de 0 à 19).
    // 20 lettres de 8 pixels = 160 pixels : toute la largeur de l'écran.
    // (si on dépasse : texte(20, 0, "U") est refusé par le compilateur,
    // qui répond « l'écran fait 20 colonnes et 18 lignes ; 20,0 est dehors »)

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur toute la première ligne, d'un bord à l'autre de l'écran.

A B C D E F G H I J K L M N O P Q R S T



l'écran, au bout de vingt images

0.4.1. L'alphabet à la main, jusqu'au bout de la ligne — de base, ailleurs

Les vingt lettres du 0.4, sur la ligne 5 au lieu de la ligne 0.

C'est le 0.4, avec un seul changement : la **ligne** : 5 au lieu de 0, pour les vingt lignes de code.

Seul le **deuxième nombre change**, dans chaque `texte` : c'est la ligne. Les colonnes restent 0 à 19 : la même ligne de lettres, plus bas.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.4.2 met les deux ensemble.

LE PROGRAMME — 1513 OCTETS

```
int main() {
    // Le changement : la ligne, 0 → 5, dans chaque texte.
    texte(0, 5, "A");
    texte(1, 5, "B");
    texte(2, 5, "C");
    texte(3, 5, "D");
    texte(4, 5, "E");
    texte(5, 5, "F");
    texte(6, 5, "G");
    texte(7, 5, "H");
    texte(8, 5, "I");
    texte(9, 5, "J");
    texte(10, 5, "K");
    texte(11, 5, "L");
    texte(12, 5, "M");
    texte(13, 5, "N");
    texte(14, 5, "O");
    texte(15, 5, "P");
    texte(16, 5, "Q");
    texte(17, 5, "R");
    texte(18, 5, "S");
    texte(19, 5, "T");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 5.

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.4.2. L'alphabet à la main, jusqu'au bout de la ligne — doublé, deux positions

La ligne A à T deux fois : en ligne 0 et en ligne 5. Écrit à la main, c'est quarante lignes — la boucle du 0.5 va l'éviter.

C'est le 0.4 et le 0.4.1 réunis : les vingt lettres en ligne 0, puis les vingt en ligne 5.

Quarante lignes presque pareilles : c'est long, et facile de se tromper. C'est exactement ce que la boucle `for` du 0.5 va raccourcir.

LE PROGRAMME — 1753 OCTETS

```
int main() {
    // 1. La ligne 0 (le 0.4)
    texte(0, 0, "A"); texte(1, 0, "B"); texte(2, 0, "C"); texte(3, 0, "D"); texte(4, 0, "E");
    texte(5, 0, "F"); texte(6, 0, "G"); texte(7, 0, "H"); texte(8, 0, "I"); texte(9, 0, "J");
    texte(10, 0, "K"); texte(11, 0, "L"); texte(12, 0, "M"); texte(13, 0, "N"); texte(14, 0, "O");
    texte(15, 0, "P"); texte(16, 0, "Q"); texte(17, 0, "R"); texte(18, 0, "S"); texte(19, 0, "T");
    // 2. La ligne 5 (le 0.4.1)
    texte(0, 5, "A"); texte(1, 5, "B"); texte(2, 5, "C"); texte(3, 5, "D"); texte(4, 5, "E");
    texte(5, 5, "F"); texte(6, 5, "G"); texte(7, 5, "H"); texte(8, 5, "I"); texte(9, 5, "J");
    texte(10, 5, "K"); texte(11, 5, "L"); texte(12, 5, "M"); texte(13, 5, "N"); texte(14, 5, "O");
    texte(15, 5, "P"); texte(16, 5, "Q"); texte(17, 5, "R"); texte(18, 5, "S"); texte(19, 5, "T");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T deux fois : sur la
ligne 0 et sur la ligne 5.

ABCDEFGHIJKLMNOPQRST

ABCDEFGHIJKLMNOPQRST

l'écran, au bout de vingt images

0.5. La même ligne, avec une boucle for

Vingt lignes presque pareilles deviennent une seule, répétée vingt fois.

`for (uint8_t i = 0; i < 20; i++)` répète ce qui est entre ses accolades **20 fois**. `i` vaut 0 au premier tour, puis 1, 2... jusqu'à 19.

`i` sert **deux fois** : comme **colonne**, pour avancer de gauche à droite, et comme **indice** dans `ALPHABET`, pour passer de A à T.

`i < 20` : on s'arrête avant la colonne 20, qui n'existe pas, exactement comme au 0.4. Le résultat est le même, en 3 lignes au lieu de 20, et le programme est plus léger.

LE PROGRAMME — 1333 OCTETS

```
int main() {  
    for (uint8_t i = 0; i < 20; i++) { // 20 tours : les colonnes 0 à 19  
        poser(i, 0, ALPHABET[i]);    // la lettre n° i, dans la colonne i  
    }  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

A à T sur toute la première ligne, comme au 0.4.

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.5.1. La même ligne, avec une boucle for — de base, ailleurs

Le 0.5 sur la ligne 5 : un seul nombre change dans la boucle.

C'est le 0.5, avec un seul changement : la **ligne** : 5 au lieu de 0, dans le `poser` de la boucle.

Un seul nombre à **changer**, au lieu de vingt au 0.4.1 : la boucle écrit la ligne entière, et la ligne n'est dite qu'une fois.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.5.2 met les deux ensemble.

LE PROGRAMME — 1334 OCTETS

```
int main() {
    // Le changement : la ligne, 0 → 5.
    for (uint8_t i = 0; i < 20; i++) { // 20 tours : les colonnes 0 à 19
        poser(i, 5, ALPHABET[i]);    // la lettre n° i, colonne i, ligne 5
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 5.

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.5.2. La même ligne, avec une boucle for — doublé, deux positions

La boucle deux fois : la ligne 0 puis la ligne 5.

C'est le 0.5 et le 0.5.1 réunis : la boucle du 0.5 (ligne 0), puis celle du 0.5.1 (ligne 5).

Deux boucles l'une après l'autre : la première finit ses vingt tours, puis la seconde commence. Chaque `for` déclare son propre `i` : il n'existe que dans sa boucle.

LE PROGRAMME — 1394 OCTETS

```
int main() {
    // 1. La ligne 0 (le 0.5)
    for (uint8_t i = 0; i < 20; i++) { // 20 tours : les colonnes 0 à 19
        poser(i, 0, ALPHABET[i]);    // la lettre n° i, colonne i, ligne 0
    }
    // 2. La ligne 5 (le 0.5.1)
    for (uint8_t i = 0; i < 20; i++) { // 20 tours : les colonnes 0 à 19
        poser(i, 5, ALPHABET[i]);    // la lettre n° i, colonne i, ligne 5
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 0 et sur la ligne 5.

A B C D E F G H I J K L M N O P Q R S T

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.6. La même ligne, avec une boucle while

La boucle `for`, décomposée en ses trois morceaux.

`while (i < 20)` répète **tant que** la condition est vraie. C'est la même boucle que `while (true)` du 0.0, mais celle-ci **s'arrête**.

Les trois morceaux que `for` réunit sur une ligne sont ici écrits **séparément** : le départ `uint8_t i = 0;` avant la boucle, la condition `i < 20` dans le `while`, et `i++`, à la fin de chaque tour.

Le piège : sans `i++`, `i` reste à 0, la condition reste vraie, et la boucle **ne s'arrête jamais**. Le programme reste bloqué dedans.

LE PROGRAMME — 1333 OCTETS

```
int main() {
    uint8_t i = 0;           // 1. on part de la colonne 0

    while (i < 20) {         // 2. tant qu'on est dans l'écran
        poser(i, 0, ALPHABET[i]); // la lettre n° i, dans la colonne i
        i++;                // 3. on passe à la suivante
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur toute la première ligne, comme au 0.5.

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.6.1. La même ligne, avec une boucle while — de base, ailleurs

Le 0.6 sur la ligne 5 : un seul nombre change dans la boucle.

C'est le 0.6, avec un seul changement : la **ligne** : 5 au lieu de 0, dans le `poser` de la boucle.

Un seul nombre à **changer**, au lieu de vingt au 0.4.1 : la boucle écrit la ligne entière, et la ligne n'est dite qu'une fois.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.6.2 met les deux ensemble.

LE PROGRAMME — 1334 OCTETS

```
int main() {
    // Le changement : la ligne, 0 → 5.
    uint8_t i = 0;           // on part de la colonne 0
    while (i < 20) {         // tant qu'on est dans l'écran
        poser(i, 5, ALPHABET[i]); // la lettre n° i, colonne i, ligne 5
        i++;                // la suivante
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 5.

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.6.2. La même ligne, avec une boucle while — doublé, deux positions

La boucle deux fois : la ligne 0 puis la ligne 5.

C'est le 0.6 et le 0.6.1 réunis : la boucle du 0.6 (ligne 0), puis celle du 0.6.1 (ligne 5).

Deux boucles l'une après l'autre, avec la même variable `i` : déclarée une fois, elle est remise à 0 (`i = 0;`) avant la seconde boucle, sans `uint8_t` — on ne déclare pas deux fois le même nom.

LE PROGRAMME — 1394 OCTETS

```
int main() {
    // 1. La ligne 0 (le 0.6)
    uint8_t i = 0;           // on part de la colonne 0
    while (i < 20) {         // tant qu'on est dans l'écran
        poser(i, 0, ALPHABET[i]); // la lettre n° i, colonne i, ligne 0
        i++;                // la suivante
    }
    // 2. La ligne 5 (le 0.6.1)
    i = 0;                  // on part de la colonne 0
    while (i < 20) {         // tant qu'on est dans l'écran
        poser(i, 5, ALPHABET[i]); // la lettre n° i, colonne i, ligne 5
        i++;                // la suivante
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 0 et sur la ligne 5.

A B C D E F G H I J K L M N O P Q R S T

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.7. La même ligne, avec `do ... while`

La boucle qui vérifie sa condition à la fin : elle fait toujours au moins un tour.

`do { ... } while (i < 20);` fait le tour **d'abord**, et vérifie la condition **ensuite**. Il fait donc **toujours au moins un tour**, même si la condition est fausse dès le départ.

Ici, le résultat est le même qu'avec `for` et `while` : les trois boucles savent faire la même chose. On choisit celle qui se lit le mieux.

Attention au **point-virgule** après `while (i < 20)` : il est obligatoire avec `do`, et seulement avec lui.

LE PROGRAMME — 1330 OCTETS

```
int main() {
    uint8_t i = 0;

    do {
        poser(i, 0, ALPHABET[i]); // la lettre n° i, dans la colonne i
        i++;
    } while (i < 20);             // la condition est vérifiée À LA FIN

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur toute la première ligne, comme au 0.5.

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.7.1. La même ligne, avec do ... while — de base, ailleurs

Le 0.7 sur la ligne 5 : un seul nombre change dans la boucle.

C'est le 0.7, avec un seul changement : la **ligne** : 5 au lieu de 0, dans le `poser` de la boucle.

Un seul nombre à **changer**, au lieu de vingt au 0.4.1 : la boucle écrit la ligne entière, et la ligne n'est dite qu'une fois.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.7.2 met les deux ensemble.

LE PROGRAMME — 1331 OCTETS

```
int main() {
    // Le changement : la ligne, 0 → 5.
    uint8_t i = 0;
    do {
        poser(i, 5, ALPHABET[i]); // la lettre n° i, colonne i, ligne 5
        i++;
    } while (i < 20);             // la condition est vérifiée À LA FIN

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 5.

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.7.2. La même ligne, avec do ... while — doublé, deux positions

La boucle deux fois : la ligne 0 puis la ligne 5.

C'est le 0.7 et le 0.7.1 réunis : la boucle du 0.7 (ligne 0), puis celle du 0.7.1 (ligne 5).

Deux boucles l'une après l'autre, avec la **même** variable `i` : déclarée une fois, elle est remise à 0 (`i = 0;`) avant la seconde boucle, sans `uint8_t` — on ne déclare pas deux fois le même nom.

LE PROGRAMME — 1388 OCTETS

```
int main() {
    // 1. La ligne 0 (le 0.7)
    uint8_t i = 0;
    do {
        poser(i, 0, ALPHABET[i]); // la lettre n° i, colonne i, ligne 0
        i++;
    } while (i < 20);              // la condition est vérifiée À LA FIN
    // 2. La ligne 5 (le 0.7.1)
    i = 0;
    do {
        poser(i, 5, ALPHABET[i]); // la lettre n° i, colonne i, ligne 5
        i++;
    } while (i < 20);              // la condition est vérifiée À LA FIN

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 0 et sur la ligne 5.

A B C D E F G H I J K L M N O P Q R S T

A B C D E F G H I J K L M N O P Q R S T

l'écran, au bout de vingt images

0.8. Tout l'alphabet, avec deux boucles

La ligne est pleine après T : une seconde boucle écrit U à Z sur la ligne d'en dessous.

Il y a **26 lettres** et seulement **20 colonnes**. La première boucle écrit A à T sur la ligne 0 ; la seconde écrit les 6 dernières, U à Z, sur la ligne 1.

Dans la seconde boucle, `i` va de 20 à 25 : c'est le bon **indice** dans `ALPHABET` (U est à l'indice 20). Mais la **colonne** doit repartir à 0 : on écrit donc `i - 20`.

LE PROGRAMME — 1399 OCTETS

```
int main() {
    for (uint8_t i = 0; i < 20; i++) {    // A à T : ligne 0, colonnes 0 à 19
        poser(i, 0, ALPHABET[i]);
    }

    for (uint8_t i = 20; i < 26; i++) {    // U à Z : ligne 1
        poser(i - 20, 1, ALPHABET[i]);    // la colonne repart à 0
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 0, U à Z en dessous.

ABCDEFGHIJKLMNPQRST
UVWXYZ

l'écran, au bout de vingt images

0.8.1. Tout l'alphabet, avec deux boucles — de base, ailleurs

Le 0.8 trois lignes plus bas : tout l'alphabet sur les lignes 5 et 6.

C'est le 0.8, avec un seul changement : les **lignes** : 5 et 6 au lieu de 0 et 1, dans les deux `poser`.

Le reste ne bouge pas : A à T sur une ligne, U à Z sur la suivante.

C'est la version de base : une seule chose, à la deuxième place. Le 0.8.2 met les deux ensemble.

LE PROGRAMME — 1400 OCTETS

```
int main() {
    // Le changement : les lignes 0 et 1 → 5 et 6.
    for (uint8_t i = 0; i < 20; i++) {    // A à T : ligne 5
        poser(i, 5, ALPHABET[i]);
    }
    for (uint8_t i = 20; i < 26; i++) {    // U à Z : ligne 6
        poser(i - 20, 6, ALPHABET[i]);
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Tout l'alphabet sur les lignes 5 et 6.

ABCDEFGHIJKLMNOPQRSTUVWXYZ
UVWXYZ

l'écran, au bout de vingt images

0.8.2. Tout l'alphabet, avec deux boucles — doublé, deux positions

Tout l'alphabet deux fois : lignes 0 et 1, puis lignes 5 et 6.

C'est le 0.8 et le 0.8.1 réunis : l'alphabet du 0.8 (lignes 0 et 1), puis celui du 0.8.1 (lignes 5 et 6).

LE PROGRAMME — 1526 OCTETS

```
int main() {
    // 1. Lignes 0 et 1 (le 0.8)
    for (uint8_t i = 0; i < 20; i++) {    // A à T : ligne 0
        poser(i, 0, ALPHABET[i]);
    }
    for (uint8_t i = 20; i < 26; i++) {    // U à Z : ligne 1
        poser(i - 20, 1, ALPHABET[i]);
    }
    // 2. Lignes 5 et 6 (le 0.8.1)
    for (uint8_t i = 0; i < 20; i++) {    // A à T : ligne 5
        poser(i, 5, ALPHABET[i]);
    }
    for (uint8_t i = 20; i < 26; i++) {    // U à Z : ligne 6
        poser(i - 20, 6, ALPHABET[i]);
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Tout l'alphabet deux fois :
en haut, et trois lignes
plus bas.

A B C D E F G H I J K L M N O P Q R S T
U V W X Y Z

A B C D E F G H I J K L M N O P Q R S T
U V W X Y Z

l'écran, au bout de vingt images

0.9. Tout l'alphabet, avec une seule boucle

Le modulo donne la colonne, la division donne la ligne.

Ce qui est nouveau ici : `sizeof`, le modulo `%` et la division `/`. Le reste (`for`, `poser`, `ALPHABET[i]`) vient des chapitres précédents.

`sizeof(ALPHABET)` veut dire « la taille de ALPHABET » : le nombre de cases du tableau, **26**. On ne l'écrit plus à la main : si le tableau changeait de taille, la boucle suivrait toute seule. `i < sizeof(ALPHABET)` s'arrête donc après `i = 25`, la dernière lettre.

`i / 20` est une **division entière** : elle garde le quotient et **jette les virgules**. $5 / 20$ donne 0 (et non 0,25) ; $19 / 20$ donne 0 ; $20 / 20$ donne 1 ; $25 / 20$ donne 1. Tant que `i` est plus petit que 20, le résultat est 0 : c'est la **ligne 0**. À partir de 20, c'est 1 : la **ligne 1**.

`i % 20` se lit « i modulo 20 » : c'est le **reste** de cette même division. $5 \% 20$ donne 5 (20 ne rentre pas dans 5, tout reste) ; $19 \% 20$ donne 19 ; $20 \% 20$ donne 0 (la division tombe juste, il ne reste rien) ; $25 \% 20$ donne 5. Le reste monte de 0 à 19 puis **repart à 0** : c'est la **colonne**.

Imagine **20 places par rang**, comme au cinéma : `i / 20` dit combien de rangs **complets** sont déjà remplis (le rang de la lettre), `i % 20` dit combien de places il **reste** une fois ces rangs retirés (sa place dans le rang).

Tour par tour : `i = 0` → colonne 0, ligne 0, **A** · `i = 19` → colonne 19, ligne 0, **T** (la ligne est pleine) · `i = 20` → colonne **0**, ligne **1**, **U** (le reste repart à 0, le quotient passe à 1) · `i = 25` → colonne 5, ligne 1, **Z**.

C'est compliqué à lire : le chapitre suivant montre `poserS`, qui fait ce calcul à notre place.

LE PROGRAMME — 1352 OCTETS

```
int main() {
    // for : répète le bloc entre { } ; i part de 0 et augmente de 1 à chaque tour.
    // sizeof(ALPHABET) = 26, la taille du tableau : 26 tours, i va de 0 à 25.
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {

        // poser(colonne, ligne, tuile) :
        //   i % 20      = le RESTE de i divisé par 20 : 0, 1 ... 19, puis 0, 1 ... 5
        //               → la colonne, qui repart à 0 après la 19
        //   i / 20      = le QUOTIENT, sans virgule : 0 jusqu'à i = 19, puis 1
        //               → la ligne : 0 pour A à T, 1 pour U à Z
        //   ALPHABET[i] = la lettre n° i : A pour i = 0, Z pour i = 25
        poser(i % 20, i / 20, ALPHABET[i]);
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 0, U à Z
en dessous, comme au
0.8.

ABCDEFGHIJKLMN O P Q R S T
U V W X Y Z

l'écran, au bout de vingt images

0.9.1. Tout l'alphabet, avec une seule boucle — de base, ailleurs

Le 0.9 trois lignes plus bas : tout l'alphabet sur les lignes 5 et 6.

C'est le 0.9, avec un seul changement : la **ligne de départ** : `5 + i / 20` au lieu de `i / 20`. `i / 20` vaut 0 puis 1 ; en ajoutant 5, on obtient les lignes 5 puis 6.

Le reste ne bouge pas : A à T sur une ligne, U à Z sur la suivante.

C'est la version de base : une seule chose, à la deuxième place. Le 0.9.2 met les deux ensemble.

LE PROGRAMME — 1354 OCTETS

```
int main() {  
    // Le changement : les lignes 0 et 1 → 5 et 6.  
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {  
        poser(i % 20, 5 + i / 20, ALPHABET[i]);    // 5 + i / 20 : la ligne 5, puis 6  
    }  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

Tout l'alphabet sur les
lignes 5 et 6.

ABCDEFGHIJKLMNOPQRSTUVWXYZ
UVWXYZ

l'écran, au bout de vingt images

0.9.2. Tout l'alphabet, avec une seule boucle — doublé, deux positions

Tout l'alphabet deux fois : lignes 0 et 1, puis lignes 5 et 6.

C'est le 0.9 et le 0.9.1 réunis : l'alphabet du 0.9 (lignes 0 et 1), puis celui du 0.9.1 (lignes 5 et 6).

LE PROGRAMME — 1433 OCTETS

```
int main() {
    // 1. Lignes 0 et 1 (le 0.9)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poser(i % 20, 0 + i / 20, ALPHABET[i]);    // 0 + i / 20 : la ligne 0, puis 1
    }
    // 2. Lignes 5 et 6 (le 0.9.1)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poser(i % 20, 5 + i / 20, ALPHABET[i]);    // 5 + i / 20 : la ligne 5, puis 6
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Tout l'alphabet deux fois :
en haut, et trois lignes
plus bas.

A B C D E F G H I J K L M N O P Q R S T
U V W X Y Z

A B C D E F G H I J K L M N O P Q R S T
U V W X Y Z

l'écran, au bout de vingt images

0.10. Tout l'alphabet, avec poserS

poserS passe à la ligne tout seul : une seule boucle pour les 26 lettres.

Ce qui est nouveau ici : `poserS`. Le reste (`for`, `sizeof`, `ALPHABET[i]`) vient des chapitres précédents : c'est le programme du 0.9, en plus simple.

`sizeof(ALPHABET)` veut dire « la taille de ALPHABET » : le nombre de cases du tableau, **26**. On ne l'écrit plus à la main : si le tableau changeait de taille, la boucle suivrait toute seule. `i < sizeof(ALPHABET)` s'arrête donc après `i = 25`, la dernière lettre.

`poserS(colonne, ligne, tuile)` fait partie de la console, comme `ALPHABET` : on s'en sert sans rien déclarer. C'est `poser()`, mais qui **passe à la ligne tout seul** : une colonne trop grande (20 ou plus) repart à gauche, une ligne plus bas.

On lui donne donc simplement `i` comme colonne, sur la ligne 0. Tour par tour : **i = 0** → A en (0, 0) · **i = 19** → T en (19, 0), la ligne est pleine · **i = 20** → la colonne 20 n'existe pas : `poserS` pose U en (0, 1) · **i = 25** → Z en (5, 1).

Comment `poserS` trouve la place, pour les curieux : la ligne est le nombre de rangées de 20 déjà pleines ($20 / 20 = 1$), et la colonne ce qui reste une fois ces rangées retirées ($25 - 20 = 5$). En C++ : `i / 20` et `i % 20`. `poserS` fait ce calcul à notre place.

Le calcul du 0.9 a disparu du programme : `poserS` le fait à notre place.

LE PROGRAMME — 1360 OCTETS

```
int main() {
    // for : répète le bloc entre { } ; i part de 0 et augmente de 1 à chaque tour.
    // sizeof(ALPHABET) = 26, la taille du tableau : 26 tours, i va de 0 à 25.
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {

        // poserS(colonne, ligne, tuile) : comme poser(), mais passe à la ligne tout seul.
        // i = la colonne : de 0 à 19 elle tient sur la ligne 0 ;
        // à partir de 20, poserS repart à gauche une ligne plus bas
        // (i = 20 → U en (0, 1), i = 25 → Z en (5, 1))
        // 0 = la ligne de départ
        // ALPHABET[i] = la lettre n° i : A pour i = 0, Z pour i = 25
        poserS(i, 0, ALPHABET[i]);
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 0, U à Z en dessous, comme au 0.8.

ABCDEFGHIJKLMNOPQRST
UUVWXYZ

l'écran, au bout de vingt images

0.10.1. Tout l'alphabet, avec poserS — de base, ailleurs

Le 0.10 trois lignes plus bas : tout l'alphabet sur les lignes 5 et 6.

C'est le 0.10, avec un seul changement : la **ligne** : 5 au lieu de 0. `poserS` passe tout seul à la ligne suivante, la 6.

Le reste ne bouge pas : A à T sur une ligne, U à Z sur la suivante.

C'est la version de base : une seule chose, à la deuxième place. Le 0.10.2 met les deux ensemble.

LE PROGRAMME — 1362 OCTETS

```
int main() {
    // Le changement : les lignes 0 et 1 → 5 et 6.
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poserS(i, 5, ALPHABET[i]); // poserS passe tout seul à la ligne 6
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Tout l'alphabet sur les lignes 5 et 6.

ABCDEFGHIJKLMNOPQRSTUVWXYZ
UVWXYZ

l'écran, au bout de vingt images

0.10.2. Tout l'alphabet, avec poserS — doublé, deux positions

Tout l'alphabet deux fois : lignes 0 et 1, puis lignes 5 et 6.

C'est le 0.10 et le 0.10.1 réunis : l'alphabet du 0.10 (lignes 0 et 1), puis celui du 0.10.1 (lignes 5 et 6).

LE PROGRAMME — 1449 OCTETS

```
int main() {
    // 1. Lignes 0 et 1 (le 0.10)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poserS(i, 0, ALPHABET[i]); // poserS passe tout seul à la ligne 1
    }
    // 2. Lignes 5 et 6 (le 0.10.1)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poserS(i, 5, ALPHABET[i]); // poserS passe tout seul à la ligne 6
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Tout l'alphabet deux fois :
en haut, et trois lignes
plus bas.

ABCDEFGHIJKLMNOPQRSTUVWXYZ
ABCDEFGHIJKLMNOPQRSTUVWXYZ
ABCDEFGHIJKLMNOPQRSTUVWXYZ

l'écran, au bout de vingt images

0.11. Tout l'alphabet, avec textS

textS passe à la ligne tout seul : plus de calcul à écrire.

textS écrit comme texte, mais **passse à la ligne** quand la ligne est pleine : après la colonne 19, la suite reprend en colonne 0 de la ligne d'en dessous.

Le calcul du 0.9 ($i \% 20$, $i / 20$) est fait à **notre place**. texte(), lui, refuserait ce mot trop long.

Après la ligne 17, textS repart **en haut**, à la ligne 0.

LE PROGRAMME — 1321 OCTETS

```
int main() {
    textS(0, 0, "ABCDEFGH IJKLMNOPQRSTU VWXYZ"); // passe à la ligne après T

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

A à T sur la ligne 0, U à Z en dessous, comme au 0.9.

ABCDEFGHIJKLMN O PQRST
U V W X Y Z

l'écran, au bout de vingt images

0.11.1. Tout l'alphabet, avec textS — de base, ailleurs

Le 0.11 trois lignes plus bas : tout l'alphabet sur les lignes 5 et 6.

C'est le 0.11, avec un seul changement : la **ligne** : 5 au lieu de 0. `textS` passe tout seul à la ligne 6 après le T.

Le reste ne bouge pas : A à T sur une ligne, U à Z sur la suivante.

C'est la version de base : une seule chose, à la deuxième place. Le 0.11.2 met les deux ensemble.

LE PROGRAMME — 1321 OCTETS

```
int main() {  
    // Le changement : les lignes 0 et 1 → 5 et 6.  
    textS(0, 5, "ABCDEFGH IJKLMNOPQRSTUVWXYZ"); // passe à la ligne 6 après T  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

Tout l'alphabet sur les
lignes 5 et 6.

ABCDEFGHIJKLMNOPQRSTUVWXYZ
UVWXYZ

l'écran, au bout de vingt images

0.11.2. Tout l'alphabet, avec textS — doublé, deux positions

Tout l'alphabet deux fois : lignes 0 et 1, puis lignes 5 et 6.

C'est le 0.11 et le 0.11.1 réunis : l'alphabet du 0.11 (lignes 0 et 1), puis celui du 0.11.1 (lignes 5 et 6).

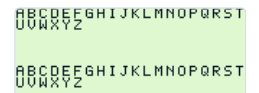
LE PROGRAMME — 1369 OCTETS

```
int main() {
    // 1. Lignes 0 et 1 (le 0.11)
    textS(0, 0, "ABCDEFGH IJKLMNOPQRSTUVWXYZ"); // passe à la ligne 1 après T
    // 2. Lignes 5 et 6 (le 0.11.1)
    textS(0, 5, "ABCDEFGH IJKLMNOPQRSTUVWXYZ"); // passe à la ligne 6 après T

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Tout l'alphabet deux fois :
en haut, et trois lignes
plus bas.



```
ABCDEFGHI JKLMNOPQRST
UVWXYZ
ABCDEFGHI JKLMNOPQRST
UVWXYZ
```

l'écran, au bout de vingt images

0.12. Une lettre toutes les secondes

Écrire **DANS** la boucle de jeu, et compter les images pour mesurer le temps.

Jusqu'ici, on écrivait **avant** la boucle de jeu, une fois pour toutes. Ici, on écrit **dedans** : à chaque image, la case (0, 0) reçoit la lettre du moment.

La console ne connaît ni les secondes ni les millisecondes : elle compte les images. Il y en a **60 par seconde** ; une image dure donc $1000 \div 60 \approx 16,7$ millisecondes. `image()` attend l'image suivante : chaque tour de boucle dure une image.

`IMAGES_PAR_SECONDE` et `SECONDES` donnent un nom à ces nombres. `SECONDES * IMAGES_PAR_SECONDE` vaut $1 \times 60 = 60$: quand `images` atteint 60, une seconde est passée. Pour changer la vitesse, on ne touche qu'à `SECONDES`.

Limite : `images` est un `uint8_t`, qui ne dépasse pas 255. On compte donc au plus 255 images, environ **4 secondes** : `SECONDES = 5` donnerait 300, trop grand. Le chapitre suivant montre `attendre()`, qui n'a pas cette limite.

Chaque nouvelle lettre **écrase** la précédente, comme au 0.3. Après Z, `lettre` revient à 0 : on repart de A.

LE PROGRAMME — 1364 OCTETS

```
const uint8_t IMAGES_PAR_SECONDE = 60; // la console affiche 60 images par seconde
const uint8_t SECONDES = 1;           // le temps entre deux lettres, en secondes

uint8_t lettre = 0; // l'indice de la lettre affichée
uint8_t images = 0; // les images comptées depuis la dernière lettre

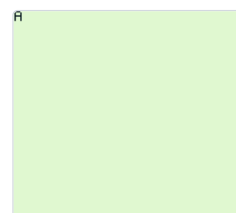
int main() {
    while (true) {
        image(); // attend l'image suivante : environ 16,7 ms
        images++; // une image de plus

        if (images == SECONDES * IMAGES_PAR_SECONDE) { // 1 x 60 = 60 images = 1 seconde
            images = 0; // on recommence à compter
            lettre++;    // la lettre suivante
            if (lettre == sizeof(ALPHABET)) lettre = 0; // après Z, on revient à A
        }

        poser(0, 0, ALPHABET[lettre]); // la même case : la lettre écrase la précédente
    }
}
```

CE QU'ON DOIT VOIR

En haut à gauche : A, puis
B une seconde plus tard,
puis C...



l'écran, au bout de vingt images

0.12.1. Une lettre toutes les secondes — de base, ailleurs

Le 0.12 en colonne 10 : la lettre qui change chaque seconde, au milieu de la ligne.

C'est le 0.12, avec un seul changement : la **colonne** du `poser` : 10 au lieu de 0.

Le **chronomètre ne change pas** : seule la case où l'on affiche la lettre a bougé.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.12.2 met les deux ensemble.

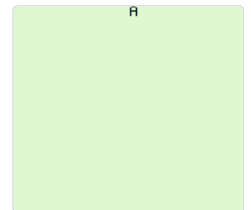
LE PROGRAMME — 1365 OCTETS

```
const uint8_t IMAGES_PAR_SECONDE = 60;
const uint8_t SECONDES = 1;
uint8_t lettre = 0;
uint8_t images = 0;

int main() {
    while (true) {
        image();
        images++;
        if (images == SECONDES * IMAGES_PAR_SECONDE) {
            images = 0;
            lettre++;
            if (lettre == sizeof(ALPHABET)) lettre = 0;
        }
        // Le changement : la colonne, 0 → 10.
        poser(10, 0, ALPHABET[lettre]);
    }
}
```

CE QU'ON DOIT VOIR

Une lettre en haut, au milieu de la ligne, qui change chaque seconde.



l'écran, au bout de vingt images

0.12.2. Une lettre toutes les secondes — doublé, deux positions

La même lettre à deux places : colonne 0 et colonne 10. Un seul chronomètre, deux poser.

C'est le 0.12 et le 0.12.1 réunis : deux `poser` de la même lettre, en colonne 0 et en colonne 10.

Les deux changent ensemble : c'est la **même** variable `lettre`, affichée à deux endroits. Pour deux rythmes différents, il faut deux variables : c'est le 0.14.

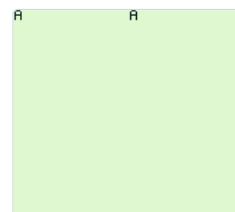
LE PROGRAMME — 1392 OCTETS

```
const uint8_t IMAGES_PAR_SECONDE = 60;
const uint8_t SECONDES = 1;
uint8_t lettre = 0;
uint8_t images = 0;

int main() {
    while (true) {
        image();
        images++;
        if (images == SECONDES * IMAGES_PAR_SECONDE) {
            images = 0;
            lettre++;
            if (lettre == sizeof(ALPHABET)) lettre = 0;
        }
        poser(0, 0, ALPHABET[lettre]); // colonne 0 (le 0.12)
        poser(10, 0, ALPHABET[lettre]); // colonne 10 (le 0.12.1) : la même lettre
    }
}
```

CE QU'ON DOIT VOIR

Deux fois la même lettre, en colonne 0 et en colonne 10, qui changent ensemble chaque seconde.



l'écran, au bout de vingt images

0.13. Une lettre toutes les secondes, avec attendre

Le temps dit directement en secondes : `attendre(1)`.

Ce qui est nouveau ici : `attendre(secondes)`. Elle fait partie de la console, comme `ALPHABET` : on s'en sert sans rien déclarer.

`attendre(1)` attend **une seconde**, puis le programme continue à la ligne suivante. À l'intérieur, elle compte les images à notre place : $1 \times 60 = 60$ images. On peut aller jusqu'à `attendre(255)`, plus de 4 minutes.

Le programme se lit dans l'ordre : **afficher** la lettre, **attendre** une seconde, passer à la **suivante**. Il n'y a plus de compteur `images`, ni d'`image()` : `attendre` s'en occupe.

La différence avec le 0.12 : pendant `attendre()`, le programme est **arrêté**. Rien d'autre ne se passe : la manette n'est pas lue. En comptant les images soi-même (0.12), la boucle de jeu continue de tourner pendant l'attente. Pour un jeu, c'est souvent ce qu'il faut ; pour faire patienter, `attendre` est plus simple.

La limite : `attendre` ne sait pas **ce qui** doit attendre : il fige **tout** le programme. Avec `attendre(10)`, aucune autre chose ne peut bouger pendant 10 secondes. Pour que plusieurs choses aient chacune leur rythme, voir le chapitre suivant.

LE PROGRAMME — 1351 OCTETS

```
uint8_t lettre = 0; // l'indice de la lettre affichée

int main() {
    while (true) {
        poser(0, 0, ALPHABET[lettre]); // 1. affiche la lettre en haut à gauche
        attendre(1);                  // 2. attend 1 seconde : le nombre entre ( ) est en SECONDES
                                     // (tout le programme s'arrête pendant ce temps)

        lettre++;                      // 3. la lettre suivante
        if (lettre == sizeof(ALPHABET)) lettre = 0; // après Z, on revient à A
    }
}
```

CE QU'ON DOIT VOIR

En haut à gauche : A, puis
B une seconde plus tard,
puis C..., comme au 0.12.

A

l'écran, au bout de vingt images

0.13.1. Une lettre toutes les secondes, avec attendre — de base, ailleurs

Le 0.13 en colonne 10.

C'est le 0.13, avec un seul changement : la **colonne** du `poser` : 10 au lieu de 0.

`attendre(1)` **ne change pas** : une seconde entre deux lettres.

C'est la version de base : une seule chose, à la deuxième place. Le 0.13.2 met les deux ensemble.

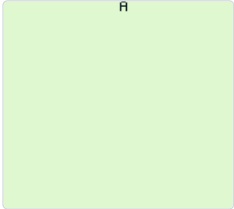
LE PROGRAMME — 1352 OCTETS

```
uint8_t lettre = 0;

int main() {
    while (true) {
        poser(10, 0, ALPHABET[lettre]); // le changement : colonne 0 → 10
        attendre(1);
        lettre++;
        if (lettre == sizeof(ALPHABET)) lettre = 0;
    }
}
```

CE QU'ON DOIT VOIR

Une lettre au milieu de la ligne du haut, qui change chaque seconde.

R

l'écran, au bout de vingt images

0.13.2. Une lettre toutes les secondes, avec attendre — doublé, deux positions

Deux poser avant attendre : la même lettre en colonne 0 et en colonne 10.

C'est le 0.13 et le 0.13.1 réunis : deux `poser`, avant le même `attendre(1)`.

`attendre` **arrête tout** : les deux lettres sont posées, puis le programme attend une seconde, et recommence.

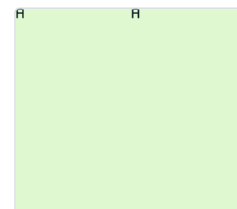
LE PROGRAMME — 1379 OCTETS

```
uint8_t lettre = 0;

int main() {
    while (true) {
        poser(0, 0, ALPHABET[lettre]);    // colonne 0 (le 0.13)
        poser(10, 0, ALPHABET[lettre]);   // colonne 10 (le 0.13.1)
        attendre(1);
        lettre++;
        if (lettre == sizeof(ALPHABET)) lettre = 0;
    }
}
```

CE QU'ON DOIT VOIR

La même lettre en
colonne 0 et en colonne
10, qui change chaque
seconde.



l'écran, au bout de vingt images

0.14. Deux choses, deux rythmes

Un compteur par chose : chacune avance à son rythme, et rien ne s'arrête.

Le problème : `attendre()` fige tout le programme. Impossible, avec lui, de faire changer une lettre toutes les secondes **et** une autre quatre fois par seconde : pendant que l'une attend, l'autre attend aussi.

La solution : on n'arrête plus jamais le programme. La boucle tourne sans cesse, un tour par image (60 par seconde), et **chaque chose a son propre compteur** d'images. C'est la méthode du 0.12, faite deux fois.

La lettre lente (ligne 0) : son compteur `imagesLente` avance d'un à chaque tour ; à **60**, une seconde est passée : elle change de lettre, et son compteur repart de 0.

La lettre rapide (ligne 2) : son compteur `imagesRapide` avance en même temps ; à **15**, un quart de seconde est passé ($60 \div 4 = 15$) : elle change, et son compteur repart de 0. Elle change donc **4 fois** pendant que la lente change **une fois**.

Les deux compteurs sont **indépendants** : on peut changer le 15 sans toucher au 60. Et comme la boucle ne s'arrête jamais, on pourrait lire la manette en même temps. C'est comme cela que tourne un jeu.

Le déroulé dans le temps : 0 s (tour 0) : A en haut, A en bas · **0,25 s** (tour 15) : `imagesRapide` atteint 15 et repart à 0, la rapide passe à **B** · **0,5 s** (tour 30) : la rapide passe à **C** · **0,75 s** (tour 45) : la rapide passe à **D** · **1 s** (tour 60) : `imagesLente` atteint 60 **et** `imagesRapide` atteint 15 : **les deux** changent, la lente passe à **B**, la rapide à **E**.

Rappels : `++` veut dire « ajoute 1 » ; `==` compare (« est égal à ? »), alors qu'un seul `=` range une valeur ; `sizeof(ALPHABET)` vaut 26, le nombre de lettres.

LE PROGRAMME — 1456 OCTETS

```
// _____
// LES VARIABLES : écrites AVANT main(), elles existent pendant
// tout le programme et gardent leur valeur d'un tour à l'autre.
// uint8_t = un nombre entier de 0 à 255.
// _____

uint8_t lente = 0;           // QUELLE lettre montre la lente : 0 = A, 1 = B ... 25 = Z
uint8_t rapide = 0;          // QUELLE lettre montre la rapide : 0 = A, 1 = B ... 25 = Z

uint8_t imagesLente = 0;     // le CHRONOMÈTRE de la lente :
                             // combien d'images depuis son dernier changement
uint8_t imagesRapide = 0;    // le CHRONOMÈTRE de la rapide, séparé de l'autre

int main() {

    // La boucle de jeu : elle ne s'arrête JAMAIS.
    // Chaque tour dure exactement une image, soit 1/60e de seconde.
    while (true) {

        image();              // attend l'image suivante (= 16,7 ms) :
                             // c'est ce qui cadence la boucle à 60 tours par seconde

        // À chaque tour, UNE image est passée : les deux chronomètres
        // avancent d'un cran, en même temps, chacun de son côté.
        imagesLente++;        // ++ veut dire « ajoute 1 »
        imagesRapide++;

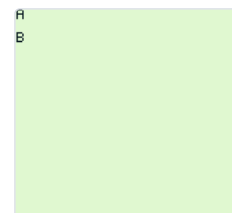
        // — LA LENTE : elle change toutes les 60 images = toutes les secondes —
        if (imagesLente == 60) {           // == veut dire « est égal à ? »
            // (un seul = voudrait dire « range »)
            imagesLente = 0;               // son chronomètre repart de zéro
            lente++;                       // elle passe à la lettre suivante
            if (lente == sizeof(ALPHABET)) lente = 0; // sizeof(ALPHABET) = 26 :
                                                // après Z (25), 26 n'existe pas → retour à A (0)
        }

        // — LA RAPIDE : elle change toutes les 15 images = 4 fois par seconde —
        // (60 images ÷ 15 = 4 changements par seconde)
        if (imagesRapide == 15) {
            imagesRapide = 0;              // son chronomètre repart de zéro
            rapide++;                      // elle passe à la lettre suivante
            if (rapide == sizeof(ALPHABET)) rapide = 0; // après Z, retour à A
        }

        // — L'AFFICHAGE : à CHAQUE tour, on repose les deux lettres —
        // Si rien n'a changé, on repose la même lettre au même endroit : ça ne se voit pas.
        // Si l'indice a changé, la nouvelle lettre écrase l'ancienne (comme au 0.3).
        poser(0, 0, ALPHABET[lente]);     // colonne 0, ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]);    // colonne 0, ligne 2 : la rapide
                                                // (la ligne 1 reste vide pour les séparer)
    }
}
```

CE QU'ON DOIT VOIR

Deux lettres : celle du bas change quatre fois pendant que celle du haut change une fois.



l'écran, au bout de vingt images

0.14.1. Deux choses, deux rythmes — de base, ailleurs

Le 0.14 en colonne 10 : la lente et la rapide, au milieu de la ligne.

C'est le 0.14, avec un seul changement : la **colonne** des deux `poser` : 10 au lieu de 0.

Les **chronomètres ne changent pas** : seules les cases où l'on affiche les deux lettres ont bougé.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.14.2 met les deux ensemble.

LE PROGRAMME — 1458 OCTETS

```
//
// LES VARIABLES : écrites AVANT main(), elles existent pendant
// tout le programme et gardent leur valeur d'un tour à l'autre.
// uint8_t = un nombre entier de 0 à 255.
//
uint8_t lente = 0;           // QUELLE lettre montre la lente : 0 = A, 1 = B ... 25 = Z
uint8_t rapide = 0;         // QUELLE lettre montre la rapide : 0 = A, 1 = B ... 25 = Z

uint8_t imagesLente = 0;    // le CHRONOMÈTRE de la lente :
                             // combien d'images depuis son dernier changement
uint8_t imagesRapide = 0;   // le CHRONOMÈTRE de la rapide, séparé de l'autre

int main() {

    // La boucle de jeu : elle ne s'arrête JAMAIS.
    // Chaque tour dure exactement une image, soit 1/60e de seconde.
    while (true) {

        image();              // attend l'image suivante (= 16,7 ms) :
                             // c'est ce qui cadence la boucle à 60 tours par seconde

        // À chaque tour, UNE image est passée : les deux chronomètres
        // avancent d'un cran, en même temps, chacun de son côté.
        imagesLente++;        // ++ veut dire « ajoute 1 »
        imagesRapide++;

        // — LA LENTE : elle change toutes les 60 images = toutes les secondes —
        if (imagesLente == 60) { // == veut dire « est égal à ? »
                                // (un seul = voudrait dire « range »)
            imagesLente = 0;    // son chronomètre repart de zéro
            lente++;           // elle passe à la lettre suivante
            if (lente == sizeof(ALPHABET)) lente = 0; // sizeof(ALPHABET) = 26 :
                                                    // après Z (25), 26 n'existe pas → retour à A (0)
        }

        // — LA RAPIDE : elle change toutes les 15 images = 4 fois par seconde —
        // (60 images ÷ 15 = 4 changements par seconde)
        if (imagesRapide == 15) {
            imagesRapide = 0;    // son chronomètre repart de zéro
            rapide++;           // elle passe à la lettre suivante
            if (rapide == sizeof(ALPHABET)) rapide = 0; // après Z, retour à A
        }

        // — L'AFFICHAGE : à CHAQUE tour, on repose les deux lettres —
        // Si rien n'a changé, on repose la même lettre au même endroit : ça ne se voit pas.
        // Si l'indice a changé, la nouvelle lettre écrase l'ancienne (comme au 0.3).
        poser(10, 0, ALPHABET[lente]); // la lente : colonne 0 → 10
        poser(10, 2, ALPHABET[rapide]); // la rapide : colonne 0 → 10
                                         // (la ligne 1 reste vide pour les séparer)
    }
}
```

CE QU'ON DOIT VOIR

La lente et la rapide, en colonne 10.

A
B

l'écran, au bout de vingt images

0.14.2. Deux choses, deux rythmes — doublé, deux positions

Les deux rythmes à deux places : en colonne 0 et en colonne 10.

C'est le 0.14 et le 0.14.1 réunis : les deux `poser` du 0.14, et les deux du 0.14.1.

Quatre `poser`, deux variables : la lente est la même en colonne 0 et en colonne 10, la rapide aussi.

LE PROGRAMME — 1513 OCTETS

```
// -----
// LES VARIABLES : écrites AVANT main(), elles existent pendant
// tout le programme et gardent leur valeur d'un tour à l'autre.
// uint8_t = un nombre entier de 0 à 255.
// -----

uint8_t lente = 0;           // QUELLE lettre montre la lente : 0 = A, 1 = B ... 25 = Z
uint8_t rapide = 0;          // QUELLE lettre montre la rapide : 0 = A, 1 = B ... 25 = Z

uint8_t imagesLente = 0;     // le CHRONOMÈTRE de la lente :
                             // combien d'images depuis son dernier changement
uint8_t imagesRapide = 0;     // le CHRONOMÈTRE de la rapide, séparé de l'autre

int main() {

    // La boucle de jeu : elle ne s'arrête JAMAIS.
    // Chaque tour dure exactement une image, soit 1/60e de seconde.
    while (true) {

        image();

        // attend l'image suivante (= 16,7 ms) :
        // c'est ce qui cadence la boucle à 60 tours par seconde

        // À chaque tour, UNE image est passée : les deux chronomètres
        // avancent d'un cran, en même temps, chacun de son côté.
        imagesLente++;        // ++ veut dire « ajoute 1 »
        imagesRapide++;

        // — LA LENTE : elle change toutes les 60 images = toutes les secondes —
        if (imagesLente == 60) {
            // == veut dire « est égal à ? »
            // (un seul = voudrait dire « range »)
            imagesLente = 0;      // son chronomètre repart de zéro
            lente++;              // elle passe à la lettre suivante
            if (lente == sizeof(ALPHABET)) lente = 0; // sizeof(ALPHABET) = 26 :
            // après Z (25), 26 n'existe pas → retour à A (0)
        }

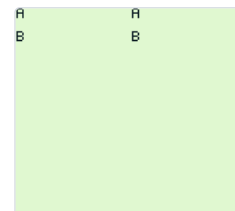
        // — LA RAPIDE : elle change toutes les 15 images = 4 fois par seconde —
        // (60 images ÷ 15 = 4 changements par seconde)
        if (imagesRapide == 15) {
            imagesRapide = 0;      // son chronomètre repart de zéro
            rapide++;              // elle passe à la lettre suivante
            if (rapide == sizeof(ALPHABET)) rapide = 0; // après Z, retour à A
        }

        // — L'AFFICHAGE : à CHAQUE tour, on repose les deux lettres —
        // Si rien n'a changé, on repose la même lettre au même endroit : ça ne se voit pas.
        // Si l'indice a changé, la nouvelle lettre écrase l'ancienne (comme au 0.3).
        poser(0, 0, ALPHABET[lente]); // colonne 0, ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]); // colonne 0, ligne 2 : la rapide
        poser(10, 0, ALPHABET[lente]); // la lente, en colonne 10 (le 0.14.1)
        poser(10, 2, ALPHABET[rapide]); // la rapide, en colonne 10 (le 0.14.1)
        // (la ligne 1 reste vide pour les séparer)

    }
}
```

CE QU'ON DOIT VOIR

La lente et la rapide, en colonne 0 et en colonne 10.



l'écran, au bout de vingt images

0.15. Deux rythmes, un seul chronomètre

Le même programme en plus court : un seul compteur, et le modulo fait le reste.

C'est le programme du 0.14, en plus simple. L'écran est exactement le même : la lettre du bas change quatre fois pendant que celle du haut change une fois.

Un seul chronomètre : au lieu de deux compteurs, `images` compte de 0 à 59, puis repart à 0. Il fait donc un tour complet **chaque seconde**.

La lente change quand le chronomètre repasse par 0 : une fois par tour, donc une fois par seconde.

La rapide change quand `images % 15 == 0` : quand `images` est un **multiple de 15**, c'est-à-dire 0, 15, 30 ou 45. Le reste de la division par 15 ne vaut 0 que pour ces nombres : $30 \% 15 = 0$, mais $31 \% 15 = 1$. Cela fait **4 fois** par tour, donc 4 fois par seconde.

Le retour à A sans `if` : `(rapide + 1) % sizeof(ALPHABET)` ajoute 1, puis garde le reste de la division par 26. Tant qu'on est sous 26, le reste est le nombre lui-même ($7 \% 26 = 7$) ; à 26, il retombe à 0 ($26 \% 26 = 0$) : après Z, A.

Ce qu'on gagne : 3 variables au lieu de 4, deux `if` d'une ligne au lieu de deux blocs, et un programme plus léger. Ce qu'on perd : les deux rythmes sont liés au même chronomètre ; au 0.14, on pouvait choisir n'importe quelle durée pour chacun, sans rapport entre eux.

LE PROGRAMME — 1438 OCTETS

```
uint8_t lente = 0;    // QUELLE lettre montre la lente : 0 = A ... 25 = Z
uint8_t rapide = 0;   // QUELLE lettre montre la rapide : 0 = A ... 25 = Z
uint8_t images = 0;   // UN SEUL chronomètre : il compte de 0 à 59, soit 1 seconde

int main() {
    while (true) {
        image();           // attend l'image suivante : 60 tours par seconde

        // Le chronomètre avance d'une image, et repart à 0 après 59 :
        // il fait un tour complet chaque seconde.
        images++;
        if (images == 60) images = 0;

        // La RAPIDE : quand images vaut 0, 15, 30 ou 45 (4 fois par seconde).
        // images % 15 = le reste de images divisé par 15 ;
        // il vaut 0 seulement pour les multiples de 15.
        // (rapide + 1) % sizeof(ALPHABET) : la lettre suivante,
        // et après Z (25 + 1 = 26), 26 % 26 = 0 : retour à A.
        if (images % 15 == 0) rapide = (rapide + 1) % sizeof(ALPHABET);

        // La LENTE : quand le chronomètre repasse par 0 (1 fois par seconde).
        if (images == 0) lente = (lente + 1) % sizeof(ALPHABET);

        poser(0, 0, ALPHABET[lente]); // ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]); // ligne 2 : la rapide
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.14 : la lettre du bas change quatre fois pendant que celle du haut change une fois.



l'écran, au bout de vingt images

0.15.1. Deux rythmes, un seul chronomètre — de base, ailleurs

Le 0.15 en colonne 10 : la lente et la rapide, au milieu de la ligne.

C'est le 0.15, avec un seul changement : la **colonne** des deux `poser` : 10 au lieu de 0.

Les **chronomètres ne changent pas** : seules les cases où l'on affiche les deux lettres ont bougé.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.15.2 met les deux ensemble.

LE PROGRAMME — 1440 OCTETS

```
uint8_t lente = 0;    // QUELLE lettre montre la lente : 0 = A ... 25 = Z
uint8_t rapide = 0;   // QUELLE lettre montre la rapide : 0 = A ... 25 = Z
uint8_t images = 0;   // UN SEUL chronomètre : il compte de 0 à 59, soit 1 seconde

int main() {
    while (true) {
        image();                // attend l'image suivante : 60 tours par seconde

        // Le chronomètre avance d'une image, et repart à 0 après 59 :
        // il fait un tour complet chaque seconde.
        images++;
        if (images == 60) images = 0;

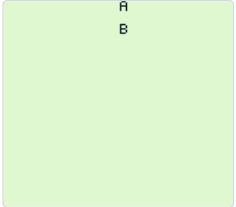
        // La RAPIDE : quand images vaut 0, 15, 30 ou 45 (4 fois par seconde).
        // images % 15 = le reste de images divisé par 15 ;
        // il vaut 0 seulement pour les multiples de 15.
        // (rapide + 1) % sizeof(ALPHABET) : la lettre suivante,
        // et après Z (25 + 1 = 26), 26 % 26 = 0 : retour à A.
        if (images % 15 == 0) rapide = (rapide + 1) % sizeof(ALPHABET);

        // La LENTE : quand le chronomètre repasse par 0 (1 fois par seconde).
        if (images == 0) lente = (lente + 1) % sizeof(ALPHABET);

        poser(10, 0, ALPHABET[lente]);    // la lente : colonne 0 → 10
        poser(10, 2, ALPHABET[rapide]);   // la rapide : colonne 0 → 10
    }
}
```

CE QU'ON DOIT VOIR

La lente et la rapide, en colonne 10.



l'écran, au bout de vingt images

0.15.2. Deux rythmes, un seul chronomètre — doublé, deux positions

Les deux rythmes à deux places : en colonne 0 et en colonne 10.

C'est le 0.15 et le 0.15.1 réunis : les deux `poser` du 0.15, et les deux du 0.15.1.

Quatre `poser`, deux variables : la lente est la même en colonne 0 et en colonne 10, la rapide aussi.

LE PROGRAMME — 1495 OCTETS

```
uint8_t lente = 0;    // QUELLE lettre montre la lente : 0 = A ... 25 = Z
uint8_t rapide = 0;   // QUELLE lettre montre la rapide : 0 = A ... 25 = Z
uint8_t images = 0;   // UN SEUL chronomètre : il compte de 0 à 59, soit 1 seconde

int main() {
    while (true) {
        image();
        // attend l'image suivante : 60 tours par seconde

        // Le chronomètre avance d'une image, et repart à 0 après 59 :
        // il fait un tour complet chaque seconde.
        images++;
        if (images == 60) images = 0;

        // La RAPIDE : quand images vaut 0, 15, 30 ou 45 (4 fois par seconde).
        // images % 15 = le reste de images divisé par 15 ;
        // il vaut 0 seulement pour les multiples de 15.
        // (rapide + 1) % sizeof(ALPHABET) : la lettre suivante,
        // et après Z (25 + 1 = 26), 26 % 26 = 0 : retour à A.
        if (images % 15 == 0) rapide = (rapide + 1) % sizeof(ALPHABET);

        // La LENTE : quand le chronomètre repasse par 0 (1 fois par seconde).
        if (images == 0) lente = (lente + 1) % sizeof(ALPHABET);

        poser(0, 0, ALPHABET[lente]); // ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]); // ligne 2 : la rapide
        poser(10, 0, ALPHABET[lente]); // la lente, en colonne 10 (le 0.15.1)
        poser(10, 2, ALPHABET[rapide]); // la rapide, en colonne 10 (le 0.15.1)
    }
}
```

CE QU'ON DOIT VOIR

La lente et la rapide, en colonne 0 et en colonne 10.

A	A
B	B

l'écran, au bout de vingt images

0.16. Deux rythmes, écrits en millisecondes

ms(1000) et ms(250) : chaque durée s'écrit en millisecondes, la console la traduit en images.

C'est le programme du 0.14, mais les durées ne sont plus des nombres d'images à calculer de tête : elles s'écrivent en **millisecondes** (millièmes de seconde). 1000 ms = 1 seconde ; 250 ms = un quart de seconde.

Ce qui est nouveau ici : `ms(durée)`. Elle fait partie de la console, comme `ALPHABET` : on s'en sert sans rien déclarer. Elle **traduit** une durée en millisecondes en nombre d'images, car la console ne sait compter que les images.

La traduction : une image dure $1000 \div 60 \approx 16,7$ ms. `ms(1000)` donne donc 60 images, `ms(250)` donne 15 images, `ms(500)` donne 30 images. Le résultat est arrondi à l'image la plus proche : `ms(100)` donne 6 images, soit 100 ms environ.

Elle ne coûte rien : la traduction est faite par le compilateur, avant que la cartouche existe. `ms(250)` est remplacé par 15, et la console ne voit qu'un nombre. C'est pour cela qu'il faut écrire la durée **en clair** : `ms(250)`, et non `ms(une_variable)`.

Les constantes `DUREE_LENTE` et `DUREE_RAPIDE` rangent les deux durées en haut du programme : pour changer un rythme, on change un seul nombre, en millisecondes.

La limite : les compteurs sont des `uint8_t`, qui s'arrêtent à 255 images. On peut donc aller jusqu'à `ms(4250)`, environ 4 secondes. Au-delà, le compilateur refuse et l'explique. En dessous de 17 ms (moins d'une image), il refuse aussi : la console ne sait pas compter plus fin.

LE PROGRAMME — 1456 OCTETS

```
// -----
// LES DURÉES, EN MILLISECONDES (1000 ms = 1 seconde).
// ms() les traduit en images (60 images = 1000 ms) :
// ms(1000) = 60 images, ms(250) = 15 images.
// -----

const uint8_t DUREE_LENTE = ms(1000); // la lente change toutes les 1000 ms (1 seconde)
const uint8_t DUREE_RAPIDE = ms(250); // la rapide change toutes les 250 ms (4 fois par seconde)

uint8_t lente = 0; // QUELLE lettre montre la lente : 0 = A ... 25 = Z
uint8_t rapide = 0; // QUELLE lettre montre la rapide : 0 = A ... 25 = Z

uint8_t imagesLente = 0; // le CHRONOMÈTRE de la lente, en images
uint8_t imagesRapide = 0; // le CHRONOMÈTRE de la rapide, en images

int main() {
    while (true) {
        image(); // attend l'image suivante : un tour = 16,7 ms

        imagesLente++; // une image de plus pour chaque chronomètre
        imagesRapide++;

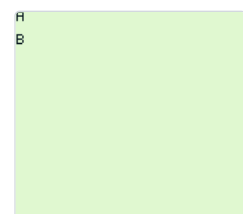
        // — LA LENTE : quand son chronomètre atteint DUREE_LENTE (1000 ms) —
        if (imagesLente == DUREE_LENTE) {
            imagesLente = 0; // son chronomètre repart de zéro
            lente++; // la lettre suivante
            if (lente == sizeof(ALPHABET)) lente = 0; // après Z, retour à A
        }

        // — LA RAPIDE : quand son chronomètre atteint DUREE_RAPIDE (250 ms) —
        if (imagesRapide == DUREE_RAPIDE) {
            imagesRapide = 0; // son chronomètre repart de zéro
            rapide++; // la lettre suivante
            if (rapide == sizeof(ALPHABET)) rapide = 0; // après Z, retour à A
        }

        poser(0, 0, ALPHABET[lente]); // ligne 0 : la lente (toutes les 1000 ms)
        poser(0, 2, ALPHABET[rapide]); // ligne 2 : la rapide (toutes les 250 ms)
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.14 : la lettre du bas change quatre fois pendant que celle du haut change une fois.



l'écran, au bout de vingt images

0.16.1. Deux rythmes, écrits en millisecondes — de base, ailleurs

Le 0.16 en colonne 10 : la lente et la rapide, au milieu de la ligne.

C'est le 0.16, avec un seul changement : la **colonne** des deux `poser` : 10 au lieu de 0.

Les **chronomètres ne changent pas** : seules les cases où l'on affiche les deux lettres ont bougé.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.16.2 met les deux ensemble.

LE PROGRAMME — 1458 OCTETS

```
// -----  
// LES DURÉES, EN MILLISECONDES (1000 ms = 1 seconde).  
// ms() les traduit en images (60 images = 1000 ms) :  
// ms(1000) = 60 images, ms(250) = 15 images.  
// -----  
  
const uint8_t DUREE_LENTE = ms(1000); // la lente change toutes les 1000 ms (1 seconde)  
const uint8_t DUREE_RAPIDE = ms(250); // la rapide change toutes les 250 ms (4 fois par seconde)  
  
uint8_t lente = 0; // QUELLE lettre montre la lente : 0 = A ... 25 = Z  
uint8_t rapide = 0; // QUELLE lettre montre la rapide : 0 = A ... 25 = Z  
  
uint8_t imagesLente = 0; // le CHRONOMÈTRE de la lente, en images  
uint8_t imagesRapide = 0; // le CHRONOMÈTRE de la rapide, en images  
  
int main() {  
    while (true) {  
        image(); // attend l'image suivante : un tour ≈ 16,7 ms  
  
        imagesLente++; // une image de plus pour chaque chronomètre  
        imagesRapide++;  
  
        // — LA LENTE : quand son chronomètre atteint DUREE_LENTE (1000 ms) —  
        if (imagesLente == DUREE_LENTE) {  
            imagesLente = 0; // son chronomètre repart de zéro  
            lente++; // la lettre suivante  
            if (lente == sizeof(ALPHABET)) lente = 0; // après Z, retour à A  
        }  
  
        // — LA RAPIDE : quand son chronomètre atteint DUREE_RAPIDE (250 ms) —  
        if (imagesRapide == DUREE_RAPIDE) {  
            imagesRapide = 0; // son chronomètre repart de zéro  
            rapide++; // la lettre suivante  
            if (rapide == sizeof(ALPHABET)) rapide = 0; // après Z, retour à A  
        }  
  
        poser(10, 0, ALPHABET[lente]); // la lente : colonne 0 → 10  
        poser(10, 2, ALPHABET[rapide]); // la rapide : colonne 0 → 10  
    }  
}
```

CE QU'ON DOIT VOIR

La lente et la rapide, en colonne 10.

A
B

l'écran, au bout de vingt images

0.16.2. Deux rythmes, écrits en millisecondes — doublé, deux positions

Les deux rythmes à deux places : en colonne 0 et en colonne 10.

C'est le 0.16 et le 0.16.1 réunis : les deux `poser` du 0.16, et les deux du 0.16.1.

Quatre `poser`, deux variables : la lente est la même en colonne 0 et en colonne 10, la rapide aussi.

LE PROGRAMME — 1513 OCTETS

```
// _____
// LES DURÉES, EN MILLISECONDES (1000 ms = 1 seconde).
// ms() les traduit en images (60 images = 1000 ms) :
// ms(1000) = 60 images, ms(250) = 15 images.
// _____

const uint8_t DUREE_LENTE = ms(1000); // la lente change toutes les 1000 ms (1 seconde)
const uint8_t DUREE_RAPIDE = ms(250); // la rapide change toutes les 250 ms (4 fois par seconde)

uint8_t lente = 0; // QUELLE lettre montre la lente : 0 = A ... 25 = Z
uint8_t rapide = 0; // QUELLE lettre montre la rapide : 0 = A ... 25 = Z

uint8_t imagesLente = 0; // le CHRONOMÈTRE de la lente, en images
uint8_t imagesRapide = 0; // le CHRONOMÈTRE de la rapide, en images

int main() {
  while (true) {
    image(); // attend l'image suivante : un tour ≈ 16,7 ms

    imagesLente++; // une image de plus pour chaque chronomètre
    imagesRapide++;

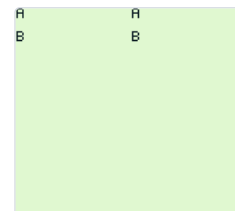
    // — LA LENTE : quand son chronomètre atteint DUREE_LENTE (1000 ms) —
    if (imagesLente == DUREE_LENTE) {
      imagesLente = 0; // son chronomètre repart de zéro
      lente++; // la lettre suivante
      if (lente == sizeof(ALPHABET)) lente = 0; // après Z, retour à A
    }

    // — LA RAPIDE : quand son chronomètre atteint DUREE_RAPIDE (250 ms) —
    if (imagesRapide == DUREE_RAPIDE) {
      imagesRapide = 0; // son chronomètre repart de zéro
      rapide++; // la lettre suivante
      if (rapide == sizeof(ALPHABET)) rapide = 0; // après Z, retour à A
    }

    poser(0, 0, ALPHABET[lente]); // ligne 0 : la lente (toutes les 1000 ms)
    poser(0, 2, ALPHABET[rapide]); // ligne 2 : la rapide (toutes les 250 ms)
    poser(10, 0, ALPHABET[lente]); // la lente, en colonne 10 (le 0.16.1)
    poser(10, 2, ALPHABET[rapide]); // la rapide, en colonne 10 (le 0.16.1)
  }
}
```

CE QU'ON DOIT VOIR

La lente et la rapide, en colonne 0 et en colonne 10.



l'écran, au bout de vingt images

0.17. Une lettre qui avance de 5 cases

Bouger, c'est effacer la lettre à sa place, puis la réécrire un peu plus loin — et s'arrêter après le nombre de pas voulu.

La lettre A avance d'une colonne toutes les **15 images**, soit quatre fois par seconde.

Pour bouger, on **efface** d'abord l'ancienne place avec `effacer(x, 0, 1)`, une case ; puis on ajoute 1 à `x`, et la lettre est réécrite à sa nouvelle place. Sans l'effacement, elle laisserait une traînée de A derrière elle.

Ce qui est nouveau ici : la variable `pas`. Elle range le **nombre de cases** à parcourir : 5. Pour que la lettre aille plus loin ou moins loin, on change ce seul nombre, en haut du programme, sans toucher au reste.

L'arrêt : la condition `if (x < pas)` compare la colonne de la lettre au nombre de pas. Au départ `x` vaut 0 : $0 < 5$ est vrai, le chronomètre tourne et la lettre avance. Après le cinquième pas, `x` vaut 5 : $5 < 5$ est **faux**, on ne rentre plus dans le bloc, `x` ne change plus... et la lettre reste en colonne 5.

La boucle `while (true)` continue pourtant de tourner : la lettre est simplement réécrite, à chaque image, à la même place.

LE PROGRAMME — 1389 OCTETS

```
uint8_t pas = 5;      // le NOMBRE DE CASES à parcourir : change-le pour aller plus ou moins loin
uint8_t x = 0;        // la colonne de la lettre : elle part de 0 (tout à gauche)
uint8_t images = 0;   // le chronomètre, en images

int main() {
    while (true) {
        image();           // attend l'image suivante (60 par seconde)

        if (x < pas) {     // tant que la lettre n'a pas fait tous ses pas...
            images++;      // ..le chronomètre tourne
            if (images == 15) { // 4 fois par seconde :
                images = 0;  // le chronomètre repart de zéro
                effacer(x, 0, 1); // 1. efface l'ancienne place
                x++;         // 2. une colonne plus loin
            }
        }                 // x vaut pas (5) : on ne rentre plus ici, elle s'arrête

        poser(x, 0, ALPHABET[0]); // 3. la lettre à sa place (nouvelle ou finale)
    }
}
```

CE QU'ON DOIT VOIR

Un A qui avance de 5 cases vers la droite, puis s'arrête en colonne 5.

A

l'écran, au bout de vingt images

0.17.1. Une lettre qui avance de 5 cases — de base, ailleurs

Le 0.17 sur la ligne 8, avec le B.

C'est le 0.17, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : la **ligne 8** au lieu de la ligne 0, dans `effacer` et dans `poser`.

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.17.2 met les deux ensemble.

LE PROGRAMME — 1391 OCTETS

```
uint8_t pas = 5;      // le nombre de cases à parcourir
uint8_t x = 0;        // la colonne de la lettre
uint8_t images = 0;   // le chronomètre, en images

int main() {
    while (true) {
        image();
        if (x < pas) {
            images++;
            if (images == 15) {
                images = 0;
                effacer(x, 8, 1);    // efface l'ancienne place, ligne 8
                x++;
            }
        }
        poser(x, 8, ALPHABET[1]);  // le B, ligne 8
    }
}
```

CE QU'ON DOIT VOIR

Un B qui avance de 5 cases sur la ligne 8.



B

l'écran, au bout de vingt images

0.17.2. Une lettre qui avance de 5 cases — doublé, deux positions

Le A sur la ligne 0 et le B sur la ligne 8 avancent ensemble : la même variable `x` pour les deux.

C'est le 0.17 et le 0.17.1 réunis : deux `effacer` et deux `poser`, un par ligne.

Une seule variable `x` pour les deux lettres : elles sont toujours à la même colonne, donc elles avancent **ensemble**, au même pas.

LE PROGRAMME — 1442 OCTETS

```
uint8_t pas = 5;      // le nombre de cases à parcourir
uint8_t x = 0;        // la colonne des deux lettres
uint8_t images = 0;   // le chronomètre, en images

int main() {
    while (true) {
        image();
        if (x < pas) {
            images++;
            if (images == 15) {
                images = 0;
                effacer(x, 0, 1);    // efface l'ancienne place, ligne 0
                effacer(x, 8, 1);    // efface l'ancienne place, ligne 8
                x++;
            }
        }
        poser(x, 0, ALPHABET[0]);   // le A, ligne 0
        poser(x, 8, ALPHABET[1]);   // le B, ligne 8
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B avancent ensemble de 5 cases, l'un sur la ligne 0, l'autre sur la ligne 8.

H

B

l'écran, au bout de vingt images

0.18. Avancer avec `deplace_x`

`deplace_x(colonne, ligne, tuile, pas)` fait tout le 0.17 en une ligne : la lettre avance de 5 cases.

C'est le programme du 0.17, mais tout le travail (le chronomètre, l'effacement, le pas de plus, la lettre reposée) est fait par **une seule fonction de la console** :

`deplace_x`.

Ce qui est nouveau ici : `deplace_x(colonne, ligne, tuile, pas)`. Elle pose la tuile en (`colonne`, `ligne`), puis la fait avancer d'une case vers la droite tous les quarts de seconde (15 images), **pas** fois.

Pourquoi `x = deplace_x(...)` ? Quand on écrit `deplace_x(x, ...)`, la fonction ne reçoit pas la variable `x` elle-même, mais une **copie** de sa valeur (0). Elle fait bouger la lettre en changeant sa copie, jamais notre `x`. À la fin, elle **rend** (`return`) la colonne d'arrivée, 5, et `x = ...` range ce 5 dans `x`.

Sans le `x =`, en C ordinaire, la lettre bougerait bien à l'écran, mais `x` vaudrait encore 0 : le programme croirait la lettre toujours au départ. Le cours suivant montre quand cela pose problème.

Elle bloque, comme `attendre()` : pendant le voyage, rien d'autre ne tourne. Au bord de l'écran (colonne 19), elle s'arrête au lieu de sortir.

LE PROGRAMME — 1595 OCTETS

```
uint8_t pas = 5;      // le nombre de cases à parcourir
uint8_t x = 0;        // la colonne de la lettre : elle part de 0 (tout à gauche)

int main() {
    // La ligne, morceau par morceau :
    //
    // x = deplace_x(x, 0, ALPHABET[0], pas);
    // | | | | |
    // | | | | |      +-- pas      : combien de cases (5)
    // | | | | |      +----- tuile : ce qui bouge, la lettre A
    // | | | | |      +----- ligne : 0, tout en haut
    // | | | | |      +----- colonne : d'où la lettre part.
    // | | | | |      La fonction reçoit une COPIE
    // | | | | |      de x (sa valeur, 0), pas x.
    // | +--- deplace_x fait avancer la lettre, puis REND (return) la
    // |      colonne d'arrivée : 5.
    // | +----- « x = » range ce 5 dans x.
    //
    // POURQUOI « x = » ? Une fonction n'a qu'une COPIE de x : elle ne peut pas
    // le changer. Pour garder le résultat, on le RANGE nous-mêmes : x = ...
    x = deplace_x(x, 0, ALPHABET[0], pas);    // après cette ligne : x vaut 5

    while (true) {
        image();      // le programme continue ; la lettre, elle, reste en colonne 5
    }
}
```

CE QU'ON DOIT VOIR

Un A qui avance de 5 cases vers la droite, puis s'arrête en colonne 5.

A

l'écran, au bout de vingt images

0.18.1. Avancer avec `deplace_x` — de base, ailleurs

Le 0.18 sur la ligne 8, avec le B.

C'est le 0.18, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : la **ligne 8** au lieu de 0 (2e réglage de `deplace_x`).

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.18.2 met les deux ensemble.

LE PROGRAMME — 1596 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;      // la colonne du B

int main() {
    // Les changements : la ligne (0 → 8) et la lettre (ALPHABET[0], le A).
    x = deplace_x(x, 8, ALPHABET[1], pas);    // x vaut 5

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui avance de 5 cases sur la ligne 8.

B

l'écran, au bout de vingt images

0.18.2. Avancer avec `deplace_x` — doublé, deux positions

Deux lettres, deux variables : le A (x) sur la ligne 0, puis le B (xb) sur la ligne 8.

C'est le 0.18 et le 0.18.1 réunis : deux lignes `deplace_x`.

Chaque lettre a SA variable : `x` pour le A, `xb` pour le B. Une seule variable ne suffirait pas : chaque `deplace_x` rend la colonne de SA lettre.

L'un après l'autre : `deplace_x` bloque ; le B part quand le A est arrivé.

LE PROGRAMME — 1619 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;      // la colonne du A
uint8_t xb = 0;     // la colonne du B : sa propre variable

int main() {
    x = deplace_x(x, 0, ALPHABET[0], pas);    // le A, ligne 0 (le 0.18)
    xb = deplace_x(xb, 8, ALPHABET[1], pas);  // puis le B, ligne 8 (le 0.18.1)

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A avance de 5 cases sur la ligne 0, puis le B sur la ligne 8.

H

l'écran, au bout de vingt images

0.19. Revenir : un pas négatif

Le 0.18, plus une ligne : `deplace_x` avec `-pas` fait revenir la lettre. C'est là que le « `x =` » sert.

C'est le 0.18, plus une ligne : le retour.

Ce qui est nouveau ici : un pas négatif. `-pas`, c'est -5 : la lettre **recule** de 5 cases vers la gauche. Le signe décide du sens : positif vers la droite, négatif vers la gauche.

Un octet ne connaît pourtant pas les nombres négatifs : -5 y est rangé comme $256 - 5 = 251$. `deplace_x` sait donc qu'au-delà de 127, c'est un recul.

Le retour part de `x`. Grâce au `x =` du 0.18, `x` vaut 5 : le retour part bien de la colonne 5, et rend 0. Sans le `x =`, en C ordinaire, `x` vaudrait encore 0 : le retour partirait du bord gauche, buterait dessus, et **ne ferait rien**. Le `x =` sert à **garder en mémoire où est la lettre**.

LE PROGRAMME — 1617 OCTETS

```
uint8_t pas = 5;      // +5 : avance de 5 cases ; -5 : recule de 5 cases
uint8_t x = 0;        // la colonne de la lettre : elle part de 0 (tout à gauche)

int main() {
    // L'ALLER (le 0.18) : 5 cases vers la droite ; x reçoit 5.
    x = deplace_x(x, 0, ALPHABET[0], pas); // après cette ligne : x vaut 5

    // LE RETOUR, la ligne ajoutée :
    //
    //   x = deplace_x(x, 0, ALPHABET[0], -pas);
    //
    //           |               |
    //           |               +--- -pas : -5, un pas NÉGATIF, donc vers
    //           |               la gauche. (Dans l'octet, -5 est rangé
    //           |               251 ; au-delà de 127, c'est un recul.)
    //           +--- x vaut 5 grâce à l'aller : le retour part de la colonne 5
    //
    // deplace_x fait reculer la lettre et rend 0 ; « x = » range ce 0 dans x.
    x = deplace_x(x, 0, ALPHABET[0], -pas); // après cette ligne : x vaut 0

    while (true) {
        image(); // la lettre reste en colonne 0
    }
}
```

CE QU'ON DOIT VOIR

Un A qui avance de 5 cases vers la droite, puis revient de 5 cases vers la gauche, et s'arrête.

H

l'écran, au bout de vingt images

0.19.1. Revenir : un pas négatif — de base, ailleurs

Le 0.19 sur la ligne 8, avec le B : aller et retour.

C'est le 0.19, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : la ligne 8, dans les deux `deplace_x`.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.19.2 met les deux ensemble.

LE PROGRAMME — 1619 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;

int main() {
    x = deplace_x(x, 8, ALPHABET[1], pas);    // l'aller, ligne 8 : x vaut 5
    x = deplace_x(x, 8, ALPHABET[1], -pas);    // le retour : x vaut 0

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui va et revient sur la ligne 8.

B

l'écran, au bout de vingt images

0.19.2. Revenir : un pas négatif — doublé, deux positions

L'aller-retour du A (ligne 0), puis celui du B (ligne 8), chacun avec sa variable.

C'est le 0.19 et le 0.19.1 réunis : les deux allers-retours, chacun avec sa variable (`x` , `xb`).

LE PROGRAMME — 1664 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;      // le A
uint8_t xb = 0;     // le B

int main() {
    x = deplace_x(x, 0, ALPHABET[0], pas);    // le A : aller...
    x = deplace_x(x, 0, ALPHABET[0], -pas);   // ...et retour (le 0.19)
    xb = deplace_x(xb, 8, ALPHABET[1], pas);  // le B : aller...
    xb = deplace_x(xb, 8, ALPHABET[1], -pas); // ...et retour (le 0.19.1)

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A va et revient sur la ligne 0, puis le B sur la ligne 8.

A

l'écran, au bout de vingt images

0.20. Voir x à l'écran : nombre

Le 0.19, plus nombre(0, 2, x) après chaque trajet : on VOIT la valeur de x changer, 005 puis 000.

C'est le 0.19, plus deux lignes identiques : nombre(0, 2, x); après chaque trajet.

Ce qui est nouveau ici : nombre(colonne, ligne, valeur). Elle écrit un nombre à l'écran, à la case (colonne, ligne), en trois chiffres : 5 s'écrit 005, 0 s'écrit 000.

À quoi ça sert : une variable ne se voit pas. x change dans la mémoire de la console, mais rien ne le montre. En l'écrivant à l'écran, on voit ce que vaut x : 005 après l'aller, 000 après le retour.

Pourquoi la ligne 2 : la lettre bouge sur la ligne 0 ; le nombre, deux lignes plus bas, ne la gêne pas.

LE PROGRAMME — 1637 OCTETS

```
uint8_t pas = 5;      // +5 : avance ; -5 : recule
uint8_t x = 0;        // la colonne de la lettre

int main() {
    x = deplace_x(x, 0, ALPHABET[0], pas);    // l'aller : x vaut 5

    // La ligne ajoutée :
    //
    //     nombre(0, 2, x);
    //           | | |
    //           | | +-- valeur : ce qu'on écrit, la valeur de x
    //           | +----- ligne : 2, sous la lettre (qui est en ligne 0)
    //           +----- colonne : 0, tout à gauche
    //
    // Le nombre s'écrit en 3 chiffres : 5 → « 005 ».
    nombre(0, 2, x);                // on VOIT x : 005

    x = deplace_x(x, 0, ALPHABET[0], -pas);    // le retour : x vaut 0
    nombre(0, 2, x);                // on VOIT x : 000

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A avance de 5 cases ;
en dessous s'écrit 005. Il
revient ; le nombre
devient 000.

H

l'écran, au bout de vingt images

0.20.1. Voir x à l'écran : nombre — de base, ailleurs

Le 0.20 sur la ligne 8, avec le B ; son x s'affiche en ligne 10.

C'est le 0.20, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : la **ligne 8** pour la lettre, et la **ligne 10** pour le nombre, juste en dessous.

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.20.2 met les deux ensemble.

LE PROGRAMME — 1639 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;

int main() {
    x = deplace_x(x, 8, ALPHABET[1], pas);    // le B, ligne 8
    nombre(0, 10, x);                          // son x, deux lignes plus bas : 005
    x = deplace_x(x, 8, ALPHABET[1], -pas);    // 000
    nombre(0, 10, x);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le B va et revient sur la ligne 8 ; en dessous, 005 puis 000.

B

l'écran, au bout de vingt images

0.20.2. Voir x à l'écran : nombre — doublé, deux positions

Les deux, chacun avec son nombre : x en ligne 2, xb en ligne 10.

C'est le 0.20 et le 0.20.1 réunis : les deux lettres et leurs deux nombres.

Deux variables, deux nombres : `x` s'affiche sous le A (ligne 2), `xb` sous le B (ligne 10).

LE PROGRAMME — 1704 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;      // le A
uint8_t xb = 0;     // le B

int main() {
    x = deplace_x(x, 0, ALPHABET[0], pas);
    nombre(0, 2, x);           // 005 sous le A
    x = deplace_x(x, 0, ALPHABET[0], -pas);
    nombre(0, 2, x);           // 000
    xb = deplace_x(xb, 8, ALPHABET[1], pas);
    nombre(0, 10, xb);         // 005 sous le B
    xb = deplace_x(xb, 8, ALPHABET[1], -pas);
    nombre(0, 10, xb);         // 000

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A puis le B vont et reviennent ; sous chacun, son nombre passe à 005 puis 000.

A

l'écran, au bout de vingt images

0.21. Descendre avec `deplace_y`

`deplace_y(colonne, ligne, tuile, pas)` : comme `deplace_x`, mais sur l'axe Y. La lettre descend de 5 lignes.

C'est le 0.18, mais à la verticale. L'écran a deux axes : **X**, l'horizontal (les colonnes, de 0 à gauche jusqu'à 19 à droite), et **Y**, le vertical (les lignes, de 0 en haut jusqu'à 17 en bas).

Ce qui est nouveau ici : `deplace_y(colonne, ligne, tuile, pas)`. Les mêmes renseignements que `deplace_x`, dans le même ordre, mais c'est la **ligne** qui change : la lettre descend d'une ligne tous les quarts de seconde, **pas** fois. Un pas positif **descend**, car les numéros de ligne grandissent vers le bas.

Elle rend la nouvelle ligne, d'où `y = deplace_y(...)`, pour la même raison qu'au 0.18 : la fonction n'a qu'une copie de `y`.

Au bord (ligne 17), elle s'arrête au lieu de sortir de l'écran.

LE PROGRAMME — 1595 OCTETS

```
uint8_t pas = 5;      // le nombre de lignes à parcourir
uint8_t y = 0;        // la LIGNE de la lettre : elle part de 0 (tout en haut)

int main() {
    // La ligne, morceau par morceau :
    //
    // y = deplace_y(0, y, ALPHABET[0], pas);
    // | |         | | |         |
    // | |         | | |         +-- pas      : combien de lignes (5, vers le bas)
    // | |         | | +----- tuile      : la lettre A
    // | |         | +----- ligne      : d'où elle part – une COPIE de y
    // | |         +----- colonne : 0, tout à gauche (elle ne change pas)
    // | +--- deplace_y fait descendre la lettre, puis REND la ligne d'arrivée : 5
    // +----- « y = » range ce 5 dans y
    y = deplace_y(0, y, ALPHABET[0], pas); // après cette ligne : y vaut 5

    while (true) {
        image();           // la lettre reste en ligne 5
    }
}
```

CE QU'ON DOIT VOIR

Un A qui descend de 5 lignes, puis s'arrête en ligne 5.

A

l'écran, au bout de vingt images

0.21.1. Descendre avec `deplace_y` — de base, ailleurs

Le 0.21 en colonne 10, avec le B.

C'est le 0.21, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : la **colonne 10** au lieu de 0 (1er réglage de `deplace_y`).

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.21.2 met les deux ensemble.

LE PROGRAMME — 1596 OCTETS

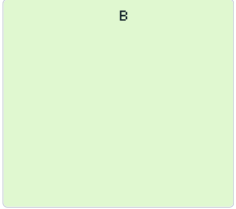
```
uint8_t pas = 5;
uint8_t y = 0;

int main() {
    y = deplace_y(10, y, ALPHABET[1], pas);    // le B descend en colonne 10 : y vaut 5

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui descend de 5 lignes, en colonne 10.

B

l'écran, au bout de vingt images

0.21.2. Descendre avec `deplace_y` — doublé, deux positions

Le A descend en colonne 0 (y), puis le B en colonne 10 (yb).

C'est le 0.21 et le 0.21.1 réunis : deux `deplace_y`, chacun avec sa variable (`y`, `yb`).

LE PROGRAMME — 1619 OCTETS

```
uint8_t pas = 5;
uint8_t y = 0;      // le A
uint8_t yb = 0;     // le B

int main() {
    y = deplace_y(0, y, ALPHABET[0], pas);    // le A, colonne 0
    yb = deplace_y(10, yb, ALPHABET[1], pas); // puis le B, colonne 10

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A descend en colonne 0, puis le B en colonne 10.

A

l'écran, au bout de vingt images

0.22. Remonter : `deplace_y` et un pas négatif

Le 0.21, plus une ligne : -pas fait remonter la lettre. Et pourquoi deux fonctions, une par axe.

C'est le 0.21, plus une ligne : la montée.

Ce qui est nouveau ici : le pas négatif **sur Y**. `-pas` fait **monter** : les numéros de ligne diminuent vers le haut. Comme au 0.19, `-5` est rangé 251, et `deplace_y` lit tout ce qui dépasse 127 comme une montée.

La montée part de `y`, qui vaut 5 grâce au `y =`. Elle rend 0, et `y =` le range.

Pourquoi deux fonctions, et pas une seule pour X et Y ? Une fonction ne peut rendre qu'une seule valeur. `deplace_x` rend la colonne, `deplace_y` rend la ligne : chacune dit exactement où la lettre s'est arrêtée.

LE PROGRAMME — 1617 OCTETS

```
uint8_t pas = 5;      // +5 : descend ; -5 : monte
uint8_t y = 0;        // la LIGNE de la lettre

int main() {
    y = deplace_y(0, y, ALPHABET[0], pas);    // la descente (le 0.21) : y vaut 5

    // La ligne ajoutée : le même appel avec -pas, donc vers le HAUT.
    // Elle part de y (5) et rend 0 ; « y = » range ce 0 dans y.
    y = deplace_y(0, y, ALPHABET[0], -pas);    // après cette ligne : y vaut 0

    while (true) {
        image();          // la lettre est revenue tout en haut
    }
}
```

CE QU'ON DOIT VOIR

Un A qui descend de 5 lignes, puis remonte de 5 lignes, et s'arrête tout en haut.

A

l'écran, au bout de vingt images

0.22.1. Remonter : deplace_y et un pas négatif — de base, ailleurs

Le 0.22 en colonne 10, avec le B : descente et montée.

C'est le 0.22, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : la **colonne 10**, dans les deux deplace_y.

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.22.2 met les deux ensemble.

LE PROGRAMME — 1619 OCTETS

```
uint8_t pas = 5;
uint8_t y = 0;

int main() {
    y = deplace_y(10, y, ALPHABET[1], pas);    // descente : y vaut 5
    y = deplace_y(10, y, ALPHABET[1], -pas);   // montée : y vaut 0

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui descend puis remonte, en colonne 10.

B

l'écran, au bout de vingt images

0.22.2. Remonter : deplace_y et un pas négatif — doublé, deux positions

La descente et la montée du A (colonne 0), puis celles du B (colonne 10).

C'est le 0.22 et le 0.22.1 réunis : les deux descentes-montées, chacune avec sa variable.

LE PROGRAMME — 1664 OCTETS

```
uint8_t pas = 5;
uint8_t y = 0;      // le A
uint8_t yb = 0;     // le B

int main() {
    y = deplace_y(0, y, ALPHABET[0], pas);
    y = deplace_y(0, y, ALPHABET[0], -pas);    // le A (le 0.22)
    yb = deplace_y(10, yb, ALPHABET[1], pas);
    yb = deplace_y(10, yb, ALPHABET[1], -pas); // le B (le 0.22.1)

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A descend et remonte en colonne 0, puis le B en colonne 10.

R

l'écran, au bout de vingt images

0.23. Sans « x = » : la console range la position

Le 0.20 sans les « x = » : `deplace_x(x, ...)`; seul sur sa ligne, et x change quand même. Les nombres le montrent.

C'est le 0.20, avec un seul changement : les deux `x =` ont disparu.

Ce qui est nouveau ici : quand l'appel est **seul sur sa ligne** et que la position donnée est une **variable** (`x`), la console **range elle-même** la position d'arrivée dans cette variable. Le compilateur réécrit la ligne en `x = deplace_x(x, ...)` : c'est exactement le 0.20, la même cartouche, octet pour octet.

La preuve : les nombres affichent toujours `005` puis `000`. Personne n'a écrit `x =`, et pourtant `x` a changé.

C'est une exception, réservée à la console. La règle du C ne change pas : une fonction reçoit une copie. Seules les fonctions de la console (`deplace_x`, `deplace_y`, et celles qui viendront) ont droit à ce rangement. Pour **tes** fonctions, il faut toujours `x =` (le 0.33 le montre).

Une condition : la position doit être une **variable**. `deplace_x(3, 0, ALPHABET[0], 5)` fait bouger la lettre, mais il n'y a nulle part où ranger la colonne.

LE PROGRAMME — 1637 OCTETS

```
uint8_t pas = 5;      // +5 : avance ; -5 : recule
uint8_t x = 0;        // la colonne de la lettre

int main() {
    // Le changement : plus de « x = » devant deplace_x.
    //
    // deplace_x(x, 0, ALPHABET[0], pas);
    //
    // |
    // +-- x est une VARIABLE : la console y range toute seule la
    //     colonne d'arrivée. Le compilateur réécrit la ligne en
    //     x = deplace_x(x, 0, ALPHABET[0], pas);
    //     C'est le 0.20, écrit plus court.
    deplace_x(x, 0, ALPHABET[0], pas); // x vaut 5 quand même...
    nombre(0, 2, x);                  // ...la preuve : 005

    deplace_x(x, 0, ALPHABET[0], -pas); // x vaut 0
    nombre(0, 2, x);                  // 000

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Exactement le 0.20 : le A avance, 005 s'écrit ; il revient, 000 s'écrit.

A

l'écran, au bout de vingt images

0.23.1. Sans « x = » : la console range la position — de base, ailleurs

Le 0.23 sur la ligne 8, avec le B : toujours sans « x = ».

C'est le 0.23, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : la ligne 8 pour la lettre, la ligne 10 pour le nombre.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.23.2 met les deux ensemble.

LE PROGRAMME — 1639 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;

int main() {
    deplace_x(x, 8, ALPHABET[1], pas); // sans « x = » : x vaut 5 quand même
    nombre(0, 10, x); // 005
    deplace_x(x, 8, ALPHABET[1], -pas); // x vaut 0
    nombre(0, 10, x); // 000

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le B va et revient sur la ligne 8 ; 005 puis 000 en dessous.

B

l'écran, au bout de vingt images

0.23.2. Sans « x = » : la console range la position — doublé, deux positions

Les deux, sans « = » : la console range x pour le A et xb pour le B.

C'est le 0.23 et le 0.23.1 réunis : les deux lettres, sans aucun `=`.

La console range chaque position dans SA variable : celle qu'on donne en premier à `deplace_x`, `x` pour le A, `xb` pour le B.

LE PROGRAMME — 1639 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;      // le A
uint8_t xb = 0;     // le B

int main() {
    deplace_x(x, 0, ALPHABET[0], pas);    // x vaut 5
    deplace_x(xb, 8, ALPHABET[1], pas);   // xb vaut 5
    nombre(0, 2, x);
    nombre(0, 10, xb);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A puis le B avancent de 5 cases ; sous chacun, 005.

A

l'écran, au bout de vingt images

0.24. Les deux axes à la suite

`deplace_x` puis `deplace_y`, sans « = » : la lettre va à droite, puis descend depuis la colonne où elle est arrivée.

On garde la forme courte du 0.23, et on utilise **les deux axes** : d'abord `deplace_x`, puis `deplace_y`.

Ce qui est nouveau ici : deux variables, `x` et `y`, et chaque fonction reçoit **les deux**. `deplace_x(x, y, ...)` avance sur la ligne `y` ; `deplace_y(x, y, ...)` descend dans la colonne `x`.

C'est pour cela qu'il faut garder `x` **à jour** : `deplace_y` part de la colonne `x`, celle où `deplace_x` a laissé la lettre (5). Si `x` était resté à 0, la descente partirait de la colonne 0, et la lettre sauterait d'un coup.

Les nombres, en bas de l'écran (ligne 17) : `x` en colonne 0, `y` en colonne 4.

LE PROGRAMME — 1928 OCTETS

```
uint8_t pas = 5;      // 5 cases, sur chaque axe
uint8_t x = 0;        // la COLONNE de la lettre (axe X)
uint8_t y = 0;        // la LIGNE de la lettre (axe Y)

int main() {
    // 1. À droite : deplace_x reçoit x ET y – elle avance sur la ligne y (0).
    deplace_x(x, y, ALPHABET[0], pas);    // x vaut 5
    nombre(0, 17, x);                      // en bas à gauche : 005

    // 2. En bas : deplace_y part de la colonne x – 5, là où la lettre est.
    //
    //   | |
    //   | +- y : la ligne de départ (0) ; la console y range l'arrivée (5)
    //   +---- x : la colonne, 5 grâce à la ligne d'avant
    deplace_y(x, y, ALPHABET[0], pas);    // y vaut 5
    nombre(4, 17, y);                      // à côté : 005

    while (true) {
        image();                          // la lettre est en (5, 5)
    }
}
```

CE QU'ON DOIT VOIR

Un A qui va de 5 cases à droite, puis descend de 5 lignes. En bas, 005 et 005.

A

l'écran, au bout de vingt images

0.24.1. Les deux axes à la suite — de base, ailleurs

Le 0.24 à partir de (10, 0), avec le B.

C'est le 0.24, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : le **départ (10, 0)** au lieu de (0, 0) : x part de 10.

La lettre va à droite jusqu'en (15, 0), puis descend jusqu'en (15, 5).

C'est la version de base : une seule lettre, à la deuxième place. Le 0.24.2 met les deux ensemble.

LE PROGRAMME — 1909 OCTETS

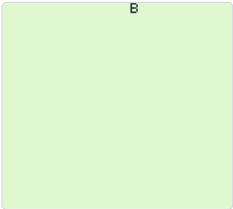
```
uint8_t pas = 5;
uint8_t x = 10;    // la colonne du B
uint8_t y = 0;     // sa ligne

int main() {
    deplace_x(x, y, ALPHABET[1], pas);
    deplace_y(x, y, ALPHABET[1], pas);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui va de (10, 0) à droite, puis descend.



l'écran, au bout de vingt images

0.24.2. Les deux axes à la suite — doublé, deux positions

Le A part de (0, 0), le B de (10, 0) : chacun ses deux variables.

C'est le 0.24 et le 0.24.1 réunis : le A (x, y) puis le B (xb, yb).

Deux lettres, quatre variables : chaque lettre a sa colonne et sa ligne.

LE PROGRAMME — 1957 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;    // 1a colonne du A
uint8_t y = 0;    // sa ligne
uint8_t xb = 10;  // 1a colonne du B
uint8_t yb = 0;   // sa ligne

int main() {
    deplace_x(x, y, ALPHABET[0], pas);
    deplace_y(x, y, ALPHABET[0], pas);
    deplace_x(xb, yb, ALPHABET[1], pas);
    deplace_y(xb, yb, ALPHABET[1], pas);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son L à gauche,
puis le B à droite.

A

l'écran, au bout de vingt images

0.25. Le carré, côté par côté

Le 0.24, plus deux côtés : à gauche, puis en haut. La lettre fait un carré et revient à (0, 0).

C'est le 0.24, plus deux lignes (et leurs nombres) : les deux côtés du retour.

Ce qui est nouveau ici : rien de plus que des pas **négatifs** sur les deux axes : `-pas` sur X ramène à gauche, `-pas` sur Y ramène en haut. Quatre côtés : **un carré**.

Chaque côté part de là où le précédent s'est arrêté, parce que `x` et `y` sont tenus à jour par la console.

LE PROGRAMME — 1994 OCTETS

```
uint8_t pas = 5;      // 5 cases, sur chaque axe
uint8_t x = 0;        // la COLONNE de la lettre
uint8_t y = 0;        // la LIGNE de la lettre

int main() {
    deplace_x(x, y, ALPHABET[0], pas);    // 1. à droite : x vaut 5
    nombre(0, 17, x);
    deplace_y(x, y, ALPHABET[0], pas);    // 2. en bas : y vaut 5
    nombre(4, 17, y);

    // Les deux lignes ajoutées : les mêmes, avec -pas.
    deplace_x(x, y, ALPHABET[0], -pas);   // 3. à gauche : x revient à 0
    nombre(0, 17, x);
    deplace_y(x, y, ALPHABET[0], -pas);   // 4. en haut : y revient à 0
    nombre(4, 17, y);

    // (0,0) → → → → (5,0)
    //   ↑           ↓      le carré : 1 à droite, 2 en bas,
    //   ↑           ↓      3 à gauche, 4 en haut
    // (0,5) ← ← ← ← (5,5)

    while (true) {
        image();      // la lettre est revenue en (0, 0)
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un carré : 5 cases à droite, 5 en bas, 5 à gauche, 5 en haut. En bas, les nombres changent après chaque côté.

A

l'écran, au bout de vingt images

0.25.1. Le carré, côté par côté — de base, ailleurs

Le carré du 0.25 à partir de (10, 0), avec le B.

C'est le 0.25, avec une autre lettre, le **B** (ALPHABET[1], la 2e lettre), et une autre place : le **départ (10, 0)** : le carré va de la colonne 10 à 15.

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.25.2 met les deux ensemble.

LE PROGRAMME — 1955 OCTETS

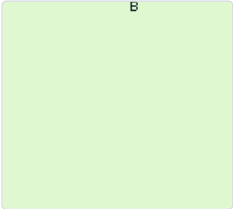
```
uint8_t pas = 5;
uint8_t x = 10;    // la colonne du B
uint8_t y = 0;     // sa ligne

int main() {
    deplace_x(x, y, ALPHABET[1], pas);
    deplace_y(x, y, ALPHABET[1], pas);
    deplace_x(x, y, ALPHABET[1], -pas);
    deplace_y(x, y, ALPHABET[1], -pas);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui fait un carré à partir de (10, 0).



l'écran, au bout de vingt images

0.25.2. Le carré, côté par côté — doublé, deux positions

Les deux carrés : le A à gauche, puis le B à droite.

C'est le 0.25 et le 0.25.1 réunis : le carré du A, puis celui du B.

LE PROGRAMME — 2049 OCTETS

```
uint8_t pas = 5;
uint8_t x = 0;    // la colonne du A
uint8_t y = 0;    // sa ligne
uint8_t xb = 10;  // la colonne du B
uint8_t yb = 0;   // sa ligne

int main() {
    deplace_x(x, y, ALPHABET[0], pas);
    deplace_y(x, y, ALPHABET[0], pas);
    deplace_x(x, y, ALPHABET[0], -pas);
    deplace_y(x, y, ALPHABET[0], -pas);
    deplace_x(xb, yb, ALPHABET[1], pas);
    deplace_y(xb, yb, ALPHABET[1], pas);
    deplace_x(xb, yb, ALPHABET[1], -pas);
    deplace_y(xb, yb, ALPHABET[1], -pas);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré à gauche, puis le B à droite.

A

l'écran, au bout de vingt images

0.26. En diagonale : deplace

`deplace(x, y, tuile, pasX, pasY)` : les deux axes en une ligne. Avec 5 et 5, la lettre descend en biais.

Une seule ligne pour les deux axes : `deplace`.

Ce qui est nouveau ici : `deplace(colonne, ligne, tuile, pasX, pasY)`. Elle a **deux** nombres de pas : `pasX` pour l'horizontale (+ droite, - gauche) et `pasY` pour la verticale (+ bas, - haut).

La diagonale : avec 5 et 5, la lettre fait à chaque quart de seconde un pas sur X **et** un pas sur Y **en même temps** : elle descend en biais, (1, 1), (2, 2)... jusqu'à (5, 5). C'est impossible avec `deplace_x` puis `deplace_y`, qui font un axe après l'autre.

Deux positions à ranger : une fonction ne rend qu'une valeur. `deplace` rend la colonne, et dépose la ligne dans une variable de la console. **Seule sur sa ligne**, la console range les deux : `x` **et** `y`. Écris-la toujours ainsi : `x = deplace(...)` ne rangerait que `x`.

LE PROGRAMME — 1806 OCTETS

```
uint8_t x = 0;      // la COLONNE de la lettre
uint8_t y = 0;      // la LIGNE de la lettre

int main() {
    // La ligne, morceau par morceau :
    //
    // deplace(x, y, ALPHABET[0], 5, 5);
    //
    //      | | |           | |
    //      | | |           | +-- pasY : +5, vers le bas
    //      | | |           +---- pasX : +5, vers la droite
    //      | | +----- tuile : la lettre A
    //      +--+----- x, y : d'où elle part ; la console y
    //                      range l'arrivée : x = 5, y = 5
    //
    //
    // A              (0, 0)
    //
    //  \
    //   \              à chaque pas : +1 sur X ET +1 sur Y
    //    \
    //     \
    //      A          (5, 5)
    deplace(x, y, ALPHABET[0], 5, 5);
    nombre(0, 17, x);    // 005
    nombre(4, 17, y);    // 005

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un A qui descend en diagonale, du coin en haut à gauche jusqu'en (5, 5). En bas, 005 et 005.

A

l'écran, au bout de vingt images

0.26.1. En diagonale : deplace — de base, ailleurs

La diagonale du 0.26 à partir de (10, 0), avec le B.

C'est le 0.26, avec une autre lettre, le **B** (`ALPHABET[1]`), la 2e lettre), et une autre place : le **départ (10, 0)** : la diagonale finit en (15, 5).

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.26.2 met les deux ensemble.

LE PROGRAMME — 1787 OCTETS

```
uint8_t x = 10;      // la colonne du B
uint8_t y = 0;       // sa ligne

int main() {
    deplace(x, y, ALPHABET[1], 5, 5);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui descend en biais
de (10, 0) à (15, 5).

B

l'écran, au bout de vingt images

0.26.2. En diagonale : deplace — doublé, deux positions

Deux diagonales : le A de (0, 0), puis le B de (10, 0).

C'est le 0.26 et le 0.26.1 réunis : deux `deplace`, chacun avec ses variables.

LE PROGRAMME — 1822 OCTETS

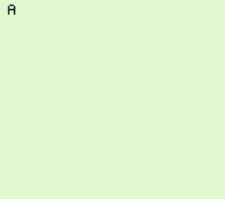
```
uint8_t x = 0;      // la colonne du A
uint8_t y = 0;      // sa ligne
uint8_t xb = 10;    // la colonne du B
uint8_t yb = 0;     // sa ligne

int main() {
    deplace(x, y, ALPHABET[0], 5, 5);
    deplace(xb, yb, ALPHABET[1], 5, 5);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A descend en biais à gauche, puis le B à droite.



l'écran, au bout de vingt images

0.27. Le carré avec deplace

Le 0.26, plus quatre lignes : avec 0 sur un axe, deplace va tout droit. Un carré à partir de (5, 5).

C'est le 0.26, plus quatre lignes : un carré, à partir de là où la diagonale s'est arrêtée.

Ce qui est nouveau ici : un 0 comme pas. `deplace(x, y, ..., 5, 0)` : 5 sur X, rien sur Y, donc tout droit vers la droite. `0, 5` : tout droit vers le bas. Avec un zéro, `deplace` fait ce que faisaient `deplace_x` ou `deplace_y`.

Le carré part de (5, 5) : à droite jusqu'en (10, 5), en bas jusqu'en (10, 10), à gauche jusqu'en (5, 10), en haut jusqu'en (5, 5).

LE PROGRAMME — 1918 OCTETS

```
uint8_t x = 0;      // la COLONNE de la lettre
uint8_t y = 0;      // la LIGNE de la lettre

int main() {
    deplace(x, y, ALPHABET[0], 5, 5);    // la diagonale (le 0.26) : (5, 5)

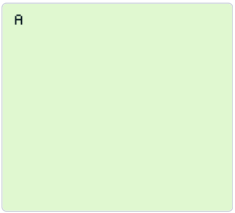
    // Les quatre lignes ajoutées. Un 0 : cet axe ne bouge pas.
    //
    //           pasX  pasY
    deplace(x, y, ALPHABET[0], 5, 0);    // à droite : (10, 5)
    deplace(x, y, ALPHABET[0], 0, 5);    // en bas   : (10, 10)
    deplace(x, y, ALPHABET[0], -5, 0);   // à gauche : (5, 10)
    deplace(x, y, ALPHABET[0], 0, -5);   // en haut  : (5, 5)

    nombre(0, 17, x);    // 005
    nombre(4, 17, y);    // 005

    while (true) {
        image();         // la lettre est revenue en (5, 5)
    }
}
```

CE QU'ON DOIT VOIR

Un A qui descend en diagonale jusqu'en (5, 5), puis fait un carré et revient en (5, 5).



l'écran, au bout de vingt images

0.27.1. Le carré avec deplace — de base, ailleurs

Le 0.27 à partir de (9, 0), avec le B : diagonale puis carré, à droite de l'écran.

C'est le 0.27, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : le **départ (9, 0)** : la diagonale finit en (14, 5), le carré va jusqu'en (19, 10), le bord droit.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.27.2 met les deux ensemble.

LE PROGRAMME — 1899 OCTETS

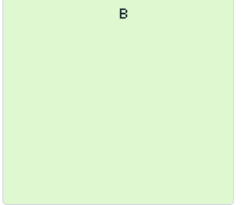
```
uint8_t x = 9;      // la colonne du B
uint8_t y = 0;      // sa ligne

int main() {
    deplace(x, y, ALPHABET[1], 5, 5);
    deplace(x, y, ALPHABET[1], 5, 0);
    deplace(x, y, ALPHABET[1], 0, 5);
    deplace(x, y, ALPHABET[1], -5, 0);
    deplace(x, y, ALPHABET[1], 0, -5);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui descend en biais puis fait un carré, contre le bord droit.

B

l'écran, au bout de vingt images

0.27.2. Le carré avec deplace — doublé, deux positions

Les deux : le A à gauche (départ (0, 0)), puis le B à droite (départ (9, 0)).

C'est le 0.27 et le 0.27.1 réunis : les dix lignes du A, puis les dix du B.

Les deux trajets ne se touchent pas : le A reste dans les colonnes 0 à 10, le B dans les colonnes 9 à 19, sur d'autres cases.

LE PROGRAMME — 2046 OCTETS

```
uint8_t x = 0;      // la colonne du A
uint8_t y = 0;      // sa ligne
uint8_t xb = 9;     // la colonne du B
uint8_t yb = 0;     // sa ligne

int main() {
    deplace(x, y, ALPHABET[0], 5, 5);
    deplace(x, y, ALPHABET[0], 5, 0);
    deplace(x, y, ALPHABET[0], 0, 5);
    deplace(x, y, ALPHABET[0], -5, 0);
    deplace(x, y, ALPHABET[0], 0, -5);
    deplace(xb, yb, ALPHABET[1], 5, 5);
    deplace(xb, yb, ALPHABET[1], 5, 0);
    deplace(xb, yb, ALPHABET[1], 0, 5);
    deplace(xb, yb, ALPHABET[1], -5, 0);
    deplace(xb, yb, ALPHABET[1], 0, -5);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait sa diagonale et son carré à gauche, puis le B à droite.

A

l'écran, au bout de vingt images

0.28. Aller à une case : `va_a`

`va_a(x, y, tuile, colonne, ligne)` : on ne compte plus les pas, on dit où aller.

Jusqu'ici, on comptait les pas. Pour aller en (10, 5) depuis (3, 2), il faudrait calculer $10 - 3 = 7$ et $5 - 2 = 3$. `va_a` fait ce calcul pour nous.

Ce qui est nouveau ici : `va_a(colonne, ligne, tuile, versColonne, versLigne)`. Les deux derniers nombres ne sont plus des pas, mais **la case d'arrivée**.

Le chemin : à chaque quart de seconde, un pas vers la colonne voulue (si elle n'y est pas) **et** un pas vers la ligne voulue (si elle n'y est pas). De (0, 0) à (10, 5) : en **diagonale** jusqu'en (5, 5), où la ligne est bonne, puis **tout droit** jusqu'en (10, 5).

Les positions sont rangées toutes seules, comme avec `deplace` : écris-la seule sur sa ligne.

LE PROGRAMME — 1668 OCTETS

```
uint8_t x = 0;      // la COLONNE de la lettre
uint8_t y = 0;      // la LIGNE de la lettre

int main() {
    // La ligne, morceau par morceau :
    //
    // va_a(x, y, ALPHABET[0], 10, 5);
    //      | | |           | |
    //      | | |           | +-- la LIGNE d'arrivée : 5
    //      | | |           +----- la COLONNE d'arrivée : 10
    //      | | +----- tuile : la lettre A
    //      +--+----- d'où elle part ; la console y range
    //                  l'arrivée : x = 10, y = 5
    //
    // A                      (0, 0)
    //  \
    //   \                      la diagonale, tant que la ligne n'est pas bonne...
    //    \
    //     \
    //      +--+--+--+ A ...puis tout droit : (5, 5) → (10, 5)
    va_a(x, y, ALPHABET[0], 10, 5);
    nombre(0, 17, x);      // 010
    nombre(4, 17, y);      // 005

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un A qui descend en biais jusqu'en (5, 5), puis file tout droit jusqu'en (10, 5).

A

l'écran, au bout de vingt images

0.28.1. Aller à une case : va_a — de base, ailleurs

Le 0.28 dix lignes plus bas : le B part de (0, 10) et va en (10, 15).

C'est le 0.28, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : le départ (0, 10) et l'arrivée (10, 15) : tout descend de 10 lignes.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.28.2 met les deux ensemble.

LE PROGRAMME — 1649 OCTETS

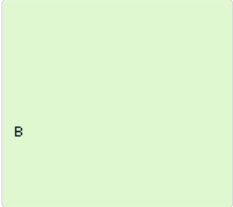
```
uint8_t x = 0;      // la colonne du B
uint8_t y = 10;     // sa ligne

int main() {
    va_a(x, y, ALPHABET[1], 10, 15);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui va de (0, 10) à (10, 15) : en biais, puis tout droit.



l'écran, au bout de vingt images

0.28.2. Aller à une case : `va_a` — doublé, deux positions

Le A va en (10, 5), puis le B en (10, 15).

C'est le 0.28 et le 0.28.1 réunis : deux `va_a`, chacun avec ses variables.

LE PROGRAMME — 1684 OCTETS

```
uint8_t x = 0;      // la colonne du A
uint8_t y = 0;      // sa ligne
uint8_t xb = 0;     // la colonne du B
uint8_t yb = 10;    // sa ligne

int main() {
    va_a(x, y, ALPHABET[0], 10, 5);
    va_a(xb, yb, ALPHABET[1], 10, 15);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A va en (10, 5), puis le B en (10, 15).

A

l'écran, au bout de vingt images

0.29. Revenir au départ avec va_a

Le 0.28, plus une ligne : `va_a(x, y, ..., 0, 0)` ramène la lettre au coin, sans compter les pas du retour.

C'est le 0.28, plus une ligne : le retour à (0, 0).

Ce qui est nouveau ici : rien qu'une autre arrivée. Pour revenir, on ne calcule pas « -10 et -5 » : on dit simplement **où** : `0, 0`.

Le chemin du retour suit la même règle : en diagonale tant que les deux axes avancent, de (10, 5) à (5, 0), puis tout droit jusqu'en (0, 0).

LE PROGRAMME — 1695 OCTETS

```
uint8_t x = 0;      // la COLONNE de la lettre
uint8_t y = 0;      // la LIGNE de la lettre

int main() {
    va_a(x, y, ALPHABET[0], 10, 5); // l'aller (le 0.28) : jusqu'en (10, 5)

    // La ligne ajoutée : l'arrivée est le coin (0, 0).
    // de (10, 5), en diagonale jusqu'en (5, 0), puis tout droit jusqu'en (0, 0)
    va_a(x, y, ALPHABET[0], 0, 0);

    nombre(0, 17, x); // 000
    nombre(4, 17, y); // 000

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A va jusqu'en (10, 5), puis revient au coin (0, 0), en biais puis tout droit.

A

l'écran, au bout de vingt images

0.29.1. Revenir au départ avec va_a — de base, ailleurs

Le 0.29 dix lignes plus bas : le B va en (10, 15) et revient en (0, 10).

C'est le 0.29, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : le départ (0, 10), l'arrivée (10, 15), et le retour en (0, 10).

C'est la version de base : une seule lettre, à la deuxième place. Le 0.29.2 met les deux ensemble.

LE PROGRAMME — 1677 OCTETS

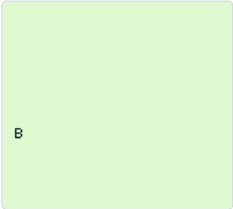
```
uint8_t x = 0;      // la colonne du B
uint8_t y = 10;     // sa ligne

int main() {
    va_a(x, y, ALPHABET[1], 10, 15);
    va_a(x, y, ALPHABET[1], 0, 10);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui va en (10, 15) et revient en (0, 10).



l'écran, au bout de vingt images

0.29.2. Revenir au départ avec va_a — doublé, deux positions

L'aller-retour du A (en haut), puis celui du B (en bas).

C'est le 0.29 et le 0.29.1 réunis : les deux allers-retours.

LE PROGRAMME — 1739 OCTETS

```
uint8_t x = 0;      // la colonne du A
uint8_t y = 0;      // sa ligne
uint8_t xb = 0;     // la colonne du B
uint8_t yb = 10;    // sa ligne

int main() {
    va_a(x, y, ALPHABET[0], 10, 5);
    va_a(x, y, ALPHABET[0], 0, 0);
    va_a(xb, yb, ALPHABET[1], 10, 15);
    va_a(xb, yb, ALPHABET[1], 0, 10);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A va et revient en haut,
puis le B en bas.

A

l'écran, au bout de vingt images

0.30. Un pas sans attendre : un_pas

un_pas fait UN pas, tout de suite, sans bloquer. C'est la boucle qui donne le rythme.

Le défaut des fonctions d'avant : `deplace_x`, `deplace_y`, `deplace` et `va_a` **bloquent**. Pendant le voyage, le programme attend : rien d'autre ne peut bouger.

Ce qui est nouveau ici : `un_pas(colonne, ligne, tuile, sensX, sensY)`. Elle fait **un seul pas**, tout de suite, et rend la main aussitôt : elle efface la lettre et la pose une case plus loin. Elle **n'attend pas**.

Seul le signe compte : `1` vers la droite (X) ou le bas (Y) ; `-1` vers la gauche ou le haut ; `0` ne bouge pas sur cet axe.

C'est la boucle qui donne le rythme, avec le chronomètre du 0.17 : `images` compte les images, et tous les 15 (quatre fois par seconde), on fait un pas.

Au bord de l'écran (colonne 19), un pas vers l'extérieur ne fait plus bouger la lettre.

LE PROGRAMME — 1616 OCTETS

```
uint8_t x = 0;      // la colonne de la lettre
uint8_t y = 0;      // sa ligne
uint8_t images = 0; // le chronomètre, en images (comme au 0.17)

int main() {
    poser(x, y, ALPHABET[0]); // la lettre à sa place de départ

    while (true) {
        image();              // attend l'image suivante (60 par seconde)
        images++;             // le chronomètre avance d'une image
        if (images == 15) {   // 15 images : un quart de seconde
            images = 0;        // le chronomètre repart de zéro

            // La ligne, morceau par morceau :
            //
            // un_pas(x, y, ALPHABET[0], 1, 0);
            //
            // | | |           | |
            // | | |           | +-- sensY : 0, ne bouge pas sur Y
            // | | |           +----- sensX : 1, un pas vers la droite
            // | | +----- tuile : la lettre A
            // +---+----- où elle est ; la console y range
            //                  sa nouvelle place
            //
            // un_pas N'ATTEND PAS : elle fait le pas et rend la main tout de suite.
            un_pas(x, y, ALPHABET[0], 1, 0);
        }
    }
}
```

CE QU'ON DOIT VOIR

Un A qui avance d'une case tous les quarts de seconde vers la droite, et s'arrête au bord.

A

l'écran, au bout de vingt images

0.30.1. Un pas sans attendre : un_pas — de base, ailleurs

Le 0.30 sur la ligne 8, avec le B.

C'est le 0.30, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : la ligne 8 : y part de 8.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.30.2 met les deux ensemble.

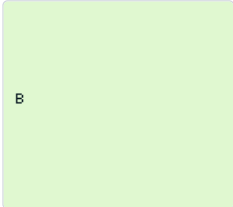
LE PROGRAMME — 1617 OCTETS

```
uint8_t x = 0;      // la colonne du B
uint8_t y = 8;      // sa ligne
uint8_t images = 0; // le chronomètre

int main() {
    poser(x, y, ALPHABET[1]);
    while (true) {
        image();
        images++;
        if (images == 15) {
            images = 0;
            un_pas(x, y, ALPHABET[1], 1, 0);
        }
    }
}
```

CE QU'ON DOIT VOIR

Un B qui avance sur la ligne 8, un pas tous les quarts de seconde, jusqu'au bord.

B

l'écran, au bout de vingt images

0.30.2. Un pas sans attendre : un_pas — doublé, deux positions

Le A (ligne 0) et le B (ligne 8) avancent EN MÊME TEMPS : deux un_pas dans la même image.

C'est le 0.30 et le 0.30.1 réunis : deux un_pas dans la même boucle.

Ensemble, cette fois : un_pas n'attend pas, les deux pas se font dans la même image. Les deux lettres avancent côte à côte.

LE PROGRAMME — 1678 OCTETS

```
uint8_t x = 0;      // la colonne du A
uint8_t y = 0;      // sa ligne
uint8_t xb = 0;     // la colonne du B
uint8_t yb = 8;     // sa ligne
uint8_t images = 0; // le chronomètre

int main() {
    poser(x, y, ALPHABET[0]);
    poser(xb, yb, ALPHABET[1]);
    while (true) {
        image();
        images++;
        if (images == 15) {
            images = 0;
            un_pas(x, y, ALPHABET[0], 1, 0);
            un_pas(xb, yb, ALPHABET[1], 1, 0);
        }
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B avancent ensemble, l'un sur la ligne 0, l'autre sur la ligne 8.

A

B

l'écran, au bout de vingt images

0.31. Deux lettres à la fois

Le 0.30, plus une deuxième lettre : deux `un_pas` dans la même boucle, et les deux bougent ensemble.

C'est le 0.30, plus une lettre, le B, qui descend pendant que le A avance.

Ce qui est nouveau ici : deux appels à `un_pas` dans la même boucle, l'un après l'autre. Comme `un_pas` n'attend pas, les deux se font **dans la même image** : les deux lettres bougent **ensemble**. Avec `deplace`, le B aurait attendu la fin du trajet du A.

Chaque lettre a ses deux variables : `xa`, `ya` pour le A ; `xb`, `yb` pour le B.

Le B s'arrête en ligne 16 : le `if (yb < 16)` ne le laisse plus avancer après, pour qu'il ne touche pas les nombres de la ligne 17.

LE PROGRAMME — 1717 OCTETS

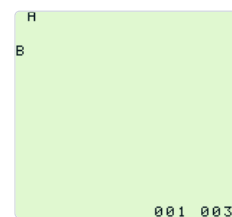
```
uint8_t xa = 0;      // le A : sa colonne...
uint8_t ya = 0;      //      ...et sa ligne. Il file vers la DROITE.
uint8_t xb = 0;      // le B : sa colonne...
uint8_t yb = 2;      //      ...et sa ligne. Il file vers le BAS.
uint8_t images = 0;  // le chronomètre, en images

int main() {
    poser(xa, ya, ALPHABET[0]); // les deux lettres à leur place de départ
    poser(xb, yb, ALPHABET[1]); // ALPHABET[1] : la 2e lettre, B

    while (true) {
        image();
        images++;
        if (images == 15) {           // 4 fois par seconde :
            images = 0;
            un_pas(xa, ya, ALPHABET[0], 1, 0); // le A : un pas à droite
            // La ligne ajoutée : le B, un pas en bas – DANS LA MÊME IMAGE que le A,
            // puisque un_pas n'attend pas. Tant que yb < 16 seulement.
            if (yb < 16) un_pas(xb, yb, ALPHABET[1], 0, 1);
            nombre(12, 17, xa); // la colonne du A, pour la voir changer
            nombre(16, 17, yb); // la ligne du B
        }
    }
}
```

CE QU'ON DOIT VOIR

Un A qui file vers la droite sur la ligne du haut, et un B qui descend en même temps sur la gauche.



l'écran, au bout de vingt images

0.31.1. Deux lettres à la fois — de base, ailleurs

Le 0.31 déplacé : le A part de la colonne 10, le B descend en colonne 10.

C'est le 0.31, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : la **colonne 10** : le A part de (10, 0) vers la droite, le B de (10, 2) vers le bas.

Leurs chemins ne se croisent pas : le A reste sur la ligne 0, le B commence à la ligne 2.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.31.2 met les deux ensemble.

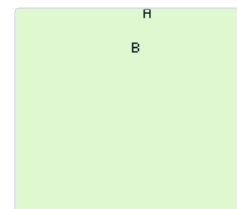
LE PROGRAMME — 1699 OCTETS

```
uint8_t xa = 10;      // la colonne du A
uint8_t ya = 0;       // sa ligne
uint8_t xb = 10;      // la colonne du B
uint8_t yb = 2;       // sa ligne
uint8_t images = 0;   // le chronomètre

int main() {
    poser(xa, ya, ALPHABET[0]);
    poser(xb, yb, ALPHABET[1]);
    while (true) {
        image();
        images++;
        if (images == 15) {
            images = 0;
            un_pas(xa, ya, ALPHABET[0], 1, 0);
            if (yb < 16) un_pas(xb, yb, ALPHABET[1], 0, 1);
        }
    }
}
```

CE QU'ON DOIT VOIR

Un A qui file à droite depuis le milieu de la ligne du haut, un B qui descend en colonne 10.



l'écran, au bout de vingt images

0.31.2. Deux lettres à la fois — doublé, deux positions

Quatre lettres, EN MÊME TEMPS : A et B à gauche (le 0.31), C et D en colonne 10.

C'est le 0.31 et le 0.31.1 réunis : les deux paires dans la même boucle.

Quatre un_pas dans la même image : les quatre lettres avancent ensemble. ALPHABET[2] est le C, ALPHABET[3] le D.

Le A s'arrête en colonne 9 (if (xa < 9)) : sans cela, il rattraperait la place du C.

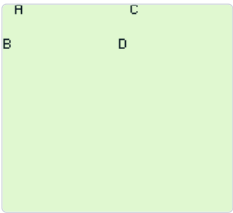
LE PROGRAMME — 1860 OCTETS

```
uint8_t xa = 0;      // la colonne du A
uint8_t ya = 0;      // sa ligne
uint8_t xb = 0;      // la colonne du B
uint8_t yb = 2;      // sa ligne
uint8_t xc = 10;     // la colonne du C
uint8_t yc = 0;      // sa ligne
uint8_t xd = 10;     // la colonne du D
uint8_t yd = 2;      // sa ligne
uint8_t images = 0;  // le chronomètre

int main() {
    poser(xa, ya, ALPHABET[0]);
    poser(xb, yb, ALPHABET[1]);
    poser(xc, yc, ALPHABET[2]);
    poser(xd, yd, ALPHABET[3]);
    while (true) {
        image();
        images++;
        if (images == 15) {
            images = 0;
            if (xa < 9) un_pas(xa, ya, ALPHABET[0], 1, 0);
            if (yb < 16) un_pas(xb, yb, ALPHABET[1], 0, 1);
            un_pas(xc, yc, ALPHABET[2], 1, 0);
            if (yd < 16) un_pas(xd, yd, ALPHABET[3], 0, 1);
        }
    }
}
```

CE QU'ON DOIT VOIR

Quatre lettres en même temps : deux vont à droite sur la ligne du haut, deux descendent.



l'écran, au bout de vingt images

0.32. Le trajet dans un tableau

Les pas du trajet rangés dans deux tableaux : une boucle for les parcourt. On change le chemin sans toucher au code.

Le carré du 0.27 répétait quatre fois la même ligne, avec d'autres nombres. Quand un programme répète la même chose avec des nombres différents, on range les nombres dans un tableau, et on écrit la ligne une seule fois, dans une boucle.

Ce qui est nouveau ici : deux tableaux, `PAS_X` et `PAS_Y`. La case `i` de chacun dit le pas du côté `i` : `PAS_X[0]` et `PAS_Y[0]` pour le premier côté (5 et 0 : à droite), `PAS_X[1]` et `PAS_Y[1]` pour le deuxième (0 et 5 : en bas), et ainsi de suite.

Des nombres négatifs dans un tableau d'octets : `-5` y est rangé comme 251, comme toujours. `deplace` sait le lire : au-delà de 127, c'est un pas en arrière.

`const` devant le tableau veut dire qu'il ne change jamais : il est gravé dans la cartouche, et ne prend pas de place dans la mémoire de travail.

La boucle `for` fait tourner `i` de 0 à 3 : `sizeof(PAS_X)` vaut 4, le nombre de cases (comme `sizeof(ALPHABET)` au 0.9). Et la boucle `while (true)` autour recommence le trajet sans fin.

Pour changer le chemin, on ne touche qu'aux tableaux. Essaie `{ 3, 3, 3, -9 }` et `{ 3, -3, 3, 0 }` : un zigzag. Ajoute des cases, la boucle suit toute seule.

LE PROGRAMME — 1836 OCTETS

```
// Le trajet : un côté par case. PAS_X[i] et PAS_Y[i] vont ensemble.
//      côté 0   côté 1   côté 2   côté 3
//      droite  bas    gauche  haut
const uint8_t PAS_X[] = { 5,    0,    -5,    0 };
const uint8_t PAS_Y[] = { 0,    5,     0,   -5 };

uint8_t x = 0;      // la colonne de la lettre
uint8_t y = 0;      // la ligne de la lettre

int main() {
    while (true) {
        // le trajet, sans fin
        for (uint8_t i = 0; i < sizeof(PAS_X); i++) { // i = 0, 1, 2, 3 : un tour par côté
            // Le côté i : on lit ses deux pas dans les tableaux.
            // i = 0 : deplace(x, y, ALPHABET[0], 5, 0);   à droite
            // i = 1 : deplace(x, y, ALPHABET[0], 0, 5);   en bas
            // i = 2 : deplace(x, y, ALPHABET[0], -5, 0);  à gauche
            // i = 3 : deplace(x, y, ALPHABET[0], 0, -5);  en haut
            deplace(x, y, ALPHABET[0], PAS_X[i], PAS_Y[i]);
        }
    }
}
```

CE QU'ON DOIT VOIR

Un A qui tourne en carré sans s'arrêter : droite, bas, gauche, haut, et on recommence.

A

l'écran, au bout de vingt images

0.32.1. Le trajet dans un tableau — de base, ailleurs

Le trajet du 0.32 à partir de (10, 0), avec le B.

C'est le 0.32, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : le **départ (10, 0)** : les mêmes tableaux, le carré à droite.

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.32.2 met les deux ensemble.

LE PROGRAMME — 1837 OCTETS

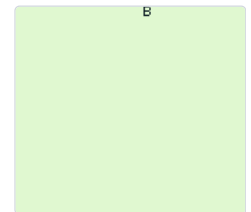
```
const uint8_t PAS_X[] = { 5, 0, -5, 0 };
const uint8_t PAS_Y[] = { 0, 5, 0, -5 };

uint8_t x = 10;      // la colonne du B
uint8_t y = 0;       // sa ligne

int main() {
    while (true) {
        for (uint8_t i = 0; i < sizeof(PAS_X); i++) {
            deplace(x, y, ALPHABET[1], PAS_X[i], PAS_Y[i]);
        }
    }
}
```

CE QU'ON DOIT VOIR

Un B qui tourne en carré sans fin, à partir de (10, 0).



l'écran, au bout de vingt images

0.32.2. Le trajet dans un tableau — doublé, deux positions

Les deux carrés, avec les MÊMES tableaux : le A et le B font chacun le côté i, à tour de rôle.

C'est le 0.32 et le 0.32.1 réunis : deux `deplace` dans la boucle, chacun avec ses variables.

Un seul trajet pour deux lettres : les tableaux disent les pas ; chaque lettre les suit avec ses variables. Le A fait un côté, puis le B le même côté, et ainsi de suite.

LE PROGRAMME — 1888 OCTETS

```
const uint8_t PAS_X[] = { 5, 0, -5, 0 };
const uint8_t PAS_Y[] = { 0, 5, 0, -5 };

uint8_t x = 0;      // la colonne du A
uint8_t y = 0;      // sa ligne
uint8_t xb = 10;    // la colonne du B
uint8_t yb = 0;     // sa ligne

int main() {
    while (true) {
        for (uint8_t i = 0; i < sizeof(PAS_X); i++) {
            deplace(x, y, ALPHABET[0], PAS_X[i], PAS_Y[i]);
            deplace(xb, yb, ALPHABET[1], PAS_X[i], PAS_Y[i]);
        }
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B font le même carré, chacun de son côté, à tour de rôle.

A

l'écran, au bout de vingt images

0.33. Écrire soi-même sa fonction

`mon_deplace_x`, écrite à la main : le 0.17 rangé dans une fonction. C'est ce qu'il y a dans `deplace_x`.

Toutes ces fonctions sont écrites dans le même langage que tes programmes. Ici, on écrit la nôtre, `mon_deplace_x` : c'est, ligne pour ligne, ce que la console a dans `deplace_x`.

Ce qui est nouveau ici : écrire une fonction. `uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) { ... }`. Le premier `uint8_t` dit ce que la fonction **rend** (un octet, la colonne) ; entre les parenthèses, ses **paramètres**, ce qu'elle reçoit ; entre les accolades, ce qu'elle fait.

Les paramètres sont des copies. Quand on appelle `mon_deplace_x(x, 0, ALPHABET[0], 5)`, `colonne` reçoit une copie de `x` (0), `pas` reçoit 5. La fonction change `colonne` et `pas` tant qu'elle veut : `x`, lui, ne bouge pas.

`return colonne;` rend la colonne d'arrivée, et termine la fonction. C'est cette valeur que `x = ...` range dans `x`.

Ici, `x =` est obligatoire. Le rangement automatique du 0.23 ne vaut que pour les fonctions de la console. `mon_deplace_x(x, ...);` seule ferait bouger la lettre, mais `x` resterait à 0 : c'est la règle normale du C.

`break` sort de la boucle `while` tout de suite : au bord de l'écran, il n'y a plus de pas possible.

LE PROGRAMME — 1604 OCTETS

```
uint8_t x = 0;          // la colonne de la lettre

// NOTRE fonction : c'est le 0.17, rangé sous un nom.
//
// uint8_t      mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas)
// |            |            |
// |            |            +-- les PARAMÈTRES : ce qu'elle reçoit. Des COPIES :
// |            |            changer colonne ne change pas x.
// |            +----- son nom
// +----- ce qu'elle REND : un octet (la colonne d'arrivée)
uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) {
    uint8_t recule = pas > 127;    // -5 est rangé 251 : au-delà de 127, on recule
    if (recule) pas = 0 - pas;      // 251 redevient 5 : le nombre de cases
    poser(colonne, ligne, tuile);  // la lettre à sa place de départ
    while (pas > 0) {              // tant qu'il reste des pas
        for (uint8_t i = 0; i < 15; i++) image(); // un quart de seconde
        if (recule && colonne == 0) break;        // bord gauche : on sort de la boucle
        if (!recule && colonne == 19) break;      // bord droit : on sort de la boucle
        effacer(colonne, ligne, 1);              // 1. efface l'ancienne place
        if (recule) colonne--;                    // 2. une colonne plus à gauche...
        else colonne++;                          // ...ou plus à droite
        poser(colonne, ligne, tuile);            // 3. la lettre à sa nouvelle place
        pas--;                                   // un pas de moins à faire
    }
    return colonne;                          // REND la colonne d'arrivée, et termine
}

int main() {
    // « x = » est OBLIGATOIRE ici : mon_deplace_x est à nous, pas à la console.
    // Elle reçoit une copie de x, et REND la colonne d'arrivée ; on la range.
    x = mon_deplace_x(x, 0, ALPHABET[0], 5);    // x vaut 5
    x = mon_deplace_x(x, 0, ALPHABET[0], -5);   // x vaut 0

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Exactement le 0.19 : un A qui avance de 5 cases, revient, et s'arrête. Mais cette fois, la fonction est écrite dans le programme.

A

l'écran, au bout de vingt images

0.33.1. Écrire soi-même sa fonction — de base, ailleurs

La fonction du 0.33, appelée pour le B sur la ligne 8.

C'est le 0.33, avec une autre lettre, le B (ALPHABET[1], la 2e lettre), et une autre place : la ligne 8 dans les deux appels.

La fonction ne change pas : c'est tout l'intérêt d'une fonction. On lui donne d'autres renseignements (la ligne, la tuile), elle fait le même travail ailleurs.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.33.2 met les deux ensemble.

LE PROGRAMME — 1606 OCTETS

```
uint8_t x = 0;

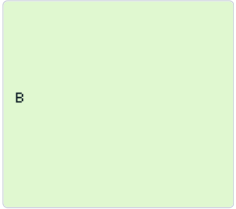
// Notre fonction, la même qu'au 0.33 (voir ses commentaires là-bas).
uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) {
    uint8_t recule = pas > 127;
    if (recule) pas = 0 - pas;
    poser(colonne, ligne, tuile);
    while (pas > 0) {
        for (uint8_t i = 0; i < 15; i++) image();
        if (recule && colonne == 0) break;
        if (!recule && colonne == 19) break;
        effacer(colonne, ligne, 1);
        if (recule) colonne--;
        else colonne++;
        poser(colonne, ligne, tuile);
        pas--;
    }
    return colonne;
}

int main() {
    x = mon_deplace_x(x, 8, ALPHABET[1], 5);    // le B, ligne 8 : x vaut 5
    x = mon_deplace_x(x, 8, ALPHABET[1], -5);   // retour : x vaut 0

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui va et revient sur la ligne 8, grâce à notre fonction.

B

l'écran, au bout de vingt images

0.33.2. Écrire soi-même sa fonction — doublé, deux positions

UNE fonction, DEUX lettres : `mon_deplace_x` sert pour le A (x) et pour le B (xb).

C'est le 0.33 et le 0.33.1 réunis : quatre appels à la même fonction.

Écris une fois, sers autant qu'on veut : la fonction ne sait rien du A ni du B ; elle bouge ce qu'on lui donne, et rend la colonne. `x =` et `xb =` rangent chaque résultat dans la bonne variable.

LE PROGRAMME — 1649 OCTETS

```
uint8_t x = 0;      // le A
uint8_t xb = 0;     // le B

// Notre fonction, la même qu'au 0.33.
uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) {
    uint8_t recule = pas > 127;
    if (recule) pas = 0 - pas;
    poser(colonne, ligne, tuile);
    while (pas > 0) {
        for (uint8_t i = 0; i < 15; i++) image();
        if (recule && colonne == 0) break;
        if (!recule && colonne == 19) break;
        effacer(colonne, ligne, 1);
        if (recule) colonne--;
        else colonne++;
        poser(colonne, ligne, tuile);
        pas--;
    }
    return colonne;
}

int main() {
    x = mon_deplace_x(x, 0, ALPHABET[0], 5);
    x = mon_deplace_x(x, 0, ALPHABET[0], -5);    // le A (le 0.33)
    xb = mon_deplace_x(xb, 8, ALPHABET[1], 5);
    xb = mon_deplace_x(xb, 8, ALPHABET[1], -5);  // le B (le 0.33.1)

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A va et revient sur la ligne 0, puis le B sur la ligne 8 : la même fonction pour les deux.

A

l'écran, au bout de vingt images

0.34. Un carré autour d’une lettre, avec carre

Une ligne, une lettre : `carre(10, 8, ALPHABET[0], 1, 1, 250, 1)` fait faire au A un tour en carré autour de la case (10, 8).

Une seule lettre, une seule ligne. Le A fait un tour complet autour de la case (10, 8), en carré, puis revient à sa place.

Ce qui est nouveau ici : `carre(x, y, tuile, taille, sens, vitesse, tours)`. Elle a sept réglages, mais pour commencer, ne regarde que les **trois premiers** :

- `10, 8` (`x, y`) : la case **autour de laquelle** on tourne, le **centre** du carré. La lettre part de là, et y revient.
- `ALPHABET[0]` (`tuile`) : ce qui tourne, la lettre A.

Les quatre autres restent simples ici : `taille 1`, `sens 1`, `vitesse 250`, `tours 1`. Ils veulent dire : le plus petit carré, dans le sens des aiguilles d’une montre, 4 pas par seconde, un seul tour. Les cours suivants les changent **un par un**.

Le trajet : le A part du centre (10, 8), fait un pas en diagonale jusqu’au coin en haut à gauche (9, 7), fait le tour du carré de **3 × 3** cases (droite, bas, gauche, haut), puis revient en diagonale au centre.

`carre` ne rend rien : la lettre finit exactement là où elle a commencé. Il n’y a ni `x =` ni variable à tenir.

LE PROGRAMME — 2082 OCTETS

```
// main : le programme commence ici. Les lignes entre { et } s'exécutent
// dans l'ordre, de haut en bas.
int main() {
    // Une seule ligne fait tout le carré. Ses sept réglages, dans l'ordre :
    //
    //   carre(10, 8, ALPHABET[0], 1, 1, 250, 1);
    //
    //   | | |           | | |
    //   | | |           | | |  +-- tours   : 1, un seul tour
    //   | | |           | | |  +----- vitesse : 250 millisecondes par pas,
    //   | | |           | | |                soit 4 pas par seconde
    //   | | |           | | |  +----- sens   : 1, comme les aiguilles
    //   | | |           | | |                d'une montre
    //   | | |           | | |  +----- taille  : 1, le plus petit carré (3 × 3)
    //   | | |           | | |                +----- tuile   : ce qui tourne ; ALPHABET[0]
    //   | | |           | | |                est la 1re lettre, A
    //   | | |           | | |  +----- y       : la LIGNE du centre, 8
    //   | | |           | | |  +----- x       : la COLONNE du centre, 10
    //
    // Pour ce premier cours, regarde surtout x, y et la tuile. Les quatre
    // autres réglages seront changés un par un dans les cours suivants.
    //
    // Ce que fait le A, un pas tous les quarts de seconde :
    //
    //   colonne :   9   10   11
    //   ligne 7 :  2 → 3 → 4
    //           ↑       ↓
    //   ligne 8 :  9   A   5
    //           ↑       ↓
    //   ligne 9 :  8 ← 7 ← 6
    //
    // 1. le A apparaît au centre (10, 8)
    // 2. un pas en diagonale : le coin en haut à gauche (9, 7)
    // 3 à 9. le tour, une case à la fois : droite, bas, gauche, haut...
    // 10. ...jusqu'au coin (9, 7), d'où il était parti
    // 11. un pas en diagonale : retour au centre (10, 8)
    carre(10, 8, ALPHABET[0], 1, 1, 250, 1);

    // Ici, carre a fini : le A est revenu au centre (10, 8), et il y reste.
    // while (true) tourne sans fin, pour que le programme ne s'arrête jamais.
    while (true) {
        image();           // attend l'image suivante (60 par seconde)
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un petit tour en carré autour de la case (10, 8), puis s'arrête au centre.



l'écran, au bout de vingt images

0.34.1. Un carré autour d’une lettre, avec carre — de base, ailleurs

Le carré du 0.34, avec une seule lettre, le B, autour d’une deuxième position : (4, 4).

C’est le carré du 0.34, mêmes réglages (taille 1, sens 1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 1 tient entier si son centre est à au moins 1 cases de chaque bord : la colonne entre 1 et 19 – 1 = 18, la ligne entre 1 et 17 – 1 = 16. (4, 4) respecte ces limites : le carré de 3 × 3 tient entier, des colonnes 3 à 5 et des lignes 3 à 5.

C’est la version de base : une seule lettre, un seul carré. Le 0.34.2 mettra les deux carrés ensemble.

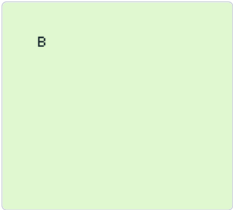
LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.34, ailleurs et avec le B :
  //
  //  carre(4, 4, ALPHABET[1], 1, 1, 250, 1);
  //      | | |
  //      | | +-- ALPHABET[1] : la 2e lettre, B
  //      +--+----- le centre (4, 4) : le carré de taille 1
  //                  y tient entier (colonnes 3 à 5, lignes 3 à 5)
  //
  // Les autres réglages sont ceux du 0.34 : taille 1, sens 1, vitesse 250, tours 1.
  carre(4, 4, ALPHABET[1], 1, 1, 250, 1);

  while (true) {
    image();          // le B est revenu à son centre
  }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.34, mais fait par un B, autour de (4, 4).



l’écran, au bout de vingt images

0.34.2. Un carré autour d'une lettre, avec carre — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C'est le 0.34 et le 0.34.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.34 ; puis le B autour de (4, 4), comme au 0.34.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l'écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

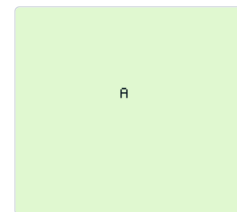
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.34.
    carre(10, 8, ALPHABET[0], 1, 1, 250, 1);

    // 2. Le B, autour de (4, 4) : le 0.34.1.
    //   Il commence quand le A a fini : carre attend la fin de son carré.
    carre(4, 4, ALPHABET[1], 1, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l'écran, au bout de vingt images

0.35. Taille 2 : un carré de 5×5

Le même carré qu'au 0.34, avec la taille 2 au lieu de 1 : 5×5 cases autour de (10, 8).

Un seul changement par rapport au 0.34 : la taille, 4e réglage, passe de 1 à 2.

Ce qui est nouveau ici : la taille 2. La lettre tourne à **2 cases** du centre. La règle $2 \times n + 1$ donne $2 \times 2 + 1 = 5$ cases de côté : un carré de 5×5 .

Où il tombe dans l'écran : autour de (10, 8), de la colonne $10 - 2 = 8$ à $10 + 2 = 12$, et de la ligne $8 - 2 = 6$ à $8 + 2 = 10$. L'écran va de la colonne 0 à 19 et de la ligne 0 à 17 : le carré tient entier.

Pourquoi le centre (10, 8) : l'écran fait 20 colonnes et 18 lignes ; son milieu est vers la colonne 10 et la ligne 8. C'est le seul genre d'endroit où **toutes** les tailles, de 1 jusqu'à 8, tiennent entières. Toutes les leçons de carré qui suivent tournent donc autour de (10, 8), et l'on voit chaque carré grandir autour du même point.

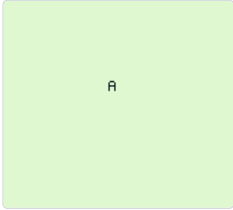
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Un seul changement par rapport au 0.34 : la taille, 1 → 2.
    //
    //   carre(10, 8, ALPHABET[0], 2, 1, 250, 1);
    //           |
    //           +-- taille 2 : à 2 cases du centre ;
    //               2 × 2 + 1 = 5 cases de côté
    //
    // Le carré, colonnes 8 à 12, lignes 6 à 10 (A : le centre, (10, 8)) :
    //
    //   ligne 6 : → → → → ↓
    //   ligne 7 : ↑ . . . ↓
    //   ligne 8 : ↑ . A . ↓
    //   ligne 9 : ↑ . . . ↓
    //   ligne 10 : ↑ ← ← ← ←
    carre(10, 8, ALPHABET[0], 2, 1, 250, 1);

    while (true) {
        image();          // le tour est fini : le A reste au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un tour en carré de 5×5 cases autour du milieu de l'écran, puis revient au centre.



l'écran, au bout de vingt images

0.35.1. Taille 2 : un carré de 5 × 5 — de base, ailleurs

Le carré du 0.35, avec une seule lettre, le B, autour d’une deuxième position : (4, 4).

C’est le carré du 0.35, mêmes réglages (taille 2, sens 1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 2 tient entier si son centre est à au moins 2 cases de chaque bord : la colonne entre 2 et 19 – 2 = 17, la ligne entre 2 et 17 – 2 = 15. (4, 4) respecte ces limites : le carré de 5 × 5 tient entier, des colonnes 2 à 6 et des lignes 2 à 6.

C’est la version de base : une seule lettre, un seul carré. Le 0.35.2 mettra les deux carrés ensemble.

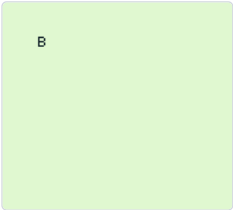
LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.35, ailleurs et avec le B :
  //
  //   carre(4, 4, ALPHABET[1], 2, 1, 250, 1);
  //
  //   | | |
  //   | | +-- ALPHABET[1] : la 2e lettre, B
  //   +--+----- le centre (4, 4) : le carré de taille 2
  //               y tient entier (colonnes 2 à 6, lignes 2 à 6)
  //
  // Les autres réglages sont ceux du 0.35 : taille 2, sens 1, vitesse 250, tours 1.
  carre(4, 4, ALPHABET[1], 2, 1, 250, 1);

  while (true) {
    image();          // le B est revenu à son centre
  }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.35, mais fait par un B, autour de (4, 4).



l’écran, au bout de vingt images

0.35.2. Taille 2 : un carré de 5×5 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C'est le 0.35 et le 0.35.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.35 ; puis le B autour de (4, 4), comme au 0.35.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l'écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

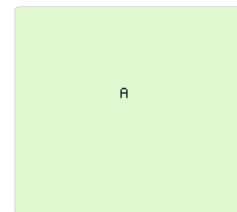
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.35.
    carre(10, 8, ALPHABET[0], 2, 1, 250, 1);

    // 2. Le B, autour de (4, 4) : le 0.35.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(4, 4, ALPHABET[1], 2, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l'écran, au bout de vingt images

0.36. Taille 3 : un carré de 7×7

Le même carré qu'au 0.35, avec la taille 3 au lieu de 2 : 7×7 cases autour de (10, 8).

Un seul changement par rapport au 0.35 : la taille, 4e réglage, passe de 2 à 3.

Ce qui est nouveau ici : la **taille 3**. La lettre tourne à **3 cases** du centre. La règle $2 \times n + 1$ donne $2 \times 3 + 1 = 7$ cases de côté : un carré de **7×7** .

Où il tombe dans l'écran : autour de (10, 8), de la colonne $10 - 3 = 7$ à $10 + 3 = 13$, et de la ligne $8 - 3 = 5$ à $8 + 3 = 11$. L'écran va de la colonne 0 à 19 et de la ligne 0 à 17 : le carré tient entier.

LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Un seul changement par rapport au 0.35 : la taille, 2 → 3.
    //
    //   carre(10, 8, ALPHABET[0], 3, 1, 250, 1);
    //           |
    //           +-- taille 3 : à 3 cases du centre ;
    //           2 × 3 + 1 = 7 cases de côté
    //
    // Le carré, colonnes 7 à 13, lignes 5 à 11 (A : le centre, (10, 8)) :
    //
    // ligne 5 : → → → → → ↓
    // ligne 6 : ↑ . . . . ↓
    // ligne 7 : ↑ . . . . ↓
    // ligne 8 : ↑ . . A . ↓
    // ligne 9 : ↑ . . . . ↓
    // ligne 10 : ↑ . . . . ↓
    // ligne 11 : ↑ ← ← ← ← ←
    carre(10, 8, ALPHABET[0], 3, 1, 250, 1);

    while (true) {
        image();          // le tour est fini : le A reste au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un tour en carré de 7×7 cases autour du milieu de l'écran, puis revient au centre.



l'écran, au bout de vingt images

0.36.1. Taille 3 : un carré de 7 × 7 — de base, ailleurs

Le carré du 0.36, avec une seule lettre, le B, autour d’une deuxième position : (4, 4).

C’est le carré du 0.36, mêmes réglages (taille 3, sens 1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 3 tient entier si son centre est à au moins 3 cases de chaque bord : la colonne entre 3 et 19 – 3 = 16, la ligne entre 3 et 17 – 3 = 14. (4, 4) respecte ces limites : le carré de 7 × 7 tient entier, des colonnes 1 à 7 et des lignes 1 à 7.

C’est la version de base : une seule lettre, un seul carré. Le 0.36.2 mettra les deux carrés ensemble.

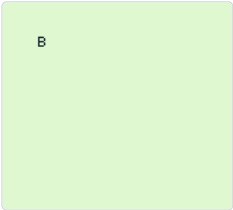
LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.36, ailleurs et avec le B :
  //
  //   carre(4, 4, ALPHABET[1], 3, 1, 250, 1);
  //
  //   | | |
  //   | | +-- ALPHABET[1] : la 2e lettre, B
  //   +--+----- le centre (4, 4) : le carré de taille 3
  //               y tient entier (colonnes 1 à 7, lignes 1 à 7)
  //
  // Les autres réglages sont ceux du 0.36 : taille 3, sens 1, vitesse 250, tours 1.
  carre(4, 4, ALPHABET[1], 3, 1, 250, 1);

  while (true) {
    image();          // le B est revenu à son centre
  }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.36, mais fait par un B, autour de (4, 4).



l’écran, au bout de vingt images

0.36.2. Taille 3 : un carré de 7 × 7 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C’est le 0.36 et le 0.36.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.36 ; puis le B autour de (4, 4), comme au 0.36.1.

L’un après l’autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l’écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

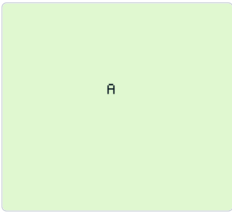
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.36.
    carre(10, 8, ALPHABET[0], 3, 1, 250, 1);

    // 2. Le B, autour de (4, 4) : le 0.36.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(4, 4, ALPHABET[1], 3, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU’ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l’écran, au bout de vingt images

0.37. Taille 4 : un carré de 9×9

Le même carré qu'au 0.36, avec la taille 4 au lieu de 3 : 9×9 cases autour de (10, 8).

Un seul changement par rapport au 0.36 : la taille, 4e réglage, passe de 3 à 4.

Ce qui est nouveau ici : la taille 4. La lettre tourne à **4 cases** du centre. La règle $2 \times n + 1$ donne $2 \times 4 + 1 = 9$ cases de côté : un carré de 9×9 .

Où il tombe dans l'écran : autour de (10, 8), de la colonne $10 - 4 = 6$ à $10 + 4 = 14$, et de la ligne $8 - 4 = 4$ à $8 + 4 = 12$. L'écran va de la colonne 0 à 19 et de la ligne 0 à 17 : le carré tient entier.

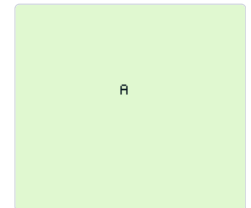
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Un seul changement par rapport au 0.36 : la taille, 3 → 4.
    //
    //   carre(10, 8, ALPHABET[0], 4, 1, 250, 1);
    //           |
    //           +-- taille 4 : à 4 cases du centre ;
    //           2 × 4 + 1 = 9 cases de côté
    //
    // Le carré, colonnes 6 à 14, lignes 4 à 12 (A : le centre, (10, 8)) :
    //
    // ligne 4 : → → → → → → → ↓
    // ligne 5 : ↑ . . . . . ↓
    // ligne 6 : ↑ . . . . . ↓
    // ligne 7 : ↑ . . . . . ↓
    // ligne 8 : ↑ . . . A . . . ↓
    // ligne 9 : ↑ . . . . . ↓
    // ligne 10 : ↑ . . . . . ↓
    // ligne 11 : ↑ . . . . . ↓
    // ligne 12 : ↑ ← ← ← ← ← ← ←
    carre(10, 8, ALPHABET[0], 4, 1, 250, 1);

    while (true) {
        image();          // le tour est fini : le A reste au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un tour en carré de 9×9 cases autour du milieu de l'écran, puis revient au centre.



l'écran, au bout de vingt images

0.37.1. Taille 4 : un carré de 9 × 9 — de base, ailleurs

Le carré du 0.37, avec une seule lettre, le B, autour d’une deuxième position : (4, 4).

C’est le carré du 0.37, mêmes réglages (taille 4, sens 1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 4 tient entier si son centre est à au moins 4 cases de chaque bord : la colonne entre 4 et 19 – 4 = 15, la ligne entre 4 et 17 – 4 = 13. (4, 4) respecte ces limites : le carré de 9 × 9 tient entier, des colonnes 0 à 8 et des lignes 0 à 8.

C’est la version de base : une seule lettre, un seul carré. Le 0.37.2 mettra les deux carrés ensemble.

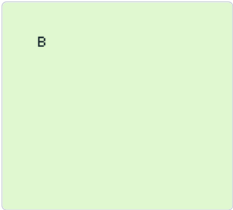
LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.37, ailleurs et avec le B :
  //
  //  carre(4, 4, ALPHABET[1], 4, 1, 250, 1);
  //      | | |
  //      | | +-- ALPHABET[1] : la 2e lettre, B
  //      +--+----- le centre (4, 4) : le carré de taille 4
  //                  y tient entier (colonnes 0 à 8, lignes 0 à 8)
  //
  // Les autres réglages sont ceux du 0.37 : taille 4, sens 1, vitesse 250, tours 1.
  carre(4, 4, ALPHABET[1], 4, 1, 250, 1);

  while (true) {
    image();          // le B est revenu à son centre
  }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.37, mais fait par un B, autour de (4, 4).



l’écran, au bout de vingt images

0.37.2. Taille 4 : un carré de 9 × 9 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C’est le 0.37 et le 0.37.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.37 ; puis le B autour de (4, 4), comme au 0.37.1.

L’un après l’autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l’écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

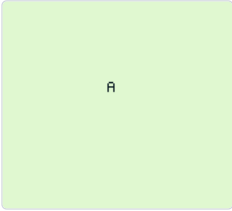
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.37.
    carre(10, 8, ALPHABET[0], 4, 1, 250, 1);

    // 2. Le B, autour de (4, 4) : le 0.37.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(4, 4, ALPHABET[1], 4, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU’ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l’écran, au bout de vingt images

0.38. Taille 5 : un carré de 11 × 11

Ce même carré qu'au 0.37, avec la taille 5 au lieu de 4 : 11 × 11 cases autour de (10, 8).

Un seul changement par rapport au 0.37 : la taille, 4e réglage, passe de 4 à 5.

Ce qui est nouveau ici : la **taille 5**. La lettre tourne à **5 cases** du centre. La règle $2 \times n + 1$ donne $2 \times 5 + 1 = 11$ cases de côté : un carré de **11 × 11**.

Où il tombe dans l'écran : autour de (10, 8), de la colonne $10 - 5 = 5$ à $10 + 5 = 15$, et de la ligne $8 - 5 = 3$ à $8 + 5 = 13$. L'écran va de la colonne 0 à 19 et de la ligne 0 à 17 : le carré tient entier.

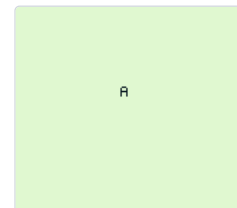
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Un seul changement par rapport au 0.37 : la taille, 4 → 5.
    //
    //   carre(10, 8, ALPHABET[0], 5, 1, 250, 1);
    //           |
    //           +-- taille 5 : à 5 cases du centre ;
    //           2 × 5 + 1 = 11 cases de côté
    //
    // Le carré, colonnes 5 à 15, lignes 3 à 13 (A : le centre, (10, 8)) :
    //
    // ligne 3 : → → → → → → → → ↓
    // ligne 4 : ↑ . . . . . ↓
    // ligne 5 : ↑ . . . . . ↓
    // ligne 6 : ↑ . . . . . ↓
    // ligne 7 : ↑ . . . . . ↓
    // ligne 8 : ↑ . . . . A . . . ↓
    // ligne 9 : ↑ . . . . . ↓
    // ligne 10 : ↑ . . . . . ↓
    // ligne 11 : ↑ . . . . . ↓
    // ligne 12 : ↑ . . . . . ↓
    // ligne 13 : ↑ ← ← ← ← ← ← ← ←
    carre(10, 8, ALPHABET[0], 5, 1, 250, 1);

    while (true) {
        image();           // le tour est fini : le A reste au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un tour en carré de 11 × 11 cases autour du milieu de l'écran, puis revient au centre.



l'écran, au bout de vingt images

0.38.1. Taille 5 : un carré de 11 × 11 — de base, ailleurs

Le carré du 0.38, avec une seule lettre, le B, autour d’une deuxième position : (5, 5).

C’est le carré du 0.38, mêmes réglages (taille 5, sens 1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(5, 5)** au lieu de (10, 8). Un carré de taille 5 tient entier si son centre est à au moins 5 cases de chaque bord : la colonne entre 5 et 19 – 5 = 14, la ligne entre 5 et 17 – 5 = 12. (5, 5) respecte ces limites : le carré de 11 × 11 tient entier, des colonnes 0 à 10 et des lignes 0 à 10.

C’est la version de base : une seule lettre, un seul carré. Le 0.38.2 mettra les deux carrés ensemble.

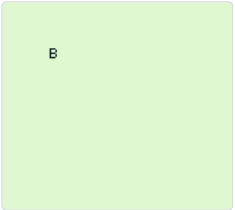
LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.38, ailleurs et avec le B :
  //
  //   carre(5, 5, ALPHABET[1], 5, 1, 250, 1);
  //       | | |
  //       | | +-- ALPHABET[1] : la 2e lettre, B
  //       +---+---- le centre (5, 5) : le carré de taille 5
  //               y tient entier (colonnes 0 à 10, lignes 0 à 10)
  //
  // Les autres réglages sont ceux du 0.38 : taille 5, sens 1, vitesse 250, tours 1.
  carre(5, 5, ALPHABET[1], 5, 1, 250, 1);

  while (true) {
    image();          // le B est revenu à son centre
  }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.38, mais fait par un B, autour de (5, 5).



l’écran, au bout de vingt images

0.38.2. Taille 5 : un carré de 11 × 11 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (5, 5).

C'est le 0.38 et le 0.38.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.38 ; puis le B autour de (5, 5), comme au 0.38.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés se croisent (taille 5 : 11 × 11 chacun). En passant sur une case, `carre` l'efface en la quittant : si le chemin du B passe sur le A arrêté, il l'efface. C'est normal : chaque lettre ne s'occupe que d'elle-même.

LE PROGRAMME — 2113 OCTETS

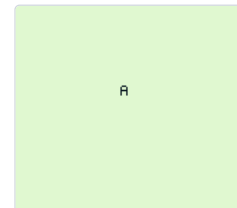
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.38.
    carre(10, 8, ALPHABET[0], 5, 1, 250, 1);

    // 2. Le B, autour de (5, 5) : le 0.38.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(5, 5, ALPHABET[1], 5, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (5, 5).



l'écran, au bout de vingt images

0.39. Taille 6 : un carré de 13 × 13

Le même carré qu'au 0.38, avec la taille 6 au lieu de 5 : 13 × 13 cases autour de (10, 8).

Un seul changement par rapport au 0.38 : la taille, 4e réglage, passe de 5 à 6.

Ce qui est nouveau ici : la taille 6. La lettre tourne à **6 cases** du centre. La règle $2 \times n + 1$ donne $2 \times 6 + 1 = 13$ cases de côté : un carré de **13 × 13**.

Où il tombe dans l'écran : autour de (10, 8), de la colonne $10 - 6 = 4$ à $10 + 6 = 16$, et de la ligne $8 - 6 = 2$ à $8 + 6 = 14$. L'écran va de la colonne 0 à 19 et de la ligne 0 à 17 : le carré tient entier.

LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Un seul changement par rapport au 0.38 : la taille, 5 → 6.
    //
    //  carre(10, 8, ALPHABET[0], 6, 1, 250, 1);
    //
    //      |
    //      +-- taille 6 : à 6 cases du centre ;
    //      2 × 6 + 1 = 13 cases de côté
    //
    // Le carré, colonnes 4 à 16, lignes 2 à 14 (A : le centre, (10, 8)) :
    //
    // ligne 2 : → → → → → → → → → ↓
    // ligne 3 : ↑ . . . . . ↓
    // ligne 4 : ↑ . . . . . ↓
    // ligne 5 : ↑ . . . . . ↓
    // ligne 6 : ↑ . . . . . ↓
    // ligne 7 : ↑ . . . . . ↓
    // ligne 8 : ↑ . . . . A . . . . ↓
    // ligne 9 : ↑ . . . . . ↓
    // ligne 10 : ↑ . . . . . ↓
    // ligne 11 : ↑ . . . . . ↓
    // ligne 12 : ↑ . . . . . ↓
    // ligne 13 : ↑ . . . . . ↓
    // ligne 14 : ↑ ← ← ← ← ← ← ← ← ←
    carre(10, 8, ALPHABET[0], 6, 1, 250, 1);

    while (true) {
        image();          // le tour est fini : le A reste au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un tour en carré de 13 × 13 cases autour du milieu de l'écran, puis revient au centre.



l'écran, au bout de vingt images

0.39.1. Taille 6 : un carré de 13 × 13 — de base, ailleurs

Le carré du 0.39, avec une seule lettre, le B, autour d’une deuxième position : (6, 6).

C’est le carré du 0.39, mêmes réglages (taille 6, sens 1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(6, 6)** au lieu de (10, 8). Un carré de taille 6 tient entier si son centre est à au moins 6 cases de chaque bord : la colonne entre 6 et 19 – 6 = 13, la ligne entre 6 et 17 – 6 = 11. (6, 6) respecte ces limites : le carré de 13 × 13 tient entier, des colonnes 0 à 12 et des lignes 0 à 12.

C’est la version de base : une seule lettre, un seul carré. Le 0.39.2 mettra les deux carrés ensemble.

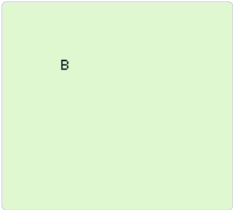
LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.39, ailleurs et avec le B :
  //
  //   carre(6, 6, ALPHABET[1], 6, 1, 250, 1);
  //       | | |
  //       | | +-- ALPHABET[1] : la 2e lettre, B
  //       +---+---- le centre (6, 6) : le carré de taille 6
  //               y tient entier (colonnes 0 à 12, lignes 0 à 12)
  //
  // Les autres réglages sont ceux du 0.39 : taille 6, sens 1, vitesse 250, tours 1.
  carre(6, 6, ALPHABET[1], 6, 1, 250, 1);

  while (true) {
    image();           // le B est revenu à son centre
  }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.39, mais fait par un B, autour de (6, 6).



l’écran, au bout de vingt images

0.39.2. Taille 6 : un carré de 13 × 13 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (6, 6).

C'est le 0.39 et le 0.39.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.39 ; puis le B autour de (6, 6), comme au 0.39.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés se croisent (taille 6 : 13 × 13 chacun). En passant sur une case, `carre` l'efface en la quittant : si le chemin du B passe sur le A arrêté, il l'efface. C'est normal : chaque lettre ne s'occupe que d'elle-même.

LE PROGRAMME — 2113 OCTETS

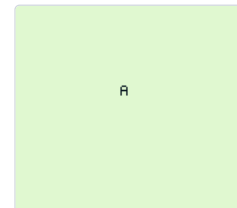
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.39.
    carre(10, 8, ALPHABET[0], 6, 1, 250, 1);

    // 2. Le B, autour de (6, 6) : le 0.39.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(6, 6, ALPHABET[1], 6, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (6, 6).



l'écran, au bout de vingt images

0.40. Taille 7 : un carré de 15 × 15

Le même carré qu'au 0.39, avec la taille 7 au lieu de 6 : 15 × 15 cases autour de (10, 8).

Un seul changement par rapport au 0.39 : la taille, 4e réglage, passe de 6 à 7.

Ce qui est nouveau ici : la **taille 7**. La lettre tourne à **7 cases** du centre. La règle $2 \times n + 1$ donne $2 \times 7 + 1 = 15$ cases de côté : un carré de **15 × 15**.

Où il tombe dans l'écran : autour de (10, 8), de la colonne $10 - 7 = 3$ à $10 + 7 = 17$, et de la ligne $8 - 7 = 1$ à $8 + 7 = 15$. L'écran va de la colonne 0 à 19 et de la ligne 0 à 17 : le carré tient entier.

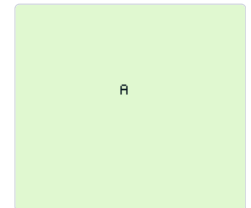
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Un seul changement par rapport au 0.39 : la taille, 6 → 7.
    //
    //  carre(10, 8, ALPHABET[0], 7, 1, 250, 1);
    //
    //      |
    //      +-- taille 7 : à 7 cases du centre ;
    //      2 × 7 + 1 = 15 cases de côté
    //
    // Le carré, colonnes 3 à 17, lignes 1 à 15 (A : le centre, (10, 8)) :
    //
    // ligne 1 : →→→→→→→→→→→→→→↓
    // ligne 2 : ↑ . . . . . ↓
    // ligne 3 : ↑ . . . . . ↓
    // ligne 4 : ↑ . . . . . ↓
    // ligne 5 : ↑ . . . . . ↓
    // ligne 6 : ↑ . . . . . ↓
    // ligne 7 : ↑ . . . . . ↓
    // ligne 8 : ↑ . . . . . A . . . . . ↓
    // ligne 9 : ↑ . . . . . ↓
    // ligne 10 : ↑ . . . . . ↓
    // ligne 11 : ↑ . . . . . ↓
    // ligne 12 : ↑ . . . . . ↓
    // ligne 13 : ↑ . . . . . ↓
    // ligne 14 : ↑ . . . . . ↓
    // ligne 15 : ↑ ←←←←←←←←←←←←←←
    carre(10, 8, ALPHABET[0], 7, 1, 250, 1);

    while (true) {
        image();          // le tour est fini : le A reste au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un tour en carré de 15 × 15 cases autour du milieu de l'écran, puis revient au centre.



l'écran, au bout de vingt images

0.40.1. Taille 7 : un carré de 15 × 15 — de base, ailleurs

Le carré du 0.40, avec une seule lettre, le B, autour d’une deuxième position : (7, 7).

C’est le carré du 0.40, mêmes réglages (taille 7, sens 1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(7, 7)** au lieu de (10, 8). Un carré de taille 7 tient entier si son centre est à au moins 7 cases de chaque bord : la colonne entre 7 et 19 – 7 = 12, la ligne entre 7 et 17 – 7 = 10. (7, 7) respecte ces limites : le carré de 15 × 15 tient entier, des colonnes 0 à 14 et des lignes 0 à 14.

C’est la version de base : une seule lettre, un seul carré. Le 0.40.2 mettra les deux carrés ensemble.

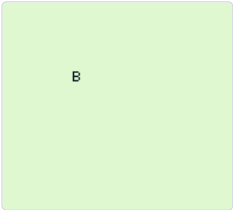
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré du 0.40, ailleurs et avec le B :
    //
    //  carre(7, 7, ALPHABET[1], 7, 1, 250, 1);
    //      | | |
    //      | | +-- ALPHABET[1] : la 2e lettre, B
    //      +---+---- le centre (7, 7) : le carré de taille 7
    //                  y tient entier (colonnes 0 à 14, lignes 0 à 14)
    //
    // Les autres réglages sont ceux du 0.40 : taille 7, sens 1, vitesse 250, tours 1.
    carre(7, 7, ALPHABET[1], 7, 1, 250, 1);

    while (true) {
        image();          // le B est revenu à son centre
    }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.40, mais fait par un B, autour de (7, 7).



l’écran, au bout de vingt images

0.40.2. Taille 7 : un carré de 15 × 15 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (7, 7).

C'est le 0.40 et le 0.40.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.40 ; puis le B autour de (7, 7), comme au 0.40.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés se croisent (taille 7 : 15 × 15 chacun). En passant sur une case, `carre` l'efface en la quittant : si le chemin du B passe sur le A arrêté, il l'efface. C'est normal : chaque lettre ne s'occupe que d'elle-même.

LE PROGRAMME — 2113 OCTETS

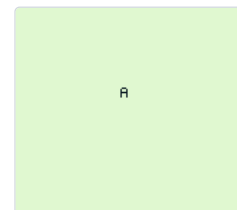
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.40.
    carre(10, 8, ALPHABET[0], 7, 1, 250, 1);

    // 2. Le B, autour de (7, 7) : le 0.40.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(7, 7, ALPHABET[1], 7, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (7, 7).



l'écran, au bout de vingt images

0.41. Taille 8 : un carré de 17 × 17

Le même carré qu'au 0.40, avec la taille 8 au lieu de 7 : 17 × 17 cases autour de (10, 8).

Un seul changement par rapport au 0.40 : la taille, 4e réglage, passe de 7 à 8.

Ce qui est nouveau ici : la **taille 8**. La lettre tourne à **8 cases** du centre. La règle $2 \times n + 1$ donne $2 \times 8 + 1 = 17$ cases de côté : un carré de **17 × 17**.

Où il tombe dans l'écran : autour de (10, 8), de la colonne $10 - 8 = 2$ à $10 + 8 = 18$, et de la ligne $8 - 8 = 0$ à $8 + 8 = 16$. L'écran va de la colonne 0 à 19 et de la ligne 0 à 17 : le carré tient entier.

C'est le plus grand possible. Avec la taille 9, le haut du carré serait à la ligne $8 - 9$, au-dessus de l'écran : il n'existe pas de ligne -1. La taille 8 touche la ligne 0 en haut, la colonne 2 à gauche et la colonne 18 à droite.

LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Un seul changement par rapport au 0.40 : la taille, 7 → 8.
    //
    //   carre(10, 8, ALPHABET[0], 8, 1, 250, 1);
    //           |
    //           +-- taille 8 : à 8 cases du centre ;
    //           2 × 8 + 1 = 17 cases de côté
    //
    // Le carré, colonnes 2 à 18, lignes 0 à 16 (A : le centre, (10, 8)) :
    //
    //   ligne 0 : → → → → → → → → → → → → → → ↓
    //   ligne 1 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 2 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 3 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 4 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 5 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 6 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 7 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 8 : ↑ . . . . . . . A . . . . . ↓
    //   ligne 9 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 10 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 11 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 12 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 13 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 14 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 15 : ↑ . . . . . . . . . . . . . ↓
    //   ligne 16 : ↑ ← ← ← ← ← ← ← ← ← ← ← ← ← ← ←
    carre(10, 8, ALPHABET[0], 8, 1, 250, 1);

    while (true) {
        image();          // le tour est fini : le A reste au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un tour en carré de 17 × 17 cases autour du milieu de l'écran, puis revient au centre.

l'écran, au bout de vingt images

0.41.1. Taille 8 : un carré de 17 × 17 — de base, ailleurs

Le carré du 0.41, avec une seule lettre, le B, autour d’une deuxième position : (8, 8).

C’est le carré du 0.41, mêmes réglages (taille 8, sens 1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(8, 8)** au lieu de (10, 8). Un carré de taille 8 tient entier si son centre est à au moins 8 cases de chaque bord : la colonne entre 8 et 19 – 8 = 11, la ligne entre 8 et 17 – 8 = 9. (8, 8) respecte ces limites : le carré de 17 × 17 tient entier, des colonnes 0 à 16 et des lignes 0 à 16.

C’est la version de base : une seule lettre, un seul carré. Le 0.41.2 mettra les deux carrés ensemble.

LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.41, ailleurs et avec le B :
  //
  //   carre(8, 8, ALPHABET[1], 8, 1, 250, 1);
  //       | | |
  //       | | +-- ALPHABET[1] : la 2e lettre, B
  //       +--+----- le centre (8, 8) : le carré de taille 8
  //                   y tient entier (colonnes 0 à 16, lignes 0 à 16)
  //
  // Les autres réglages sont ceux du 0.41 : taille 8, sens 1, vitesse 250, tours 1.
  carre(8, 8, ALPHABET[1], 8, 1, 250, 1);

  while (true) {
    image();          // le B est revenu à son centre
  }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.41, mais fait par un B, autour de (8, 8).



l’écran, au bout de vingt images

0.41.2. Taille 8 : un carré de 17 × 17 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (8, 8).

C'est le 0.41 et le 0.41.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.41 ; puis le B autour de (8, 8), comme au 0.41.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés se croisent (taille 8 : 17 × 17 chacun). En passant sur une case, `carre` l'efface en la quittant : si le chemin du B passe sur le A arrêté, il l'efface. C'est normal : chaque lettre ne s'occupe que d'elle-même.

LE PROGRAMME — 2113 OCTETS

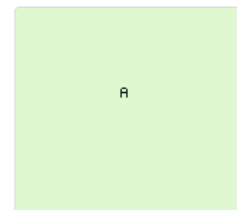
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.41.
    carre(10, 8, ALPHABET[0], 8, 1, 250, 1);

    // 2. Le B, autour de (8, 8) : le 0.41.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(8, 8, ALPHABET[1], 8, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (8, 8).



l'écran, au bout de vingt images

0.42. Dans l'autre sens : sens -1

Le carré de taille 2, avec un seul changement : le sens passe de 1 à -1. La lettre tourne dans l'autre sens.

On reprend le carré de taille 2 (le 0.35), avec un seul changement : le 5e réglage, le **sens**, passe de 1 à -1.

Ce qui est nouveau ici : le sens. 1 tourne comme les **aiguilles d'une montre** : droite, bas, gauche, haut. -1 tourne dans l'**autre sens** : bas, droite, haut, gauche.

Le coin de départ ne change pas : en haut à gauche. Seule la première direction change : avec 1, la lettre part vers la **droite** ; avec -1, vers le **bas**.

-1 dans un octet est rangé 255 : `carre` lit tout ce qui dépasse 127 comme « l'autre sens ».

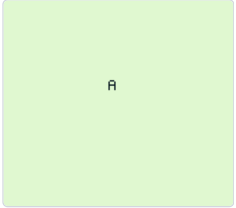
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré de taille 2 (le 0.35), avec un seul changement : le sens, 1 → -1.
    //
    //   carre(10, 8, ALPHABET[0], 2, -1, 250, 1);
    //           |
    //           +-- sens : -1, dans l'autre sens (1 : comme les aiguilles)
    //
    // sens 1 (le 0.35) :      sens -1 (ici) :
    //
    //   → → → ↓           ↓ ← ← ←
    //   ↑ . . ↓           ↓ . . ↑
    //   ↑ . A . ↓         ↓ . A . ↑
    //   ↑ . . ↓           ↓ . . ↑
    //   ↑ ← ← ←           → → → ↑
    carre(10, 8, ALPHABET[0], 2, -1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le carré de taille 2, mais la lettre part vers le bas et tourne dans l'autre sens.

A

l'écran, au bout de vingt images

0.42.1. Dans l'autre sens : sens -1 — de base, ailleurs

Le carré du 0.42, avec une seule lettre, le B, autour d'une deuxième position : (4, 4).

C'est le carré du 0.42, mêmes réglages (taille 2, sens -1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l'alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 2 tient entier si son centre est à au moins 2 cases de chaque bord : la colonne entre 2 et $19 - 2 = 17$, la ligne entre 2 et $17 - 2 = 15$. (4, 4) respecte ces limites : le carré de 5×5 tient entier, des colonnes 2 à 6 et des lignes 2 à 6.

C'est la version de base : une seule lettre, un seul carré. Le 0.42.2 mettra les deux carrés ensemble.

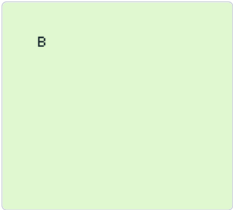
LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.42, ailleurs et avec le B :
  //
  //   carre(4, 4, ALPHABET[1], 2, -1, 250, 1);
  //
  //   | | |
  //   | | +-- ALPHABET[1] : la 2e lettre, B
  //   +--+----- le centre (4, 4) : le carré de taille 2
  //               y tient entier (colonnes 2 à 6, lignes 2 à 6)
  //
  // Les autres réglages sont ceux du 0.42 : taille 2, sens -1, vitesse 250, tours 1.
  carre(4, 4, ALPHABET[1], 2, -1, 250, 1);

  while (true) {
    image();          // le B est revenu à son centre
  }
}
```

CE QU'ON DOIT VOIR

Le même carré qu'au 0.42, mais fait par un B, autour de (4, 4).



l'écran, au bout de vingt images

0.42.2. Dans l'autre sens : sens -1 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C'est le 0.42 et le 0.42.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.42 ; puis le B autour de (4, 4), comme au 0.42.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l'écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.42.
    carre(10, 8, ALPHABET[0], 2, -1, 250, 1);

    // 2. Le B, autour de (4, 4) : le 0.42.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(4, 4, ALPHABET[1], 2, -1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l'écran, au bout de vingt images

0.43. Plus lentement : vitesse 500

Le carré de taille 2, avec un seul changement : la vitesse passe de 250 à 500 millisecondes par pas. Deux fois plus lent.

On reprend le carré de taille 2 (le 0.35), avec un seul changement : le 6e réglage, la **vitesse**, passe de 250 à 500.

Ce qui est nouveau ici : la **vitesse 500**. C'est le temps d'un **pas**, en **millisecondes** (millièmes de seconde). 500 : un pas toutes les demi-secondes, **2 pas par seconde**. C'est **deux fois plus lent** que 250.

Plus le nombre est petit, plus ça va vite, puisque c'est le temps d'attente entre deux pas. 500 ms, c'est 30 images de la console (elle en montre 60 par seconde).

Elle s'écrit en clair (500, pas une variable) : le compilateur la traduit en images avant le jeu, comme ms(500).

LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré de taille 2 (le 0.35), avec un seul changement : la vitesse, 250 → 500.
    //
    //   carre(10, 8, ALPHABET[0], 2, 1, 500, 1);
    //
    //               |
    //               +-- vitesse : 500 ms par pas (30 images)
    //               500 → 2 pas par seconde ; 250 → 4 (le 0.35)
    //
    carre(10, 8, ALPHABET[0], 2, 1, 500, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le carré de taille 2, deux fois plus lentement.



l'écran, au bout de vingt images

0.43.1. Plus lentement : vitesse 500 — de base, ailleurs

Le carré du 0.43, avec une seule lettre, le B, autour d’une deuxième position : (4, 4).

C’est le carré du 0.43, mêmes réglages (taille 2, sens 1, vitesse 500, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 2 tient entier si son centre est à au moins 2 cases de chaque bord : la colonne entre 2 et $19 - 2 = 17$, la ligne entre 2 et $17 - 2 = 15$. (4, 4) respecte ces limites : le carré de 5×5 tient entier, des colonnes 2 à 6 et des lignes 2 à 6.

C’est la version de base : une seule lettre, un seul carré. Le 0.43.2 mettra les deux carrés ensemble.

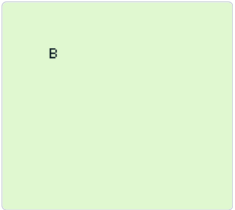
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré du 0.43, ailleurs et avec le B :
    //
    //   carre(4, 4, ALPHABET[1], 2, 1, 500, 1);
    //       | | |
    //       | | +-- ALPHABET[1] : la 2e lettre, B
    //       +--+----- le centre (4, 4) : le carré de taille 2
    //                   y tient entier (colonnes 2 à 6, lignes 2 à 6)
    //
    // Les autres réglages sont ceux du 0.43 : taille 2, sens 1, vitesse 500, tours 1.
    carre(4, 4, ALPHABET[1], 2, 1, 500, 1);

    while (true) {
        image();          // le B est revenu à son centre
    }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.43, mais fait par un B, autour de (4, 4).



l’écran, au bout de vingt images

0.43.2. Plus lentement : vitesse 500 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C'est le 0.43 et le 0.43.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.43 ; puis le B autour de (4, 4), comme au 0.43.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l'écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.43.
    carre(10, 8, ALPHABET[0], 2, 1, 500, 1);

    // 2. Le B, autour de (4, 4) : le 0.43.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(4, 4, ALPHABET[1], 2, 1, 500, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l'écran, au bout de vingt images

0.44. Plus vite : vitesse 100

Le carré de taille 2, avec un seul changement : la vitesse passe de 250 à 100 millisecondes par pas. Deux fois et demie plus vite.

On reprend le carré de taille 2 (le 0.35), avec un seul changement : le 6e réglage, la **vitesse**, passe de 250 à 100.

Ce qui est nouveau ici : la **vitesse 100**. C'est le temps d'un pas, en **millisecondes** (millièmes de seconde). 100 : un pas tous les dixièmes de seconde, **10 pas par seconde**. C'est **deux fois et demie plus vite** que 250.

Plus le nombre est petit, plus ça va vite, puisque c'est le temps d'attente entre deux pas. 100 ms, c'est 6 images de la console (elle en montre 60 par seconde).

Elle s'écrit en clair (100, pas une variable) : le compilateur la traduit en images avant le jeu, comme ms(100).

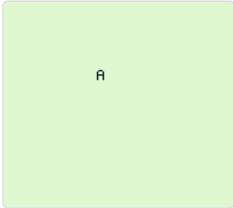
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré de taille 2 (le 0.35), avec un seul changement : la vitesse, 250 → 100.
    //
    //   carre(10, 8, ALPHABET[0], 2, 1, 100, 1);
    //
    //                                     |
    //                                     +-- vitesse : 100 ms par pas (6 images)
    //                                     100 → 10 pas par seconde ; 250 → 4 (le 0.35)
    //
    carre(10, 8, ALPHABET[0], 2, 1, 100, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le carré de taille 2, deux fois et demie plus vite.

A

l'écran, au bout de vingt images

0.44.1. Plus vite : vitesse 100 — de base, ailleurs

Le carré du 0.44, avec une seule lettre, le B, autour d’une deuxième position : (4, 4).

C’est le carré du 0.44, mêmes réglages (taille 2, sens 1, vitesse 100, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 2 tient entier si son centre est à au moins 2 cases de chaque bord : la colonne entre 2 et $19 - 2 = 17$, la ligne entre 2 et $17 - 2 = 15$. (4, 4) respecte ces limites : le carré de 5×5 tient entier, des colonnes 2 à 6 et des lignes 2 à 6.

C’est la version de base : une seule lettre, un seul carré. Le 0.44.2 mettra les deux carrés ensemble.

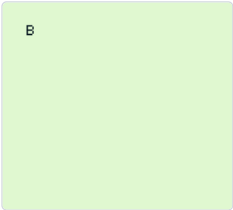
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré du 0.44, ailleurs et avec le B :
    //
    //  carre(4, 4, ALPHABET[1], 2, 1, 100, 1);
    //      | | |
    //      | | +-- ALPHABET[1] : la 2e lettre, B
    //      +--+----- le centre (4, 4) : le carré de taille 2
    //                  y tient entier (colonnes 2 à 6, lignes 2 à 6)
    //
    // Les autres réglages sont ceux du 0.44 : taille 2, sens 1, vitesse 100, tours 1.
    carre(4, 4, ALPHABET[1], 2, 1, 100, 1);

    while (true) {
        image();          // le B est revenu à son centre
    }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.44, mais fait par un B, autour de (4, 4).



l’écran, au bout de vingt images

0.44.2. Plus vite : vitesse 100 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C'est le 0.44 et le 0.44.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.44 ; puis le B autour de (4, 4), comme au 0.44.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l'écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

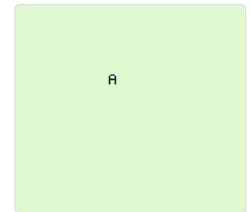
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.44.
    carre(10, 8, ALPHABET[0], 2, 1, 100, 1);

    // 2. Le B, autour de (4, 4) : le 0.44.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(4, 4, ALPHABET[1], 2, 1, 100, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l'écran, au bout de vingt images

0.45. Très vite : vitesse 50

Le carré de taille 2, avec un seul changement : la vitesse passe de 250 à 50 millisecondes par pas. Cinq fois plus vite.

On reprend le carré de taille 2 (le 0.35), avec un seul changement : le 6e réglage, la **vitesse**, passe de 250 à 50.

Ce qui est nouveau ici : la **vitesse 50**. C'est le temps d'un **pas**, en **millisecondes** (millièmes de seconde). 50 : un pas tous les vingtièmes de seconde, **20 pas par seconde**. C'est **cinq fois plus vite** que 250 : l'œil a du mal à suivre.

Plus le nombre est petit, plus ça va vite, puisque c'est le temps d'attente entre deux pas. 50 ms, c'est 3 images de la console (elle en montre 60 par seconde).

Elle s'écrit en clair (50, pas une variable) : le compilateur la traduit en images avant le jeu, comme ms(50).

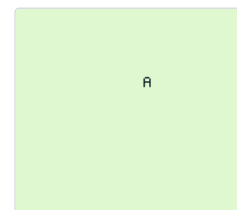
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré de taille 2 (le 0.35), avec un seul changement : la vitesse, 250 → 50.
    //
    //   carre(10, 8, ALPHABET[0], 2, 1, 50, 1);
    //
    //               |
    //               +-- vitesse : 50 ms par pas (3 images)
    //               50 → 20 pas par seconde ; 250 → 4 (le 0.35)
    //
    carre(10, 8, ALPHABET[0], 2, 1, 50, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le carré de taille 2, cinq fois plus vite : la lettre file.



l'écran, au bout de vingt images

0.45.1. Très vite : vitesse 50 — de base, ailleurs

Le carré du 0.45, avec une seule lettre, le B, autour d’une deuxième position : (4, 4).

C’est le carré du 0.45, mêmes réglages (taille 2, sens 1, vitesse 50, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 2 tient entier si son centre est à au moins 2 cases de chaque bord : la colonne entre 2 et $19 - 2 = 17$, la ligne entre 2 et $17 - 2 = 15$. (4, 4) respecte ces limites : le carré de 5×5 tient entier, des colonnes 2 à 6 et des lignes 2 à 6.

C’est la version de base : une seule lettre, un seul carré. Le 0.45.2 mettra les deux carrés ensemble.

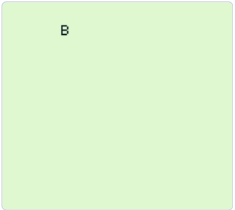
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré du 0.45, ailleurs et avec le B :
    //
    //  carre(4, 4, ALPHABET[1], 2, 1, 50, 1);
    //      | | |
    //      | | +-- ALPHABET[1] : la 2e lettre, B
    //      +--+----- le centre (4, 4) : le carré de taille 2
    //                  y tient entier (colonnes 2 à 6, lignes 2 à 6)
    //
    // Les autres réglages sont ceux du 0.45 : taille 2, sens 1, vitesse 50, tours 1.
    carre(4, 4, ALPHABET[1], 2, 1, 50, 1);

    while (true) {
        image();          // le B est revenu à son centre
    }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.45, mais fait par un B, autour de (4, 4).



l’écran, au bout de vingt images

0.45.2. Très vite : vitesse 50 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C'est le 0.45 et le 0.45.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.45 ; puis le B autour de (4, 4), comme au 0.45.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l'écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

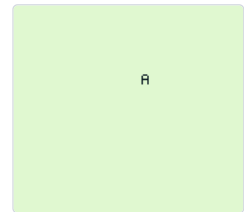
```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.45.
    carre(10, 8, ALPHABET[0], 2, 1, 50, 1);

    // 2. Le B, autour de (4, 4) : le 0.45.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(4, 4, ALPHABET[1], 2, 1, 50, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l'écran, au bout de vingt images

0.46. Deux tours

Le carré de taille 2, avec un seul changement : 2 tours au lieu d'un, avant de revenir au centre.

On reprend le carré de taille 2 (le 0.35), avec un seul changement : le 7e et dernier réglage, les **tours**, passe de 1 à 2.

Ce qui est nouveau ici : 2 tours. La lettre va au coin, fait 2 fois le tour du carré d'affilée, puis revient au centre.

On le voit au coin en bas à droite (12, 10) : la lettre y passe 2 fois.

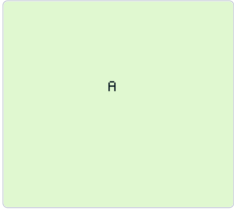
LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré de taille 2 (le 0.35), avec un seul changement : les tours, 1 → 2.
    //
    //  carre(10, 8, ALPHABET[0], 2, 1, 250, 2);
    //
    //          |
    //          +-- tours : 2, le tour 2 fois d'affilée
    //
    carre(10, 8, ALPHABET[0], 2, 1, 250, 2);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le carré de taille 2, fait 2 fois d'affilée avant le retour au centre.

A green square with the letter 'A' in the center.

l'écran, au bout de vingt images

0.46.1. Deux tours — de base, ailleurs

Le carré du 0.46, avec une seule lettre, le B, autour d’une deuxième position : (4, 4).

C’est le carré du 0.46, mêmes réglages (taille 2, sens 1, vitesse 250, 2 tours), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 2 tient entier si son centre est à au moins 2 cases de chaque bord : la colonne entre 2 et $19 - 2 = 17$, la ligne entre 2 et $17 - 2 = 15$. (4, 4) respecte ces limites : le carré de 5×5 tient entier, des colonnes 2 à 6 et des lignes 2 à 6.

C’est la version de base : une seule lettre, un seul carré. Le 0.46.2 mettra les deux carrés ensemble.

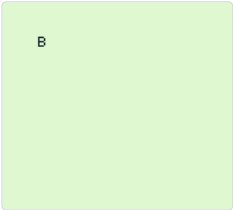
LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.46, ailleurs et avec le B :
  //
  //   carre(4, 4, ALPHABET[1], 2, 1, 250, 2);
  //       | | |
  //       | | +-- ALPHABET[1] : la 2e lettre, B
  //       +---+---- le centre (4, 4) : le carré de taille 2
  //               y tient entier (colonnes 2 à 6, lignes 2 à 6)
  //
  // Les autres réglages sont ceux du 0.46 : taille 2, sens 1, vitesse 250, tours 2.
  carre(4, 4, ALPHABET[1], 2, 1, 250, 2);

  while (true) {
    image();          // le B est revenu à son centre
  }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.46, mais fait par un B, autour de (4, 4).



l’écran, au bout de vingt images

0.46.2. Deux tours — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C'est le 0.46 et le 0.46.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.46 ; puis le B autour de (4, 4), comme au 0.46.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l'écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

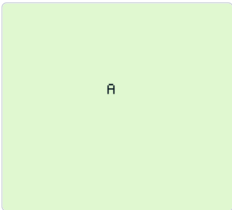
```
int main() {
  // 1. Le A, autour de (10, 8) : le 0.46.
  carre(10, 8, ALPHABET[0], 2, 1, 250, 2);

  // 2. Le B, autour de (4, 4) : le 0.46.1.
  //   Il commence quand le A a fini : carre attend la fin de son carré.
  carre(4, 4, ALPHABET[1], 2, 1, 250, 2);

  while (true) {
    image();
  }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l'écran, au bout de vingt images

0.47. Trois tours

Le carré de taille 2, avec un seul changement : 3 tours au lieu d'un, avant de revenir au centre.

On reprend le carré de taille 2 (le 0.35), avec un seul changement : le 7e et dernier réglage, les **tours**, passe de 1 à 3.

Ce qui est nouveau ici : 3 tours. La lettre va au coin, fait 3 fois le tour du carré d'affilée, puis revient au centre.

On le voit au coin en bas à droite (12, 10) : la lettre y passe 3 fois.

LE PROGRAMME — 2082 OCTETS

```
int main() {
    // Le carré de taille 2 (le 0.35), avec un seul changement : les tours, 1 → 3.
    //
    //  carre(10, 8, ALPHABET[0], 2, 1, 250, 3);
    //
    //              |
    //              +-- tours : 3, le tour 3 fois d'affilée
    //
    carre(10, 8, ALPHABET[0], 2, 1, 250, 3);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le carré de taille 2, fait 3 fois d'affilée avant le retour au centre.

A green square with the letter 'A' in the center.

l'écran, au bout de vingt images

0.47.1. Trois tours — de base, ailleurs

Le carré du 0.47, avec une seule lettre, le B, autour d’une deuxième position : (4, 4).

C’est le carré du 0.47, mêmes réglages (taille 2, sens 1, vitesse 250, 3 tours), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l’alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : **(4, 4)** au lieu de (10, 8). Un carré de taille 2 tient entier si son centre est à au moins 2 cases de chaque bord : la colonne entre 2 et $19 - 2 = 17$, la ligne entre 2 et $17 - 2 = 15$. (4, 4) respecte ces limites : le carré de 5×5 tient entier, des colonnes 2 à 6 et des lignes 2 à 6.

C’est la version de base : une seule lettre, un seul carré. Le 0.47.2 mettra les deux carrés ensemble.

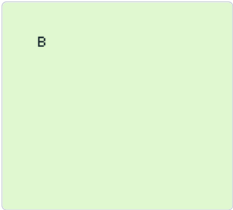
LE PROGRAMME — 2082 OCTETS

```
int main() {
  // Le carré du 0.47, ailleurs et avec le B :
  //
  //   carre(4, 4, ALPHABET[1], 2, 1, 250, 3);
  //
  //   | | |
  //   | | +-- ALPHABET[1] : la 2e lettre, B
  //   +--+----- le centre (4, 4) : le carré de taille 2
  //               y tient entier (colonnes 2 à 6, lignes 2 à 6)
  //
  // Les autres réglages sont ceux du 0.47 : taille 2, sens 1, vitesse 250, tours 3.
  carre(4, 4, ALPHABET[1], 2, 1, 250, 3);

  while (true) {
    image();          // le B est revenu à son centre
  }
}
```

CE QU’ON DOIT VOIR

Le même carré qu’au 0.47, mais fait par un B, autour de (4, 4).



l’écran, au bout de vingt images

0.47.2. Trois tours — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

C'est le 0.47 et le 0.47.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.47 ; puis le B autour de (4, 4), comme au 0.47.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l'écran. À la fin, on voit les deux lettres, chacune à son centre.

LE PROGRAMME — 2113 OCTETS

```
int main() {
    // 1. Le A, autour de (10, 8) : le 0.47.
    carre(10, 8, ALPHABET[0], 2, 1, 250, 3);

    // 2. Le B, autour de (4, 4) : le 0.47.1.
    //    Il commence quand le A a fini : carre attend la fin de son carré.
    carre(4, 4, ALPHABET[1], 2, 1, 250, 3);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).



l'écran, au bout de vingt images

0.48. Les réglages sous un nom : Carre

Carre ronde = { ... }; range les sept réglages sous un nom, en haut du programme ; carre(ronde); s'en sert.

Sept nombres à la suite se lisent mal : lequel est la taille, lequel la vitesse ? On les range donc **sous un nom**, en haut du programme, comme `Mot` au chapitre 1.

Ce qui est nouveau ici : le type `Carre`. `Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 2 };` range les sept réglages, **dans le même ordre** que `carre(...)`, sous le nom `ronde`.

`carre(ronde);` fait le carré avec ces réglages. Le compilateur remplace `carre(ronde)` par `carre(10, 8, ALPHABET[0], 2, 1, 250, 2)` : rien n'est rangé en mémoire, c'est gratuit.

L'avantage : pour changer le carré, on ne touche qu'à la ligne du haut, où chaque réglage a son titre au-dessus. Le programme, lui, ne dit plus que « fais le carré ronde ».

Il faut les sept réglages, ni plus ni moins. Il en manque un ? Le compilateur le dit, et donne un exemple complet.

LE PROGRAMME — 2082 OCTETS

```
// Carre : un type pour ranger les sept réglages d'un carré sous UN nom,
// comme Mot range la place et le texte d'un mot (chapitre 1).
//
//   Carre  ronde  =  { 10, 8, ALPHABET[0], 2, 1, 250, 2 };
//   |      |      |
//   |      |      +-- les sept réglages, DANS LE MÊME ORDRE que carre(...),
//   |      |      entre accolades { }, séparés par des virgules
//   |      +----- le nom qu'on choisit : ronde
//   +----- le type : Carre (avec une majuscule)
//
//           x  y  tuile      taille  sens  vitesse  tours
Carre ronde = { 10,  8, ALPHABET[0], 2,      1,    250,    2 };
// autour de (10, 8), la lettre A, 5 × 5, sens des aiguilles, 250 ms par pas, 2 tours

int main() {
    // carre(ronde) veut dire : « fais le carré avec les réglages de ronde ».
    // Le compilateur la remplace par carre(10, 8, ALPHABET[0], 2, 1, 250, 2) :
    // c'est exactement le même programme, mais plus facile à lire.
    carre(ronde);

    // Pour changer le carré, on ne touche QUE la ligne de « ronde », en haut :
    // chaque réglage a son titre juste au-dessus.
    while (true) {
        image();           // attend l'image suivante (60 par seconde)
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait deux tours de 5 × 5 autour de (10, 8), et revient au centre.



A

l'écran, au bout de vingt images

0.48.1. Les réglages sous un nom : Carre — de base, ailleurs

Un deuxième Carre, « coin », rangé sous son nom : le B autour de (4, 4).

C'est le 0.48, avec un autre Carre, nommé coin : le B autour de (4, 4).

C'est la version de base : la deuxième place, seule. Le 0.48.2 met les deux ensemble.

LE PROGRAMME — 2082 OCTETS

```
//      x  y  tuile      taille  sens  vitesse  tours
Carre coin = {  4, 4, ALPHABET[1], 2,    1,   250,    2 };

int main() {
    carre(coin);      // le carré, avec les réglages de « coin »

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le B fait deux tours de 5
× 5 autour de (4, 4).



l'écran, au bout de vingt images

0.48.2. Les réglages sous un nom : Carre — doublé, deux positions

Deux Carre, deux noms : ronde et coin.

C'est le 0.48 et le 0.48.1 réunis : deux `Carre`, chacun avec son nom.

Chaque nom range ses sept réglages : `carre(ronde)` et `carre(coin)` ne se mélangent pas.

LE PROGRAMME — 2113 OCTETS

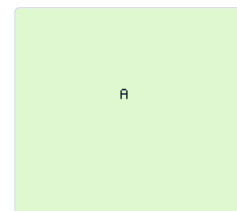
```
//      x   y   tuile      taille  sens  vitesse  tours
Carre ronde = { 10,  8, ALPHABET[0], 2,    1,   250,    2 };
Carre coin  = {  4,  4, ALPHABET[1], 2,    1,   250,    2 };

int main() {
    carre(ronde);    // le A, autour de (10, 8)
    carre(coin);     // puis le B, autour de (4, 4)

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A fait deux tours
autour de (10, 8), puis le B
autour de (4, 4).



l'écran, au bout de vingt images

0.49. De plus en plus grand : 1, puis 2

Deux carrés autour du même centre, le second un cran plus grand. Le centre (10, 8) est choisi pour que même le plus grand tienne dans l'écran.

Deux lignes, une seule différence : la taille. D'abord 1, puis 2. Le second carré entoure le premier.

Ce qui est nouveau ici : le placement. Pour des carrés de plus en plus grands, il faut un centre où **ils tiennent tous**. L'écran fait **20 colonnes** (de 0 à 19) et **18 lignes** (de 0 à 17). Son milieu est vers la colonne **10** et la ligne **8**.

Pourquoi (10, 8) est le meilleur endroit : le plus grand carré possible a la taille **8** (17×17). Autour de (10, 8), il va de la colonne $10 - 8 = 2$ à $10 + 8 = 18$, et de la ligne $8 - 8 = 0$ à $8 + 8 = 16$: il tient **entier**. Tous les carrés, de la taille 1 à la taille 8, peuvent donc tourner autour du **même** centre, emboîtés comme des cadres.

Chaque carré revient au centre avant que le suivant commence : le second repart donc bien de (10, 8).

LE PROGRAMME — 2113 OCTETS

```
int main() {
    // LE CENTRE, pour tous les carrés : (10, 8), le milieu de l'écran.
    //
    // l'écran : 20 colonnes (0 à 19), 18 lignes (0 à 17)
    // le plus grand carré (taille 8) autour de (10, 8) :
    // colonnes 10 - 8 = 2 à 10 + 8 = 18 → il tient (19 est le bord)
    // lignes 8 - 8 = 0 à 8 + 8 = 16 → il tient (17 est le bord)
    // Donc TOUTES les tailles, de 1 à 8, tiennent autour de ce centre.
    //
    // Les deux carrés de ce programme, emboîtés :
    //
    // colonne : 8 9 10 11 12
    // ligne 6 : 2 2 2 2 2      1 : le carré de taille 1 (3 × 3)
    // ligne 7 : 2 1 1 1 2      2 : le carré de taille 2 (5 × 5)
    // ligne 8 : 2 1 A 1 2
    // ligne 9 : 2 1 1 1 2
    // ligne 10 : 2 2 2 2 2

    carre(10, 8, ALPHABET[0], 1, 1, 250, 1); // taille 1 : 3 × 3
    carre(10, 8, ALPHABET[0], 2, 1, 250, 1); // taille 2 : 5 × 5, un cran plus loin

    while (true) {
        image(); // le A est revenu au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un petit tour autour du milieu de l'écran, puis un tour un peu plus grand autour du même point.



l'écran, au bout de vingt images

0.49.1. De plus en plus grand : 1, puis 2 — de base, ailleurs

Le 0.49 fait par le B, autour de (4, 4).

C'est le 0.49, avec le B, autour de (4, 4) au lieu de (10, 8).

C'est la version de base : la deuxième place, seule. Le 0.49.2 met les deux ensemble.

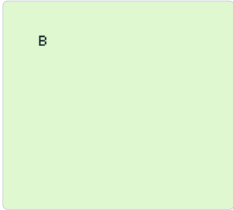
LE PROGRAMME — 2113 OCTETS

```
int main() {
    // Le B, autour de (4, 4) :
    carre(4, 4, ALPHABET[1], 1, 1, 250, 1);
    carre(4, 4, ALPHABET[1], 2, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Les mêmes carrés qu'au 0.49, faits par le B autour de (4, 4).

B

l'écran, au bout de vingt images

0.49.2. De plus en plus grand : 1, puis 2 — doublé, deux positions

Les carrés du A autour de (10, 8), puis ceux du B autour de (4, 4).

C'est le 0.49 et le 0.49.1 réunis : les carrés du A, puis ceux du B.

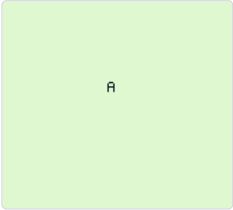
LE PROGRAMME — 2175 OCTETS

```
int main() {
    // 1. Le A, autour de (10, 8)
    carre(10, 8, ALPHABET[0], 1, 1, 250, 1);
    carre(10, 8, ALPHABET[0], 2, 1, 250, 1);
    // 2. Le B, autour de (4, 4)
    carre(4, 4, ALPHABET[1], 1, 1, 250, 1);
    carre(4, 4, ALPHABET[1], 2, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Les carrés du A, puis ceux du B.



l'écran, au bout de vingt images

0.50. De plus en plus grand : et 3

Le 0.49, plus une ligne : un troisième carré, de taille 3, autour du même centre.

C'est le 0.49, plus une ligne : le carré de taille 3 (7 × 7).

Ce qui est nouveau ici : rien qu'une taille de plus. On voit la règle se répéter : chaque carré a **2 cases de plus** de côté que le précédent (3, 5, 7), puisqu'il gagne une case de chaque côté.

On pourrait continuer ainsi jusqu'à 8, en écrivant huit lignes presque pareilles. Le cours suivant montre comment l'écrire une seule fois.

LE PROGRAMME — 2144 OCTETS

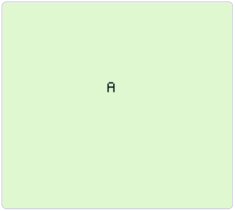
```
int main() {
    // Toujours le même centre, (10, 8) : les tailles 1 à 8 y tiennent toutes.
    //
    //     colonne :   7   8   9  10 11 12 13
    // ligne 5 :   3   3   3   3   3   3   3   chaque carré a 2 cases de plus
    // ligne 6 :   3   2   2   2   2   2   3   de côté que le précédent :
    // ligne 7 :   3   2   1   1   1   2   3   taille 1 : 3 × 3
    // ligne 8 :   3   2   1   A   1   2   3   taille 2 : 5 × 5
    // ligne 9 :   3   2   1   1   1   2   3   taille 3 : 7 × 7
    // ligne 10 :  3   2   2   2   2   2   3
    // ligne 11 :  3   3   3   3   3   3   3

    carre(10, 8, ALPHABET[0], 1, 1, 250, 1); // taille 1 : 3 × 3
    carre(10, 8, ALPHABET[0], 2, 1, 250, 1); // taille 2 : 5 × 5
    carre(10, 8, ALPHABET[0], 3, 1, 250, 1); // la ligne ajoutée : taille 3, 7 × 7

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois tours autour du milieu de l'écran, chacun plus grand que le précédent.



l'écran, au bout de vingt images

0.50.1. De plus en plus grand : et 3 — de base, ailleurs

Le 0.50 fait par le B, autour de (4, 4).

C'est le 0.50, avec le B, autour de (4, 4) au lieu de (10, 8).

C'est la version de base : la deuxième place, seule. Le 0.50.2 met les deux ensemble.

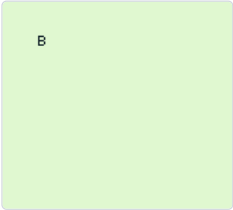
LE PROGRAMME — 2144 OCTETS

```
int main() {
    // Le B, autour de (4, 4) :
    carre(4, 4, ALPHABET[1], 1, 1, 250, 1);
    carre(4, 4, ALPHABET[1], 2, 1, 250, 1);
    carre(4, 4, ALPHABET[1], 3, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Les mêmes carrés qu'au 0.50, faits par le B autour de (4, 4).



l'écran, au bout de vingt images

0.50.2. De plus en plus grand : et 3 — doublé, deux positions

Les carrés du A autour de (10, 8), puis ceux du B autour de (4, 4).

C'est le 0.50 et le 0.50.1 réunis : les carrés du A, puis ceux du B.

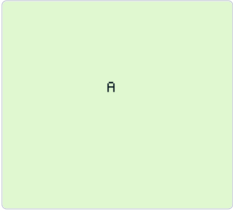
LE PROGRAMME — 2237 OCTETS

```
int main() {
    // 1. Le A, autour de (10, 8)
    carre(10, 8, ALPHABET[0], 1, 1, 250, 1);
    carre(10, 8, ALPHABET[0], 2, 1, 250, 1);
    carre(10, 8, ALPHABET[0], 3, 1, 250, 1);
    // 2. Le B, autour de (4, 4)
    carre(4, 4, ALPHABET[1], 1, 1, 250, 1);
    carre(4, 4, ALPHABET[1], 2, 1, 250, 1);
    carre(4, 4, ALPHABET[1], 3, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Les carrés du A, puis ceux du B.



l'écran, au bout de vingt images

0.51. Jusqu'au plus grand : une boucle

Une boucle **for** fait grandir la taille de 1 à 8 : huit carrés emboîtés, jusqu'au plus grand que l'écran tienne.

Le 0.50 répétait la même ligne, avec une taille qui augmente de 1 à chaque fois. Quand seul un nombre change, et toujours de la même façon, on écrit la ligne **une seule fois**, dans une **boucle**.

Ce qui est nouveau ici : la taille est une **variable**, `taille`, que la boucle `for` fait passer par 1, 2, 3... jusqu'à 8. À chaque tour de boucle, `carre` reçoit la taille du moment.

`taille <= 8` veut dire « tant que la taille est plus petite **ou égale** à 8 » : la boucle fait donc aussi le tour de taille 8, puis s'arrête.

Pourquoi 8 : c'est le plus grand carré que l'écran tienne. Autour de (10, 8), il va de la colonne 2 à 18 et de la ligne 0 à 16. Le centre a été choisi pour ça (voir le 0.49).

La vitesse, 50 ms par pas, est plus rapide : huit carrés à 250 ms dureraient plus d'une minute. Elle s'écrit toujours en clair ; la taille, elle, peut être une variable.

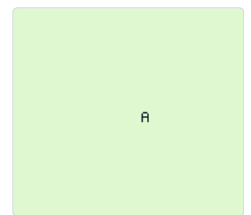
LE PROGRAMME — 2115 OCTETS

```
int main() {
    // La boucle, morceau par morceau :
    //
    //   for (uint8_t taille = 1; taille <= 8; taille++)
    //   |           |           |
    //   |           |           +-- après chaque tour : taille + 1
    //   |           +----- tant que taille ≤ 8 (8 compris)
    //   +----- on commence à la taille 1
    //
    //   tour 1 : carre(10, 8, ALPHABET[0], 1, 1, 50, 1);    3 × 3
    //   tour 2 : carre(10, 8, ALPHABET[0], 2, 1, 50, 1);    5 × 5
    //   ...
    //   tour 8 : carre(10, 8, ALPHABET[0], 8, 1, 50, 1);    17 × 17, le plus grand
    //
    // Le plus grand, autour du centre (10, 8) :
    //   colonnes 2 à 18, lignes 0 à 16 : il tient entier dans l'écran
    //   (20 colonnes de 0 à 19, 18 lignes de 0 à 17).
    for (uint8_t taille = 1; taille <= 8; taille++) {
        carre(10, 8, ALPHABET[0], taille, 1, 50, 1);    // 50 ms par pas : plus vite
    }

    while (true) {
        image();    // les huit carrés sont faits : le A est au centre
    }
}
```

CE QU'ON DOIT VOIR

Huit tours autour du milieu de l'écran, de plus en plus grands, jusqu'à un carré qui touche presque les bords.



l'écran, au bout de vingt images

0.51.1. Jusqu'au plus grand : une boucle — de base, ailleurs

Le 0.51 fait par le B, autour de (9, 9).

C'est le 0.51, avec le B, autour de (9, 9) au lieu de (10, 8).

Pourquoi (9, 9) : la taille 8 ne tient que si le centre est entre les colonnes 8 et 11 et les lignes 8 et 9. (9, 9) est l'un des rares autres centres possibles.

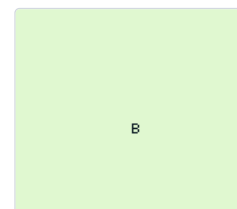
C'est la version de base : la deuxième place, seule. Le 0.51.2 met les deux ensemble.

LE PROGRAMME — 2115 OCTETS

```
int main() {  
    // Le B, autour de (9, 9) :  
    for (uint8_t taille = 1; taille <= 8; taille++) {  
        carre(9, 9, ALPHABET[1], taille, 1, 50, 1);  
    }  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

Les mêmes carrés qu'au 0.51, faits par le B autour de (9, 9).



l'écran, au bout de vingt images

0.51.2. Jusqu'au plus grand : une boucle — doublé, deux positions

Les carrés du A autour de (10, 8), puis ceux du B autour de (9, 9).

C'est le 0.51 et le 0.51.1 réunis : les carrés du A, puis ceux du B.

Ils se croisent : en passant, les carrés du B effacent le A resté au centre. Chaque lettre ne s'occupe que d'elle-même.

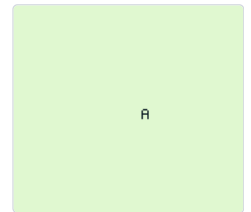
LE PROGRAMME — 2179 OCTETS

```
int main() {
    // 1. Le A, autour de (10, 8)
    for (uint8_t taille = 1; taille <= 8; taille++) {
        carre(10, 8, ALPHABET[0], taille, 1, 50, 1);
    }
    // 2. Le B, autour de (9, 9)
    for (uint8_t taille = 1; taille <= 8; taille++) {
        carre(9, 9, ALPHABET[1], taille, 1, 50, 1);
    }

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Les carrés du A, puis ceux du B.



l'écran, au bout de vingt images

0.52. Deux carrés à la suite

Deux appels à `carre`, l'un après l'autre : un petit carré lent, puis un grand carré rapide. Le second attend la fin du premier.

Deux carrés différents, l'un après l'autre. Chacun a été vu seul dans les leçons d'avant ; on les enchaîne.

Ce qui est nouveau ici : deux `carre` l'un après l'autre. `carre` **bloque** : elle attend la fin de son carré avant de rendre la main. La deuxième ligne ne commence donc que quand la lettre est revenue au centre.

Le premier est celui du 0.34 (taille 1). **Le second** change quatre réglages, chacun vu dans sa leçon : taille 3 (le 0.36), sens -1 (le 0.42), vitesse 100 (le 0.44), 3 tours (le 0.47).

LE PROGRAMME — 2113 OCTETS

```
int main() {
    // 1. Le petit carré (le 0.34) : taille 1, sens 1, 250 ms, 1 tour.
    carre(10, 8, ALPHABET[0], 1, 1, 250, 1);

    // 2. Le grand carré. Il commence quand le petit est FINI :
    //   carre attend la fin de son tour avant de passer à la ligne suivante.
    //   taille 3, sens -1, vitesse 100, 3 tours
    carre(10, 8, ALPHABET[0], 3, -1, 100, 3);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un petit tour lent autour de (10, 8), puis trois grands tours rapides dans l'autre sens.



l'écran, au bout de vingt images

0.52.1. Deux carrés à la suite — de base, ailleurs

Le 0.52 fait par le B, autour de (4, 4).

C'est le 0.52, avec le B, autour de (4, 4) au lieu de (10, 8).

C'est la version de base : la deuxième place, seule. Le 0.52.2 met les deux ensemble.

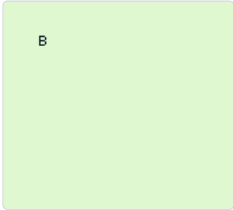
LE PROGRAMME — 2113 OCTETS

```
int main() {
    // Le B, autour de (4, 4) :
    carre(4, 4, ALPHABET[1], 1, 1, 250, 1);
    carre(4, 4, ALPHABET[1], 3, -1, 100, 3);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Les mêmes carrés qu'au 0.52, faits par le B autour de (4, 4).

B

l'écran, au bout de vingt images

0.52.2. Deux carrés à la suite — doublé, deux positions

Les carrés du A autour de (10, 8), puis ceux du B autour de (4, 4).

C'est le 0.52 et le 0.52.1 réunis : les carrés du A, puis ceux du B.

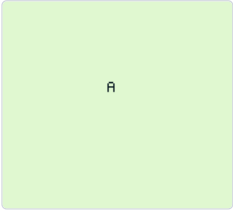
LE PROGRAMME — 2175 OCTETS

```
int main() {
    // 1. Le A, autour de (10, 8)
    carre(10, 8, ALPHABET[0], 1, 1, 250, 1);
    carre(10, 8, ALPHABET[0], 3, -1, 100, 3);
    // 2. Le B, autour de (4, 4)
    carre(4, 4, ALPHABET[1], 1, 1, 250, 1);
    carre(4, 4, ALPHABET[1], 3, -1, 100, 3);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Les carrés du A, puis ceux du B.



l'écran, au bout de vingt images

0.53. Deux lettres, deux carrés

Deux lettres, chacune autour de son propre centre : un A petit et lent, un B grand et rapide.

Deux lettres, deux centres. Jusqu'ici, tous les carrés tournaient autour de (10, 8). Ici, le A tourne autour de (4, 4), le B autour de (9, 9).

Ce qui est nouveau ici : une **deuxième lettre**, `ALPHABET[1]`, le **B**. Chaque appel à `carre` a sa propre tuile et son propre centre : les deux carrés ne se touchent pas.

Ils se font l'un après l'autre, comme au 0.52 : le B commence quand le A est revenu à son centre. À la fin, on voit les deux lettres, chacune à sa place.

LE PROGRAMME — 2113 OCTETS

```
int main() {
    // ALPHABET[0] est le A, ALPHABET[1] le B : on compte à partir de 0.

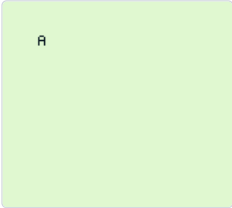
    // 1. Le A : autour de (4, 4), taille 1 (3 × 3), sens 1, 250 ms, 1 tour.
    carre(4, 4, ALPHABET[0], 1, 1, 250, 1);

    // 2. Le B, la ligne nouvelle : autour de (9, 9), taille 3 (7 × 7),
    //    l'autre sens, 100 ms, 1 tour. Il commence quand le A a fini.
    //
    //    colonne : 3 4 5 6 ... 9 ... 12
    //    ligne 4 : . A .                le carré du A : colonnes 3 à 5
    //    ligne 9 : .   B .              celui du B : colonnes 6 à 12
    //
    //                                - ils ne se touchent pas
    carre(9, 9, ALPHABET[1], 3, -1, 100, 1);

    while (true) {
        image();          // A en (4, 4), B en (9, 9)
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait un petit carré autour de (4, 4), puis un B qui fait un grand carré rapide autour de (9, 9). Chacun reste à son centre.



l'écran, au bout de vingt images

0.53.1. Deux lettres, deux carrés — de base, ailleurs

Deux autres lettres, deux autres places : le C autour de (15, 4), le D autour de (14, 12).

C'est le 0.53, avec deux autres lettres, le C et le D, à deux places libres de l'écran : (15, 4) et (14, 12).

C'est la version de base : la deuxième place, seule. Le 0.53.2 met les deux ensemble.

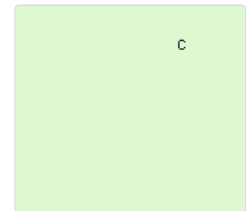
LE PROGRAMME — 2113 OCTETS

```
int main() {
    carre(15, 4, ALPHABET[2], 1, 1, 250, 1);    // le C, en haut à droite
    carre(14, 12, ALPHABET[3], 3, -1, 100, 1);   // le D, en bas à droite

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le C fait un petit carré en haut à droite, puis le D un grand en bas à droite.



l'écran, au bout de vingt images

0.53.2. Deux lettres, deux carrés — doublé, deux positions

Quatre lettres, quatre carrés : A, B, C et D, chacun à sa place.

C'est le 0.53 et le 0.53.1 réunis : les quatre carrés, l'un après l'autre.

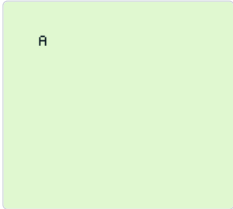
LE PROGRAMME — 2175 OCTETS

```
int main() {
    carre(4, 4, ALPHABET[0], 1, 1, 250, 1);    // le A
    carre(9, 9, ALPHABET[1], 3, -1, 100, 1);    // le B
    carre(15, 4, ALPHABET[2], 1, 1, 250, 1);    // le C
    carre(14, 12, ALPHABET[3], 3, -1, 100, 1);  // le D

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Quatre carrés, l'un après l'autre, aux quatre coins de l'écran.



l'écran, au bout de vingt images

0.54. Trois lettres, trois carrés

Le 0.53, plus une ligne : un C, avec les réglages rangés sous le nom « ronde ».

C'est le 0.53, plus un carré : le C, dont les réglages sont rangés en haut sous le nom `ronde`, comme au 0.48.

Ce qui est nouveau ici : rien qu'on ne connaisse, mais **ensemble** : deux carrés écrits en entier, et un troisième par son nom. `carre(ronde);` est remplacé par `carre(15, 8, ALPHABET[2], 2, 1, 250, 2);`.

Les **trois centres** sont assez loin l'un de l'autre pour que les carrés ne se touchent pas : (4, 4), (9, 9), (15, 8). À la fin, on voit les trois lettres.

LE PROGRAMME — 2144 OCTETS

```
// Les réglages du C, rangés sous un nom (le 0.48) :
//      x  y  tuile      taille  sens  vitesse  tours
Carre ronde = { 15,  8,  ALPHABET[2], 2,      1,    250,    2 };
// autour de (15, 8), la lettre C, 5 × 5, sens des aiguilles, 250 ms par pas, 2 tours

int main() {
  // 1. Le A : autour de (4, 4) ; taille 1 (3 × 3) ; sens 1 ; 250 ms ; 1 tour.
  carre(4, 4, ALPHABET[0], 1, 1, 250, 1);

  // 2. Le B : autour de (9, 9) ; taille 3 (7 × 7) ; sens -1 ; 100 ms ; 1 tour.
  carre(9, 9, ALPHABET[1], 3, -1, 100, 1);

  // 3. La ligne ajoutée : le C, avec les réglages de « ronde ».
  //   Le compilateur la remplace par carre(15, 8, ALPHABET[2], 2, 1, 250, 2).
  carre(ronde);

  while (true) {
    image();          // A en (4, 4), B en (9, 9), C en (15, 8)
  }
}
```

CE QU'ON DOIT VOIR

Le A fait un petit carré, le B un grand carré rapide, puis le C deux tours autour de (15, 8). Chacun reste à son centre.

A

l'écran, au bout de vingt images

0.54.1. Trois lettres, trois carrés — de base, ailleurs

Un quatrième Carre, « bas », pour le D autour de (15, 14).

C'est le 0.54, avec une quatrième lettre, le D, rangée dans un Carre nommé bas, autour de (15, 14).

C'est la version de base : la deuxième place, seule. Le 0.54.2 met les deux ensemble.

LE PROGRAMME — 2082 OCTETS

```
Carre bas = { 15, 14, ALPHABET[3], 2, 1, 250, 1 }; // le D, en bas à droite

int main() {
    carre(bas);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le D fait un tour autour de (15, 14).



l'écran, au bout de vingt images

0.54.2. Trois lettres, trois carrés — doublé, deux positions

Les trois carrés du 0.54, puis celui du D : quatre lettres.

C'est le 0.54 et le 0.54.1 réunis : les trois carrés, puis le quatrième.

LE PROGRAMME — 2175 OCTETS

```
Carre ronde = { 15, 8, ALPHABET[2], 2, 1, 250, 2 }; // le C
Carre bas   = { 15, 14, ALPHABET[3], 2, 1, 250, 1 }; // le D

int main() {
    carre(4, 4, ALPHABET[0], 1, 1, 250, 1); // le A
    carre(9, 9, ALPHABET[1], 3, -1, 100, 1); // le B
    carre(ronde); // le C
    carre(bas);   // le D

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A, le B, le C, puis le D
font leur carré.

A

l'écran, au bout de vingt images

0.55. La vitesse des déplacements : vitesse

`vitesse(ms)` règle le temps d'un pas de `deplace_x`, `deplace_y`, `deplace` et `va_a`. `vitesse(100)` : 10 pas par seconde.

`carre` a sa vitesse, mais `deplace_x`, `deplace_y`, `deplace` et `va_a` n'en ont pas : elles avancent toujours d'un pas tous les quarts de seconde (250 millisecondes).

Ce qui est nouveau ici : `vitesse(ms)`. Elle règle le temps d'un pas, en millisecondes, pour tous les déplacements qui viennent après elle. Elle ne fait rien bouger elle-même : elle change le rythme.

Comme pour `carre` : plus le nombre est petit, plus ça va vite. `vitesse(100)` : 10 pas par seconde. Le nombre s'écrit en clair.

Le reste est le 0.18 : `deplace_x` fait avancer la lettre de 5 cases. Mais cette fois, deux fois et demie plus vite.

LE PROGRAMME — 1595 OCTETS

```
uint8_t x = 0;          // la colonne de la lettre : 0, tout à gauche

int main() {
    // La ligne nouvelle :
    //
    // vitesse(100);
    //
    // +--- le temps d'UN pas, en millisecondes, pour tous les
    //      déplacements qui viennent APRÈS :
    //      250 → 4 pas par seconde (sans vitesse(), c'est ce rythme)
    //      100 → 10 pas par seconde (ici)
    //
    // vitesse() ne fait rien bouger : elle change le rythme.
    vitesse(100);

    deplace_x(x, 0, ALPHABET[0], 5);    // 5 cases à droite, à 100 ms par pas

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un A qui file de 5 cases vers la droite, bien plus vite qu'au 0.18.

A

l'écran, au bout de vingt images

0.55.1. La vitesse des déplacements : vitesse — de base, ailleurs

Le 0.55 sur la ligne 8, avec le B.

C'est le 0.55, avec le B, sur la ligne 8.

C'est la version de base : la deuxième place, seule. Le 0.55.2 met les deux ensemble.

LE PROGRAMME — 1596 OCTETS

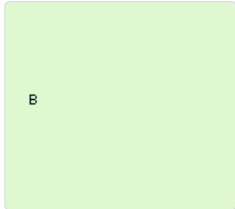
```
uint8_t x = 0;

int main() {
    vitesse(100);           // 10 pas par seconde
    deplace_x(x, 8, ALPHABET[1], 5); // le B, ligne 8

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui file de 5 cases
sur la ligne 8.

B

l'écran, au bout de vingt images

0.55.2. La vitesse des déplacements : vitesse — doublé, deux positions

Un seul `vitesse(100)` pour les deux : il vaut pour tous les déplacements qui suivent.

C'est le 0.55 et le 0.55.1 réunis : le A puis le B, avec le même `vitesse(100)`.

Un seul réglage suffit : `vitesse` reste réglée jusqu'au suivant, pour toutes les lettres.

LE PROGRAMME — 1619 OCTETS

```
uint8_t x = 0;      // le A
uint8_t xb = 0;     // le B

int main() {
    vitesse(100);
    deplace_x(x, 0, ALPHABET[0], 5);
    deplace_x(xb, 8, ALPHABET[1], 5);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A file sur la ligne 0,
puis le B sur la ligne 8, à
la même vitesse.

A

l'écran, au bout de vingt images

0.56. Changer de vitesse en route

Le 0.55, plus deux lignes : vitesse(500), puis le retour. La même lettre va vite, puis lentement.

C'est le 0.55, plus deux lignes : une nouvelle vitesse, et le retour.

Ce qui est nouveau ici : un deuxième vitesse(...). La vitesse reste réglée jusqu'au prochain vitesse : l'aller se fait à 100, et après vitesse(500), le retour se fait à 500 millisecondes par pas, 2 pas par seconde.

On peut changer de vitesse autant de fois qu'on veut, entre deux déplacements.

LE PROGRAMME — 1619 OCTETS

```
uint8_t x = 0;          // la colonne de la lettre

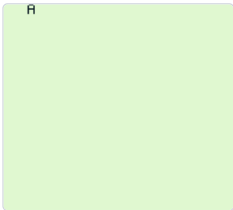
int main() {
    vitesse(100);          // 10 pas par seconde...
    deplace_x(x, 0, ALPHABET[0], 5); // ..pour l'aller (le 0.55)

    // Les deux lignes ajoutées :
    vitesse(500);          // désormais : 500 ms par pas, 2 par seconde
    deplace_x(x, 0, ALPHABET[0], -5); // le retour, lentement

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un A qui file vite vers la droite, puis revient lentement vers la gauche.



l'écran, au bout de vingt images

0.56.1. Changer de vitesse en route — de base, ailleurs

Le 0.56 sur la ligne 8, avec le B : vite à l'aller, lentement au retour.

C'est le 0.56, avec le B, sur la ligne 8.

C'est la version de base : la deuxième place, seule. Le 0.56.2 met les deux ensemble.

LE PROGRAMME — 1621 OCTETS

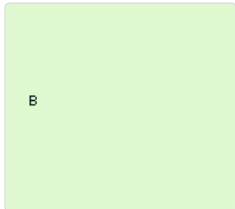
```
uint8_t x = 0;

int main() {
    vitesse(100);
    deplace_x(x, 8, ALPHABET[1], 5);    // vite
    vitesse(500);
    deplace_x(x, 8, ALPHABET[1], -5);   // lentement

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un B qui file à droite sur la ligne 8, puis revient lentement.

B

l'écran, au bout de vingt images

0.56.2. Changer de vitesse en route — doublé, deux positions

Le A vite puis lentement, et le B aussi : chaque vitesse vaut jusqu'à la suivante.

C'est le 0.56 et le 0.56.1 réunis : les deux lettres, chacune avec ses deux vitesses.

LE PROGRAMME — 1664 OCTETS

```
uint8_t x = 0;      // le A
uint8_t xb = 0;     // le B

int main() {
    vitesse(100);
    deplace_x(x, 0, ALPHABET[0], 5);
    deplace_x(xb, 8, ALPHABET[1], 5); // toujours à 100
    vitesse(500);
    deplace_x(x, 0, ALPHABET[0], -5);
    deplace_x(xb, 8, ALPHABET[1], -5); // toujours à 500

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le A puis le B filent à droite ; puis le A et le B reviennent lentement.

Fi

l'écran, au bout de vingt images

0.57. Un carré sur la pointe : losange

`losange(x, y, tuile, taille, sens, vitesse, tours)` : le carré tourné, ses côtés en diagonale. Les mêmes réglages que `carre`.

Un **losange**, c'est un **carré posé sur la pointe**. Ses quatre côtés ne sont plus droits : ce sont des **diagonales**. À chaque pas, la lettre bouge d'une case sur X **et** d'une case sur Y en même temps, comme `deplace(x, y, ..., 5, 5)` au 0.26.

Ce qui est nouveau ici : `losange(x, y, tuile, taille, sens, vitesse, tours)`. Les **mêmes sept réglages que** `carre`, dans le même ordre.

- `x`, `y` : le centre. `taille` : la distance du centre à chaque **pointe**. Avec 2, les pointes sont à 2 cases en haut, à droite, en bas et à gauche du centre.
- `sens` : `1` part vers la droite depuis la pointe du haut (comme les aiguilles d'une montre) ; `-1` part vers la gauche.
- `vitesse` en millisecondes par pas, `tours` : comme pour `carre`.

Le trajet : du centre, la lettre monte **tout droit** jusqu'à la pointe du haut ; elle fait le tour en diagonale ; puis elle redescend tout droit au centre.

Un Carre marche aussi : puisque les réglages sont les mêmes, `losange(ronde);` accepte un `Carre ronde = { ... }`, comme `carre(ronde);`.

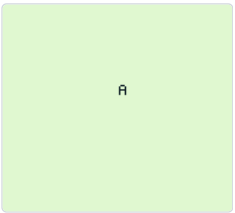
LE PROGRAMME — 2065 OCTETS

```
int main() {
    // Un losange autour de la case (10, 8). Mêmes réglages que carre :
    //
    //   losange(10, 8, ALPHABET[0], 2, 1, 250, 1);
    //
    //   | | |           | | | |
    //   | | |           | | |   +-- tours   : 1 tour
    //   | | |           | | +----- vitesse : 250 ms par pas
    //   | | |           | +----- sens     : 1, vers la droite d'abord
    //   | | |           +----- taille    : 2, les pointes à 2 cases
    //   | | +----- tuile    : la lettre A
    //   +----- x, y       : le centre (10, 8)
    //
    // Le chemin (sens 1). Chaque flèche est UN pas en diagonale :
    //
    //   colonne :  8  9 10 11 12
    //   ligne 6 :           2           2 : la pointe du haut
    //   ligne 7 :      ↗           ↘
    //   ligne 8 :  ↑      A      4      A : le centre, où tout commence
    //   ligne 9 :      ↘      ↗           et où tout finit
    //   ligne 10 :          6
    //
    //   1. du centre (10, 8), tout droit vers le haut, jusqu'à la pointe (10, 6)
    //   2. en diagonale, bas-droite, jusqu'à la pointe de droite (12, 8)
    //   3. bas-gauche, jusqu'à la pointe du bas (10, 10)
    //   4. haut-gauche, jusqu'à la pointe de gauche (8, 8)
    //   5. haut-droite, retour à la pointe du haut (10, 6)
    //   6. tout droit vers le bas, retour au centre (10, 8)
    losange(10, 8, ALPHABET[0], 2, 1, 250, 1);

    while (true) {
        image();           // le tour est fini : le A reste au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui monte de 2 cases, fait un tour en diagonale autour de (10, 8), puis revient au centre.



l'écran, au bout de vingt images

0.57.1. Un carré sur la pointe : losange — de base, ailleurs

Le 0.57 fait par le B, autour de (4, 4).

C'est le 0.57, avec le B, autour de (4, 4) au lieu de (10, 8).

La forme tient entière autour de (4, 4) : elle ne s'approche jamais à moins d'une case du bord.

C'est la version de base : la deuxième place, seule. Le 0.57.2 met les deux ensemble.

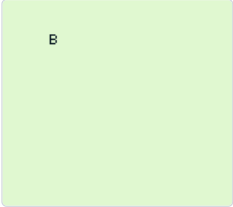
LE PROGRAMME — 2065 OCTETS

```
int main() {
    // Le B, en haut à gauche :
    losange(4, 4, ALPHABET[1], 2, 1, 250, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

La même forme qu'au 0.57, faite par le B en haut à gauche.



l'écran, au bout de vingt images

0.57.2. Un carré sur la pointe : losange — doublé, deux positions

La forme du A autour de (10, 8), puis celle du B autour de (4, 4).

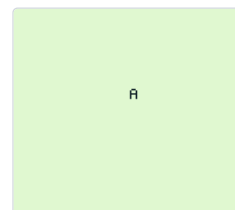
C'est le 0.57 et le 0.57.1 réunis : la même forme à deux places, l'une après l'autre.

LE PROGRAMME — 2096 OCTETS

```
int main() {  
    // 1. Le A, au milieu  
    losange(10, 8, ALPHABET[0], 2, 1, 250, 1);  
    // 2. Le B, en haut à gauche  
    losange(4, 4, ALPHABET[1], 2, 1, 250, 1);  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

La forme du A au milieu,
puis celle du B en haut à
gauche.



l'écran, au bout de vingt images

0.58. Plus large que haut : rectangle

`rectangle(x, y, tuile, largeur, hauteur, sens, vitesse, tours)` : un carré avec deux tailles, une pour X et une pour Y.

Un rectangle, c'est un carré qui n'a pas la même taille sur les deux axes. Au lieu d'une `taille`, il en a **deux** : la **largeur** (sur X) et la **hauteur** (sur Y).

Ce qui est nouveau ici : `rectangle(x, y, tuile, largeur, hauteur, sens, vitesse, tours)`, **huit** réglages : ceux de `carre`, avec `largeur` et `hauteur` à la place de `taille`.

• `largeur` et `hauteur` se comptent comme la taille d'un carré : la distance du centre au bord. La règle $2 \times n + 1$ vaut pour chacune : largeur 4 donne **9** cases de large, hauteur 2 donne **5** cases de haut, un rectangle de **9 × 5**.

• Avec la même largeur et la même hauteur, `rectangle` fait **un carré** : `rectangle(10, 8, A, 2, 2, ...)` fait le même tour que `carre(10, 8, A, 2, ...)`.

Le trajet : du centre, en diagonale, puis tout droit jusqu'au coin en haut à gauche ; le tour ; puis le même chemin à l'envers.

Les plus grandes tailles : largeur 9 (19 colonnes) et hauteur 8 (17 lignes).

LE PROGRAMME — 2301 OCTETS

```
int main() {
    // Un rectangle autour de la case (10, 8) :
    //
    //   rectangle(10, 8, ALPHABET[0], 4, 2, 1, 250, 1);
    //
    //   | | | | | | | |
    //   | | | | | | | | +- tours : 1 tour
    //   | | | | | | | | +----- vitesse : 250 ms par pas
    //   | | | | | | | | | +----- sens : 1, comme les aiguilles
    //   | | | | | | | | | +----- hauteur : 2 → 2 × 2 + 1 = 5 cases de haut
    //   | | | | | | | | | +----- largeur : 4 → 2 × 4 + 1 = 9 cases de large
    //   | | | | | | | | | +----- tuile : la lettre A
    //   +---+----- x, y : le centre (10, 8)
    //
    //
    //   colonne :  6  7  8  9 10 11 12 13 14
    //   ligne 6 :  → → → → → → → ↓      les côtés du haut et du bas :
    //   ligne 7 :  ↑                ↓      8 pas (2 × largeur)
    //   ligne 8 :  ↑          A      ↓
    //   ligne 9 :  ↑                ↓      les côtés de gauche et de droite :
    //   ligne 10 : ↑ ← ← ← ← ← ← ← ←  4 pas (2 × hauteur)
    //
    //
    // Le coin de départ est en haut à gauche, (6, 6) : le A y va du centre en
    // diagonale (2 pas), puis tout droit (2 pas), et en revient de même.
    rectangle(10, 8, ALPHABET[0], 4, 2, 1, 250, 1);

    while (true) {
        image();
        // le tour est fini : le A reste au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait le tour d'un rectangle de 9 cases de large et 5 de haut autour de (10, 8), puis revient au centre.



l'écran, au bout de vingt images

0.58.1. Plus large que haut : rectangle — de base, ailleurs

Le 0.58 fait par le B, autour de (4, 4).

C'est le 0.58, avec le B, autour de (4, 4) au lieu de (10, 8).

La forme tient entière autour de (4, 4) : elle ne s'approche jamais à moins d'une case du bord.

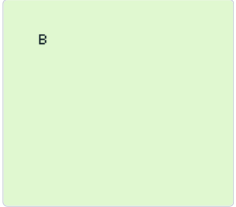
C'est la version de base : la deuxième place, seule. Le 0.58.2 met les deux ensemble.

LE PROGRAMME — 2301 OCTETS

```
int main() {  
    // Le B, en haut à gauche :  
    rectangle(4, 4, ALPHABET[1], 4, 2, 1, 250, 1);  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

La même forme qu'au 0.58, faite par le B en haut à gauche.



B

l'écran, au bout de vingt images

0.58.2. Plus large que haut : rectangle — doublé, deux positions

La forme du A autour de (10, 8), puis celle du B autour de (4, 4).

C'est le 0.58 et le 0.58.1 réunis : la même forme à deux places, l'une après l'autre.

LE PROGRAMME — 2336 OCTETS

```
int main() {  
    // 1. Le A, au milieu  
    rectangle(10, 8, ALPHABET[0], 4, 2, 1, 250, 1);  
    // 2. Le B, en haut à gauche  
    rectangle(4, 4, ALPHABET[1], 4, 2, 1, 250, 1);  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

La forme du A au milieu,
puis celle du B en haut à
gauche.



l'écran, au bout de vingt images

0.59. Tourner en s'éloignant : spirale

spirale(x, y, tuile, taille, sens, vitesse) : la lettre part du centre et tourne en s'éloignant, jusqu'au bord d'un carré de cette taille.

Une spirale tourne en s'éloignant du centre. Chaque branche est un peu plus longue que la précédente.

Ce qui est nouveau ici : **spirale(x, y, tuile, taille, sens, vitesse)**, **six** réglages : ceux de **carre**, sans les **tours** (une spirale ne se répète pas : elle s'élargit jusqu'au bout).

La règle des branches : 1 pas, 1 pas, 2 pas, 2 pas, 3 pas, 3 pas... La longueur **augmente d'un pas toutes les deux branches**. La spirale s'arrête quand elle a atteint le bord du carré de **taille**, soit des branches de $2 \times \text{taille}$ pas, et une dernière branche ferme le carré.

Le sens : **1** tourne comme les aiguilles d'une montre (droite, bas, gauche, haut, et on recommence) ; **-1** dans l'autre sens (bas, droite, haut, gauche).

À la fin, la lettre revient au centre en diagonale.

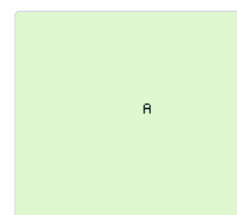
LE PROGRAMME — 2096 OCTETS

```
int main() {
    // Une spirale autour de la case (10, 8) :
    //
    //   spirale(10, 8, ALPHABET[0], 3, 1, 250);
    //
    //   | | |           | | |
    //   | | |           | | +-- vitesse : 250 ms par pas
    //   | | |           | +---- sens   : 1, droite, bas, gauche, haut...
    //   | | |           +----- taille : 3, jusqu'au bord d'un carré 7 x 7
    //   | | +-----+-----+ tuile   : la lettre A
    //   +---+-----+-----+ x, y    : le centre (10, 8)
    //
    // Les branches, avec taille 3 (le plus long :  $2 \times 3 = 6$  pas) :
    //
    //   branche : 1 2 3 4 5 6 7 8 9 10 11 12 13
    //   longueur : 1 1 2 2 3 3 4 4 5 5 6 6 6
    //   vers :    → ↓ ← ↑ → ↓ ← ↑ → ↓ ← ↑ →
    //
    // Le début, chaque case marquée du numéro de sa branche :
    //
    //   colonne : 9 10 11 12
    //   ligne 7 : 4 5 5 5      A : le centre (10, 8)
    //   ligne 8 : 4 A 1 .      1 : un pas à droite
    //   ligne 9 : 3 3 2 .      2 : un pas en bas
    //                               3 : deux pas à gauche
    //                               4 : deux pas en haut
    //                               5 : trois pas à droite... et ça grandit
    spirale(10, 8, ALPHABET[0], 3, 1, 250);

    while (true) {
        image(); // la spirale est finie : le A est revenu au centre
    }
}
```

CE QU'ON DOIT VOIR

Un A qui tourne autour de (10, 8) en s'éloignant un peu plus à chaque tour, puis revient au centre.



l'écran, au bout de vingt images

0.59.1. Tourner en s'éloignant : spirale — de base, ailleurs

Le 0.59 fait par le B, autour de (4, 4).

C'est le 0.59, avec le B, autour de (4, 4) au lieu de (10, 8).

La forme tient entière autour de (4, 4) : elle ne s'approche jamais à moins d'une case du bord.

C'est la version de base : la deuxième place, seule. Le 0.59.2 met les deux ensemble.

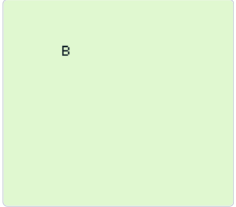
LE PROGRAMME — 2096 OCTETS

```
int main() {
    // Le B, en haut à gauche :
    spirale(4, 4, ALPHABET[1], 3, 1, 250);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

La même forme qu'au 0.59, faite par le B en haut à gauche.

B

l'écran, au bout de vingt images

0.59.2. Tourner en s'éloignant : spirale — doublé, deux positions

La forme du A autour de (10, 8), puis celle du B autour de (4, 4).

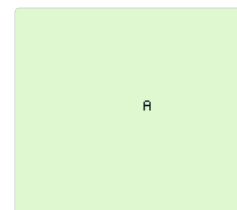
C'est le 0.59 et le 0.59.1 réunis : la même forme à deux places, l'une après l'autre.

LE PROGRAMME — 2123 OCTETS

```
int main() {  
    // 1. Le A, au milieu  
    spirale(10, 8, ALPHABET[0], 3, 1, 250);  
    // 2. Le B, en haut à gauche  
    spirale(4, 4, ALPHABET[1], 3, 1, 250);  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

La forme du A au milieu,
puis celle du B en haut à
gauche.



l'écran, au bout de vingt images

0.60. Aller et revenir : aller_retour

`aller_retour(x, y, tuile, pasX, pasY, vitesse, fois)` : aller jusqu'à un point, revenir, et recommencer.

Un va-et-vient : la lettre va jusqu'à un point, revient à sa place, et recommence.

Ce qui est nouveau ici : `aller_retour(x, y, tuile, pasX, pasY, vitesse, fois)`.

- `x`, `y` : la place de départ, où la lettre revient à chaque fois.
- `pasX`, `pasY` : où est l'autre bout, comme pour `deplace` : `5, 0` veut dire 5 cases à droite, rien sur Y.
- `vitesse` : en millisecondes par pas, comme pour `carre`. `fois` : combien d'allers-retours.

Au bord de l'écran, l'autre bout est ramené dans l'écran.

LE PROGRAMME — 2101 OCTETS

```
int main() {
    // La ligne, morceau par morceau :
    //
    // aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2);
    //
    //      | | |      | | | +- fois   : 2 allers-retours
    //      | | |      | | +----- vitesse : 250 ms par pas
    //      | | |      | +----- pasY   : 0, ne bouge pas sur Y
    //      | | |      +----- pasX   : 5, l'autre bout est
    //      | | |      5 cases à droite
    //      | | +----- tuile       : la lettre A
    //      +----- x, y       : le départ (10, 8)
    //
    // colonne : 10 11 12 13 14 15
    // ligne 8 : A → → → → → l'aller : de (10, 8) à (15, 8)
    //           ← ← ← ← ← le retour : de (15, 8) à (10, 8)
    aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2);

    while (true) {
        image();          // le A est revenu à (10, 8)
    }
}
```

CE QU'ON DOIT VOIR

Un A qui fait deux allers-retours de 5 cases vers la droite, et s'arrête à sa place.



l'écran, au bout de vingt images

0.60.1. Aller et revenir : aller_retour — de base, ailleurs

Le 0.60 fait par le B, autour de (4, 4).

C'est le 0.60, avec le B, autour de (4, 4) au lieu de (10, 8).

La forme tient entière autour de (4, 4) : elle ne s'approche jamais à moins d'une case du bord.

C'est la version de base : la deuxième place, seule. Le 0.60.2 met les deux ensemble.

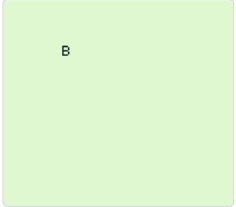
LE PROGRAMME — 2101 OCTETS

```
int main() {
    // Le B, en haut à gauche :
    aller_retour(4, 4, ALPHABET[1], 5, 0, 250, 2);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

La même forme qu'au 0.60, faite par le B en haut à gauche.

A large green square with the letter 'B' in the top-left corner.

l'écran, au bout de vingt images

0.60.2. Aller et revenir : aller_retour — doublé, deux positions

La forme du A autour de (10, 8), puis celle du B autour de (4, 4).

C'est le 0.60 et le 0.60.1 réunis : la même forme à deux places, l'une après l'autre.

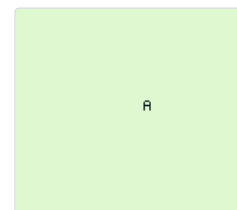
LE PROGRAMME — 2131 OCTETS

```
int main() {
    // 1. Le A, au milieu
    aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2);
    // 2. Le B, en haut à gauche
    aller_retour(4, 4, ALPHABET[1], 5, 0, 250, 2);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

La forme du A au milieu,
puis celle du B en haut à
gauche.



l'écran, au bout de vingt images

0.61. Aller et revenir en diagonale

Le 0.60, plus une ligne : aller_retour avec 4 et 4, un va-et-vient en biais.

C'est le 0.60, plus une ligne : un aller-retour en diagonale.

Ce qui est nouveau ici : pasX et pasY à la fois : 4, 4. L'autre bout est 4 cases à droite et 4 cases en bas, en (14, 12). À chaque pas, la lettre avance d'une case sur X et d'une case sur Y : elle va **en biais**, comme `deplace` au 0.26.

Plus vite, et une seule fois : vitesse 100, fois 1.

LE PROGRAMME — 2132 OCTETS

```
int main() {
    aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2); // tout droit (le 0.60)

    // La ligne ajoutée : 4 à droite ET 4 en bas → en diagonale.
    //
    //      A                                de (10, 8)...
    //      ↘
    //      ↘                                à chaque pas : +1 sur X ET +1 sur Y
    //      ↘
    //      ↘                                ...à (14, 12), puis retour par le même chemin
    aller_retour(10, 8, ALPHABET[0], 4, 4, 100, 1);

    while (true) {
        image(); // le A est revenu à (10, 8)
    }
}
```

CE QU'ON DOIT VOIR

Deux allers-retours vers la droite, puis un aller-retour rapide en diagonale vers le bas.



l'écran, au bout de vingt images

0.61.1. Aller et revenir en diagonale — de base, ailleurs

Le 0.61 fait par le B, autour de (4, 4).

C'est le 0.61, avec le B, autour de (4, 4) au lieu de (10, 8).

La forme tient entière autour de (4, 4) : elle ne s'approche jamais à moins d'une case du bord.

C'est la version de base : la deuxième place, seule. Le 0.61.2 met les deux ensemble.

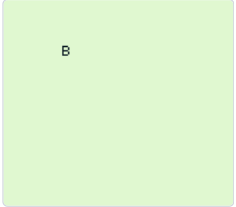
LE PROGRAMME — 2132 OCTETS

```
int main() {
    // Le B, en haut à gauche :
    aller_retour(4, 4, ALPHABET[1], 5, 0, 250, 2);
    aller_retour(4, 4, ALPHABET[1], 4, 4, 100, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

La même forme qu'au 0.61, faite par le B en haut à gauche.

A green square with the letter 'B' in the top-left corner.

l'écran, au bout de vingt images

0.61.2. Aller et revenir en diagonale — doublé, deux positions

La forme du A autour de (10, 8), puis celle du B autour de (4, 4).

C'est le 0.61 et le 0.61.1 réunis : la même forme à deux places, l'une après l'autre.

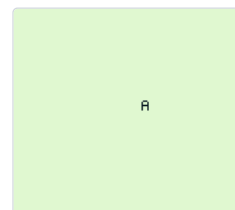
LE PROGRAMME — 2193 OCTETS

```
int main() {
    // 1. Le A, au milieu
    aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2);
    aller_retour(10, 8, ALPHABET[0], 4, 4, 100, 1);
    // 2. Le B, en haut à gauche
    aller_retour(4, 4, ALPHABET[1], 5, 0, 250, 2);
    aller_retour(4, 4, ALPHABET[1], 4, 4, 100, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

La forme du A au milieu,
puis celle du B en haut à
gauche.



l'écran, au bout de vingt images

0.62. Deux formes ensemble

Un losange et un rectangle : deux lettres, deux formes, chacune dans son coin de l'écran.

On commence à réunir les formes : un losange (le 0.57) pour le A, un rectangle (le 0.58) pour le B.

Ce qui est nouveau ici : deux formes **différentes** dans le même programme, avec des centres choisis pour qu'elles ne se touchent pas : le A en haut à gauche, le B en haut à droite.

Elles se font l'une après l'autre : le B commence quand le A est revenu à son centre. Toutes deux à 100 ms par pas, pour que le tout ne dure pas trop.

LE PROGRAMME — 3093 OCTETS

```
int main() {
    // colonnes 2 à 6          colonnes 10 à 18
    // +-----+          +-----+
    // |  A : losange  |    |  B : rectangle  |    lignes 2 à 6
    // | autour (4, 4) |    | autour (14, 4)  |
    // +-----+          +-----+

    // 1. Le A : un losange, taille 2, sens 1, 100 ms par pas, 1 tour.
    losange(4, 4, ALPHABET[0], 2, 1, 100, 1);

    // 2. Le B : un rectangle de 9 x 5 (largeur 4, hauteur 2), dans l'autre
    // sens (-1), 100 ms par pas, 1 tour.
    rectangle(14, 4, ALPHABET[1], 4, 2, -1, 100, 1);

    while (true) {
        image();          // A en (4, 4), B en (14, 4)
    }
}
```

CE QU'ON DOIT VOIR

Le A fait un losange en haut à gauche, puis le B un rectangle en haut à droite.

A

l'écran, au bout de vingt images

0.62.1. Deux formes ensemble — de base, ailleurs

Les deux formes du 0.62, en bas de l'écran, faites par le C et le D.

C'est le 0.62, avec les deux mêmes formes **en bas** de l'écran, par le C et le D.

C'est la version de base : la deuxième place, seule. Le 0.62.2 met les deux ensemble.

LE PROGRAMME — 3093 OCTETS

```
int main() {
    losange(4, 12, ALPHABET[2], 2, 1, 100, 1);    // le C, en bas à gauche
    rectangle(14, 12, ALPHABET[3], 4, 2, -1, 100, 1); // le D, en bas à droite

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un losange en bas à gauche, puis un rectangle en bas à droite.



l'écran, au bout de vingt images

0.62.2. Deux formes ensemble — doublé, deux positions

Quatre formes : les deux du haut, puis les deux du bas.

C'est le 0.62 et le 0.62.1 réunis : les formes du haut, puis celles du bas.

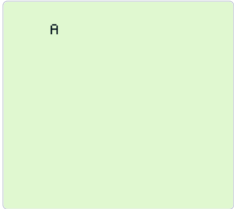
LE PROGRAMME — 3159 OCTETS

```
int main() {
  losange(4, 4, ALPHABET[0], 2, 1, 100, 1);      // le A, en haut à gauche
  rectangle(14, 4, ALPHABET[1], 4, 2, -1, 100, 1); // le B, en haut à droite
  losange(4, 12, ALPHABET[2], 2, 1, 100, 1);      // le C, en bas à gauche
  rectangle(14, 12, ALPHABET[3], 4, 2, -1, 100, 1); // le D, en bas à droite

  while (true) {
    image();
  }
}
```

CE QU'ON DOIT VOIR

Losange et rectangle en haut, puis losange et rectangle en bas.

A small screenshot of the program output showing the letter 'A'.

l'écran, au bout de vingt images

0.63. Trois formes ensemble

Le 0.62, plus une spirale : le C tourne en bas à gauche.

C'est le 0.62, plus une ligne : une spirale (le 0.59) pour le C, en bas à gauche de l'écran.

Ce qui est nouveau ici : une troisième forme, sous les deux autres. `ALPHABET[2]` est la 3e lettre, le **C**.

LE PROGRAMME — 3916 OCTETS

```
int main() {
  // +-----+           +-----+
  // |  A : losange  |       | B : rectangle  |   lignes 2 à 6
  // +-----+           +-----+
  // +-----+
  // |  C : spirale  |                               lignes 10 à 14
  // | autour (4,12) |
  // +-----+

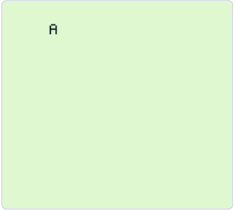
  losange(4, 4, ALPHABET[0], 2, 1, 100, 1);           // 1. le A (le 0.62)
  rectangle(14, 4, ALPHABET[1], 4, 2, -1, 100, 1);    // 2. le B (le 0.62)

  // 3. La ligne ajoutée : le C, une spirale jusqu'à la taille 2 (5 × 5),
  //    sens des aiguilles, 100 ms par pas.
  spirale(4, 12, ALPHABET[2], 2, 1, 100);

  while (true) {
    image();           // A en (4, 4), B en (14, 4), C en (4, 12)
  }
}
```

CE QU'ON DOIT VOIR

Le losange du A, le rectangle du B, puis la spirale du C en bas à gauche.



l'écran, au bout de vingt images

0.63.1. Trois formes ensemble — de base, ailleurs

Une spirale de plus, en bas à droite : le D autour de (14, 12).

C'est le 0.63, avec une spirale **en bas à droite**, par le D.

C'est la version de base : la deuxième place, seule. Le 0.63.2 met les deux ensemble.

LE PROGRAMME — 2096 OCTETS

```
int main() {
    spirale(14, 12, ALPHABET[3], 2, 1, 100);           // le D, en bas à droite

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Une spirale en bas à droite.



l'écran, au bout de vingt images

0.63.2. Trois formes ensemble — doublé, deux positions

Les trois formes du 0.63, puis la spirale du D.

C'est le 0.63 et le 0.63.1 réunis : quatre formes, dont deux spirales.

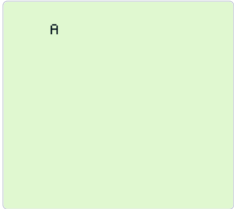
LE PROGRAMME — 3943 OCTETS

```
int main() {
  losange(4, 4, ALPHABET[0], 2, 1, 100, 1);      // le A, en haut à gauche
  rectangle(14, 4, ALPHABET[1], 4, 2, -1, 100, 1); // le B, en haut à droite
  spirale(4, 12, ALPHABET[2], 2, 1, 100);         // le C, en bas à gauche
  spirale(14, 12, ALPHABET[3], 2, 1, 100);        // le D, en bas à droite

  while (true) {
    image();
  }
}
```

CE QU'ON DOIT VOIR

Losange, rectangle,
spirale, puis une seconde
spirale en bas à droite.



l'écran, au bout de vingt images

0.64. Quatre formes ensemble

Le 0.63, plus un aller-retour : le D va et vient en bas à droite. Quatre lettres, quatre formes.

C'est le 0.63, plus une ligne : un aller-retour (le 0.60) pour le D, en bas à droite.

Ce qui est nouveau ici : la quatrième forme. `ALPHABET[3]` est la 4e lettre, le **D** : on compte à partir de 0 (A = 0, B = 1, C = 2, D = 3).

Change un nombre et relance : chaque forme a sa taille, son sens et sa vitesse. Tout ce qu'on a vu depuis le 0.34 est là.

LE PROGRAMME — 4744 OCTETS

```
int main() {
    // colonnes 2 à 6          colonnes 10 à 18
    // +-----+             +-----+
    // |  A : losange  |       |  B : rectangle  |   lignes 2 à 6
    // +-----+             +-----+
    // +-----+             +-----+
    // |  C : spirale  |       |  D : aller-retour |   lignes 10 à 14
    // +-----+             +-----+

    losange(4, 4, ALPHABET[0], 2, 1, 100, 1);      // 1. le A
    rectangle(14, 4, ALPHABET[1], 4, 2, -1, 100, 1); // 2. le B
    spirale(4, 12, ALPHABET[2], 2, 1, 100);         // 3. le C (le 0.63)

    // 4. La ligne ajoutée : le D (ALPHABET[3], la 4e lettre), un aller-retour
    //   de 7 cases vers la droite, 100 ms par pas, 1 fois.
    aller_retour(10, 13, ALPHABET[3], 7, 0, 100, 1);

    while (true) {
        image();          // les quatre lettres sont revenues à leur place
    }
}
```

CE QU'ON DOIT VOIR

Le A fait un losange, le B un rectangle, le C une spirale, le D un aller-retour. Chacun finit à sa place.

A

l'écran, au bout de vingt images

0.64.1. Quatre formes ensemble — de base, ailleurs

Un aller-retour de plus, sur la ligne 8 : le E, de (3, 8) à (17, 8).

C'est le 0.64, avec un aller-retour **sur la ligne 8**, entre les formes du haut et celles du bas, par le E (`ALPHABET[4]`).

C'est la **version de base** : la deuxième place, seule. Le 0.64.2 met les deux ensemble.

LE PROGRAMME — 2101 OCTETS

```
int main() {  
    aller_retour(3, 8, ALPHABET[4], 14, 0, 100, 1);    // le E, ligne 8  
  
    while (true) {  
        image();  
    }  
}
```

CE QU'ON DOIT VOIR

Un E qui va de (3, 8) à (17, 8) et revient.

A large green square representing the letter 'E' on a grid. The letter 'E' is formed by a horizontal bar at the top and a vertical bar on the left side, with a small gap between them. The letter is centered within the square.

l'écran, au bout de vingt images

0.64.2. Quatre formes ensemble — doublé, deux positions

Cinq lettres, cinq formes : les quatre du 0.64 et l'aller-retour du E au milieu.

C'est le 0.64 et le 0.64.1 réunis : les quatre formes, puis le E au milieu.

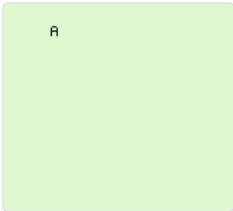
LE PROGRAMME — 4774 OCTETS

```
int main() {
    losange(4, 4, ALPHABET[0], 2, 1, 100, 1);      // le A, en haut à gauche
    rectangle(14, 4, ALPHABET[1], 4, 2, -1, 100, 1); // le B, en haut à droite
    spirale(4, 12, ALPHABET[2], 2, 1, 100);        // le C, en bas à gauche
    aller_retour(10, 13, ALPHABET[3], 7, 0, 100, 1); // le D, en bas à droite
    aller_retour(3, 8, ALPHABET[4], 14, 0, 100, 1); // le E, ligne 8

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Les quatre formes, puis le
E fait un aller-retour au
milieu de l'écran.



l'écran, au bout de vingt images

0.65. Une lettre qui avance

Bouger, c'est effacer la lettre à sa place, puis la réécrire un peu plus loin.

Maintenant, la lettre ne s'arrête plus : il n'y a plus de `pas`, elle avance tant que la console tourne.

La lettre A avance d'une colonne toutes les **15 images**, soit quatre fois par seconde.

Pour bouger, on **efface** d'abord l'ancienne place avec `effacer(x, 0, 1)`, une case ; puis on change `x`, et la lettre est réécrite à sa nouvelle place. Sans l'effacement, elle laisserait une traînée de A derrière elle.

Au bout de la ligne, `x` revient à 0 : la lettre repart de la gauche.

LE PROGRAMME — 1388 OCTETS

```
uint8_t x = 0;           // la colonne de la lettre
uint8_t images = 0;

int main() {
    while (true) {
        image();
        images++;

        if (images == 15) {           // 4 fois par seconde
            images = 0;
            effacer(x, 0, 1);         // 1. efface l'ancienne place
            x++;                      // 2. une colonne plus loin
            if (x == 20) x = 0;       // au bout de la ligne, retour à gauche
        }

        poser(x, 0, ALPHABET[0]);    // 3. la lettre à sa nouvelle place
    }
}
```

CE QU'ON DOIT VOIR

Un A qui avance vers la droite, et repart de la gauche au bout de la ligne.

A

l'écran, au bout de vingt images

0.65.1. Une lettre qui avance — de base, ailleurs

Le 0.65 sur la ligne 8, avec le B.

C'est le 0.65, avec le B, sur la ligne 8.

C'est la version de base : la deuxième place, seule. Le 0.65.2 met les deux ensemble.

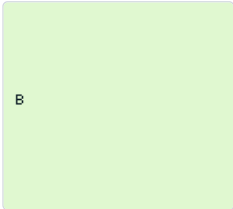
LE PROGRAMME — 1390 OCTETS

```
uint8_t x = 0;          // la colonne de la lettre
uint8_t images = 0;

int main() {
    while (true) {
        image();
        images++;
        if (images == 15) {
            images = 0;
            effacer(x, 8, 1);
            x++;
            if (x == 20) x = 0;
        }
        poser(x, 8, ALPHABET[1]); // le B, ligne 8
    }
}
```

CE QU'ON DOIT VOIR

Un B qui avance sans fin
sur la ligne 8.

B

l'écran, au bout de vingt images

0.65.2. Une lettre qui avance — doublé, deux positions

Le A et le B avancent ensemble : la même x, deux lignes.

C'est le 0.65 et le 0.65.1 réunis : deux `effacer`, deux `poser`, la même `x`.

LE PROGRAMME — 1441 OCTETS

```
uint8_t x = 0;          // la colonne des deux lettres
uint8_t images = 0;

int main() {
    while (true) {
        image();
        images++;
        if (images == 15) {
            images = 0;
            effacer(x, 0, 1);
            effacer(x, 8, 1);
            x++;
            if (x == 20) x = 0;
        }
        poser(x, 0, ALPHABET[0]); // le A, ligne 0
        poser(x, 8, ALPHABET[1]); // le B, ligne 8
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B avancent ensemble, sur la ligne 0 et sur la ligne 8.

A

B

l'écran, au bout de vingt images

0.65.3. Une lettre qui avance — en simple

Le 0.65 en une ligne : `defile(0, 0, 0, ALPHABET[0], 1, 250)`. La lettre file à droite et repart de la gauche, sans fin.

C'est le 0.65, en plus simple : le chronomètre, l'effacement, `x++` et le retour à 0 sont remplacés par une seule ligne, `defile`.

`defile(numero, x, y, tuile, sens, vitesse)`, une fonction de la console (voir le 0.76.12) : au premier appel, la lettre apparaît en (`x`, `y`) ; ensuite, un pas toutes les `vitesse` ms, à droite (`sens` 1) ou à gauche (-1). Au bord, elle repart de l'autre côté : exactement le `if (x == 20) x = 0;` du 0.65.

Plus de variables à déclarer : la console retient elle-même où en est la lettre, grâce au numéro 0.

LE PROGRAMME — 1695 OCTETS

```
int main() {
    while (true) {
        image();
        // Tout le 0.65 en une ligne :
        //  numero 0, départ (0, 0), le A, vers la droite (1), 250 ms par pas
        defile(0, 0, 0, ALPHABET[0], 1, 250);
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.65 : un A qui avance vers la droite et repart de la gauche.

A

l'écran, au bout de vingt images

0.66. La lettre bouge avec la croix

Le joueur agit, l'écran répond : c'est déjà un jeu.

`bouton(DROITE)` vaut vrai **tant que** la flèche droite est enfoncée. Même chose pour `GAUCHE`, `HAUT` et `BAS`.

Bouger à chaque image ferait traverser l'écran en un tiers de seconde. `attente` impose donc **8 images** entre deux pas : en gardant la flèche enfoncée, la lettre avance d'une case environ sept fois par seconde.

Les conditions `x < 19`, `x > 0`, `y < 17` et `y > 0` empêchent de sortir de l'écran.

Voilà les trois ingrédients d'un jeu : la **boucle**, le **joueur qui agit**, et l'**écran qui répond**.

LE PROGRAMME — 1602 OCTETS

```
uint8_t x = 9;          // la lettre part du milieu de l'écran
uint8_t y = 8;
uint8_t attente = 0;    // les images à attendre avant le prochain pas

int main() {
    while (true) {
        image();

        if (attente > 0) attente--;

        if (attente == 0) {
            effacer(x, y, 1);          // efface l'ancienne place
            if (bouton(DROITE) && x < 19) { x++; attente = 8; }
            if (bouton(GAUCHE) && x > 0) { x--; attente = 8; }
            if (bouton(BAS) && y < 17) { y++; attente = 8; }
            if (bouton(HAUT) && y > 0) { y--; attente = 8; }
        }

        poser(x, y, ALPHABET[0]);      // la lettre à sa place
    }
}
```

CE QU'ON DOIT VOIR

Un A au milieu de l'écran, qui se déplace avec les flèches.



l'écran, au bout de vingt images

0.66.1. La lettre bouge avec la croix — de base, ailleurs

Le 0.66 avec le B, qui part de (4, 4).

C'est le 0.66, avec le B, qui part de (4, 4) au lieu du milieu.

C'est la version de base : la deuxième place, seule. Le 0.66.2 met les deux ensemble.

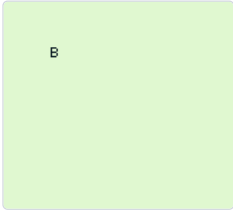
LE PROGRAMME — 1562 OCTETS

```
uint8_t x = 4;      // la colonne du B
uint8_t y = 4;      // sa ligne
uint8_t attente = 0;

int main() {
    while (true) {
        image();
        if (attente > 0) attente--;
        if (attente == 0) {
            effacer(x, y, 1);
            if (bouton(DROITE)) { if (x < 19) x++; attente = 8; }
            if (bouton(GAUCHE)) { if (x > 0) x--; attente = 8; }
            if (bouton(BAS))    { if (y < 17) y++; attente = 8; }
            if (bouton(HAUT))   { if (y > 0) y--; attente = 8; }
        }
        poser(x, y, ALPHABET[1]);
    }
}
```

CE QU'ON DOIT VOIR

Un B en haut à gauche, qui se déplace avec les flèches.

B

l'écran, au bout de vingt images

0.66.2. La lettre bouge avec la croix — doublé, deux positions

Le A et le B suivent la MÊME croix : ils bougent ensemble, chacun depuis sa place.

C'est le 0.66 et le 0.66.1 réunis : les deux lettres, commandées par la même croix.

Une flèche, deux pas : chaque bouton fait avancer les deux lettres ; chacune garde sa place et s'arrête à SON bord.

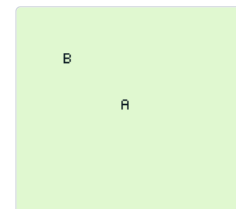
LE PROGRAMME — 1725 OCTETS

```
uint8_t x = 9;          // la colonne du A
uint8_t y = 8;          // sa ligne
uint8_t xb = 4;         // la colonne du B
uint8_t yb = 4;         // sa ligne
uint8_t attente = 0;

int main() {
    while (true) {
        image();
        if (attente > 0) attente--;
        if (attente == 0) {
            effacer(x, y, 1);
            effacer(xb, yb, 1);
            if (bouton(DROITE)) { if (x < 19) x++; if (xb < 19) xb++; attente = 8; }
            if (bouton(GAUCHE)) { if (x > 0) x--; if (xb > 0) xb--; attente = 8; }
            if (bouton(BAS)) { if (y < 17) y++; if (yb < 17) yb++; attente = 8; }
            if (bouton(HAUT)) { if (y > 0) y--; if (yb > 0) yb--; attente = 8; }
        }
        poser(x, y, ALPHABET[0]);
        poser(xb, yb, ALPHABET[1]);
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B bougent ensemble avec les flèches.



l'écran, au bout de vingt images

0.66.3. La lettre bouge avec la croix — en simple

Le 0.66 en une ligne : `deplace_croix(x, y, ALPHABET[0], 133)`. Plus d'attente ni d'effacer à écrire.

C'est le 0.66, en plus simple : tout le bloc de la croix (l'attente, l'effacement, les quatre `if`, le `poser`) est remplacé par une ligne, `deplace_croix` (le 0.71).

La vitesse, 133 ms : c'est 8 images, exactement l'`attente = 8` du 0.66. La lettre va donc à la même vitesse.

Et elle ne clignote plus : `deplace_croix` n'efface l'ancienne case que si la lettre a bougé. Le 0.66 effaçait et reposait à chaque pas possible.

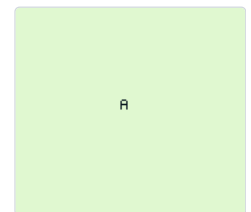
LE PROGRAMME — 1748 OCTETS

```
uint8_t x = 9;          // la lettre part du milieu de l'écran
uint8_t y = 8;

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 133); // tout le bloc du 0.66 : 133 ms = 8 images
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.66 : un A au milieu, qui se déplace avec les flèches.



l'écran, au bout de vingt images

0.67. Les variables de la lettre dans leur propre fichier

Le 0.66, rangé en deux : `x`, `y` et `attente` déménagent dans « `variables.h` » ; `principal.cpp` n'a plus que la boucle du jeu.

C'est exactement le programme du 0.66, avec un seul changement : les **trois variables** du haut (`x`, `y`, `attente`) ne sont plus dans `principal.cpp`. Elles sont dans un **second fichier**, `variables.h` : c'est l'**onglet** à côté de `principal.cpp`, au-dessus du code.

Ce qui est nouveau ici : un programme en deux fichiers. `principal.cpp` est le fichier **principal** : c'est par lui que la compilation commence. `variables.h` est un fichier **voisin** : il ne fait rien tout seul, il attend qu'on le verse quelque part.

`#include "variables.h"` veut dire : « **verse ici** tout ce qu'il y a dans `variables.h` ». Avant de compiler, la console remplace cette ligne par le contenu du fichier, mot pour mot. Le programme compilé est donc **exactement le même** qu'au 0.66 : il est seulement rangé en deux.

Pourquoi tout en haut : une variable doit être déclarée **avant** qu'on s'en serve. `main()` utilise `x`, `y` et `attente` : le `#include` qui les apporte vient donc avant `main()`.

Le `.h` est une habitude du C : il veut dire « **header** », en-tête, un fichier fait pour être versé en haut d'un autre. Les guillemets autour du nom disent que le fichier est à **côté** du programme.

À quoi ça sert : d'un côté **l'état du jeu** (où est la lettre, combien attendre), de l'autre **ce qu'on en fait** (la boucle, les boutons). Pour changer la place de départ de la lettre, on n'ouvre que `variables.h`, sans toucher à la boucle.

LE PROGRAMME — 1602 OCTETS

`variables.h`

```
// variables.h : l'ÉTAT de la lettre qui bouge.
// Ce fichier ne fait rien tout seul : principal.cpp le verse chez lui
// avec #include "variables.h", tout en haut, avant main().

uint8_t x = 9;      // la COLONNE de la lettre : elle part du milieu (9)
uint8_t y = 8;      // la LIGNE de la lettre : elle part du milieu (8)
uint8_t attente = 0; // les images à attendre avant le prochain pas (0 : tout de suite)
```

`principal.cpp`

```
// Le changement : les trois variables ne sont plus ici.
//
// #include "variables.h"
// |
// |   ++ le fichier à verser : l'onglet « variables.h », à côté
// |   +----- « verse ici » : avant la compilation, cette ligne est
// |             remplacée par tout le contenu de variables.h. Tout se passe
// |             comme si x, y et attente étaient écrits ici, comme au 0.66.
#include "variables.h"

int main() {
    // La boucle du jeu : exactement celle du 0.66. Elle se sert de x, y et
    // attente, qui viennent de variables.h.
    while (true) {
        image();                                // attend l'image suivante

        if (attente > 0) attente--;              // le temps passe

        if (attente == 0) {                      // on peut faire un pas
            effacer(x, y, 1);                    // efface l'ancienne place
            if (bouton(DROITE) && x < 19) { x++; attente = 8; }
            if (bouton(GAUCHE) && x > 0) { x--; attente = 8; }
            if (bouton(BAS) && y < 17) { y++; attente = 8; }
            if (bouton(HAUT) && y > 0) { y--; attente = 8; }
        }

        poser(x, y, ALPHABET[0]);                // la lettre à sa place
    }
}
```

CE QU'ON DOIT VOIR

Exactement le 0.66 : un A au milieu de l'écran, qui se déplace avec les flèches. Mais le programme est rangé en deux onglets.

A

l'écran, au bout de vingt images

0.67.1. Les variables de la lettre dans leur propre fichier — de base, ailleurs

Le 0.67 avec le **B** : seul `variables.h` change (le départ en (4, 4)).

C'est le **0.67**, avec le **B**, qui part de (4, 4) : le départ change dans `variables.h`.

C'est l'intérêt du fichier à part : pour changer la place de départ, on n'a touché que `variables.h`.

C'est la version de base : la deuxième place, seule. Le 0.67.2 met les deux ensemble.

LE PROGRAMME — 1562 OCTETS

`variables.h`

```
// variables.h : l'ÉTAT des lettres qui bougent.
uint8_t x = 4;      // la colonne du B
uint8_t y = 4;      // sa ligne
uint8_t attente = 0;
```

`principal.cpp`

```
// voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();
        if (attente > 0) attente--;
        if (attente == 0) {
            effacer(x, y, 1);
            if (bouton(DROITE)) { if (x < 19) x++; attente = 8; }
            if (bouton(GAUCHE)) { if (x > 0) x--; attente = 8; }
            if (bouton(BAS))    { if (y < 17) y++; attente = 8; }
            if (bouton(HAUT))   { if (y > 0) y--; attente = 8; }
        }
        poser(x, y, ALPHABET[1]);
    }
}
```

CE QU'ON DOIT VOIR

Un B qui part de (4, 4) et se déplace avec les flèches.

B

l'écran, au bout de vingt images

0.67.2. Les variables de la lettre dans leur propre fichier — doublé, deux positions

Deux lettres, et leurs quatre variables dans `variables.h`.

C'est le 0.67 et le 0.67.1 réunis : les deux lettres ; `variables.h` porte leurs quatre variables.

LE PROGRAMME — 1725 OCTETS

`variables.h`

```
// variables.h : l'ÉTAT des lettres qui bougent.
uint8_t x = 9;      // la colonne du A
uint8_t y = 8;      // sa ligne
uint8_t xb = 4;     // la colonne du B
uint8_t yb = 4;     // sa ligne
uint8_t attente = 0;
```

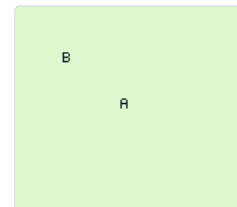
`principal.cpp`

```
// voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();
        if (attente > 0) attente--;
        if (attente == 0) {
            effacer(x, y, 1);
            effacer(xb, yb, 1);
            if (bouton(DROITE)) { if (x < 19) x++; if (xb < 19) xb++; attente = 8; }
            if (bouton(GAUCHE)) { if (x > 0) x--; if (xb > 0) xb--; attente = 8; }
            if (bouton(BAS))    { if (y < 17) y++; if (yb < 17) yb++; attente = 8; }
            if (bouton(HAUT))   { if (y > 0) y--; if (yb > 0) yb--; attente = 8; }
        }
        poser(x, y, ALPHABET[0]);
        poser(xb, yb, ALPHABET[1]);
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B bougent
ensemble avec les flèches.



l'écran, au bout de vingt images

0.67.3. Les variables de la lettre dans leur propre fichier — en simple

Le 0.67 en simple : `variables.h` ne garde que `x` et `y` ; `principal.cpp`, une ligne dans la boucle.

C'est le 0.67, en plus simple : la boucle de la croix est remplacée par une ligne, `deplace_croix`.

`variables.h` rétrécit aussi : plus besoin d'`attente`, la console la tient elle-même. Il ne reste que `x` et `y`, la place de la lettre.

LE PROGRAMME — 1748 OCTETS

`variables.h`

```
// variables.h : la place de la lettre. L'attente, la console la tient.
uint8_t x = 9;
uint8_t y = 8;
```

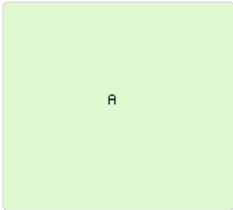
`principal.cpp`

```
// x et y : voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 133);
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.67 : un A qui se déplace avec les flèches, programme en deux onglets.

A

l'écran, au bout de vingt images

0.68. Voir la position du A en direct

Le 0.67, plus deux lignes dans la boucle : `nombre(0, 17, x)` et `nombre(4, 17, y)`. La position du A s'affiche à chaque image, pendant qu'on le déplace.

C'est le 0.67, plus deux lignes dans la boucle : `nombre(0, 17, x);` et `nombre(4, 17, y);`.

Ce qui est nouveau ici : `nombre()` **DANS la boucle**. Au 0.20, `nombre()` était appelé une fois, après un trajet : il montrait la valeur à ce moment-là. Ici, il est dans `while (true)` : il est rappelé à **chaque image**, 60 fois par seconde. L'écran montre donc toujours la valeur **du moment** : c'est de l'affichage **en temps réel**.

Ce qu'on voit : en bas à gauche, trois chiffres pour `x` (la colonne du A), et à côté, trois chiffres pour `y` (sa ligne). Appuie sur les flèches : les nombres changent en même temps que la lettre bouge. Au départ, `009` et `008`, le milieu de l'écran.

Pourquoi avant `poser` : si le A va sur la ligne 17, il passe **par-dessus** les nombres, parce qu'il est posé après eux. Les nombres sont réécrits à l'image suivante, dès que le A repart.

Sans rien écrire, la page le montre aussi : sous la console, ouvre «  Inspecteur », onglet **Variables** : `x`, `y` et `attente` y sont, par leur nom, mis à jour 4 fois par seconde.

LE PROGRAMME — 1622 OCTETS

`variables.h`

```
// variables.h : l'ÉTAT de la lettre qui bouge (comme au 0.67).

uint8_t x = 9;           // la COLONNE de la lettre : elle part du milieu (9)
uint8_t y = 8;           // la LIGNE de la lettre : elle part du milieu (8)
uint8_t attente = 0;     // les images à attendre avant le prochain pas
```

`principal.cpp`

```
// x, y, attente : voir l'onglet variables.h. (#include veut être seul sur sa ligne.)
#include "variables.h"

int main() {
    while (true) {
        image();                                     // attend l'image suivante

        if (attente > 0) attente--;

        if (attente == 0) {
            effacer(x, y, 1);                         // efface l'ancienne place
            if (bouton(DROITE) && x < 19) { x++; attente = 8; }
            if (bouton(GAUCHE) && x > 0) { x--; attente = 8; }
            if (bouton(BAS) && y < 17) { y++; attente = 8; }
            if (bouton(HAUT) && y > 0) { y--; attente = 8; }
        }

        // Les deux lignes ajoutées : la position du A, réécrite à CHAQUE image.
        //
        //   nombre(0, 17, x);           nombre(4, 17, y);
        //   | | |                       | | |
        //   | | +-- x : la colonne | | +-- y : la ligne du A
        //   | +----- ligne 17 : tout en bas de l'écran
        //   +----- colonne 0 et colonne 4 : côte à côte
        //
        //   En bas de l'écran : 009 008 ← le A au départ, en (9, 8)
        //                       012 010 ← après 3 pas à droite et 2 en bas
        nombre(0, 17, x);           // la colonne, en direct
        nombre(4, 17, y);           // la ligne, en direct

        poser(x, y, ALPHABET[0]);           // la lettre, par-dessus tout
    }
}
```

CE QU'ON DOIT VOIR

Le A qui se déplace avec les flèches ; en bas à gauche, sa colonne et sa ligne changent en même temps.



l'écran, au bout de vingt images

0.68.1. Voir la position du A en direct — de base, ailleurs

Le 0.68 avec le B, parti de (4, 4) : sa position en direct.

C'est le 0.68, avec le B, parti de (4, 4) ; ses nombres en bas à gauche.

C'est la version de base : la deuxième place, seule. Le 0.68.2 met les deux ensemble.

LE PROGRAMME — 1582 OCTETS

variables.h

```
// variables.h : l'ÉTAT des lettres qui bougent.
uint8_t x = 4;      // la colonne du B
uint8_t y = 4;      // sa ligne
uint8_t attente = 0;
```

principal.cpp

```
// voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();
        if (attente > 0) attente--;
        if (attente == 0) {
            effacer(x, y, 1);
            if (bouton(DROITE)) { if (x < 19) x++; attente = 8; }
            if (bouton(GAUCHE)) { if (x > 0) x--; attente = 8; }
            if (bouton(BAS)) { if (y < 17) y++; attente = 8; }
            if (bouton(HAUT)) { if (y > 0) y--; attente = 8; }
        }
        nombre(0, 17, x);
        nombre(4, 17, y);
        poser(x, y, ALPHABET[1]);
    }
}
```

CE QU'ON DOIT VOIR

Un B qui bouge avec les flèches ; sa colonne et sa ligne en bas à gauche.

B

004 004

l'écran, au bout de vingt images

0.68.2. Voir la position du A en direct — doublé, deux positions

Les deux positions en direct : le A à gauche, le B à droite de la ligne 17.

C'est le 0.68 et le 0.68.1 réunis : deux lettres, quatre nombres.

LE PROGRAMME — 1765 OCTETS

variables.h

```
// variables.h : l'ÉTAT des lettres qui bougent.
uint8_t x = 9;      // la colonne du A
uint8_t y = 8;      // sa ligne
uint8_t xb = 4;     // la colonne du B
uint8_t yb = 4;     // sa ligne
uint8_t attente = 0;
```

principal.cpp

```
// voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();
        if (attente > 0) attente--;
        if (attente == 0) {
            effacer(x, y, 1);
            effacer(xb, yb, 1);
            if (bouton(DROITE)) { if (x < 19) x++; if (xb < 19) xb++; attente = 8; }
            if (bouton(GAUCHE)) { if (x > 0) x--; if (xb > 0) xb--; attente = 8; }
            if (bouton(BAS)) { if (y < 17) y++; if (yb < 17) yb++; attente = 8; }
            if (bouton(HAUT)) { if (y > 0) y--; if (yb > 0) yb--; attente = 8; }
        }
        nombre(0, 17, x);
        nombre(4, 17, y);
        nombre(10, 17, xb);
        nombre(14, 17, yb);
        poser(x, y, ALPHABET[0]);
        poser(xb, yb, ALPHABET[1]);
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B bougent ; en bas, leurs deux positions.

009 008 004 004

l'écran, au bout de vingt images

0.68.3. Voir la position du A en direct — en simple

Le 0.68 en simple : `deplace_croix`, puis les deux `nombre`. Trois lignes dans la boucle.

C'est le 0.68, en plus simple : la croix tient en une ligne, `deplace_croix` ; il reste les deux `nombre` qui montrent la position.

Trois lignes font tout : bouger, montrer la colonne, montrer la ligne.

LE PROGRAMME — 1768 OCTETS

`variables.h`

```
uint8_t x = 9;
uint8_t y = 8;
```

`principal.cpp`

```
// x et y : voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 133); // bouger
        nombre(0, 17, x);                      // la colonne, en direct
        nombre(4, 17, y);                      // la ligne, en direct
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.68 : le A bouge, sa position s'affiche en bas.

A

009 008

l'écran, au bout de vingt images

0.69. Chercher le A sur l'écran : lire

Le 0.68, plus une recherche : quand on appuie sur A, le programme regarde chaque case de l'écran avec lire(), et affiche où il a trouvé la lettre.

C'est le 0.68, plus un bloc : quand on appuie sur le bouton A, le programme **cherche** la lettre sur l'écran, case par case, et affiche où il l'a trouvée.

Ce qui est nouveau ici : lire(colonne, ligne). Elle rend la tuile affichée dans une case de l'écran. lire(9, 8) == ALPHABET[0] est vrai si la case (9, 8) montre un A.

La recherche : deux boucles for, l'une dans l'autre, passent sur **toutes les cases** : l fait les 18 lignes (0 à 17), et pour chaque ligne, c fait les 20 colonnes (0 à 19). Soit $18 \times 20 = 360$ cases. Quand une case contient le A, on retient sa colonne dans trouveX et sa ligne dans trouveY.

Deux façons de connaître la position. x et y : le programme la **retient** lui-même, à chaque pas. lire() : il la **retrouve** en regardant l'écran, sans rien avoir retenu. Les deux nombres affichés (en bas à gauche, et à droite après avoir appuyé sur A) sont **les mêmes** : c'est la preuve que les deux façons s'accordent.

À quoi sert lire() : à savoir **ce qu'il y a à un endroit**, pas seulement où est sa propre lettre. Plus tard, c'est ainsi qu'on saura si une case est un mur, une pièce, un ennemi.

Pourquoi seulement quand on appuie sur A : regarder 360 cases prend du temps. Le faire à chaque image ralentirait le jeu ; on ne le fait que quand on le demande.

LE PROGRAMME — 1774 OCTETS

variables.h

```
// variables.h : l'ÉTAT de la lettre qui bouge, et ce que la recherche a trouvé.

uint8_t x = 9;      // la COLONNE de la lettre : elle part du milieu (9)
uint8_t y = 8;      // la LIGNE de la lettre : elle part du milieu (8)
uint8_t attente = 0; // les images à attendre avant le prochain pas

uint8_t trouveX = 0; // la colonne où lire() a trouvé le A (nouveau)
uint8_t trouveY = 0; // la ligne où lire() a trouvé le A (nouveau)
```

principal.cpp

```
// x, y, attente, trouveX, trouveY : voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();

        if (attente > 0) attente--;

        if (attente == 0) {
            effacer(x, y, 1);
            if (bouton(DROITE) && x < 19) { x++; attente = 8; }
            if (bouton(GAUCHE) && x > 0) { x--; attente = 8; }
            if (bouton(BAS) && y < 17) { y++; attente = 8; }
            if (bouton(HAUT) && y > 0) { y--; attente = 8; }
        }

        nombre(0, 17, x); // ce que le programme RETIENT : x...
        nombre(4, 17, y); // ...et y (le 0.68)
        poser(x, y, ALPHABET[0]);

        // Le bloc ajouté : quand on appuie sur A, on CHERCHE la lettre sur l'écran.
        if (bouton(A)) {
            for (uint8_t l = 0; l < 18; l++) { // chaque ligne, de 0 à 17...
                for (uint8_t c = 0; c < 20; c++) { // ...et dans chaque ligne, chaque colonne
                    // lire(c, l) : la tuile affichée dans la case (c, l).
                    //
                    // if (lire(c, l) == ALPHABET[0])
                    // | | | |
                    // | | | +-- la tuile du A
                    // | +--+----- la case regardée : colonne c, ligne l
                    // +----- rend ce que l'écran montre là
                    if (lire(c, l) == ALPHABET[0]) {
                        trouveX = c; // trouvé : on retient la colonne...
                        trouveY = l; // ...et la ligne
                    }
                }
            }
            nombre(10, 17, trouveX); // ce que lire() a TROUVÉ, à droite :
            nombre(14, 17, trouveY); // les mêmes nombres que x et y
        }
    }
}
```

CE QU'ON DOIT VOIR

Le A se déplace avec les flèches. Quand on appuie sur A, sa position trouvée sur l'écran s'affiche à droite, en bas : la même que celle de gauche.



l'écran, au bout de vingt images

0.69.1. Chercher le A sur l'écran : lire — de base, ailleurs

On cherche le B, cette fois : `lire(c, l) == ALPHABET[1]`.

C'est le 0.69, avec le B, parti de (4, 4) ; la recherche compare à `ALPHABET[1]`.

C'est la version de base : la deuxième place, seule. Le 0.69.2 met les deux ensemble.

LE PROGRAMME — 1734 OCTETS

variables.h

```
// variables.h : l'ÉTAT des lettres qui bougent.
uint8_t x = 4;      // la colonne du B
uint8_t y = 4;      // sa ligne
uint8_t attente = 0;
uint8_t trouveX = 0;
uint8_t trouveY = 0;
```

principal.cpp

```
// voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();
        if (attente > 0) attente--;
        if (attente == 0) {
            effacer(x, y, 1);
            if (bouton(DROITE)) { if (x < 19) x++; attente = 8; }
            if (bouton(GAUCHE)) { if (x > 0) x--; attente = 8; }
            if (bouton(BAS))    { if (y < 17) y++; attente = 8; }
            if (bouton(HAUT))   { if (y > 0) y--; attente = 8; }
        }
        nombre(0, 17, x);
        nombre(4, 17, y);
        poser(x, y, ALPHABET[1]);
        if (bouton(A)) {
            for (uint8_t l = 0; l < 18; l++) {
                for (uint8_t c = 0; c < 20; c++) {
                    if (lire(c, l) == ALPHABET[1]) { trouveX = c; trouveY = l; } // cherche le B
                }
            }
            nombre(10, 17, trouveX);
            nombre(14, 17, trouveY);
        }
    }
}
```

CE QU'ON DOIT VOIR

Un B qui bouge ; avec A, sa position trouvée s'affiche à droite.

B

004 004

l'écran, au bout de vingt images

0.69.2. Chercher le A sur l'écran : lire — doublé, deux positions

Deux lettres, et la recherche du B : lire() distingue les lettres.

C'est le 0.69 et le 0.69.1 réunis : le A et le B bougent ; la recherche ne trouve que le B.

lire compare des tuiles : ALPHABET[1] n'est égal qu'au B. Le A, sur une autre case, est ignoré.

LE PROGRAMME — 1897 OCTETS

variables.h

```
// variables.h : l'ÉTAT des lettres qui bougent.
uint8_t x = 9;      // la colonne du A
uint8_t y = 8;      // sa ligne
uint8_t xb = 4;     // la colonne du B
uint8_t yb = 4;     // sa ligne
uint8_t attente = 0;
uint8_t trouveX = 0;
uint8_t trouveY = 0;
```

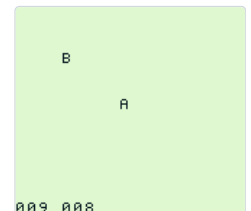
principal.cpp

```
// voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();
        if (attente > 0) attente--;
        if (attente == 0) {
            effacer(x, y, 1);
            effacer(xb, yb, 1);
            if (bouton(DROITE)) { if (x < 19) x++; if (xb < 19) xb++; attente = 8; }
            if (bouton(GAUCHE)) { if (x > 0) x--; if (xb > 0) xb--; attente = 8; }
            if (bouton(BAS)) { if (y < 17) y++; if (yb < 17) yb++; attente = 8; }
            if (bouton(HAUT)) { if (y > 0) y--; if (yb > 0) yb--; attente = 8; }
        }
        nombre(0, 17, x);
        nombre(4, 17, y);
        poser(x, y, ALPHABET[0]);
        poser(xb, yb, ALPHABET[1]);
        if (bouton(A)) {
            for (uint8_t l = 0; l < 18; l++) {
                for (uint8_t c = 0; c < 20; c++) {
                    if (lire(c, l) == ALPHABET[1]) { trouveX = c; trouveY = l; } // cherche le B
                }
            }
            nombre(10, 17, trouveX);
            nombre(14, 17, trouveY);
        }
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B bougent ;
avec A, c'est la position
du B qui s'affiche à droite.



l'écran, au bout de vingt images

0.69.3. Chercher le A sur l'écran : lire — en simple

Le 0.69 en simple : la croix en une ligne ; la recherche avec lire() reste, c'est elle qu'on apprend ici.

C'est le 0.69, en plus simple : la croix est remplacée par `deplace_croix`.

La recherche, elle, reste écrite à la main : c'est ce que la leçon apprend, `lire()` et les deux boucles. Simplifier, ce n'est pas tout cacher : on ne cache que ce qu'on sait déjà faire.

LE PROGRAMME — 1920 OCTETS

variables.h

```
uint8_t x = 9;
uint8_t y = 8;
uint8_t trouveX = 0; // la colonne où lire() a trouvé le A
uint8_t trouveY = 0; // sa ligne
```

principal.cpp

```
// x, y, trouveX, trouveY : voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 133);
        nombre(0, 17, x);
        nombre(4, 17, y);

        if (bouton(A)) { // on cherche le A sur l'écran
            for (uint8_t l = 0; l < 18; l++) {
                for (uint8_t c = 0; c < 20; c++) {
                    if (lire(c, l) == ALPHABET[0]) {
                        trouveX = c;
                        trouveY = l;
                    }
                }
            }
            nombre(10, 17, trouveX);
            nombre(14, 17, trouveY);
        }
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.69 : avec A, la position trouvée s'affiche à droite, la même qu'à gauche.



009 008

l'écran, au bout de vingt images

0.70. Plus fluide : la lettre au pixel près

`sprite(0, px, py, ALPHABET[0])` : la lettre devient un lutin, qui avance d'UN pixel par image au lieu de sauter d'une case. Le mouvement devient fluide.

Pourquoi la lettre saute : jusqu'ici, elle était **posée sur la grille** avec `poser`. Une case fait **8 × 8 pixels** : chaque pas la faisait donc sauter de 8 pixels d'un coup, environ 7 fois par seconde. L'œil voit des sauts.

Ce qui est nouveau ici : un lutin, avec `sprite(numero, x, y, tuile)`. Un lutin n'est **pas** sur la grille : il flotte au-dessus du décor, et se place **au pixel près**. On le fait avancer d'un **seul pixel par image**, 60 fois par seconde : le mouvement devient **fluide**.

- `numero` : le numéro du lutin, de 0 à 39 (la console en a 40). Ici, `0`, le premier.
- `px`, `py` : sa place **en pixels**, et plus en cases. L'écran fait **160 pixels** de large et **144** de haut. Le milieu, pour une lettre de 8 pixels : $(160 - 8) \div 2 = 76$, et $(144 - 8) \div 2 = 68$.
- `tuile` : ce qu'il montre, `ALPHABET[0]`, le A.

Plus besoin d'effacer ni d'attente : un lutin se **déplace**, il ne se réécrit pas. On lui donne sa nouvelle place à chaque image, et la console le dessine là, sans rien laisser derrière. Et comme un pixel est petit, on peut avancer à chaque image, sans attendre.

Les bords : `px` va de 0 à **152** ($160 - 8$, pour que la lettre reste entière), `py` de 0 à **136** ($144 - 8$).

LE PROGRAMME — 1510 OCTETS

variables.h

```
// variables.h : la place de la lettre, maintenant EN PIXELS.

uint8_t px = 76;      // la colonne en pixels : 76 = le milieu ((160 - 8) / 2)
uint8_t py = 68;      // la ligne en pixels : 68 = le milieu ((144 - 8) / 2)
```

principal.cpp

```
// px, py : voir l'onglet variables.h
#include "variables.h"

int main() {
    while (true) {
        image(); // 60 fois par seconde

        // UN pixel par image, tant que la flèche est tenue : 60 pixels par
        // seconde, sans à-coups. (Avec poser, c'était 8 pixels d'un coup.)
        if (bouton(DROITE) && px < 152) px++; // 152 = 160 - 8 : le bord droit
        if (bouton(GAUCHE) && px > 0) px--;
        if (bouton(BAS) && py < 136) py++; // 136 = 144 - 8 : le bord du bas
        if (bouton(HAUT) && py > 0) py--;

        // Le lutin, morceau par morceau :
        //
        // sprite(0, px, py, ALPHABET[0]);
        // | | | |
        // | | | +-- tuile : ce qu'il montre, la lettre A
        // | +---+----- px, py : sa place EN PIXELS (et non en cases)
        // +----- numero : le lutin n° 0 (la console en a 40, de 0 à 39)
        //
        // Pas d'effacer : un lutin se DÉPLACE, il ne laisse rien derrière lui.
        sprite(0, px, py, ALPHABET[0]);
    }
}
```

CE QU'ON DOIT VOIR

Un A au milieu de l'écran, qui glisse doucement avec les flèches, pixel par pixel, sans sauter de case en case.



l'écran, au bout de vingt images

0.70.1. Plus fluide : la lettre au pixel près — de base, ailleurs

Le 0.70 avec le B, parti de (20, 20).

C'est le 0.70, avec le B, parti de (20, 20) pixels, en haut à gauche.

C'est la version de base : la deuxième place, seule. Le 0.70.2 met les deux ensemble.

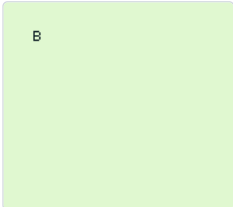
LE PROGRAMME — 1510 OCTETS

```
uint8_t px = 20;      // le B, en pixels
uint8_t py = 20;

int main() {
    while (true) {
        image();
        if (bouton(DROITE) && px < 152) px++;
        if (bouton(GAUCHE) && px > 0)  px--;
        if (bouton(BAS) && py < 136)  py++;
        if (bouton(HAUT) && py > 0)   py--;
        sprite(0, px, py, ALPHABET[1]); // le lutin n° 0
    }
}
```

CE QU'ON DOIT VOIR

Un B en haut à gauche,
qui glisse avec les flèches.

B

l'écran, au bout de vingt images

0.70.2. Plus fluide : la lettre au pixel près — doublé, deux positions

Deux lutins, 0 et 1 : le A et le B glissent ensemble.

C'est le 0.70 et le 0.70.1 réunis : deux lutins, chacun avec son numéro (0 et 1).

Chaque lutin a son numéro : le même numéro ferait bouger le même lutin. Ici, 0 pour le A, 1 pour le B.

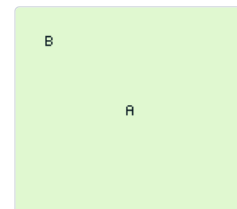
LE PROGRAMME — 1747 OCTETS

```
uint8_t px = 76;      // le A, en pixels
uint8_t py = 68;
uint8_t pxb = 20;     // le B, en pixels
uint8_t pyb = 20;

int main() {
    while (true) {
        image();
        if (bouton(DROITE) && px < 152) px++;
        if (bouton(GAUCHE) && px > 0)  px--;
        if (bouton(BAS) && py < 136)  py++;
        if (bouton(HAUT) && py > 0)    py--;
        if (bouton(DROITE) && pxb < 152) pxb++;
        if (bouton(GAUCHE) && pxb > 0)  pxb--;
        if (bouton(BAS) && pyb < 136)  pyb++;
        if (bouton(HAUT) && pyb > 0)    pyb--;
        sprite(0, px, py, ALPHABET[0]); // le lutin n° 0
        sprite(1, pxb, pyb, ALPHABET[1]); // le lutin n° 1
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B glissent ensemble avec les flèches.



l'écran, au bout de vingt images

0.70.3. Plus fluide : la lettre au pixel près — en simple

Le 0.70 en une ligne : `glisse_croix(0, px, py, ALPHABET[0], 1)`.

C'est le 0.70, en plus simple : les quatre `if` des flèches et le `sprite` sont remplacés par une ligne, `glisse_croix` (le 0.73).

La vitesse, 1 : un pixel par image, comme au 0.70.

LE PROGRAMME — 1592 OCTETS

```
uint8_t px = 76;      // en pixels : le milieu
uint8_t py = 68;

int main() {
    while (true) {
        image();
        glisse_croix(0, px, py, ALPHABET[0], 1); // tout le 0.70
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.70 : un A qui glisse pixel par pixel avec les flèches.



l'écran, au bout de vingt images

0.71. La croix en une ligne : deplace_croix

`deplace_croix(x, y, tuile, vitesse)`, fonction de la console : tout le bloc de la croix du 0.66 en une seule ligne, dans la boucle, à la vitesse qu'on choisit.

C'est le programme du 0.66, avec un seul changement : tout le bloc qui lisait la croix (l'attente, l'effacement, les quatre `if`, le `poser`) est remplacé par **une ligne**.

Ce qui est nouveau ici : `deplace_croix(x, y, tuile, vitesse)`. C'est une fonction **de la console**, comme `ALPHABET` est un tableau de la console : on s'en sert **sans rien déclarer**, et elle n'est ajoutée à la cartouche que si on l'appelle.

La vitesse, 4e réglage, est obligatoire : c'est le temps entre deux pas, en **millisecondes**. `250` : un pas tous les quarts de seconde, **4 cases par seconde**. Plus le nombre est petit, plus la lettre va vite (le 0.72 essaie `100`). Il s'écrit **en clair** : le compilateur le traduit en images avant le jeu, comme `ms(250)` = 15 images.

Ce qu'elle fait, à chaque appel : elle lit la croix ; si une flèche est tenue et que le temps d'attente est passé, elle fait **un pas** d'une case (en restant dans l'écran), efface l'ancienne case, et attend le temps de la vitesse avant le pas suivant. Puis elle pose la lettre à sa place.

Elle ne bloque pas : un appel, au plus un pas, et elle rend la main. On l'appelle donc **dans la boucle**, une fois par image, après `image()`.

Pas de `x =` : comme `deplace` au 0.26, elle est seule sur sa ligne, et la console range la nouvelle colonne dans `x` et la nouvelle ligne dans `y`.

Elle ne clignote pas : elle n'efface l'ancienne case **que si la lettre a bougé**. Immobile, la lettre est simplement reposée à la même place.

LE PROGRAMME — 1748 OCTETS

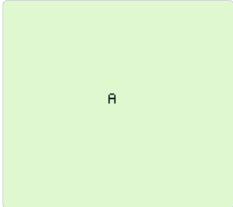
```
uint8_t x = 9;           // la COLONNE de la lettre : elle part du milieu
uint8_t y = 8;           // la LIGNE de la lettre

int main() {
    while (true) {
        image();          // une fois par image, 60 fois par seconde

        // La ligne qui remplace tout le bloc du 0.66 :
        //
        //   deplace_croix(x, y, ALPHABET[0], 250);
        //
        //           | | | |
        //           | | | +-- vitesse : 250 ms entre deux pas,
        //           | | |                soit 4 cases par seconde
        //           | | |
        //           | | +----- tuile   : ce qui bouge, la lettre A
        //           +---+----- x, y    : où elle est ; la console
        //                               y range sa nouvelle place
        //
        // Ce qu'elle fait pour nous, à chaque image :
        // - lit la croix (DROITE, GAUCHE, HAUT, BAS)
        // - au plus UN pas d'une case, puis attend 250 ms
        // - reste dans l'écran (colonnes 0 à 19, lignes 0 à 17)
        // - efface l'ancienne case, SEULEMENT si la lettre a bougé
        // - pose la lettre à sa place
        deplace_croix(x, y, ALPHABET[0], 250);
    }
}
```

CE QU'ON DOIT VOIR

Un A au milieu de l'écran, qui se déplace case par case avec les flèches, 4 cases par seconde. Le programme tient en quelques lignes.



l'écran, au bout de vingt images

0.71.1. La croix en une ligne : `deplace_croix` — de base, ailleurs

Le 0.71 avec le **B**, parti de (4, 4).

C'est le 0.71, avec le **B**, parti de (4, 4).

C'est la version de base : la deuxième place, seule. Le 0.71.2 met les deux ensemble.

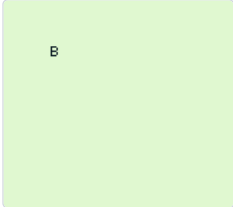
LE PROGRAMME — 1748 OCTETS

```
uint8_t x = 4;      // le B
uint8_t y = 4;

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[1], 250);
    }
}
```

CE QU'ON DOIT VOIR

Un B qui suit la croix,
depuis le haut à gauche.



B

l'écran, au bout de vingt images

0.71.2. La croix en une ligne : `deplace_croix` — doublé, deux positions

Deux lettres, la même croix : deux appels, dans la même image.

C'est le 0.71 et le 0.71.1 réunis : deux appels, un par lettre.

Les deux bougent dans la même image : la console ne fait passer le temps d'attente qu'une fois par image ; chaque appel de cette image a le droit de faire son pas.

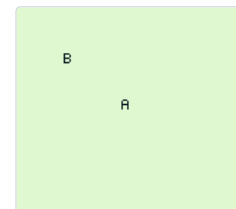
LE PROGRAMME — 1781 OCTETS

```
uint8_t x = 9;      // le A
uint8_t y = 8;
uint8_t xb = 4;     // le B
uint8_t yb = 4;

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250);
        deplace_croix(xb, yb, ALPHABET[1], 250);
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B suivent la croix ensemble.



l'écran, au bout de vingt images

0.72. Plus vite sur la grille : deplace_croix à 100

Le 0.71 avec un seul nombre changé : la vitesse passe de 250 à 100 millisecondes. La lettre fait 10 cases par seconde au lieu de 4.

C'est le 0.71, avec un seul changement : la vitesse, 4e réglage de `deplace_croix`, passe de `250` à `100`.

Ce qui est nouveau ici : choisir sa vitesse. `100` millisecondes entre deux pas : **10 cases par seconde**, deux fois et demie plus vite qu'au 0.71. `500` ferait 2 cases par seconde, très lentement.

Plus le nombre est petit, plus ça va vite, puisque c'est le temps d'attente entre deux pas.

Trop vite, c'est difficile à diriger : à `50` (20 cases par seconde), la lettre traverse l'écran en une seconde, et il devient dur de s'arrêter sur la bonne case. Essaie, et choisis ce qui convient à ton jeu.

LE PROGRAMME — 1748 OCTETS

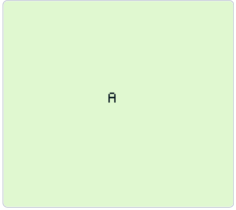
```
uint8_t x = 9;      // la COLONNE de la lettre
uint8_t y = 8;      // la LIGNE de la lettre

int main() {
    while (true) {
        image();

        // Le changement : la vitesse, 250 → 100.
        //
        // deplace_croix(x, y, ALPHABET[0], 100);
        //
        // +-- vitesse : le temps entre deux pas.
        // Plus PETIT = plus VITE :
        // 500 → 2 cases par seconde
        // 250 → 4 cases par seconde (le 0.71)
        // 100 → 10 cases par seconde (ici)
        // 50 → 20 cases par seconde
        deplace_croix(x, y, ALPHABET[0], 100);
    }
}
```

CE QU'ON DOIT VOIR

Le même A qu'au 0.71, mais qui file sur la grille : 10 cases par seconde.



l'écran, au bout de vingt images

0.72.1. Plus vite sur la grille : deplace_croix à 100 — de base, ailleurs

Le 0.72 avec le **B**, parti de (4, 4).

C'est le 0.72, avec le **B**, parti de (4, 4).

C'est la version de base : la deuxième place, seule. Le 0.72.2 met les deux ensemble.

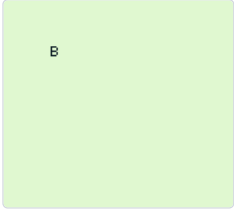
LE PROGRAMME — 1748 OCTETS

```
uint8_t x = 4;      // le B
uint8_t y = 4;

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[1], 100);
    }
}
```

CE QU'ON DOIT VOIR

Un B qui suit la croix,
depuis le haut à gauche.



B

l'écran, au bout de vingt images

0.72.2. Plus vite sur la grille : `deplace_croix` à 100 — doublé, deux positions

Deux lettres, la même croix : deux appels, dans la même image.

C'est le 0.72 et le 0.72.1 réunis : deux appels, un par lettre.

Les deux bougent dans la même image : la console ne fait passer le temps d'attente qu'une fois par image ; chaque appel de cette image a le droit de faire son pas.

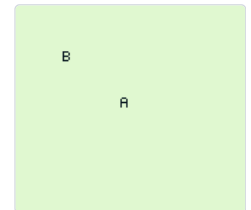
LE PROGRAMME — 1781 OCTETS

```
uint8_t x = 9;          // le A
uint8_t y = 8;
uint8_t xb = 4;         // le B
uint8_t yb = 4;

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 100);
        deplace_croix(xb, yb, ALPHABET[1], 100);
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B suivent la croix ensemble.



l'écran, au bout de vingt images

0.73. Glisser en une ligne : glisse_croix

glisse_croix(0, px, py, tuile, vitesse), fonction de la console : la lettre-lutin du 0.70 suit la croix au pixel près, en une seule ligne.

C'est le programme du 0.70, avec un seul changement : les quatre `if` des flèches et le `sprite` sont remplacés par **une ligne**.

Ce qui est nouveau ici : `glisse_croix(numero, px, py, tuile, vitesse)`. La même idée que `deplace_croix` au 0.71, mais **au pixel près**, avec un lutin.

- `numero` : le numéro du lutin, `0`. • `px`, `py` : sa place en pixels ; la console y range la nouvelle place. • `tuile` : ce qu'il montre.
- `vitesse`, obligatoire : combien de **pixels par image** tant qu'une flèche est tenue. `1` : un pixel par image, **60 pixels par seconde**, le mouvement le plus doux. Ce n'est pas une durée comme pour `deplace_croix` : ici, **plus le nombre est grand, plus ça va vite** (le 0.74 essaie `3`).

Elle reste dans l'écran : `px` de 0 à 152, `py` de 0 à 136, pour que la lettre de 8 pixels reste entière dans l'écran de 160 × 144.

Laquelle choisir ? `deplace_croix` pour un jeu **sur la grille** (un labyrinthe, un plateau), où l'on avance case par case. `glisse_croix` pour un personnage **qui glisse** librement.

LE PROGRAMME — 1592 OCTETS

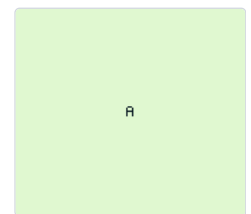
```
uint8_t px = 76;      // la colonne en PIXELS : le milieu
uint8_t py = 68;      // la ligne en PIXELS   : le milieu

int main() {
    while (true) {
        image();        // une fois par image, 60 fois par seconde

        // La ligne qui remplace les quatre if et le sprite du 0.70 :
        //
        //      glisse_croix(0, px, py, ALPHABET[0], 1);
        //
        //      | | | | |
        //      | | | | |      +-- vitesse : 1 pixel par image,
        //      | | | | |      60 pixels par seconde
        //      | | | +----- tuile   : la lettre A
        //      | +---+----- px, py  : sa place EN PIXELS ; la
        //      |                   console y range la nouvelle
        //      +----- numero   : le lutin n° 0
        glisse_croix(0, px, py, ALPHABET[0], 1);
    }
}
```

CE QU'ON DOIT VOIR

Exactement le 0.70 : un A qui glisse doucement avec les flèches, pixel par pixel. Mais en une ligne.



l'écran, au bout de vingt images

0.73.1. Glisser en une ligne : glisse_croix — de base, ailleurs

Le 0.73 avec le B, parti de (20, 20) pixels.

C'est le 0.73, avec le B, parti de (20, 20) pixels.

C'est la version de base : la deuxième place, seule. Le 0.73.2 met les deux ensemble.

LE PROGRAMME — 1592 OCTETS

```
uint8_t px = 20;      // le B, en pixels
uint8_t py = 20;

int main() {
    while (true) {
        image();
        glisse_croix(0, px, py, ALPHABET[1], 1);
    }
}
```

CE QU'ON DOIT VOIR

Un B qui suit la croix,
depuis le haut à gauche.

B

l'écran, au bout de vingt images

0.73.2. Glisser en une ligne : glisse_croix — doublé, deux positions

Deux lettres, la même croix : deux appels, dans la même image.

C'est le 0.73 et le 0.73.1 réunis : deux appels, un par lettre.

Deux lutins, deux numéros (0 et 1) : ils glissent ensemble.

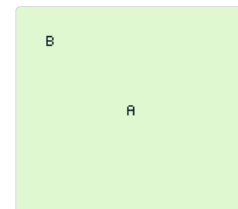
LE PROGRAMME — 1629 OCTETS

```
uint8_t px = 76;      // le A, en pixels
uint8_t py = 68;
uint8_t pxb = 20;     // le B, en pixels
uint8_t pyb = 20;

int main() {
    while (true) {
        image();
        glisse_croix(0, px, py, ALPHABET[0], 1);
        glisse_croix(1, pxb, pyb, ALPHABET[1], 1);
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B suivent la croix ensemble.



l'écran, au bout de vingt images

0.74. Glisser plus vite : glisse_croix à 3

Le 0.73 avec un seul nombre changé : la vitesse passe de 1 à 3 pixels par image. La lettre glisse trois fois plus vite.

C'est le 0.73, avec un seul changement : la vitesse, 5e réglage de `glisse_croix`, passe de 1 à 3.

Ce qui est nouveau ici : **plusieurs pixels par image**. 3 : trois pixels à chaque image, **180 pixels par seconde**. La lettre traverse l'écran (152 pixels) en moins d'une seconde.

Toujours fluide : le lutin se déplace à **chaque** image, 60 fois par seconde ; il fait seulement des pas un peu plus grands. Au-delà de 4 ou 5, l'œil commence à voir des sauts.

Au bord, un pas qui dépasserait s'arrête pile au bord : la lettre ne sort jamais de l'écran.

Elle peut être une variable : contrairement à la vitesse de `deplace_croix`, celle-ci n'est pas une durée. On pourra la changer en plein jeu, par exemple aller plus vite tant qu'on tient B.

LE PROGRAMME — 1592 OCTETS

```
uint8_t px = 76;      // la colonne en PIXELS
uint8_t py = 68;      // la ligne en PIXELS

int main() {
    while (true) {
        image();

        // Le changement : la vitesse, 1 → 3.
        //
        // glisse_croix(0, px, py, ALPHABET[0], 3);
        //
        // |
        // +-- vitesse : pixels par image.
        //
        // Plus GRAND = plus VITE :
        // 1 → 60 pixels par seconde (le 0.73)
        // 3 → 180 pixels par seconde (ici)
        glisse_croix(0, px, py, ALPHABET[0], 3);
    }
}
```

CE QU'ON DOIT VOIR

Le même A qu'au 0.73, mais qui glisse trois fois plus vite, toujours sans à-coups.



l'écran, au bout de vingt images

0.74.1. Glisser plus vite : glisse_croix à 3 — de base, ailleurs

Le 0.74 avec le B, parti de (20, 20) pixels.

C'est le 0.74, avec le B, parti de (20, 20) pixels.

C'est la version de base : la deuxième place, seule. Le 0.74.2 met les deux ensemble.

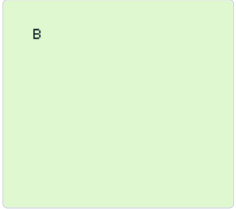
LE PROGRAMME — 1592 OCTETS

```
uint8_t px = 20;      // le B, en pixels
uint8_t py = 20;

int main() {
    while (true) {
        image();
        glisse_croix(0, px, py, ALPHABET[1], 3);
    }
}
```

CE QU'ON DOIT VOIR

Un B qui suit la croix,
depuis le haut à gauche.



B

l'écran, au bout de vingt images

0.74.2. Glisser plus vite : glisse_croix à 3 — doublé, deux positions

Deux lettres, la même croix : deux appels, dans la même image.

C'est le 0.74 et le 0.74.1 réunis : deux appels, un par lettre.

Deux lutins, deux numéros (0 et 1) : ils glissent ensemble.

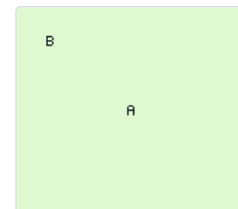
LE PROGRAMME — 1629 OCTETS

```
uint8_t px = 76;      // le A, en pixels
uint8_t py = 68;
uint8_t pxb = 20;     // le B, en pixels
uint8_t pyb = 20;

int main() {
    while (true) {
        image();
        glisse_croix(0, px, py, ALPHABET[0], 3);
        glisse_croix(1, pxb, pyb, ALPHABET[1], 3);
    }
}
```

CE QU'ON DOIT VOIR

Le A et le B suivent la croix ensemble.



l'écran, au bout de vingt images

0.75. Arrivé en (0, 0), le A devient B

Le A part de (10, 0) et suit la croix ; sa position s'affiche en direct. Quand il arrive en (0, 0), il se transforme en B : `if (x == 0 && y == 0)`.

Le A part de (10, 0), en haut de l'écran, et se déplace avec la croix, comme au 0.71. Sa position s'affiche **en direct** en bas de l'écran, comme au 0.68 : `x` à gauche, `y` à côté.

Ce qui est nouveau ici : une condition sur la position. `if (x == 0 && y == 0)` veut dire : « si `x` vaut 0 **ET** si `y` vaut 0 ». `&&` se lit « et » : il faut que **les deux** soient vrais en même temps. En (0, 5), `x == 0` est vrai mais `y == 0` est faux : rien ne se passe. En (0, 0), les deux sont vrais : la condition est vraie.

La lettre dans une variable : `lettre` vaut 0 au départ, et la croix pose `ALPHABET[lettre]`, donc `ALPHABET[0]`, le A. Quand la condition est vraie, `lettre = 1` ; à l'image suivante, `deplace_croix` pose `ALPHABET[1]`, le B.

Le B reste un B : rien ne remet `lettre` à 0. Même en quittant la case (0, 0), la lettre reste transformée.

Deux façons d'écrire le `if` : sur une ligne, `if (x == 0 && y == 0) lettre = 1` ; ou **avec des accolades** : `if (x == 0 && y == 0) { lettre = 1 ; }`. Les deux font exactement la même chose. Sans accolades, le `if` ne commande **qu'une seule** instruction, celle qui le suit. Avec des accolades, il commande **tout ce qui est entre elles** : on peut y mettre plusieurs lignes (changer la lettre ET jouer un son, par exemple). Le programme l'écrit avec des accolades, et montre l'autre forme en commentaire juste au-dessus.

Pour y arriver : la flèche **gauche**, dix pas. Les nombres en bas montrent `x` qui descend : 010, 009... jusqu'à 000. Et la lettre change.

LE PROGRAMME — 1822 OCTETS

```
uint8_t x = 10;      // la COLONNE : le A part de la colonne 10...
uint8_t y = 0;       // ...et de la ligne 0, en haut de l'écran
uint8_t lettre = 0;  // QUELLE lettre : 0 = A, 1 = B

int main() {
    while (true) {
        image();

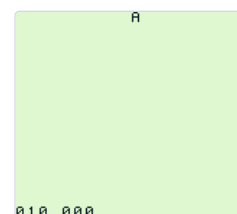
        // La croix déplace la lettre ; ALPHABET[lettre] : le A, puis le B.
        deplace_croix(x, y, ALPHABET[lettre], 250);

        // Le contrôleur de position : x et y, en direct, en bas de l'écran.
        nombre(0, 17, x);
        nombre(4, 17, y);

        // La condition, morceau par morceau :
        //
        // if (x == 0 && y == 0) lettre = 1;
        // |   |   |   |
        // |   |   |   +-- alors : la lettre devient ALPHABET[1], le B
        // |   |   +----- y vaut-il 0 ? (tout en haut)
        // |   +----- && : ET - les DEUX doivent être vrais
        // +----- x vaut-il 0 ? (tout à gauche)
        //
        // en (10, 0) : x == 0 faux          → rien
        // en (0, 5)  : x == 0 vrai, y == 0 faux → rien
        // en (0, 0)  : les deux vrais        → le A devient B
        // Sur une ligne : if (x == 0 && y == 0) lettre = 1;
        // Avec des accolades, la même chose :
        if (x == 0 && y == 0) {
            lettre = 1;
        }
    }
}
```

CE QU'ON DOIT VOIR

Un A en haut de l'écran ; avec la flèche gauche, il va jusqu'au coin (0, 0), et là il devient un B. En bas, sa position en direct.



l'écran, au bout de vingt images

0.75.1. Arrivé en (0, 0), le A devient B — de base, ailleurs

Le 0.75 avec un autre départ : le A part de (10, 8), et devient toujours B en (0, 0). Il faut aller à gauche, puis monter.

C'est le 0.75, avec un seul changement : le **départ**. Le A part de **(10, 8)**, au milieu de l'écran, au lieu de (10, 0).

La case de transformation ne change pas : **(0, 0)**, le coin en haut à gauche. La condition est la même : `if (x == 0 && y == 0)`.

Le chemin est plus long : la flèche **gauche** amène le A en (0, 8), au bord gauche. Là, `x == 0` est vrai, mais `y == 0` est faux (y vaut 8) : il reste un A. Puis la flèche **haut** le fait monter : 7, 6... et en (0, 0), les deux sont vrais. Il devient B.

C'est ce que montre `&&` : arriver dans la bonne colonne ne suffit pas, il faut aussi la bonne ligne. Regarde les nombres en bas : `000 008` au bord gauche, encore un A ; `000 000` au coin, un B.

C'est la version de base : une seule lettre, un autre départ. Le 0.75.2 met les deux ensemble.

LE PROGRAMME — 1823 OCTETS

```
uint8_t x = 10;      // le changement : le départ, (10, 8), au milieu
uint8_t y = 8;
uint8_t lettre = 0;  // 0 = A, 1 = B

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[lettre], 250);
        nombre(0, 17, x);          // la position, en direct
        nombre(4, 17, y);

        // La même case qu'au 0.75 : (0, 0), le coin en haut à gauche.
        //
        // en (0, 8) : x == 0 vrai, y == 0 faux → encore un A (il faut monter)
        // en (0, 0) : les deux vrais          → le A devient B
        // Sur une ligne : if (x == 0 && y == 0) lettre = 1;
        // Avec des accolades, la même chose :
        if (x == 0 && y == 0) {
            lettre = 1;
        }
    }
}
```

CE QU'ON DOIT VOIR

Un A au milieu de l'écran ;
la flèche gauche l'amène
au bord (0, 8), encore un
A ; la flèche haut le fait
monter jusqu'au coin (0,
0), où il devient B.



l'écran, au bout de vingt images

0.75.2. Arrivé en (0, 0), le A devient B — doublé, deux positions

Deux A, deux départs, la même case d'arrivée : chacun devient B en arrivant en (0, 0).

C'est le 0.75 et le 0.75.1 réunis : deux A, l'un parti de (10, 0), l'autre de (10, 8). La même croix les déplace ensemble.

Chacun a ses variables : `x`, `y`, `lettre` pour le premier ; `xb`, `yb`, `lettreB` pour le second. Et chacun a **sa condition**, sur **sa** position, mais vers la **même** case : (0, 0).

Ils n'y arrivent pas en même temps. Avec la flèche gauche, le premier arrive en (0, 0) et devient B ; le second, en (0, 8), reste un A. Avec la flèche haut, le premier ne peut plus monter (il est déjà en haut) ; le second monte, arrive en (0, 0), et devient B à son tour.

À la fin, les deux B sont sur la même case, (0, 0) : on n'en voit qu'un. Chaque lettre ne connaît que sa propre position ; rien ne les empêche de se retrouver au même endroit.

LE PROGRAMME — 1930 OCTETS

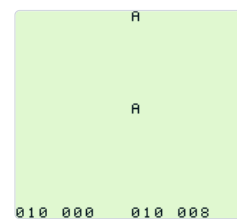
```
uint8_t x = 10;      // le premier A : départ (10, 0)
uint8_t y = 0;
uint8_t lettre = 0;
uint8_t xb = 10;     // le second A : départ (10, 8)
uint8_t yb = 8;
uint8_t lettreB = 0;

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[lettre], 250); // le premier
        deplace_croix(xb, yb, ALPHABET[lettreB], 250); // le second, dans la même image
        nombre(0, 17, x); // la position du premier
        nombre(4, 17, y);
        nombre(10, 17, xb); // celle du second
        nombre(14, 17, yb);

        // La même case pour les deux : (0, 0). Chacun regarde SA position.
        // 1. Le premier devient B quand il arrive en (0, 0).
        // Sur une ligne : if (x == 0 && y == 0) lettre = 1;
        // Avec des accolades, la même chose :
        if (x == 0 && y == 0) {
            lettre = 1;
        }
        // 2. Le second aussi, quand il y arrive à son tour.
        // Sur une ligne : if (xb == 0 && yb == 0) lettreB = 1;
        // Avec des accolades, la même chose :
        if (xb == 0 && yb == 0) {
            lettreB = 1;
        }
    }
}
```

CE QU'ON DOIT VOIR

Deux A ; à gauche, le premier arrive en (0, 0) et devient B ; en montant, le second y arrive aussi et devient B.



l'écran, au bout de vingt images

0.76. Un déclencheur en (0, 0) : actif

Le A suit la croix depuis (10, 0) ; en arrivant en (0, 0), la variable actif passe de 0 à 1. C'est un déclencheur : il servira aux leçons 0.76.1 à 0.76.6.

Le A part de (10, 0) et suit la croix, avec sa position en direct, comme au 0.75. Mais cette fois, il ne se transforme pas : en arrivant en (0, 0), il **déclenche** quelque chose.

Ce qui est nouveau ici : un drapeau, `actif`. C'est une variable qui dit **oui ou non** : 0 = « pas encore », 1 = « le A est passé par (0, 0) ». On la voit à l'écran, en bas, à la colonne 10 : `000`, puis `001`.

Une condition à trois morceaux : `if (x == 0 && y == 0 && actif == 0)`. Le A est en (0, 0) **et** le drapeau vaut encore 0. La troisième partie fait que le bloc ne s'exécute **qu'une seule fois** : dès qu'`actif` vaut 1, elle devient fausse, même si le A reste sur la case.

Pourquoi un drapeau : les leçons suivantes font apparaître des lettres et les mettent en mouvement **à partir de ce moment-là**. Le drapeau retient que le moment est passé.

LE PROGRAMME — 1858 OCTETS

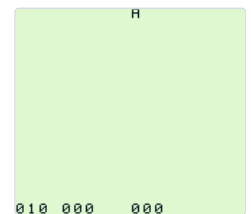
```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);
        nombre(10, 17, actif);                 // le drapeau : 000, puis 001

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1;                          // une seule fois : ensuite actif vaut 1
        }
    }
}
```

CE QU'ON DOIT VOIR

Le A suit la croix ; en bas, sa position et le drapeau. En (0, 0), le drapeau passe à 001.



l'écran, au bout de vingt images

0.76.1. Un déclencheur en (0, 0) : le B apparaît au milieu

Le 0.76, plus une ligne dans le bloc du déclencheur : `poser(10, 8, ALPHABET[1]);`. En (0, 0), un B apparaît au milieu de l'écran.

C'est le 0.76, plus une ligne dans le bloc du déclencheur.

Ce qui est nouveau ici : une action déclenchée. `poser(10, 8, ALPHABET[1]);` pose un B au milieu de l'écran. Comme le bloc ne s'exécute **qu'une fois** (grâce à `actif == 0`), le B n'est posé qu'une fois, au moment où le A arrive en (0, 0).

Avant, le milieu est vide ; après, le B y reste. Le A, lui, continue de suivre la croix.

LE PROGRAMME — 1875 OCTETS

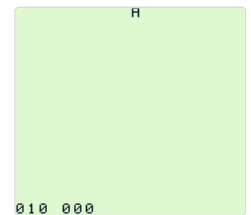
```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1;                          // une seule fois : ensuite actif vaut 1
            poser(10, 8, ALPHABET[1]);          // le B apparaît au milieu
        }
    }
}
```

CE QU'ON DOIT VOIR

Le A suit la croix ; quand il arrive en (0, 0), un B apparaît au milieu de l'écran.



l'écran, au bout de vingt images

0.76.2. Un déclencheur en (0, 0) : le B tourne en carré

Le 0.76.1, plus un bloc : une fois apparu, le B fait un carré sans fin à partir du milieu, un pas tous les quarts de seconde, sans bloquer le A.

C'est le 0.76.1, plus un bloc : les mouvements, qui ne commencent que quand `actif` vaut 1.

Pourquoi pas `carre` : `carre` bloque. Pendant son tour, le A ne pourrait plus bouger. On fait donc le carré **pas à pas**, avec `un_pas`, qui n'attend pas (le 0.30).

Le tour, compté par `bpas` : un carré de 4 cases de côté, c'est 16 pas. `bpas` va de 0 à 15 : de 0 à 3, un pas **à droite** ; de 4 à 7, **en bas** ; de 8 à 11, **à gauche** ; de 12 à 15, **en haut**. Après le pas 15, `bpas` revient à 0 : le tour recommence, sans fin.

`else if` enchaîne les cas : on ne teste le suivant que si le précédent était faux. Un seul des quatre `un_pas` est fait à chaque pas.

Le rythme : `bimages` compte les images ; à 15 (un quart de seconde), un pas, et on recompte. Le B part du milieu (10, 8) et tourne : (14, 8), (14, 12), (10, 12), puis retour au milieu.

LE PROGRAMME — 2387 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)
uint8_t bx = 10;     // le B : sa colonne...
uint8_t by = 8;      // ...et sa ligne. Il apparaît au milieu, (10, 8).
uint8_t bpas = 0;    // où il en est dans son tour : du pas 0 au pas 15
uint8_t bimages = 0; // le chronomètre du B, en images

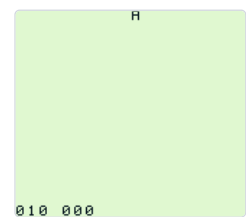
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1; // une seule fois : ensuite actif vaut 1
            poser(10, 8, ALPHABET[1]); // le B apparaît au milieu
        }

        // Les mouvements, une fois le déclencheur passé.
        if (actif == 1) {
            bimages++; // le chronomètre avance
            if (bimages == 15) { // toutes les 15 images : un pas
                bimages = 0;
                if (bpas < 4) { un_pas(bx, by, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas < 8) { un_pas(bx, by, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas < 12) { un_pas(bx, by, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx, by, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas++;
                if (bpas == 16) { bpas = 0; } // le tour est fini : on recommence
            }
        }
    }
}
```

CE QU'ON DOIT VOIR

En (0, 0), le B apparaît au milieu, puis tourne en carré sans fin, pendant que le A suit toujours la croix.



l'écran, au bout de vingt images

0.76.3. Un déclencheur en (0, 0) : le carré plus rapide

Le 0.76.2 avec un seul nombre changé : le B fait un pas toutes les 5 images au lieu de 15. Son carré va trois fois plus vite.

C'est le 0.76.2, avec un seul changement : `if (bimages == 5)` au lieu de 15.

Ce qui est nouveau ici : la vitesse du chronomètre. Un pas toutes les **5 images**, c'est 12 pas par seconde, **trois fois plus vite**. Le tour de 16 pas dure alors moins d'une seconde et demie.

Plus le nombre est petit, plus ça va vite, comme pour toutes les vitesses vues jusqu'ici : c'est le temps d'attente entre deux pas.

LE PROGRAMME — 2387 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)
uint8_t bx = 10;     // le B : sa colonne...
uint8_t by = 8;      // ...et sa ligne. Il apparaît au milieu, (10, 8).
uint8_t bpas = 0;    // où il en est dans son tour : du pas 0 au pas 15
uint8_t bimages = 0; // le chronomètre du B, en images

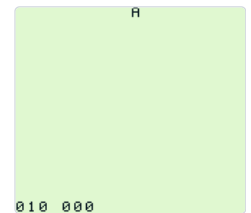
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1; // une seule fois : ensuite actif vaut 1
            poser(10, 8, ALPHABET[1]); // le B apparaît au milieu
        }

        // Les mouvements, une fois le déclencheur passé.
        if (actif == 1) {
            bimages++; // le chronomètre avance
            if (bimages == 5) { // toutes les 5 images : un pas
                bimages = 0;
                if (bpas < 4) { un_pas(bx, by, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas < 8) { un_pas(bx, by, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas < 12) { un_pas(bx, by, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx, by, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas++;
                if (bpas == 16) { bpas = 0; } // le tour est fini : on recommence
            }
        }
    }
}
```

CE QU'ON DOIT VOIR

Le même carré qu'au 0.76.2, mais trois fois plus rapide.



010 000

l'écran, au bout de vingt images

0.76.4. Un déclencheur en (0, 0) : deux B

Le 0.76.3, plus un second B : il part du coin opposé du carré, et les deux B se poursuivent.

C'est le 0.76.3, plus un second B.

Ce qui est nouveau ici : deux lettres sur le même carré. Le second B a ses variables (`bx2`, `by2`, `bpas2`). Il apparaît au coin opposé, (14, 12), avec `bpas2 = 8` : il est déjà à mi-tour, donc il commence par aller à gauche.

Le même chronomètre fait avancer les deux : à chaque tic, un pas pour chacun. Toujours à 8 pas l'un de l'autre, ils se poursuivent sans jamais se rattraper.

LE PROGRAMME — 2633 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)
uint8_t bx = 10;     // le B : sa colonne...
uint8_t by = 8;      // ...et sa ligne. Il apparaît au milieu, (10, 8).
uint8_t bpas = 0;    // où il en est dans son tour : du pas 0 au pas 15
uint8_t bimages = 0; // le chronomètre du B, en images
uint8_t bx2 = 14;    // le second B : il part du coin opposé, (14, 12)...
uint8_t by2 = 12;
uint8_t bpas2 = 8;   // ...donc déjà à mi-tour : il commence par aller à gauche

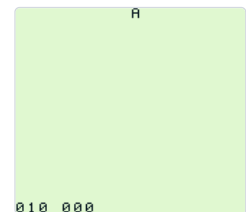
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1; // une seule fois : ensuite actif vaut 1
            poser(10, 8, ALPHABET[1]); // le B apparaît au milieu
            poser(14, 12, ALPHABET[1]); // le second B, au coin opposé
        }

        // Les mouvements, une fois le déclencheur passé.
        if (actif == 1) {
            bimages++; // le chronomètre avance
            if (bimages == 5) { // toutes les 5 images : un pas
                bimages = 0;
                if (bpas < 4) { un_pas(bx, by, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas < 8) { un_pas(bx, by, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas < 12) { un_pas(bx, by, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx, by, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas++;
                if (bpas == 16) { bpas = 0; } // le tour est fini : on recommence
                if (bpas2 < 4) { un_pas(bx2, by2, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas2 < 8) { un_pas(bx2, by2, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas2 < 12) { un_pas(bx2, by2, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx2, by2, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas2++;
                if (bpas2 == 16) { bpas2 = 0; } // le tour est fini : on recommence
            }
        }
    }
}
```

CE QU'ON DOIT VOIR

En (0, 0), deux B apparaissent, au milieu et au coin opposé, et tournent ensemble en se poursuivant.



l'écran, au bout de vingt images

0.76.5. Un déclencheur en (0, 0) : le C file à gauche

Le 0.76.4, plus un C : il apparaît au bord droit, sur la ligne 3, et file vers la gauche ; au bord, il repart de droite.

C'est le 0.76.4, plus un C (ALPHABET[2]).

Ce qui est nouveau ici : un mouvement vers la GAUCHE, qui recommence. À chaque tic du chronomètre, le C efface sa case, recule d'une colonne (cx--), et se repose. Arrivé à la colonne 0, il **repart de la colonne 19** : `if (cx == 0) { cx = 19; } else { cx--; }`.

`if ... else` avec accolades : si `cx` vaut 0, on fait le premier bloc (repartir à droite) ; sinon, le second (reculer d'une case). Jamais les deux.

LE PROGRAMME — 2753 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)
uint8_t bx = 10;     // le B : sa colonne...
uint8_t by = 8;      // ...et sa ligne. Il apparaît au milieu, (10, 8).
uint8_t bpas = 0;     // où il en est dans son tour : du pas 0 au pas 15
uint8_t bimages = 0;  // le chronomètre du B, en images
uint8_t bx2 = 14;     // le second B : il part du coin opposé, (14, 12)...
uint8_t by2 = 12;
uint8_t bpas2 = 8;    // ...donc déjà à mi-tour : il commence par aller à gauche
uint8_t cx = 19;      // le C : il apparaît en (19, 3), au bord droit
uint8_t cy = 3;

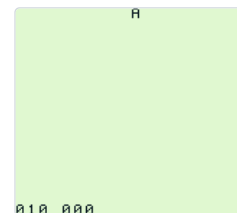
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1;                          // une seule fois : ensuite actif vaut 1
            poser(10, 8, ALPHABET[1]);           // le B apparaît au milieu
            poser(14, 12, ALPHABET[1]);          // le second B, au coin opposé
            poser(19, 3, ALPHABET[2]);           // le C apparaît au bord droit
        }

        // Les mouvements, une fois le déclencheur passé.
        if (actif == 1) {
            bimages++;                          // le chronomètre avance
            if (bimages == 5) {                  // toutes les 5 images : un pas
                bimages = 0;
                if (bpas < 4) { un_pas(bx, by, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas < 8) { un_pas(bx, by, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas < 12) { un_pas(bx, by, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx, by, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas++;
                if (bpas == 16) { bpas = 0; } // le tour est fini : on recommence
                if (bpas2 < 4) { un_pas(bx2, by2, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas2 < 8) { un_pas(bx2, by2, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas2 < 12) { un_pas(bx2, by2, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx2, by2, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas2++;
                if (bpas2 == 16) { bpas2 = 0; } // le tour est fini : on recommence
                effacer(cx, cy, 1); // le C : un pas à GAUCHE...
                if (cx == 0) { cx = 19; } else { cx--; } // ...et au bord gauche, il repart de droite
                poser(cx, cy, ALPHABET[2]);
            }
        }
    }
}
```

CE QU'ON DOIT VOIR

En (0, 0), les deux B tournent, et un C file vers la gauche en haut de l'écran, sans fin.



l'écran, au bout de vingt images

0.76.6. Un déclencheur en (0, 0) : le D file à droite

Le 0.76.5, plus un D : il apparaît au bord gauche, sur la ligne 14, et file vers la droite ; au bord, il repart de gauche.

C'est le 0.76.5, plus un D (ALPHABET[3]).

Ce qui est nouveau ici : le mouvement inverse du C. Le D avance d'une colonne (dx++) à chaque tic, et arrivé à la colonne 19, il **repart de la colonne 0** : if (dx == 19) { dx = 0; } else { dx++; }.

Tout part du même déclencheur : en (0, 0), le A fait apparaître quatre lettres, et chacune a son mouvement. Le A, lui, suit toujours la croix.

LE PROGRAMME — 2871 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)
uint8_t bx = 10;     // le B : sa colonne...
uint8_t by = 8;      // ...et sa ligne. Il apparaît au milieu, (10, 8).
uint8_t bpa = 0;     // où il en est dans son tour : du pas 0 au pas 15
uint8_t bima = 0;    // le chronomètre du B, en images
uint8_t bx2 = 14;    // le second B : il part du coin opposé, (14, 12)...
uint8_t by2 = 12;
uint8_t bpa2 = 8;    // ...donc déjà à mi-tour : il commence par aller à gauche
uint8_t cx = 19;     // le C : il apparaît en (19, 3), au bord droit
uint8_t cy = 3;
uint8_t dx = 0;      // le D : il apparaît en (0, 14), au bord gauche
uint8_t dy = 14;

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1;                          // une seule fois : ensuite actif vaut 1
            poser(10, 8, ALPHABET[1]);           // le B apparaît au milieu
            poser(14, 12, ALPHABET[1]);         // le second B, au coin opposé
            poser(19, 3, ALPHABET[2]);          // le C apparaît au bord droit
            poser(0, 14, ALPHABET[3]);          // le D apparaît au bord gauche
        }

        // Les mouvements, une fois le déclencheur passé.
        if (actif == 1) {
            bima++;                             // le chronomètre avance
            if (bima == 5) {                    // toutes les 5 images : un pas
                bima = 0;
                if (bpa < 4) { un_pas(bx, by, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpa < 8) { un_pas(bx, by, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpa < 12) { un_pas(bx, by, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx, by, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpa++;
                if (bpa == 16) { bpa = 0; } // le tour est fini : on recommence
                if (bpa2 < 4) { un_pas(bx2, by2, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpa2 < 8) { un_pas(bx2, by2, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpa2 < 12) { un_pas(bx2, by2, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx2, by2, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpa2++;
                if (bpa2 == 16) { bpa2 = 0; } // le tour est fini : on recommence
                effacer(cx, cy, 1);           // le C : un pas à GAUCHE...
                if (cx == 0) { cx = 19; } else { cx--; } // ...et au bord gauche, il repart de droite
                poser(cx, cy, ALPHABET[2]);
                effacer(dx, dy, 1);           // le D : un pas à DROITE...
                if (dx == 19) { dx = 0; } else { dx++; } // ...et au bord droit, il repart de gauche
                poser(dx, dy, ALPHABET[3]);
            }
        }
    }
}
```

CE QU'ON DOIT VOIR

En (0, 0), deux B tourment, un C file à gauche en haut, un D file à droite en bas.

A

010 000

l'écran, au bout de vingt images

0.76.7. Un déclencheur en (0, 0) : actif — en simple

Le 0.76 en plus simple : `if (x == 0 && y == 0) { actif = 1; }`, sans le troisième morceau.

C'est le 0.76, écrit plus simplement. La condition perd son troisième morceau : `if (x == 0 && y == 0) { actif = 1; }`.

Pourquoi on peut l'enlever : au 0.76, `&& actif == 0` servait à ne faire le bloc **qu'une fois**. Mais ici, le bloc ne fait que mettre `actif` à 1 : le refaire à chaque image ne change rien, puisqu'il vaut déjà 1. Le troisième morceau n'est utile que si le bloc fait quelque chose qu'on ne veut **pas** répéter.

La règle pour simplifier : une condition qui ne change rien au résultat peut disparaître. Moins de code, moins d'erreurs.

LE PROGRAMME — 1830 OCTETS

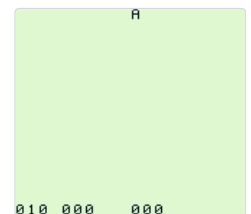
```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;    // 0 : pas encore ; 1 : le A est passé par (0, 0)

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }
        nombre(10, 17, actif);                  // le drapeau : 000, puis 001
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.76 : en (0, 0), le drapeau passe à 001.



l'écran, au bout de vingt images

0.76.8. Un déclencheur en (0, 0) : le B apparaît au milieu — en simple

Le 0.76.1 en plus simple : tant qu'actif vaut 1, on pose le B au milieu, à chaque image.

C'est le 0.76.1, écrit plus simplement. Au lieu de poser le B **une fois**, au moment exact du déclencheur, on le pose à **chaque image** tant qu'`actif` vaut 1 : `if (actif == 1) { poser(10, 8, ALPHABET[1]); }`.

Pourquoi ça marche : poser la même lettre à la même place ne se voit pas, et ne clignote pas (rien ne l'efface). Le résultat à l'écran est le même, et il n'y a plus besoin de penser au « une seule fois ».

LE PROGRAMME — 1866 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)

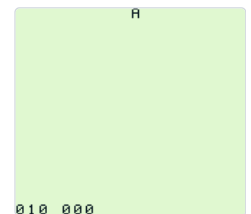
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }

        // Tout ce qui se passe après le déclencheur, une ligne par lettre :
        if (actif == 1) {
            poser(10, 8, ALPHABET[1]);          // le B au milieu, à chaque image
        }
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.76.1 : en (0, 0), un B apparaît au milieu.



l'écran, au bout de vingt images

0.76.9. Un déclencheur en (0, 0) : le B tourne en carré — en simple

Le 0.76.2 en une ligne : tourne_carre(0, 10, 8, ALPHABET[1], 4, 250). Plus de bpas, bx, bimages : la console s'en occupe.

C'est le 0.76.2, en une ligne. Tout le bloc des mouvements (le chronomètre bimages, le compteur de tour bpas, les quatre un_pas, bx, by) est remplacé par **une fonction de la console**.

Ce qui est nouveau ici : tourne_carre(numero, x, y, tuile, cote, vitesse). À appeler à **chaque image**. Au premier appel, la lettre apparaît en (x, y), un coin du carré ; ensuite, toutes les vitesse millisecondes, elle fait **un pas** : cote pas à droite, puis en bas, puis à gauche, puis en haut, et elle recommence, sans fin.

- **numero** (0 à 3) : la console retient, pour chaque numéro, **où en est** la lettre (sa place, son pas dans le tour, son attente). C'est ce qui remplace bx, by, bpas et bimages.
- **cote** : 4, un côté de 4 pas, comme au 0.76.2. • **vitesse** : 250 ms par pas, écrite en clair.

Elle ne bloque pas : comme un_pas, elle fait au plus un pas et rend la main. Le A suit toujours la croix.

Elle remplace aussi le poser du 0.76.1 : c'est son premier appel qui fait apparaître le B.

LE PROGRAMME — 2536 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)

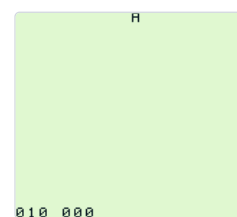
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }

        // Tout ce qui se passe après le déclencheur, une ligne par lettre :
        if (actif == 1) {
            tourne_carre(0, 10, 8, ALPHABET[1], 4, 250); // le B tourne en carré
        }
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.76.2 : en (0, 0), le B apparaît au milieu et tourne en carré.



l'écran, au bout de vingt images

0.76.10. Un déclencheur en (0, 0) : le carré plus rapide — en simple

Le 0.76.3 en simple : un seul nombre change, la vitesse de tourne_carre, 250 → 80.

C'est le 0.76.3, en simple : la vitesse, 6e réglage de tourne_carre, passe de 250 à 80 millisecondes.

80 ms, c'est 5 images : exactement la vitesse du 0.76.3 (bimages == 5). Mais ici, on la dit en millisecondes, comme partout ailleurs.

LE PROGRAMME — 2536 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)

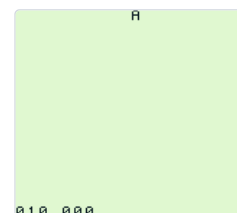
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }

        // Tout ce qui se passe après le déclencheur, une ligne par lettre :
        if (actif == 1) {
            tourne_carre(0, 10, 8, ALPHABET[1], 4, 80); // le B tourne en carré
        }
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.76.3 : le carré tourne trois fois plus vite.



l'écran, au bout de vingt images

0.76.11. Un déclencheur en (0, 0) : deux B — en simple

Le 0.76.4 en simple : une ligne de plus, tourne_carre(1, 14, 12, ALPHABET[1], -4, 80). Un côté négatif part du coin opposé.

C'est le 0.76.4, en simple : une ligne de plus, pour le second B.

Ce qui est nouveau ici : le numéro 1, et un côté négatif. Le **numéro 1** : la console retient cette lettre à part de celle du numéro 0. Le **côté -4** : le même carré de 4, mais la lettre part vers la **gauche** puis vers le haut, comme le second B du 0.76.4, qui partait du coin opposé (14, 12).

LE PROGRAMME — 2563 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)

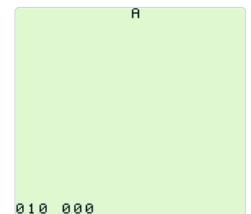
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }

        // Tout ce qui se passe après le déclencheur, une ligne par lettre :
        if (actif == 1) {
            tourne_carre(0, 10, 8, ALPHABET[1], 4, 80); // le B tourne en carré
            tourne_carre(1, 14, 12, ALPHABET[1], -4, 80); // le second B, depuis le coin opposé
        }
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.76.4 : deux B tournent sur le même carré, en se poursuivant.



l'écran, au bout de vingt images

0.76.12. Un déclencheur en (0, 0) : le C file à gauche — en simple

Le 0.76.5 en simple : une ligne de plus, `defile(0, 19, 3, ALPHABET[2], -1, 80)`. Le C file à gauche, sans fin.

C'est le 0.76.5, en simple : une ligne de plus, pour le C.

Ce qui est nouveau ici : `defile(numero, x, y, tuile, sens, vitesse)`. Au premier appel, la lettre apparaît en `(x, y)` ; ensuite, toutes les `vitesse` ms, elle fait un pas sur sa ligne : `sens` -1 vers la gauche, 1 vers la droite. Au bord, elle repart de l'autre côté, comme au 0.76.5.

Son numéro, 0, est à elle : les numéros de `defile` et ceux de `tourne_carre` sont séparés. Le C est le numéro 0 de `defile`, le B le numéro 0 de `tourne_carre`.

LE PROGRAMME — 2987 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)

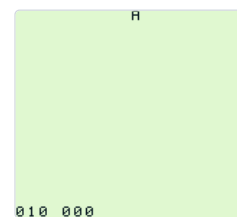
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }

        // Tout ce qui se passe après le déclencheur, une ligne par lettre :
        if (actif == 1) {
            tourne_carre(0, 10, 8, ALPHABET[1], 4, 80); // le B tourne en carré
            tourne_carre(1, 14, 12, ALPHABET[1], -4, 80); // le second B, depuis le coin opposé
            defile(0, 19, 3, ALPHABET[2], -1, 80);       // le C file à gauche
        }
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.76.5 : deux B tournent, un C file à gauche.



l'écran, au bout de vingt images

0.76.13. Un déclencheur en (0, 0) : le D file à droite — en simple

Le 0.76.6 en simple : une ligne de plus, `defile(1, 0, 14, ALPHABET[3], 1, 80)`. Tout le 0.76.6, en quelques lignes.

C'est le 0.76.6, en simple : une ligne de plus, pour le D : `defile` avec le **numéro 1** et le **sens 1**, vers la droite.

Compare avec le 0.76.6 : là-bas, plus de cinquante lignes, onze variables, trois chronomètres cachés dans les `if`. Ici, quatre lignes dans le bloc `if (actif == 1)`, une par lettre. C'est le même jeu, à l'écran.

Pourquoi avoir appris la version longue : pour savoir ce que ces fonctions font **pour nous**. `tourne_carre` et `defile` contiennent exactement ce qu'on a écrit à la main : un chronomètre, un compteur, des pas.

LE PROGRAMME — 3013 OCTETS

```
uint8_t x = 10;      // le A part de (10, 0)
uint8_t y = 0;
uint8_t actif = 0;   // 0 : pas encore ; 1 : le A est passé par (0, 0)

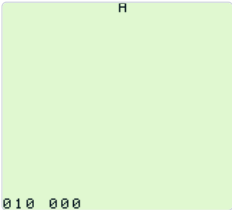
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }

        // Tout ce qui se passe après le déclencheur, une ligne par lettre :
        if (actif == 1) {
            tourne_carre(0, 10, 8, ALPHABET[1], 4, 80); // le B tourne en carré
            tourne_carre(1, 14, 12, ALPHABET[1], -4, 80); // le second B, depuis le coin opposé
            defile(0, 19, 3, ALPHABET[2], -1, 80); // le C file à gauche
            defile(1, 0, 14, ALPHABET[3], 1, 80); // le D file à droite
        }
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.76.6 : deux B tournent, un C file à gauche, un D file à droite.



l'écran, au bout de vingt images

0.77. Plusieurs rythmes sans compteur : chaque

if (chaque(250)) { ... } : le bloc se fait toutes les 250 ms, sans rien arrêter et sans chronomètre à écrire. Le 0.16 en quatre lignes.

C'est le 0.16 (la lente et la rapide), **sans les chronomètres**. Au 0.16, chaque lettre avait sa variable qui comptait les images, et son if pour la remettre à zéro. Ici, une fonction de la console fait tout ça.

Ce qui est nouveau ici : chaque(ms). Elle répond 1 (oui) quand le temps est passé depuis la dernière fois, 0 (non) le reste du temps. if (chaque(1000)) { ... } : le bloc se fait une fois par seconde. if (chaque(250)) { ... } : quatre fois par seconde.

Elle n'arrête rien, contrairement à attendre() : le reste de la boucle continue de tourner à chaque image. On peut donc avoir plusieurs rythmes dans le même programme.

Chaque chaque a son propre chronomètre : la console les distingue toute seule, par leur place dans le programme. Les deux lignes ci-dessous ne se gênent pas. On peut en écrire jusqu'à huit.

(lente + 1) % sizeof(ALPHABET) passe à la lettre suivante, et revient à 0 après la 26e (le Z) : le % du 0.9.

LE PROGRAMME — 1584 OCTETS

```
uint8_t lente = 0;    // la lettre de la lente : 0 = A ... 25 = Z
uint8_t rapide = 0;   // la lettre de la rapide

int main() {
    while (true) {
        image();

        // La fonction, morceau par morceau :
        //
        //   if (chaque(1000)) { ... }
        //   |       |
        //   |       +-+ toutes les 1000 millisecondes (1 seconde)
        //   +----- répond 1 quand c'est l'heure, 0 sinon : le bloc
        //               ne se fait qu'aux bons moments, rien n'est arrêté
        if (chaque(1000)) {
            lente = (lente + 1) % sizeof(ALPHABET);    // la lente : 1 fois par seconde
        }
        if (chaque(250)) {
            rapide = (rapide + 1) % sizeof(ALPHABET);  // la rapide : 4 fois par seconde
        }

        poser(0, 0, ALPHABET[lente]);    // ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]);    // ligne 2 : la rapide
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.16 : en haut, une lettre change chaque seconde ; en dessous, une autre quatre fois plus vite.

A
B

l'écran, au bout de vingt images

0.78. Le rebond : une vitesse qui change de signe

Une lettre qui va et vient d'un bord à l'autre, toute seule : $x = x + vx$, et au bord, vx change de signe.

Une balle qui rebondit : la lettre O avance vers la droite ; au bord droit, elle repart vers la gauche ; au bord gauche, vers la droite. Sans fin, et sans compter les pas.

Ce qui est nouveau ici : une VITESSE, vx . Elle dit de combien bouge la lettre à chaque pas : 1 , une case à droite ; -1 , une case à gauche. À chaque pas, $x = x + vx$;

Le rebond : au bord droit, $vx = -1$; au bord gauche, $vx = 1$;. On ne change **pas** la position, seulement la **direction** : le pas suivant se fait dans l'autre sens.

-1 dans un octet : il est rangé 255. $x + 255$ « tourne » et revient à $x - 1$: c'est pour cela que $x = x + vx$ recule quand vx vaut -1.

Le rythme, avec $\text{chaque}(50)$ (le 0.77) : un pas toutes les 50 ms, 20 cases par seconde.

C'est la base des jeux de balle : Pong, casse-briques. Le 0.78.1 fait rebondir sur les deux axes.

LE PROGRAMME — 1595 OCTETS

```
uint8_t x = 0;           // la colonne de la balle
uint8_t vx = 1;          // sa VITESSE : 1 = vers la droite, -1 = vers la gauche

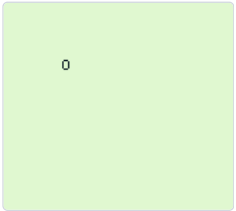
int main() {
    while (true) {
        image();
        if (chaque(50)) {           // 20 fois par seconde
            effacer(x, 5, 1);        // 1. efface l'ancienne place
            x = x + vx;               // 2. avance de vx : +1 ou -1

            // 3. Le rebond : au bord, la vitesse change de signe.
            //
            //   0  1  2 ... 18 19
            //   0 + + + + + 0      au bord droit (19) : vx = -1, elle repart à gauche
            //   0 + + + + + 0      au bord gauche (0) : vx = 1, elle repart à droite
            if (x == 19) {
                vx = -1;
            }
            if (x == 0) {
                vx = 1;
            }

            poser(x, 5, ALPHABET[14]); // 4. la balle : ALPHABET[14], le O
        }
    }
}
```

CE QU'ON DOIT VOIR

Un O qui file d'un bord à l'autre sur la ligne 5, et rebondit sans fin.



l'écran, au bout de vingt images

0.78.1. Le rebond : une vitesse qui change de signe — en diagonale

Le 0.78 sur les deux axes : vx ET vy. La balle rebondit sur les quatre bords.

C'est le 0.78, sur les deux axes : une vitesse `vy` pour la ligne, en plus de `vx` pour la colonne.

Ce qui est nouveau ici : deux vitesses. À chaque pas, `x = x + vx;` et `y = y + vy;` : la balle va **en diagonale**. Chaque axe rebondit à **part** : aux bords gauche et droit, `vx` change de signe ; en haut et en bas, `vy`.

La balle part de (0, 0) et ne s'arrête jamais : elle rebondit dans tout l'écran (lignes 0 à 16, pour laisser la ligne 17 libre).

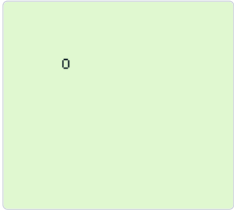
LE PROGRAMME — 1657 OCTETS

```
uint8_t x = 0;      // la colonne de la balle
uint8_t y = 0;      // sa ligne
uint8_t vx = 1;     // vitesse en colonne : 1 à droite, -1 à gauche
uint8_t vy = 1;     // vitesse en ligne : 1 en bas, -1 en haut

int main() {
    while (true) {
        image();
        if (chaque(50)) {
            effacer(x, y, 1);
            x = x + vx;           // un pas en diagonale :
            y = y + vy;           // une colonne ET une ligne
            if (x == 19) { vx = -1; } // les bords gauche et droit : vx
            if (x == 0) { vx = 1; }
            if (y == 16) { vy = -1; } // les bords haut et bas : vy
            if (y == 0) { vy = 1; }
            poser(x, y, ALPHABET[14]);
        }
    }
}
```

CE QU'ON DOIT VOIR

Un O qui rebondit en diagonale sur les quatre bords de l'écran.



l'écran, au bout de vingt images

0.79. Suivre une autre lettre

Le A suit la croix ; le B le poursuit tout seul : à chaque pas, il avance vers le A, de ± 1 sur chaque axe.

Un poursuivant : tu déplaces le A avec la croix ; le B avance tout seul **vers le A**, un pas tous les **600 ms**. C'est le premier « ennemi ».

Ce qui est nouveau ici : choisir la direction en comparant. Pour la colonne : si le A est plus à droite (`x > bx`), un pas à droite (`sx = 1`) ; plus à gauche, un pas à gauche (`sx = -1`) ; même colonne, rien (`sx = 0`). Pareil pour la ligne avec `sy`.

`un_pas(bx, by, ALPHABET[1], sx, sy)` (le 0.30) fait ce pas. Avec `sx` et `sy` à la fois, le B va **en diagonale** vers le A.

Attrapé : quand le B arrive sur la case du A, le mot `PRIS` s'écrit en bas.

Pourquoi 600 ms : le A fait un pas tous les 250 ms, plus de **deux fois plus vite** que le B. On peut donc lui échapper. Mais le B avance aussi en diagonale : dans un coin, il finit par nous coincer. Plus rapide (300 ms), il serait presque impossible à fuir. Le 0.79.1 le fait accélérer peu à peu.

L'ordre compte : le B bouge **avant** que la croix ne repose le A. Ainsi le A est dessiné par-dessus, et reste visible même quand le B le touche.

LE PROGRAMME — 2446 OCTETS

```
uint8_t x = 10;      // le A (la croix)
uint8_t y = 8;
uint8_t bx = 0;      // le B (le poursuivant) : il part du coin (0, 0)
uint8_t by = 0;
uint8_t sx = 0;      // la direction du prochain pas du B, en colonne...
uint8_t sy = 0;      // ...et en ligne : 1, -1 ou 0

int main() {
    while (true) {
        image();

        if (chaque(600)) {                // un pas du B tous les 600 ms : plus lent que le A
            // La direction : vers le A, sur chaque axe.
            sx = 0;
            if (x > bx) { sx = 1; }        // le A est à droite : un pas à droite
            if (x < bx) { sx = -1; }       // à gauche : un pas à gauche
            sy = 0;
            if (y > by) { sy = 1; }        // plus bas : un pas en bas
            if (y < by) { sy = -1; }       // plus haut : un pas en haut
            if (sx != 0 || sy != 0) {      // pas déjà sur lui (|| : OU)
                un_pas(bx, by, ALPHABET[1], sx, sy);
            }
        }

        deplace_croix(x, y, ALPHABET[0], 250); // le A, dessiné après : par-dessus

        if (bx == x && by == y) {         // le B est sur la case du A
            texte(0, 17, "PRIS");
        }
    }
}
```

CE QU'ON DOIT VOIR

Un A au milieu, un B dans le coin qui s'approche tout seul ; déplace le A avec les flèches pour lui échapper. S'il t'attrape, PRIS.



l'écran, au bout de vingt images

0.79.1. Suivre une autre lettre — de plus en plus vite

Le 0.79, mais le B accélère : toutes les 3 secondes, il attend un peu moins entre deux pas. Facile au début, de plus en plus dur.

C'est le 0.79, avec un B qui accélère. Au début, il est lent (600 ms par pas) : on lui échappe facilement. Toutes les 3 secondes, il devient un peu plus rapide, jusqu'à 200 ms par pas : là, il est presque impossible à fuir.

Ce qui est nouveau ici : une vitesse qui **CHANGE pendant le jeu**. `chaque(600)` ne peut pas le faire : son nombre est écrit en clair, traduit avant le jeu, et ne bouge plus. On compte donc les images nous-mêmes : `compte` augmente à chaque image, et quand il atteint `lenteur`, le B fait un pas.

`lenteur` est le nombre d'images entre deux pas : 36 au départ (600 ms). Toutes les 3 secondes, `if (chaque(3000))`, on lui retire 6 : 30, 24, 18... jusqu'à 12 (200 ms), le plus rapide. Le `if (lenteur > 12)` empêche de descendre plus bas.

`compte >= lenteur` (plus grand **ou égal**) : si `lenteur` diminue juste quand `compte` était déjà plus grand, le pas se fait quand même, au lieu d'attendre que `compte` fasse le tour des 255.

En bas à droite, `lenteur` s'affiche : on la voit descendre, 036, 030, 024...

LE PROGRAMME — 2517 OCTETS

```
uint8_t x = 10;      // le A (la croix)
uint8_t y = 8;
uint8_t bx = 0;      // le B (le poursuivant)
uint8_t by = 0;
uint8_t sx = 0;      // la direction du prochain pas du B
uint8_t sy = 0;
uint8_t lenteur = 36; // les images entre deux pas du B : 36 = 600 ms au départ
uint8_t compte = 0;  // les images comptées depuis son dernier pas

int main() {
    while (true) {
        image();

        // Toutes les 3 secondes, le B accélère : 6 images de moins entre deux pas.
        if (chaque(3000)) {
            if (lenteur > 12) { // 12 images (200 ms) : le plus rapide
                lenteur = lenteur - 6;
            }
        }

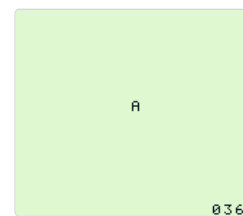
        // Le pas du B, quand compte atteint lenteur.
        compte++;
        if (compte >= lenteur) {
            compte = 0;
            sx = 0;
            if (x > bx) { sx = 1; }
            if (x < bx) { sx = -1; }
            sy = 0;
            if (y > by) { sy = 1; }
            if (y < by) { sy = -1; }
            if (sx != 0 || sy != 0) {
                un_pas(bx, by, ALPHABET[1], sx, sy);
            }
        }

        deplace_croix(x, y, ALPHABET[0], 250); // le A, par-dessus
        nombre(17, 17, lenteur);                // la lenteur du B : elle descend

        if (bx == x && by == y) {
            texte(0, 17, "PRIS");
        }
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.79, mais le B accélère toutes les 3 secondes ; en bas à droite, sa lenteur descend de 036 à 012.



l'écran, au bout de vingt images

0.80. x et y rangés ensemble : struct

`struct Position { uint8_t x, y; }` : la colonne et la ligne d'une lettre sous un seul nom, `joueur.x` et `joueur.y`.

Deux variables qui vont toujours ensemble — la colonne et la ligne d'une lettre — peuvent être rangées **sous un seul nom**.

Ce qui est nouveau ici : `struct`. `struct Position { uint8_t x, y; }` crée un **nouveau type**, `Position`, qui contient deux octets : `x` et `y`. Puis `Position joueur;` crée une variable de ce type.

Le point `.` atteint une partie : `joueur.x` est la colonne du joueur, `joueur.y` sa ligne. On s'en sert comme de n'importe quelle variable : `joueur.x = 10;`, `deplace_croix(joueur.x, joueur.y, ...)`.

Pourquoi c'est utile : avec plusieurs lettres, `Position joueur;` et `Position ennemi;` se lisent mieux que `x`, `y`, `xb`, `yb`. On ne mélange plus les variables d'une lettre avec celles d'une autre.

La console range toujours la position dans `joueur.x` et `joueur.y` : ce qu'elle sait faire avec une variable (le 0.23), elle le sait avec une partie de `struct`.

LE PROGRAMME — 1775 OCTETS

```
// Le nouveau type, morceau par morceau :
//
// struct Position { uint8_t x, y; };
// |         |         |
// |         |         +-- ses parties : deux octets, x et y
// |         +----- son nom : Position
// +----- « struct » : on crée un type à nous
struct Position { uint8_t x, y; };

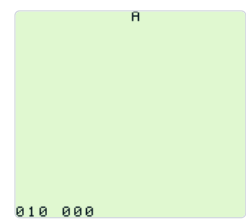
Position joueur;      // une variable de type Position : joueur.x et joueur.y

int main() {
    joueur.x = 10;      // le point atteint une partie : la colonne...
    joueur.y = 0;       // ...et la ligne. Départ : (10, 0).

    while (true) {
        image();
        deplace_croix(joueur.x, joueur.y, ALPHABET[0], 250); // la console range dans joueur.x, joueur.y
        nombre(0, 17, joueur.x);
        nombre(4, 17, joueur.y);
    }
}
```

CE QU'ON DOIT VOIR

Un A en haut de l'écran, qui suit la croix ; sa position, `joueur.x` et `joueur.y`, en bas.



l'écran, au bout de vingt images

0.80.1. x et y rangés ensemble : struct — la boucle dans un fichier

Le 0.80, avec l'intérieur du `while` dans un autre fichier, `boucle.h`, versé au milieu de la boucle par `#include`.

C'est le 0.80, avec un seul changement : les trois lignes qui étaient dans le `while` (`image`, `deplace_croix`, les `nombre`) sont maintenant dans un **second fichier**, `boucle.h` : l'onglet à côté de `principal.cpp`.

Ce qui est nouveau ici : un `#include` au MILIEU du programme. Au 0.67, il était tout en haut. Mais `#include "boucle.h"` veut seulement dire « **verse ici** le contenu de ce fichier ». On peut donc le mettre **entre les accolades du `while`** : avant la compilation, la ligne est remplacée par les trois lignes de `boucle.h`, exactement là.

Le programme compilé est le même qu'au 0.80, au mot près : il est seulement rangé en deux.

`#include` doit être seul sur sa ligne, sans rien avant ni après : c'est pour cela qu'il est collé au bord gauche, et que son commentaire est au-dessus.

C'est possible, mais ce n'est pas la meilleure façon : en lisant `boucle.h` seul, on ne sait pas qu'il est versé dans une boucle, ni d'où vient `joueur`. Le 0.80.2 montre la façon propre, avec une fonction.

LE PROGRAMME — 1775 OCTETS

`boucle.h`

```
// boucle.h : l'intérieur de la boucle du jeu.
// Ces lignes sont versées DANS le while de principal.cpp, par #include.
image();                               // attend l'image suivante
deplace_croix(joueur.x, joueur.y, ALPHABET[0], 250); // le A suit la croix
nombre(0, 17, joueur.x);                // sa position, en direct
nombre(4, 17, joueur.y);
```

`principal.cpp`

```
// Le nouveau type : la colonne et la ligne sous un seul nom (le 0.80).
struct Position { uint8_t x, y; };

Position joueur; // joueur.x et joueur.y

int main() {
    joueur.x = 10; // départ : (10, 0)
    joueur.y = 0;

    while (true) {
        // Le changement : l'intérieur de la boucle est dans l'onglet boucle.h.
        // #include le verse ICI, à chaque tour, comme si ses lignes étaient
        // écrites entre ces accolades. (#include veut être seul sur sa ligne.)
        #include "boucle.h"
    }
}
```

CE QU'ON DOIT VOIR

Exactement le 0.80 : un A qui suit la croix, sa position en bas. Mais la boucle est dans l'onglet `boucle.h`.

H

010 000

l'écran, au bout de vingt images

0.80.2. x et y rangés ensemble : struct — une fonction dans un fichier

Le 0.80.1, en mieux : l'intérieur de la boucle devient une fonction, `tour_de_jeu()`, rangée dans `jeu.h`. La boucle n'a plus qu'une ligne.

C'est le 0.80.1, en mieux : au lieu de verser des lignes en vrac au milieu de la boucle, on les range dans une **fonction**, `tour_de_jeu()`, dans le fichier `jeu.h`.

Ce qui est nouveau ici : `void`. Au 0.33, notre fonction **rendait** une valeur (`uint8_t`, puis `return colonne;`). Celle-ci ne rend rien : elle **fait** quelque chose. On l'écrit `void tour_de_jeu() { ... }` : `void` veut dire « rien », et les parenthèses vides veulent dire qu'elle ne reçoit rien.

Le `#include "jeu.h"` **revient tout en haut**, avant `main`, comme au 0.67 : il apporte la **définition** de la fonction. La boucle, elle, se contente de l'**appeler** : `tour_de_jeu();`, une ligne.

Pourquoi c'est mieux : en ouvrant `jeu.h`, on voit une fonction complète, avec son nom et ses accolades : on sait ce qu'elle est. Et `main` se lit comme une phrase : « tant que le jeu tourne, fais un tour de jeu ».

L'ordre compte : `joueur` est déclaré **avant** le `#include "jeu.h"`, parce que la fonction s'en sert. Une variable doit exister avant qu'on écrive quelque chose qui l'utilise.

LE PROGRAMME — 1779 OCTETS

`jeu.h`

```
// jeu.h : un tour de jeu, rangé dans une fonction.
//
// void tour_de_jeu() { ... }
// |   |           |
// |   |           +-- () : elle ne reçoit rien
// |   +----- son nom
// +----- void : elle ne rend rien, elle FAIT quelque chose
void tour_de_jeu() {
    image(); // attend l'image suivante
    deplace_croix(joueur.x, joueur.y, ALPHABET[0], 250); // le A suit la croix
    nombre(0, 17, joueur.x); // sa position, en direct
    nombre(4, 17, joueur.y);
}
```

`principal.cpp`

```
// Le nouveau type : la colonne et la ligne sous un seul nom (le 0.80).
struct Position { uint8_t x, y; };

Position joueur; // joueur.x et joueur.y

// La fonction tour_de_jeu : voir l'onglet jeu.h. Elle se sert de joueur,
// déclaré juste au-dessus : l'ordre compte.
#include "jeu.h"

int main() {
    joueur.x = 10; // départ : (10, 0)
    joueur.y = 0;

    while (true) {
        tour_de_jeu(); // le changement : UNE ligne, l'appel de la fonction
    }
}
```

CE QU'ON DOIT VOIR

Exactement le 0.80 : un A qui suit la croix. La boucle ne contient plus qu'un appel : `tour_de_jeu()`.

H

010 000

l'écran, au bout de vingt images

0.81. Ramasser une pièce : le P

Une pièce, la lettre P, en (15, 8). Quand le A arrive dessus, le score augmente de 1, et la pièce s'en va.

La pièce est une lettre, le P (ALPHABET[15], la 16e lettre). On la pose une fois, au début, en (15, 8).

Ce qui est nouveau ici : comparer DEUX positions. Au 0.75, on comparait la position du A à une case fixe, (0, 0). Ici, on la compare à celle de la pièce, rangée dans px et py : if (x == px && y == py). Le A est sur la pièce quand sa colonne et sa ligne sont celles de la pièce.

Ramassée : le score augmente (score = score + 1;), et la pièce part hors de l'écran, px = 20; (il n'y a pas de colonne 20). Ainsi le A, en restant sur la case, ne la ramasse pas une deuxième fois.

Elle disparaît de l'écran toute seule : en arrivant sur la case, deplace_croix pose le A par-dessus le P ; en repartant, il efface la case. Il ne reste rien.

Le score s'affiche en bas, après le mot SCORE.

LE PROGRAMME — 1870 OCTETS

```
// ---- LES VARIABLES : tout ce que le jeu doit retenir ----

uint8_t x = 5;          // la COLONNE du A (0 à gauche ... 19 à droite) : il part de la colonne 5
uint8_t y = 8;          // la LIGNE du A (0 en haut ... 17 en bas) : il part de la ligne 8

uint8_t px = 15;        // la colonne de la PIÈCE. La pièce est une LETTRE : le P.
uint8_t py = 8;         // sa ligne. Elle est donc en (15, 8), sur la même ligne que le A.

uint8_t score = 0;      // combien de pièces on a ramassées : 0 au départ

int main() {
    // ---- AVANT LA BOUCLE : ce qui ne se fait qu'une fois ----

    // La pièce est posée une seule fois, au départ.
    //  poser(px, py, ALPHABET[15]);
    //      |   |   |
    //      |   |   +-- ALPHABET[15] : la 16e lettre (on compte à partir de 0), le P
    //      +----- sa place : (px, py) = (15, 8)
    poser(px, py, ALPHABET[15]);

    texte(0, 17, "SCORE");          // le mot SCORE, en bas à gauche (ligne 17)

    // ---- LA BOUCLE DU JEU : 60 fois par seconde ----
    while (true) {
        image();                    // 1. attend l'image suivante

        deplace_croix(x, y, ALPHABET[0], 250); // 2. le A suit la croix (le 0.71)

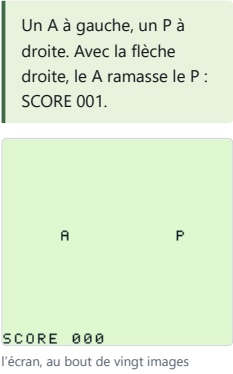
        // 3. Le A est-il sur la pièce ?
        //
        //   if (x == px && y == py)
        //   |       |
        //   |       +-- la LIGNE du A est-elle celle du P ?
        //   +----- la COLONNE du A est-elle celle du P ?
        //   Les DEUX doivent être vraies (&&) : le A est alors sur la case du P.
        //
        //   A en (14, 8) : x == px faux (14 ≠ 15)           → rien
        //   A en (15, 8) : x == px vrai ET y == py vrai     → ramassée !
        if (x == px && y == py) {
            score = score + 1;          // le score augmente de 1 : 0 → 1

            // La pièce part HORS de l'écran : la colonne 20 n'existe pas (0 à 19).
            // Sans cette ligne, le A resté sur la case la ramasserait encore à
            // l'image suivante, et le score monterait sans fin.
            px = 20;
        }

        // Le P a déjà disparu de l'écran : en arrivant, deplace_croix a posé le A
        // PAR-DESSUS ; en repartant, il a effacé la case. Il ne reste rien.

        nombre(6, 17, score);          // 4. le score, après le mot SCORE : 000, 001...
    }
}
```

CE QU'ON DOIT VOIR



0.81.1. Ramasser une pièce : le P — elle réapparaît ailleurs

Le 0.81, mais la pièce ramassée réapparaît à la place suivante d’un tableau : on peut en ramasser sans fin.

C’est le 0.81, mais au lieu de partir hors de l’écran, la pièce réapparaît ailleurs.

Ce qui est nouveau ici : des places rangées dans deux tableaux, `PX` et `PY` (comme les pas du 0.32). La pièce n° `k` est en (`PX[k]`, `PY[k]`). Ramassée, on passe à la suivante : `k = (k + 1) % 5;` — après la 5e, on revient à la 1re.

On la repose aussitôt à sa nouvelle place, avec `poser`.

LE PROGRAMME — 1944 OCTETS

```
// ---- LES VARIABLES : tout ce que le jeu doit retenir ----

uint8_t x = 5;      // la COLONNE du A (0 à gauche ... 19 à droite) : il part de la colonne 5
uint8_t y = 8;      // la LIGNE du A (0 en haut ... 17 en bas) : il part de la ligne 8

uint8_t px = 15;    // la colonne de la PIÈCE. La pièce est une LETTRE : le P.
uint8_t py = 8;     // sa ligne. Elle est donc en (15, 8), sur la même ligne que le A.

uint8_t score = 0;  // combien de pièces on a ramassées : 0 au départ

// Les places des pièces, l'une après l'autre, dans deux tableaux (le 0.32) :
//
//  pièce n° :      0   1   2   3   4
//  PX (colonne): 15,   3, 17,   8, 12
//  PY (ligne) :   8,   2, 14,   5, 11
//
//  la pièce n° 0 est en (15, 8), la n° 1 en (3, 2), la n° 2 en (17, 14)...
const uint8_t PX[] = { 15,  3, 17,  8, 12 };
const uint8_t PY[] = {  8,  2, 14,  5, 11 };

uint8_t k = 0;      // le numéro de la pièce EN COURS : 0, 1, 2, 3, 4, puis de nouveau 0

int main() {
    poser(px, py, ALPHABET[15]);
    texte(0, 17, "SCORE");

    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250);

        if (x == px && y == py) {          // le A est sur la pièce (le 0.81)
            score = score + 1;             // une pièce de plus

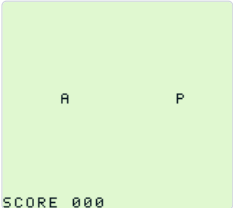
            // La pièce SUIVANTE du tableau :
            //
            //  k = (k + 1) % 5;
            //  |       |
            //  |       +-- % 5 : le reste de la division par 5. 5 % 5 = 0 :
            //  |       après la pièce n° 4, on revient à la n° 0.
            //  +----- k + 1 : le numéro suivant
            //
            //  k : 0 → 1 → 2 → 3 → 4 → 0 → 1 ... sans fin
            k = (k + 1) % 5;

            px = PX[k];                    // la colonne de la pièce n° k...
            py = PY[k];                    // ..et sa ligne
            poser(px, py, ALPHABET[15]);   // on y pose le P : la nouvelle pièce apparaît
        }

        nombre(6, 17, score);
    }
}
```

CE QU’ON DOIT VOIR

Le A ramasse le P ; un autre P apparaît aussitôt ailleurs, et ainsi de suite.



SCORE 000

l’écran, au bout de vingt images

0.81.2. Ramasser une pièce : le P — au hasard

Le 0.81.1, mais la nouvelle place est tirée au hasard : `hasard() % 20` pour la colonne, `hasard() % 17` pour la ligne.

C'est le 0.81.1, sans le tableau : la nouvelle place est tirée **au hasard**.

Ce qui est nouveau ici : `hasard()`. Elle rend un nombre imprévisible, de 0 à 255. `hasard() % 20` en garde le **reste** de la division par 20 : un nombre de 0 à 19, une colonne. `hasard() % 17` : une ligne de 0 à 16 (la ligne 17 est celle du score).

On ne sait jamais où la pièce va tomber : c'est ce qui rend un jeu différent à chaque partie.

LE PROGRAMME — 1919 OCTETS

```
// ---- LES VARIABLES : tout ce que le jeu doit retenir ----

uint8_t x = 5;      // la COLONNE du A (0 à gauche ... 19 à droite) : il part de la colonne 5
uint8_t y = 8;      // la LIGNE du A (0 en haut ... 17 en bas) : il part de la ligne 8

uint8_t px = 15;    // la colonne de la PIÈCE. La pièce est une LETTRE : le P.
uint8_t py = 8;     // sa ligne. Elle est donc en (15, 8), sur la même ligne que le A.

uint8_t score = 0;  // combien de pièces on a ramassées : 0 au départ

int main() {
    poser(px, py, ALPHABET[15]);
    texte(0, 17, "SCORE");

    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250);

        if (x == px && y == py) {          // le A est sur la pièce (le 0.81)
            score = score + 1;

            // La nouvelle place, AU HASARD :
            //
            // px = hasard() % 20;
            // |           |
            // |           +-- % 20 : le reste de la division par 20, de 0 à 19.
            // |           Exemple : hasard() rend 137 ; 137 = 6 x 20 + 17 ;
            // |           le reste est 17 → la colonne 17.
            // +----- un nombre imprévisible, de 0 à 255
            px = hasard() % 20;             // une colonne : 0 à 19
            py = hasard() % 17;             // une ligne : 0 à 16 (la 17, c'est le score)

            poser(px, py, ALPHABET[15]);   // la nouvelle pièce apparaît là
        }

        nombre(6, 17, score);
    }
}
```

CE QU'ON DOIT VOIR

Le A ramasse le P ; un autre P apparaît quelque part au hasard.

A P

SCORE 000

l'écran, au bout de vingt images

0.82. Un mur qu'on ne traverse pas : le M

Un mur de M en colonne 10. Avant chaque pas, lire() regarde la case d'arrivée : si c'est un M, on ne bouge pas.

Le mur est fait de lettres M (ALPHABET[12]), posées l'une sous l'autre en colonne 10, des lignes 4 à 12.

Ce qui est nouveau ici : regarder AVANT de bouger. `deplace_croix` ne connaît pas les murs : on refait donc le déplacement à la main. On calcule d'abord la case d'arrivée, `nx` et `ny`. Puis `lire(nx, ny)` (le 0.69) dit ce qu'il y a dessus. Si ce n'est **pas** un M (`!=`), on avance ; sinon, on reste.

`!=` veut dire « n'est pas égal à ». C'est le contraire de `==`.

`chaque(250)` (le 0.77) donne le rythme : au plus un pas tous les 250 ms.

Pour passer, il faut contourner : le mur s'arrête à la ligne 12. En descendant jusqu'à la ligne 13, on peut passer de l'autre côté.

LE PROGRAMME — 1938 OCTETS

```
// ---- LES VARIABLES ----
uint8_t x = 3;      // la colonne du A : il part de (3, 8), à GAUCHE du mur
uint8_t y = 8;      // sa ligne
uint8_t nx = 0;     // la colonne d'ARRIVÉE : où le A irait s'il bougeait
uint8_t ny = 0;     // la ligne d'arrivée. On la calcule AVANT de bouger.

int main() {
  // ---- LE MUR : des lettres M, posées l'une sous l'autre ----
  //
  //   colonne :   3 ... 9 10
  //   ligne 4 :           M
  //   ligne 5 :           M
  //   ...
  //   ligne 8 :   A     M   le mur : colonne 10, lignes 4 à 12
  //   ...
  //   ligne 12 :          M   le A part à gauche du mur
  //   ligne 13 :          M
  //   ligne 13 :          M   ← sous le mur, la ligne 13 est libre : on passe
  //
  // 1 va de 4 à 12 (<= : « plus petit OU ÉGAL », donc 12 compris).
  for (uint8_t l = 4; l <= 12; l++) {
    poser(10, l, ALPHABET[12]); // ALPHABET[12] : la 13e lettre, le M
  }

  poser(x, y, ALPHABET[0]);      // le A à sa place de départ

  // ---- LA BOUCLE DU JEU ----
  while (true) {
    image();                     // attend l'image suivante
    if (chaque(250)) {           // au plus un pas tous les 250 ms (le 0.77)

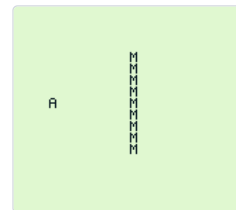
      // ÉTAPE 1 : calculer la case d'ARRIVÉE, sans encore bouger.
      // On part de la case actuelle : si aucune flèche n'est tenue,
      // l'arrivée reste la case où l'on est.
      nx = x;
      ny = y;
      if (bouton(DROITE) && x < 19) { nx = x + 1; } // une colonne à droite
      if (bouton(GAUCHE) && x > 0) { nx = x - 1; } // une colonne à gauche
      if (bouton(BAS) && y < 16) { ny = y + 1; } // une ligne plus bas
      if (bouton(HAUT) && y > 0) { ny = y - 1; } // une ligne plus haut

      // ÉTAPE 2 : REGARDER ce qu'il y a sur la case d'arrivée, avec lire() :
      //
      //   lire(nx, ny) != ALPHABET[12]
      //   |           | |
      //   |           | +- le M, la lettre des murs
      //   |           | +---- != : « n'est pas »
      //   +----- ce que montre l'écran à la case d'arrivée
      //
      // Exemple : le A est en (9, 8), la flèche droite est tenue.
      // L'arrivée est (10, 8). lire(10, 8) rend le M du mur :
      // « n'est pas un M » est FAUX → le A ne bouge pas.
      if (nx != x || ny != y) { // une flèche est tenue (|| : OU)
        if (lire(nx, ny) != ALPHABET[12]) {

          // ÉTAPE 3 : ce n'est pas un mur, on avance.
          effacer(x, y, 1); // efface le A à l'ancienne case
          x = nx;           // la case d'arrivée devient
          y = ny;           // la case du A
          poser(x, y, ALPHABET[0]); // pose le A à la nouvelle case
        }
      }
    }
  }
}
```

CE QU'ON DOIT VOIR

Un A à gauche d'un mur de M. La flèche droite le bloque contre le mur ; en descendant sous le mur, on passe.



l'écran, au bout de vingt images

0.82.1. Un mur qu'on ne traverse pas : le M — un labyrinthe

Des M tout autour et un mur au milieu : un labyrinthe. La pièce P est derrière le mur ; il faut faire le tour pour l'attraper.

C'est le 0.82, avec plus de murs : un cadre de M tout autour, et le mur du milieu, ouvert en bas. La pièce P (le 0.81) attend derrière.

Ce qui est nouveau ici : dessiner des murs avec `texte`. Une ligne de vingt M, c'est `texte(0, 2, "MMMMMMMMMMMMMMMMMMMM");` : un seul appel. Les côtés, eux, se posent un par un dans une boucle.

Le déplacement ne change pas : c'est `lire()` qui fait tout le travail. Il ne sait pas où sont les murs à l'avance : il **regarde l'écran**. Ajoute des M où tu veux, le A ne les traversera pas.

Le chemin : descendre sous le mur (lignes 11 à 13), passer à droite, remonter jusqu'au P.


```
// ---- LES VARIABLES ----
uint8_t x = 3;           // le A : il part de (3, 5), à gauche du mur du milieu
uint8_t y = 5;
uint8_t nx = 0;          // la case d'arrivée, calculée avant chaque pas (le 0.82)
uint8_t ny = 0;
uint8_t score = 0;

int main() {
    // ---- LE LABYRINTHE : tout en lettres M ----
    //
    // ligne 2 : M M M M M M M M M M M M M M M M M M M M le haut
    // ligne 3 : M M M M M M M M M M M M M M M M M M M M
    // ligne 5 : M A M M M M M M M M M M M M M M M M M M A : départ, P : la pièce
    // ... M M M M M M M M M M M M M M M M M M M M
    // ligne 10 : M M M M M M M M M M M M M M M M M M M M le mur du milieu s'arrête ici...
    // ligne 11 : M M M M M M M M M M M M M M M M M M M M ...les lignes 11 à 13
    // ligne 13 : M M M M M M M M M M M M M M M M M M M M sont libres : on passe
    // ligne 14 : M M M M M M M M M M M M M M M M M M M M le bas

    // Le haut et le bas : une ligne de 20 M, en UN seul texte.
    texte(0, 2, "MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM");
    texte(0, 14, "MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM");

    // Les deux côtés : un M par ligne, à gauche (colonne 0) et à droite (colonne 19).
    for (uint8_t l = 3; l < 14; l++) {
        poser(0, l, ALPHABET[12]);
        poser(19, l, ALPHABET[12]);
    }

    // Le mur du milieu, en colonne 10, des lignes 3 à 10 seulement : ouvert en bas.
    for (uint8_t l = 3; l <= 10; l++) {
        poser(10, l, ALPHABET[12]);
    }

    poser(16, 5, ALPHABET[15]); // la pièce P, de l'autre côté du mur
    poser(x, y, ALPHABET[0]);   // le A, à son départ
    texte(0, 17, "SCORE");

    // ---- LA BOUCLE DU JEU ----
    while (true) {
        image();
        if (chaque(250)) { // au plus un pas tous les 250 ms (le 0.77)

            // ÉTAPE 1 : calculer la case d'ARRIVÉE, sans encore bouger.
            // On part de la case actuelle : si aucune flèche n'est tenue,
            // l'arrivée reste la case où l'on est.
            nx = x;
            ny = y;
            if (bouton(DROITE) && x < 19) { nx = x + 1; } // une colonne à droite
            if (bouton(GAUCHE) && x > 0) { nx = x - 1; } // une colonne à gauche
            if (bouton(BAS) && y < 16) { ny = y + 1; } // une ligne plus bas
            if (bouton(HAUT) && y > 0) { ny = y - 1; } // une ligne plus haut

            // ÉTAPE 2 : REGARDER ce qu'il y a sur la case d'arrivée, avec lire() :
            //
            // lire(nx, ny) != ALPHABET[12]
            // | |
            // | | +-+ le M, la lettre des murs
            // | +----- != : « n'est pas »
            // +----- ce que montre l'écran à la case d'arrivée
            //
            // Exemple : le A est en (9, 8), la flèche droite est tenue.
            // L'arrivée est (10, 8). lire(10, 8) rend le M du mur :
            // « n'est pas un M » est FAUX → le A ne bouge pas.
            if (nx != x || ny != y) { // une flèche est tenue (|| : OU)
                if (lire(nx, ny) != ALPHABET[12]) {

                    // ÉTAPE 3 : ce n'est pas un mur, on avance.
                    effacer(x, y, 1); // efface le A à l'ancienne case
                    x = nx;           // la case d'arrivée devient
                    y = ny;           // la case du A
                    poser(x, y, ALPHABET[0]); // pose le A à la nouvelle case
                }
            }
        }
    }

    // La pièce, en (16, 5) : le A est dessus ?
    if (x == 16 && y == 5) {
        if (score == 0) { // seulement la première fois :
            score = 1; // le score passe à 1, une fois pour toutes
        }
        nombre(6, 17, score);
    }
}
```

Un labyrinthe de M ; le A doit descendre, passer sous le mur, et remonter jusqu'au P.

A screenshot of the Atari 2600 game 'Pong'. The game is displayed on a black background. Two vertical white paddles are positioned on the left and right sides. The left paddle is labeled 'A' and the right paddle is labeled 'P'. A small white ball is in the center of the screen. The score '000' is displayed at the bottom left.

l'écran, au bout de vingt images

0.83. Un son quand on ramasse : note

Le 0.81.1, plus une note : `note(1, DO5, 10, 12)` quand le A ramasse la pièce.

C'est le 0.81.1 (la pièce qui réapparaît), **plus un son** au moment où on la ramasse.

Ce qui est nouveau ici : `note(voix, hauteur, duree, volume)`. Elle joue une note sur la **voix 1** (la console en a deux qui chantent, 1 et 2).

• `DO5` : la hauteur, un do aigu (5 : la 5e octave). • `10` : la durée, en images, un sixième de seconde. • `12` : le volume, de 0 (muet) à 15 (le plus fort).

Elle ne bloque pas : la note joue toute seule pendant que le jeu continue.

LE PROGRAMME — 2000 OCTETS

```
// ---- LES VARIABLES : tout ce que le jeu doit retenir ----

uint8_t x = 5;      // la COLONNE du A (0 à gauche ... 19 à droite) : il part de la colonne 5
uint8_t y = 8;      // la LIGNE du A (0 en haut ... 17 en bas) : il part de la ligne 8

uint8_t px = 15;    // la colonne de la PIÈCE. La pièce est une LETTRE : le P.
uint8_t py = 8;     // sa ligne. Elle est donc en (15, 8), sur la même ligne que le A.

uint8_t score = 0;  // combien de pièces on a ramassées : 0 au départ

// Les places des pièces, l'une après l'autre, dans deux tableaux (le 0.32) :
//
//   pièce n° :    0   1   2   3   4
//   PX (colonne): 15,   3, 17,   8, 12
//   PY (ligne) :   8,   2, 14,   5, 11
//
//   la pièce n° 0 est en (15, 8), la n° 1 en (3, 2), la n° 2 en (17, 14)...
const uint8_t PX[] = { 15,   3, 17,   8, 12 };
const uint8_t PY[] = {   8,   2, 14,   5, 11 };

uint8_t k = 0;      // le numéro de la pièce EN COURS : 0, 1, 2, 3, 4, puis de nouveau 0

int main() {
    poser(px, py, ALPHABET[15]);
    texte(0, 17, "SCORE");

    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250);

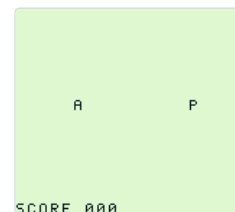
        if (x == px && y == py) {
            score = score + 1;
            // Le son, morceau par morceau :
            //
            //   note(1, DO5, 10, 12);
            //   | | | |
            //   | | | +-- volume : 12 (de 0, muet, à 15, le plus fort)
            //   | | +----- durée : 10 images, un sixième de seconde
            //   | +----- hauteur : DO5, un do aigu (5 : la 5e octave)
            //   +----- voix : la 1 (la console en a deux qui chantent, 1 et 2)
            //
            // La note joue TOUTE SEULE : le jeu continue pendant qu'elle sonne.
            note(1, DO5, 10, 12);

            k = (k + 1) % 5;          // la pièce suivante (le 0.81.1)
            px = PX[k];
            py = PY[k];
            poser(px, py, ALPHABET[15]);
        }

        nombre(6, 17, score);
    }
}
```

CE QU'ON DOIT VOIR

Comme au 0.81.1, mais chaque pièce ramassée fait « ding ».



l'écran, au bout de vingt images

0.83.1. Un son quand on ramasse : note — et un bruit contre le mur

Le labyrinthe du 0.82.1, plus un bruit quand on se cogne : `bruit(4, 8)`, sur la voix du bruit.

C'est le labyrinthe du 0.82.1, plus un son quand le A se **cogne** contre un mur.

Ce qui est nouveau ici : `bruit(duree, volume)`. Elle frappe sur la **voix du bruit** (la 4e) : pas une note, un « toc ». `bruit(4, 8)` : 4 images, volume 8.

Où le mettre : dans le `else` du test du mur. Si la case d'arrivée **n'est pas** un M, on avance ; **sinon** (`else`), c'en est un : on reste, et on fait « toc ».

Et une note quand on ramasse la pièce, comme au 0.83.

```
// ---- LES VARIABLES les mêmes qu'au 0.82.1 ----
uint8_t x = 3;
uint8_t y = 5;
uint8_t nx = 0;
uint8_t ny = 0;
uint8_t score = 0;

int main() {
    // ---- LE LABYRINTHE : tout en lettres M ----
    //
    // ligne 2 :   M M M M M M M M M M M M M M M M M M M le haut
    // ligne 3 :   M                               M       M
    // ligne 5 :   M     A                       M         P     M   A : départ, P : la pièce
    // ...        M                               M       M
    // ligne 10 :  M                               M         M   le mur du milieu s'arrête ici...
    // ligne 11 :  M                               M         M   ...les lignes 11 à 13
    // ligne 13 :  M                               M         M   sont libres : on passe
    // ligne 14 :  M M M M M M M M M M M M M M M M M M M le bas

    // Le haut et le bas : une ligne de 20 M, en UN seul texte.
    texte(0, 2, "MMMMMMMMMMMMMMMMMMMM");
    texte(0, 14, "MMMMMMMMMMMMMMMMMMMM");

    // Les deux côtés : un M par ligne, à gauche (colonne 0) et à droite (colonne 19).
    for (uint8_t l = 3; l < 14; l++) {
        poser(0, l, ALPHABET[12]);
        poser(19, l, ALPHABET[12]);
    }

    // Le mur du milieu, en colonne 10, des lignes 3 à 10 seulement : ouvert en bas.
    for (uint8_t l = 3; l <= 10; l++) {
        poser(10, l, ALPHABET[12]);
    }

    poser(16, 5, ALPHABET[15]);           // la pièce P, de l'autre côté du mur
    poser(x, y, ALPHABET[0]);             // le A, à son départ
    texte(0, 17, "SCORE");

    // ---- LA BOUCLE DU JEU ----
    while (true) {
        image();
        if (chaque(250)) {                 // au plus un pas tous les 250 ms (le 0.77)

            // ÉTAPE 1 : calculer la case d'ARRIVÉE, sans encore bouger.
            // On part de la case actuelle : si aucune flèche n'est tenue,
            // l'arrivée reste la case où l'on est.
            nx = x;
            ny = y;
            if (bouton(DROITE) && x < 19) { nx = x + 1; } // une colonne à droite
            if (bouton(GAUCHE) && x > 0) { nx = x - 1; } // une colonne à gauche
            if (bouton(BAS) && y < 16) { ny = y + 1; } // une ligne plus bas
            if (bouton(HAUT) && y > 0) { ny = y - 1; } // une ligne plus haut

            // ÉTAPE 2 : REGARDER ce qu'il y a sur la case d'arrivée, avec lire() :
            //
            // lire(nx, ny) != ALPHABET[12]
            // |               |
            // |               | +- le M, la lettre des murs
            // |               | +---- != : « n'est pas »
            // +----- ce que montre l'écran à la case d'arrivée
            //
            // Exemple : le A est en (9, 8), la flèche droite est tenue.
            // L'arrivée est (10, 8). lire(10, 8) rend le M du mur :
            // « n'est pas un M » est FAUX → le A ne bouge pas.
            if (nx != x || ny != y) {      // une flèche est tenue (!| : OU)
                if (lire(nx, ny) != ALPHABET[12]) {

                    // ÉTAPE 3 : ce n'est pas un mur, on avance.
                    effacer(x, y, 1);       // efface le A à l'ancienne case
                    x = nx;                  // la case d'arrivée devient
                    y = ny;                  // la case du A
                    poser(x, y, ALPHABET[0]); // pose le A à la nouvelle case
                } else {
                    // ÉTAPE 3 bis : c'EST un mur. Le A ne bouge pas, et on fait « toc ».
                    //
                    bruit(4, 8);
                    // | |
                    // | +- volume : 8 (de 0 à 15)
                    // +---- durée : 4 images, un bruit très court
                    //
                    // bruit joue sur la VOIX DU BRUIT, la 4e : pas une note, un choc.
                    bruit(4, 8);
                }
            }
        }
    }

    // La pièce : une seule fois (score == 0), avec un « ding » (le 0.83).
    if (x == 16 && y == 5 && score == 0) {
        score = 1;
        note(1, D05, 10, 12);
    }
    nombre(6, 17, score);
}
}
```

Le labyrinthe ; contre un mur, un « toc » ; sur la pièce, un « ding ».

SCORE 000

l'écran, au bout de vingt images

0.84. Des vies et une fin de partie

Le poursuivant du 0.79, et 3 vies. Attrapé : une vie de moins, et tout le monde repart. À 0 : PERDU, et START pour recommencer.

C'est le poursuivant du 0.79, avec des vies. On en a 3. Chaque fois que le B nous attrape, on en perd une, et le A comme le B **repartent de leur place de départ**.

Ce qui est nouveau ici : l'ÉTAT du jeu. La variable `etat` dit **où on en est** : 0, on joue ; 1, on a perdu. Tout le programme est coupé en deux par `if (etat == 0) { ... } else { ... }` : en jeu, on bouge ; perdu, on attend.

Perdu : quand `vies` arrive à 0, `etat = 1` ; et le mot PERDU s'écrit au milieu. Dans l'état 1, plus rien ne bouge.

Recommencer : dans l'état 1, le bouton **START** remet 3 vies et `etat = 0` ; on rejoue.

Repartir du départ après une prise : on efface la case (le A et le B y sont tous les deux), puis on remet `x`, `y`, `bx`, `by` à leurs valeurs de départ.

LE PROGRAMME — 2600 OCTETS

```
// ---- LES VARIABLES ----
uint8_t x = 10;      // le A : il part du milieu, (10, 8)
uint8_t y = 8;
uint8_t bx = 0;      // le B, le poursuivant : il part du coin, (0, 0)
uint8_t by = 0;
uint8_t sx = 0;      // la direction du prochain pas du B (le 0.79)
uint8_t sy = 0;
uint8_t vies = 3;    // les vies qui restent : 3 au départ
uint8_t etat = 0;    // L'ÉTAT DU JEU : 0 = on joue, 1 = perdu

int main() {
    texte(0, 17, "VIES");          // le mot VIES, en bas à gauche

    while (true) {
        image();

        // ---- LE PROGRAMME EST COUPÉ EN DEUX PAR L'ÉTAT ----
        //
        // if (etat == 0) { ... on joue ... } else { ... perdu : on attend START ... }
        //
        // Un seul des deux blocs se fait à chaque image, jamais les deux.
        if (etat == 0) {            // ===== ON JOUE

            // Le B avance vers le A, un pas tous les 600 ms (le 0.79).
            if (chaque(600)) {
                sx = 0;
                if (x > bx) { sx = 1; }          // le A est à droite : un pas à droite
                if (x < bx) { sx = -1; }         // à gauche : un pas à gauche
                sy = 0;
                if (y > by) { sy = 1; }          // plus bas : un pas en bas
                if (y < by) { sy = -1; }         // plus haut : un pas en haut
                if (sx != 0 || sy != 0) {        // pas déjà sur lui
                    un_pas(bx, by, ALPHABET[1], sx, sy);
                }
            }

            deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix

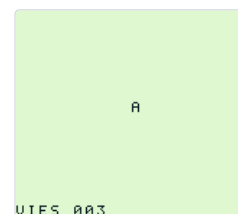
            // ATTRAPÉ : le B est sur la case du A.
            if (bx == x && by == y) {
                vies = vies - 1;                // 1. une vie de moins : 3 + 2 + 1 + 0
                effacer(x, y, 1);               // 2. efface la case (le A et le B y sont)
                x = 10;                         // 3. tout le monde repart du départ :
                y = 8;                          // le A au milieu...
                bx = 0;                         // ...le B dans le coin
                by = 0;

                if (vies == 0) {                // 4. plus aucune vie ?
                    etat = 1;                  // on passe dans l'état « perdu »...
                    texte(7, 8, "PERDU");      // ...et on l'écrit au milieu
                }
            }

            nombre(5, 17, vies);               // les vies, après le mot VIES
        } else {                             // ===== PERDU
            // Plus rien ne bouge : on attend seulement le bouton START.
            if (bouton(START)) {
                effacer(7, 8, 5);              // efface les 5 lettres de PERDU
                vies = 3;                      // de nouveau 3 vies...
                etat = 0;                      // ...et on rejoue
            }
        }
    }
}
```

CE QU'ON DOIT VOIR

Le B poursuit le A ; à chaque prise, une vie de moins et tout repart. Au bout de trois : PERDU. START relance la partie.



l'écran, au bout de vingt images

0.85. Plusieurs ennemis : un tableau de struct

Trois B, rangés dans Position ennemis[3], partis d'un endroit au hasard ; une boucle for les fait tous avancer vers le A.

Trois poursuivants au lieu d'un. Plutôt que `bx`, `by`, `bx2`, `by2`, `bx3`, `by3`, on les range dans un tableau de `struct` : `Position ennemis[3]`; (le type `Position` du 0.80).

Ce qui est nouveau ici : `ennemis[i].x`. D'abord la **case** du tableau (`[i]`), l'ennemi n° i, **puis** la partie (`.x`), sa colonne). `ennemis[0].x` est la colonne du premier, `ennemis[2].y` la ligne du troisième.

Une boucle `for` passe sur les trois : le même code fait avancer chacun vers le A (le 0.79). Pour quatre ennemis, on changerait seulement le 3 en 4.

Au départ, au hasard (le 0.81.2) : chaque ennemi prend une colonne `hasard() % 20` sur les lignes du haut. Chaque partie commence différemment.

Ils finissent par se rejoindre sur le A : ils visent tous la même case.

LE PROGRAMME — 2620 OCTETS

```

---- LE TYPE Position (le 0.80) : une colonne et une ligne
struct Position { uint8_t x, y; };

// ---- UN TABLEAU DE Position : trois ennemis ----
//
// Position ennemis[3];
// |           |           |
// |           |           | +-- [3] : trois cases, numérotées 0, 1 et 2
// |           | +----- le nom du tableau
// +----- chaque case est une Position (un x et un y)
//
// ennemis[0].x ennemis[0].y ← le 1er ennemi
// ennemis[1].x ennemis[1].y ← le 2e
// ennemis[2].x ennemis[2].y ← le 3e
Position ennemis[3];

uint8_t x = 10;           // le A : en bas, (10, 12)
uint8_t y = 12;
uint8_t sx = 0;           // la direction du prochain pas d'un ennemi
uint8_t sy = 0;

int main() {
    // ---- LE DÉPART : chaque ennemi au hasard, en haut de l'écran ----
    // i vaut 0, puis 1, puis 2 : la boucle s'occupe de chaque ennemi à son tour.
    for (uint8_t i = 0; i < 3; i++) {
        ennemis[i].x = hasard() % 20;           // sa colonne : de 0 à 19 (le 0.81.2)
        ennemis[i].y = hasard() % 3;           // sa ligne : de 0 à 2, tout en haut
        poser(ennemis[i].x, ennemis[i].y, ALPHABET[1]); // on l'y pose : un B
    }

    // ---- LA BOUCLE DU JEU ----
    while (true) {
        image();

        if (chaque(700)) { // tous les 700 ms...
            for (uint8_t i = 0; i < 3; i++) { // ...chaque ennemi fait un pas vers le A
                // ennemis[i].x se lit en deux temps :
                // ennemis[i] : d'abord la CASE du tableau, l'ennemi n° i
                // .x           puis la PARTIE, sa colonne
                //
                // La direction, comme au 0.79, mais avec les parties de ennemis[i] :
                sx = 0;
                if (x > ennemis[i].x) { sx = 1; }
                if (x < ennemis[i].x) { sx = -1; }
                sy = 0;
                if (y > ennemis[i].y) { sy = 1; }
                if (y < ennemis[i].y) { sy = -1; }
                if (sx != 0 || sy != 0) {
                    // un_pas range la nouvelle place dans ennemis[i].x et ennemis[i].y :
                    // la console sait ranger dans une partie de struct (le 0.80).
                    un_pas(ennemis[i].x, ennemis[i].y, ALPHABET[1], sx, sy);
                }
            }
        }
    }

    deplace_croix(x, y, ALPHABET[0], 250); // le A, dessiné en dernier : par-dessus
}

```

CE QU'ON DOIT VOIR

Trois B partis d'en haut, à des places différentes à chaque partie, qui descendent tous vers le A.

l'écran, au bout de vingt images

1. Écrire à l'écran

Une cartouche qui affiche un mot, et rien d'autre.

Le plus petit programme qui fait quelque chose. Comme tout programme C++, il commence à `int main()` : c'est là que la console arrive.

`texte(colonne, ligne, "...")` écrit à partir d'une case : l'écran en fait **20 de large sur 18 de haut**, et la case (0, 0) est en haut à gauche.

`while (true) image();` n'est pas décoratif. Un programme qui se termine laisse le processeur partir n'importe où ; ici, il attend l'image suivante, soixante fois par seconde, et l'écran reste affiché.

LE PROGRAMME — 1291 OCTETS

```
int main() {  
    texte(5, 6, "BONJOUR");  
  
    while (true) image();  
}
```

CE QU'ON DOIT VOIR

« BONJOUR » à la
colonne 5, ligne 6.

BONJOUR

l'écran, au bout de vingt images

1.1. Effacer avec des espaces

Il n'y a pas de gomme : on écrit du vide par-dessus.

L'espace est une case vide. Écrire des espaces par-dessus un mot, c'est l'effacer.

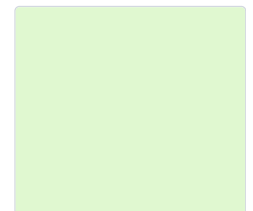
Il en faut **autant que de lettres** : `BONJOUR` en a sept, donc sept espaces. Un de moins laisse le `R` tout seul.

LE PROGRAMME — 1309 OCTETS

```
int main() {  
    texte(5, 6, "BONJOUR");  
    texte(5, 6, "      ");  
  
    while (true) image();  
}
```

CE QU'ON DOIT VOIR

« BONJOUR » est écrit
puis recouvert d'espaces :
l'écran reste vide.



l'écran, au bout de vingt images

1.2. Effacer sans compter

`effacer()` compte les lettres à ta place.

`effacer(5, 6, "BONJOUR")` efface **autant de cases que le mot a de lettres** : sept. Plus d'espaces à compter.

Il faut effacer **à la même place** qu'on a écrit : ici, (5, 6) deux fois.

LE PROGRAMME — 1299 OCTETS

```
int main() {
    texte(5, 6, "BONJOUR");
    effacer(5, 6, "BONJOUR");

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » est écrit
puis effacé : l'écran reste
vide.

l'écran, au bout de vingt images

1.3. Ranger le mot sous un nom

Le mot est écrit une seule fois, en haut, et porte un nom.

`const char MOT[] = "BONJOUR";` range le mot sous le nom `MOT`, au-dessus de `int main()`. C'est la chaîne de caractères de cette console.

`texte(5, 6, MOT)` écrit ce que `MOT` contient.

LE PROGRAMME — 1291 OCTETS

```
const char MOT[] = "BONJOUR";

int main() {
    texte(5, 6, MOT);

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » à la
colonne 5, ligne 6.

BONJOUR

l'écran, au bout de vingt images

1.4. Effacer le mot rangé sous un nom

Le compilateur connaît la longueur de MOT : on l'efface par son nom.

`effacer(5, 6, MOT)` efface les sept cases de `MOT`. Si l'on change le mot, l'effacement suit tout seul.

LE PROGRAMME — 1299 OCTETS

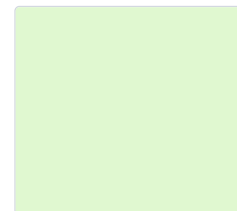
```
const char MOT[] = "BONJOUR";

int main() {
    texte(5, 6, MOT);
    effacer(5, 6, MOT);

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » est écrit
puis effacé : l'écran reste
vide.



l'écran, au bout de vingt images

1.5. La place du mot dans deux variables

La colonne et la ligne sont rangées, elles aussi, sous un nom.

`uint8_t x = 5;` range un nombre sous le nom `x` : la **colonne**. La **ligne** va dans `y`.

`texte(x, y, MOT)` écrit le mot à cette place. Pour le déplacer, on ne change que `x` ou `y`.

LE PROGRAMME — 1316 OCTETS

```
const char MOT[] = "BONJOUR";
uint8_t x = 5;
uint8_t y = 6;

int main() {
    texte(x, y, MOT);

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » en (x, y) :
colonne 5, ligne 6.

BONJOUR

l'écran, au bout de vingt images

1.6. Effacer à la place rangée

`texte()` et `effacer()` lisent les mêmes `x` et `y` : ils visent forcément la même case.

`effacer(x, y, MOT)` efface là où `texte(x, y, MOT)` a écrit : les deux lisent les mêmes variables.

LE PROGRAMME — 1341 OCTETS

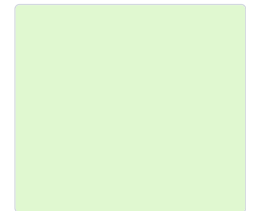
```
const char MOT[] = "BONJOUR";
uint8_t x = 5;
uint8_t y = 6;

int main() {
    texte(x, y, MOT);
    effacer(x, y, MOT);

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » est écrit
puis effacé en (x, y) :
l'écran reste vide.



l'écran, au bout de vingt images

1.7. Le mot et sa place sous un seul nom

Mot range la colonne, la ligne et le texte : on n'écrit plus que le nom.

`Mot mot_xy = { 5, 6, "BONJOUR" };` range **trois choses sous un seul nom** : la colonne, la ligne, et le texte.

`texte(mot_xy)` l'écrit. Plus rien à répéter : ni la place, ni le mot.

LE PROGRAMME — 1291 OCTETS

```
Mot mot_xy = { 5, 6, "BONJOUR" };

int main() {
    texte(mot_xy);

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » à la
colonne 5, ligne 6.

BONJOUR

l'écran, au bout de vingt images

1.8. Effacer un Mot

`effacer(mot_xy)` : le nom suffit.

`effacer(mot_xy)` efface la place et la longueur que `mot_xy` retient : exactement ce que `texte(mot_xy)` a écrit.

LE PROGRAMME — 1299 OCTETS

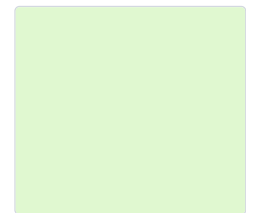
```
Mot mot_xy = { 5, 6, "BONJOUR" };

int main() {
    texte(mot_xy);
    effacer(mot_xy);

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » est écrit
puis effacé : l'écran reste
vide.



l'écran, au bout de vingt images

1.9. Un Mot fait de variables créées d'avance

x, y et le texte sont créés d'abord ; le Mot les rassemble sous un seul nom.

On crée d'abord `x`, `y` et `MOT`, **avant** le `Mot` : il ne peut se servir que de ce qui existe déjà.

`Mot mot_xy = { x, y, MOT };` ne recopie pas 5 et 6 : il **relit** `x` et `y` à chaque `texte()`. Changer `x` déplace donc le mot.

LE PROGRAMME — 1316 OCTETS

```
uint8_t x = 5;
uint8_t y = 6;
const char MOT[] = "BONJOUR";
Mot mot_xy = { x, y, MOT };

int main() {
    texte(mot_xy);

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » en (x, y) :
colonne 5, ligne 6.

BONJOUR

l'écran, au bout de vingt images

1.10. Effacer un Mot fait de variables

Le même Mot, effacé par son seul nom.

`effacer(mot_xy)` relit `x`, `y` et `MOT` : il efface exactement ce que `texte(mot_xy)` a écrit.

LE PROGRAMME — 1341 OCTETS

```
uint8_t x = 5;
uint8_t y = 6;
const char MOT[] = "BONJOUR";
Mot mot_xy = { x, y, MOT };

int main() {
    texte(mot_xy);
    effacer(mot_xy);

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » est écrit
puis effacé en (x, y) :
l'écran reste vide.

l'écran, au bout de vingt images

1.11. Les variables dans leur propre fichier

x, y, MOT et le Mot déménagent dans « variables.h » ; main() n'a plus que les gestes.

Les quatre lignes du haut de la 1.10 sont maintenant dans un **second fichier**, variables.h : c'est l'onglet à côté de principal.cpp.

#include "variables.h" **verse ce fichier à cet endroit**, avant la compilation. Le programme est exactement le même que celui de la 1.10 ; il est seulement rangé en deux.

D'un côté ce qu'on affiche et où, de l'autre ce qu'on en fait. Pour changer le mot ou sa place, on n'ouvre que variables.h.

LE PROGRAMME — 1341 OCTETS

variables.h

```
uint8_t x = 5;
uint8_t y = 6;
const char MOT[] = "BONJOUR";
Mot mot_xy = { x, y, MOT };
```

principal.cpp

```
#include "variables.h"

int main() {
    texte(mot_xy);
    effacer(mot_xy);

    while (true) image();
}
```

CE QU'ON DOIT VOIR

« BONJOUR » est écrit
puis effacé en (x, y) :
l'écran reste vide.

l'écran, au bout de vingt images

2. Effacer quand on appuie sur A

Le même effacement, déclenché par un bouton.

`if (bouton(A))` exécute ce qui suit **tant que A est enfoncé**. On y recouvre la ligne d'espaces.

« BONJOUR APPUIE SUR A » fait vingt caractères, espaces compris : il faut donc vingt espaces. C'est toute la largeur de l'écran.

LE PROGRAMME — 1376 OCTETS

```
int main() {
    texte(0, 4, "BONJOUR APPUIE SUR A");
    texte(3, 8, "APPUIE SUR A");

    while (true) {
        image();

        if (bouton(A)) {
            texte(0, 4, "                "); // vingt espaces, autant que la phrase
        }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

A efface la ligne du haut ;
« APPUIE SUR A » reste.

BONJOUR APPUIE SUR A

APPUIE SUR A

l'écran, au bout de vingt images

2.1. Changer de message avec A et B

Écrire un mot par-dessus un autre le remplace.

B écrit « APPUIE SUR A », A remet « APPUIE SUR B ». Chaque message **recouvre** le précédent, à la même place.

Les deux font **douze** caractères : le nouveau couvre exactement l'ancien, et il ne reste aucune lettre de trop.

LE PROGRAMME — 1394 OCTETS

```
int main() {  
  
    texte(1, 8, "APPUIE SUR B");  
    while (true) {  
        image();  
        if (bouton(A)) {  
            texte(6, 4, "  
                "); // sept espaces, autant que BONJOUR  
            texte(1, 8, "APPUIE SUR B");  
        }  
  
        if (bouton(B)) {  
  
            texte(1, 8, "APPUIE SUR A");  
        }  
    }  
  
    return 0;  
}
```

CE QU'ON DOIT VOIR

B affiche « APPUIE SUR A
» ; A remet « APPUIE SUR
B ».

APPUIE SUR B

l'écran, au bout de vingt images

2.2. Effacer ce qu'on a écrit

Il n'y a pas de gomme : on écrit du vide par-dessus.

La carte de fond garde ce qu'on y a mis, tant qu'on ne le remplace pas. **L'espace est la tuile 0**, c'est-à-dire du vide : écrire des espaces par-dessus **est** l'effacement. C'est ce que fait ce programme, et c'est ce qu'il faut avoir compris une fois.

Le piège tient en un mot : il faut **autant d'espaces que de lettres**. `BONJOUR` en compte sept ; six espaces laisseraient un `R` tout seul au bout, et cette lettre orpheline est l'erreur la plus courante du débutant.

C'est pour cela que `effacer(6, 4, "BONJOUR")` **existe** : il ôte exactement la longueur du texte, comptée par le compilateur. Les deux lignes font la même chose, mais l'une ne peut pas se tromper d'un espace. La leçon « Effacer sans compter les lettres » y revient.

Pour effacer une zone plus large, `poser(colonne, ligne, 0)` fait la même chose case par case, et se met dans une boucle.

LE PROGRAMME — 1430 OCTETS

```
int main() {
    texte(6, 4, "BONJOUR");
    texte(3, 8, "APPUIE SUR A");

    while (true) {
        image();

        if (bouton(A)) {
            texte(6, 4, " "); // sept espaces, autant que BONJOUR
            texte(3, 8, "MERCI ");
        }

        if (bouton(B)) {
            texte(6, 4, "BONJOUR");
            texte(3, 8, "APPUIE SUR A");
        }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

A efface « BONJOUR » ; B le remet.

BONJOUR
APPUIE SUR A

l'écran, au bout de vingt images

2.3. Les messages dans leur propre fichier

Les messages et leur place vont dans « `variables.h` » ; `main()` ne garde que les boutons.

Comme en 1.11 : chaque message devient un `Mot`, rangé dans `variables.h`. C'est l'onglet à côté de `principal.cpp`.

`principal.cpp` ne parle plus que des **boutons** : A efface `bonjour`, B le remet. Les places et les textes ne s'y écrivent plus du tout.

LE PROGRAMME — 1372 OCTETS

`variables.h`

```
Mot bonjour = { 6, 4, "BONJOUR" };
Mot consigne = { 3, 8, "APPUIE SUR A" };
```

`principal.cpp`

```
#include "variables.h"

int main() {
    texte(bonjour);
    texte(consigne);

    while (true) {
        image();

        if (bouton(A)) effacer(bonjour);
        if (bouton(B)) texte(bonjour);
    }
}
```

CE QU'ON DOIT VOIR

A efface « BONJOUR » ; B le remet. La consigne reste.

BONJOUR
APPUIE SUR A

l'écran, au bout de vingt images

3. Un carré de 8 × 8

Une case de l'écran est un carré de 8 × 8 pixels : on en pose un.

Chaque case de l'écran est un **carré de 8 × 8 pixels**, qu'on appelle une **tuile**. Une lettre est une tuile ; ce carré aussi.

Tuile CARRE = { ... }; dessine la tuile : **8 lignes de 8 chiffres**, un chiffre par pixel. 3 est la nuance la plus foncée, 0 la plus claire.

poser(colonne, ligne, CARRE) pose la tuile dans une case. L'écran fait 20 × 18 cases, soit **160 × 144 pixels**.

LE PROGRAMME — 1316 OCTETS

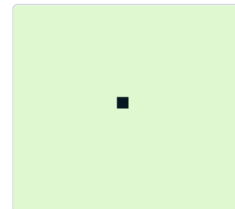
```
// Un carré de 8 × 8 pixels : 8 lignes de 8 chiffres.
// 3 = pixel le plus foncé.
Tuile CARRE = {
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
  "33333333",
};

int main() {
  poser(9, 8, CARRE); // colonne 9, ligne 8 : le milieu de l'écran

  while (true) image(); // garde l'écran affiché
}
```

CE QU'ON DOIT VOIR

Un carré noir de 8 × 8 au milieu de l'écran.



l'écran, au bout de vingt images

4. Écrire à plusieurs endroits

La position est un calcul comme un autre.

Trois appels, trois endroits. Rien de neuf — sauf ceci : la colonne et la ligne **ne sont pas obligées d'être écrites en clair**.

`texte(4 + 2, 8, "...")` marche, et `texte(x, y, "...")` avec des variables aussi. C'est ce qui rendra possible tout ce qui bouge.

LE PROGRAMME — 1342 OCTETS

```
int main() {
    texte(0, 0, "EN HAUT A GAUCHE");
    texte(4, 8, "AU MILIEU");
    texte(2, 17, "TOUT EN BAS");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois lignes, en haut, au milieu et tout en bas.

EN HAUT A GAUCHE

AU MILIEU

TOUT EN BAS

l'écran, au bout de vingt images

5. Une constante plutôt qu'un nombre

Nommer ne coûte rien du tout — et c'est la seule chose ici qui soit gratuite.

Un nombre écrit trois fois, ce sont **trois occasions de le changer à deux endroits**. Une `const` lui donne un nom.

Et elle **ne prend aucun octet de mémoire**. Elle est connue à la compilation : le compilateur remplace `COLONNE` par `5` dans le code machine, et il n'en reste rien dans la cartouche.

Comparer les deux compilations le montre — le nombre de variables annoncé ne bouge pas.

LE PROGRAMME — 1308 OCTETS

```
const uint8_t COLONNE = 5;
const uint8_t LIGNE = 6;

int main() {
    texte(COLONNE, LIGNE, "BONJOUR");
    texte(COLONNE, LIGNE + 2, "ENCORE");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« BONJOUR » puis «
ENCORE », deux lignes
plus bas.

BONJOUR
ENCORE

l'écran, au bout de vingt images

6. Réécrire au bout d'une seconde

Compter des images est l'horloge la plus simple qu'on ait.

La console affiche **soixante images par seconde**. Une boucle qui appelle `image()` soixante fois a donc laissé passer une seconde — sans minuteur, sans rien d'autre.

L'écran ne s'efface pas tout seul : pour ôter un mot, on écrit **autant d'espaces qu'il avait de lettres**. Un espace de moins, et la dernière lettre reste toute seule.

LE PROGRAMME — 1376 OCTETS

```
int main() {
    texte(4, 6, "PREMIER MOT");
    for (uint8_t i = 0; i < 60; i++) image();

    texte(4, 6, "          ");
    texte(4, 6, "SECOND MOT");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« PREMIER MOT » une
seconde, puis « SECOND
MOT » à sa place.

PREMIER MOT

l'écran, au bout de vingt images

7. Un nombre à l'écran

`texte` écrit des lettres ; `nombre` écrit un calcul.

`texte()` ne sait écrire que ce qui est entre guillemets, décidé à la compilation. **Un score qui change veut** `nombre()`.

`nombre(colonne, ligne, valeur)` écrit en base dix, sur **trois chiffres par défaut** — `042`. Un quatrième argument dit combien on en veut : `nombre(8, 4, vies, 1)` écrit `3` tout seul.

LE PROGRAMME — 1332 OCTETS

```
int main() {
    uint8_t vies = 3;
    uint8_t score = 42;

    texte(2, 4, "VIES");
    nombre(8, 4, vies, 1);

    texte(2, 6, "SCORE");
    nombre(8, 6, score);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« VIES 3 » et « SCORE 042 ».

VIES 3
SCORE 042

l'écran, au bout de vingt images

8. Effacer sans compter les lettres

Le compilateur connaît la longueur du texte. Autant qu'il la compte.

Écrire des espaces par-dessus **est** l'effacement — l'espace est la tuile 0. Mais il en faut **autant que de lettres** : un de moins laisse la dernière orpheline, un de plus mange la case d'à côté. Ni l'un ni l'autre n'est signalé, puisque écrire des espaces est parfaitement légal.

Or le compilateur **connaît** cette longueur : il la lit dans les guillemets, ou dans la table nommée. La compter soi-même, c'était se donner une occasion de se tromper que rien n'obligeait à prendre.

`effacer(colonne, ligne, quoi)` accepte les **trois** façons de dire combien : le texte lui-même, le nom d'un `const char`, ou un nombre — calculé s'il le faut. Et

`effacer(..., 0)` n'efface **aucune** case, ce qui n'allait pas de soi : sur un octet, décompter à partir de zéro donne 255.

`effacerPanneau()` fait de même sur le panneau — sans argument il le vide entièrement, à trois arguments il n'ôte qu'un mot.

LE PROGRAMME — 1414 OCTETS

```
const char FIN[] = "GAME OVER";

int main() {
    texte(2, 4, "BONJOUR");
    texte(2, 6, FIN);
    texte(2, 8, "ABCDEF");
    texte(2, 10, "JE RESTE");

    for (uint8_t i = 0; i < 90; i++) image();

    effacer(2, 4, "BONJOUR"); // sept cases, comptées pour nous
    effacer(2, 6, FIN);      // neuf, lues dans la table nommée
    uint8_t combien = 3;
    effacer(2, 8, combien);  // trois, décidées à l'exécution

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois lignes s'effacent au bout d'une seconde et demie ; « JE RESTE » ne bouge pas, et « DEF » survit.

```
BONJOUR
GAME OVER
ABCDEF
JE RESTE
```

l'écran, au bout de vingt images

9. Une variable, un octet

Retenir un nombre, et le montrer.

Une variable s'écrit `uint8_t` : un entier non signé de huit bits. C'est le seul nombre que cette machine connaisse — de 0 à 255, pas de virgule, pas de négatif. `int`, `char` et `auto` sont acceptés, et désignent la même chose ; `uint16_t`, `long` ou `float` sont **refusés avec leur numéro de ligne**, plutôt que traduits en douce sur un octet.

`attente++` est la façon C++ d'écrire `attente = attente + 1`. `+=`, `--`, `%=` existent aussi.

Pour afficher un chiffre calculé, on ne peut pas se servir de `texte()`, qui ne prend que des mots écrits en clair. On pose la **tuile** du chiffre : celle du 0 porte le numéro 27, et les neuf suivantes se suivent. D'où `27 + compte`.

LE PROGRAMME — 1382 OCTETS

```
int main() {
    uint8_t compte = 0;
    uint8_t attente = 0;

    texte(4, 4, "COMPTE");

    while (true) {
        image();

        attente++;

        if (attente > 30) {
            attente = 0;

            if (compte < 9) compte++;
        }

        poser(11, 4, 27 + compte);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Un chiffre qui monte de 0 à 9, une fois par demi-seconde.

COMPTE 0

l'écran, au bout de vingt images

10. Lire un bouton

Réagir à la manette.

bouton(A) rend 1 tant que le bouton est enfoncé. Les huit noms existent : A B HAUT BAS GAUCHE DROITE START SELECT.

Ici on lit le **niveau** : tant que A est tenu, le texte change. C'est ce qu'on veut pour courir ou viser — mais pas pour un menu : c'est la leçon « le front » qui règle ce cas.

&&, || et ! s'écrivent comme en C++, et **s'arrêtent dès que la réponse est connue** : dans `i < n && t[i] == 0`, la case n'est pas lue si l'index est hors du tableau.

LE PROGRAMME — 1410 OCTETS

```
int main() {
    texte(3, 5, "APPUIE SUR A");

    while (true) {
        image();

        if (bouton(A) && !bouton(B)) {
            texte(3, 5, "BRAVO ");
        }

        if (bouton(B)) {
            texte(3, 5, "APPUIE SUR A");
        }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

« BRAVO » pendant qu'on tient A ; B remet le texte de départ.

APPUIE SUR A

l'écran, au bout de vingt images

11. Un compteur qui déborde à 255

Ce n'est pas un bogue : c'est ce que fait un octet.

`uint8_t` est **un octet** : de 0 à 255. À 255, `compteur++` ramène à 0 — et tout le projet est bâti là-dessus.

L'ordre compte. `image()` est **à la fin** : on dessine, puis on attend. Mettre l'attente en premier ferait sauter la première image.

Laisser tourner quelques secondes suffit à voir le compteur repasser par zéro.

LE PROGRAMME — 1293 OCTETS

```
int main() {
    uint8_t compteur = 0;

    while (true) {
        nombre(8, 8, compteur);
        compteur++;
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un nombre qui monte, et qui repart de 000 après 255.

016

l'écran, au bout de vingt images

12. Un dé, et le hasard du matériel

hasard() ne tire pas pile ou face : il lit un compteur qui tourne tout seul.

hasard() rend un octet imprévisible, de 0 à 255 — pas déjà réparti entre 1 et 6. Le reste d'une division, **hasard() % 6**, ramène ça entre 0 et 5 ; il suffit d'ajouter 1 pour une face de dé.

semer(images()) est appelé une seule fois, au tout début. Sans lui, la console partirait toujours du même point : son horloge interne n'a pas encore tourné à l'instant où le jeu démarre, et deux parties de suite tireraient la même suite.

Le dé ne se relance qu'au front de A, comme la leçon « Se souvenir de l'état d'avant » — sans lui, tenir A ferait tourner le dé soixante fois par seconde, bien trop vite pour le lire.

LE PROGRAMME — 1470 OCTETS

```
uint8_t de = 1;
uint8_t avantA = 0;
uint8_t lances = 0;

uint8_t lancerDe() {
    return hasard() % 6 + 1;
}

void dessinerDe(uint8_t face) {
    effacer(8, 6, "0");
    nombre(8, 6, face, 1);
}

int main() {
    semer(images());

    texte(2, 2, "A: LANCER LE DE");
    texte(2, 4, "FACE:");
    texte(2, 9, "LANCES:");
    dessinerDe(de);

    while (true) {
        uint8_t a = bouton(A);

        if (a && !avantA) {
            de = lancerDe();
            dessinerDe(de);
            lances++;
            nombre(10, 9, lances);
        }
        avantA = a;

        image();
    }
}
```

CE QU'ON DOIT VOIR

Un dé de 1 à 6 sous FACE, qui change à chaque appui sur A ; LANCES compte les appuis.

```
A : LANCER LE DE
FACE :
      1
LANCES :
```

l'écran, au bout de vingt images

13. Deux mots de même longueur

Le piège du texte qui change : la fin de l'ancien reste.

bouton(A) rend 1 tant que le bouton est enfoncé, 0 sinon. Les huit noms sont A B HAUT BAS GAUCHE DROITE START SELECT .

Noter l'espace dans "APPUYE ". Sans lui, le mot fait six lettres et RELACHE en fait sept : le E final resterait affiché sous APPUYE , et l'on chercherait longtemps pourquoi.

Deux textes qui se remplacent doivent faire la même longueur. C'est une règle, pas une précaution.

LE PROGRAMME — 1328 OCTETS

```
int main() {
    while (true) {
        if (bouton(A)) texte(6, 8, "APPUYE ");
        else          texte(6, 8, "RELACHE");

        image();
    }
}
```

CE QU'ON DOIT VOIR

« RELACHE », et « APPUYE » tant qu'on tient A.

RELACHE

l'écran, au bout de vingt images

14. Se souvenir de l'état d'avant

Compter un appui, et non soixante par seconde.

Sans les deux lignes du milieu, `combien` monterait de **soixante par seconde** tant qu'on garde le doigt dessus : la boucle tourne soixante fois, et le bouton est enfoncé à chacun de ces tours.

Garder l'état d'avant, et ne compter que le **changement** — c'est ce qu'on appelle un front. On s'en sert pour tout ce qui doit arriver **une fois** : sauter, tirer, valider un menu.

La leçon « Le front, ou pourquoi un appui compte trois fois » montre ce qui arrive quand on l'oublie.

LE PROGRAMME — 1361 OCTETS

```
int main() {
    uint8_t combien = 0;
    uint8_t avant = 0;

    while (true) {
        uint8_t maintenant = bouton(A);

        /* Le FRONT : enfoncé maintenant, et pas au tour d'avant. */
        if (maintenant && !avant) combien++;
        avant = maintenant;

        texte(2, 6, "APPUIS");
        nombre(10, 6, combien);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le compteur monte de un par appui, même si l'on tient la touche.

APPUIS 000

l'écran, au bout de vingt images

15. Un choix, puis plusieurs

`&&` s'arrête dès que la réponse est connue.

`if ... else if ... else` enchaîne les cas. Rien que de très ordinaire — sauf la dernière ligne.

`bouton(B) && vies > 0` : si `bouton(B)` rend 0, **la partie droite n'est même pas calculée**. Ce n'est pas une finesse de compilation, c'est ce qui permet plus tard d'écrire

`if (i < n && table[i] == 0)` sans lire une case hors du tableau.

LE PROGRAMME — 1428 OCTETS

```
int main() {
    uint8_t vies = 3;

    while (true) {
        if (vies == 0)     texte(4, 8, "PERDU  ");
        else if (vies == 1) texte(4, 8, "DERNIERE");
        else               texte(4, 8, "CA VA  ");

        if (bouton(B) && vies > 0) vies--;
        image();
    }
}
```

CE QU'ON DOIT VOIR

« CA VA », puis «
DERNIERE », puis «
PERDU » — B après B.

CA VA

l'écran, au bout de vingt images

16. Deux touches, deux effets

Un `enum` donne des noms à des nombres, et `switch` les range côte à côte.

`TITRE` vaut 0, `JEU` vaut 1, `FIN` vaut 2 — mais **on ne l'écrit jamais**, et c'est tout l'intérêt : ajouter une scène au milieu ne renumérote rien à la main.

`Scene` devient un **type d'un octet**. `Scene ou = TITRE;` se relit mieux que `uint8_t ou = 0;`, et dit ce que la variable a le droit de contenir.

C'est la forme que prend presque tout jeu : quelques scènes, et un `switch` qui dit laquelle est en cours.

LE PROGRAMME — 1441 OCTETS

```
enum Scene { TITRE, JEU, FIN };

int main() {
    Scene ou = TITRE;

    while (true) {
        switch (ou) {
            case TITRE:
                texte(4, 8, "APPUIE START");
                if (bouton(START)) { texte(4, 8, " "); ou = JEU; }
                break;
            case JEU:
                texte(4, 8, "ON JOUE ");
                if (bouton(SELECT)) ou = FIN;
                break;
            case FIN:
                texte(4, 8, "C EST FINI ");
                break;
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

START passe au jeu,
SELECT y met fin.

APPUIE START

l'écran, au bout de vingt images

17. Dessiner sa propre tuile

Sortir des lettres — en chiffres, ou à la souris.

La police suffit à écrire, pas à jouer. Un **Tuile** prend **huit rangées de huit chiffres** : la nuance de chaque pixel, de 0 (le plus clair) à 3 (le plus sombre). Un **Perso** en prend seize sur seize.

Ce qui détermine le numéro d'une tuile, c'est l'ordre des déclarations. La police occupe les numéros 0 à 43 — les lettres, les chiffres, le carré plein. Le premier dessin du programme prend donc le **44**, le suivant le 45, et ainsi de suite. Le compilateur l'annonce à chaque fois, et l'atelier l'écrit sous la grille.

Le nom sert partout où un numéro est attendu : `poser(x, y, BLOC)` sur le fond, `sprite(n, x, y, BLOC)` en lutin, `lire(x, y) == BLOC` pour relire ce qui est affiché, et jusque dans une table de décor — `const uint8_t LIGNE[] = { BLOC, 0, BLOC };`. Il se calcule aussi : `BLOC + 1` est la tuile d'à côté. **Un Perso de seize, lui, compte pour quatre numéros** — c'est l'objet de la leçon suivante.

C'est exactement **pourquoi on écrit le nom et jamais le numéro**. Ajoute un **Perso** avant ta tuile, et elle passe de 44 à 48 : un `poser(2, 2, 44)` écrit à la main afficherait soudain autre chose, sans que rien ne le signale. Le nom, lui, reste juste — c'est le compilateur qui traduit, et c'est son travail.

Les deux façons de faire, et c'est la même chose. En code : change un **0** en **3** dans le programme, et le bloc change. À la souris : clique un pixel dans la grille ci-dessous. Le programme se réécrit tout seul sous tes yeux — c'est le même texte, écrit autrement.

Les chiffres ne sont pas la seule écriture. Huit chiffres à la file ne ressemblent pas à un dessin ; les signes `. - + #` disent exactement la même chose — le point est le vide, le dièse est le plein — et se relisent de loin : `"#.++++.#"`. Les deux donnent les mêmes octets. On n'en mélange pas deux dans une même tuile, et l'atelier rend à chacune la sienne quand tu peins.

C'est vrai partout dans ce projet : **le programme reste la seule vérité**. L'atelier ne garde aucun dessin de son côté ; il lit les `Tuile NOM = {...}` du texte, et il les réécrit. Un éditeur graphique qui tiendrait ses propres dessins finirait par ne plus dire la même chose que le code.

LE PROGRAMME — 1373 OCTETS

```
Tuile BLOC = {
  "33333333",
  "30000003",
  "30222203",
  "30222203",
  "30222203",
  "30222203",
  "30222203",
  "30000003",
  "33333333",
};

int main() {
  texte(4, 2, "MA TUILE");

  for (uint8_t x = 0; x < 8; x++) {
    poser(6 + x, 8, BLOC);
  }

  while (true) {
    image();
  }

  return 0;
}
```

CE QU'ON DOIT VOIR

Une rangée de huit blocs à cadre noir, au milieu de l'écran.

MA TUILE

□□□□□□□□

l'écran, au bout de vingt images

La même chose, à la souris

chercher un dessin : nom, étiquette, ★

BLOC

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BLOC

tuile n° 44 — poser(x, y, BLOC)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

BLOC ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

BLOC peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre...

18. Le dessin écrit là où on le pose

Les huit rangées dans le « poser » lui-même — sans nom, et sans numéro.

Une case de l'écran, c'est **huit rangées de huit pixels**, jamais autre chose. La leçon d'avant leur donnait un nom — `Tuile BLOC = {...}` — puis posait ce nom. Mais un nom, à l'arrivée, **c'est un numéro** : `poser(2, 2, BLOC)` écrit 44 dans la carte de fond.

Ces huit rangées peuvent s'écrire **dans le `poser` lui-même**. Le compilateur grave la tuile, lui trouve son numéro, et l'écrit à la case demandée — il n'y a plus ni nom à inventer ni numéro à suivre, et le dessin est **là où il apparaît**.

Ce n'est pas un remplacement, c'est un choix. Ce qui sert PARTOUT mérite son nom : le sol ci-dessous est posé vingt fois, et `SOL` le dit à chaque ligne. Ce qui ne sert QU'À UN ENDROIT n'apprend rien de plus en s'appelant `TUILE3`, et oblige à lire deux endroits pour en comprendre un seul.

Le même dessin ne se grave qu'une fois. Écris deux fois le même nuage à dix lignes d'écart : le compilateur reconnaît qu'il l'a déjà, et ne dépense qu'une tuile. Il reconnaît aussi un dessin déjà déclaré en `Tuile`, **à travers les deux écritures** — `"21111112"` et `"+-----"` sont les mêmes pixels. La console n'en tient que 256 : les gaspiller ne se verrait qu'au jour où il n'y en aurait plus.

Cela marche aux quatre endroits où un numéro de tuile est attendu : `poser`, `poserPanneau`, `sprite`, et `sprite16` — qui veut alors **seize rangées de seize signes**, et les découpe lui-même en quatre tuiles. Ailleurs, des accolades restent une faute, et le compilateur le dit avec sa ligne.

Une chose que le dessin sur place ne fait pas : **il n'apparaît pas dans l'atelier**. Celui-ci lit les `Tuile NOM = {...}` du texte, et il ne saurait pas où réécrire un dessin qui n'a pas de nom. Ce qu'on veut peindre à la souris, on le nomme.

LE PROGRAMME — 1440 OCTETS

```
Tuile SOL = {
  "22222222",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "22222222",
};

int main() {
  texte(4, 2, "SUR PLACE");

  // Le sol sert partout : il garde son nom.
  for (uint8_t x = 0; x < 20; x++) {
    poser(x, 12, SOL);
  }

  // La porte ne sert qu'ici : ses huit rangées sont ici.
  poser(9, 11, {
    "#####",
    "#-----#",
    "#-####-#",
    "#-#-#-#-",
    "#-#-#-#-",
    "#-#-#-#-",
    "#-#-#-#-",
    "#-#-#-#-",
    "#####",
  });

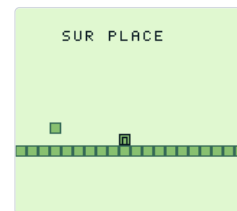
  // Le dessin de SOL, redit dans l'autre écriture : aucune tuile de plus.
  poser(3, 10, {
    "+++++++",
    "+-----+",
    "+-----+",
    "+-----+",
    "+-----+",
    "+-----+",
    "+-----+",
    "+++++++",
  });

  while (true) {
    image();
  }

  return 0;
}
```

CE QU'ON DOIT VOIR

Un sol de vingt cases, une porte posée au-dessus, et à gauche un pavé qui reprend le dessin du sol.



l'écran, au bout de vingt images

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

SOL

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

SOL

tuile n° 44 — poser(x, y, SO

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

SOL ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

SOL peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

19. Intégrer une carte dans son projet

Une carte dessinée à la souris est une fonction du programme : on l'appelle, et le décor apparaît.

Ce programme a les trois sortes de dessins du jeu : une **tuile de 8 × 8** (`Tuile ARBRE`), un **personnage de 16 × 16** (`Perso HEROS` — quatre tuiles d'un coup), et une **carte** (`VILLAGE` , tout un écran). Les deux premiers se peignent dans  **Les tuiles** ; la carte, dans  **La carte**. Essaie sur chacun les palettes, la sélection et le déplacement.

Une **carte**, c'est un décor entier dessiné à la souris, dans l'onglet  La carte. L'atelier ne la garde pas de son côté : il l'**écrit dans ton programme**, sous la forme d'une **fonction** — ici `void VILLAGE()`, entre deux commentaires `/* --- carte "VILLAGE" --- */` qui la marquent. C'est grâce à eux qu'il la retrouve et la réécrit quand tu dessines.

La créer : onglet  La carte, « + carte », et un nom en majuscules — VILLAGE, MAISON, NIVEAU1... Puis dessine : chaque carré de 8 × 8 que tu remplis devient une ligne `poser(colonne, ligne, {...})`, avec ses huit rangées. Un carré laissé vide n'est pas écrit : il ne coûte rien.

Lire le code de la carte : dans `poser(9, 10, {...})`, **9 est la colonne** (0 tout à gauche, 19 tout à droite) et **10 la ligne** (0 tout en haut, 17 tout en bas). Une case fait 8 × 8 pixels : la case (9, 10) commence donc au pixel x = 72, y = 80. Les **huit textes entre accolades** sont les huit rangées de pixels du carré, de haut en bas, et **chaque chiffre est un pixel** : 0 le plus clair, 1, 2, puis 3 le plus sombre. Regarde le toit, case (9, 9) : ses « 3 » dessinent la pente, rangée après rangée. En couleur, une ligne `teindre(colonne, ligne, palette)` dit en plus quelle palette prend la case.

Lire la carte pendant le jeu : `lire(colonne, ligne)` rend le **numéro de la tuile** affichée à cet endroit — 0 si la case est vide. C'est ainsi qu'un jeu sait si le joueur arrive sur un mur, sur le sol ou dans le vide. Le programme le montre en bas de l'écran : il relit la case (9, 11), du sol, et l'écrit avec `nombre()` ; la case (5, 11), vide, donne 0.

Une fonction ne fait rien tant qu'on ne l'appelle pas. C'est l'oubli le plus fréquent : la carte est dessinée, le programme compile, et l'écran reste vide. Il manque `VILLAGE();` dans `main`, **avant** la boucle du jeu — une seule fois : un décor reste à l'écran tant qu'on ne l'efface pas.

Essaie : mets `//` devant `VILLAGE();`. Le titre reste, la maison disparaît. Enlève-le : elle revient. Tout le décor tient dans cette seule ligne.

Plusieurs cartes : chacune a sa fonction. Pour passer de l'une à l'autre, on efface puis on appelle l'autre : `effacer();` puis `MAISON();`. Les huit carrés du sol, identiques, ne prennent **qu'une tuile** : le compilateur reconnaît un dessin qu'il a déjà.

Copier, déplacer : dans l'atelier de la carte, l'outil  **Sélection** choisit une zone — un rectangle **libre** tracé à la souris, ou d'un clic **un carré de 8 × 8** ou **un bloc de 16 × 16**, calés sur la grille. **Attrape** la zone et **glisse-la** : elle se déplace, et retombe sur la case la plus proche ; sa place redevient le fond. **Ctrl+C**, **Ctrl+X** et **Ctrl+V** la copient, la coupent et la collent. Dans l'atelier des tuiles,  **Sélectionner et déplacer** fait de même à l'intérieur d'une tuile de 8 × 8 ou d'un personnage de 16 × 16, et  **Copier le dessin**,  **Coller ici** et  **Dupliquer** copient un dessin entier. Chaque geste s'annule avec **Ctrl+Z**.

Un jeu, sans écrire la boucle : dans  La carte, « ► Jouer la scène » écrit `main` pour toi — la carte, les murs, le joueur qui bouge, les acteurs et les événements. Tes propres lignes de `main` sont alors remplacées ; l'atelier demande avant de le faire.

Poser une tuile, poser un personnage : `poser(6, 10, ARBRE);` met l'arbre dans la case (6, 10), et `teindre(6, 10, 1);` lui donne la palette 1 — celle de la forêt. `sprite16(0, 60, 72, HEROS);` pose le personnage **au pixel près**, en x = 60, y = 72 : un personnage n'est pas sur la grille, il se déplace librement par-dessus le décor.

À toi : descends à l'atelier de la carte, dessine un arbre à côté de la maison, et regarde un nouveau `poser(...)` apparaître dans `VILLAGE()`.

```
/* Une TUILE de 8 × 8 : un arbre. Elle se peint dans ■ Les tuiles. */
Tuile ARBRE = {
  "00111100",
  "01122110",
  "11222211",
  "12222221",
  "11222211",
  "01133110",
  "00033000",
  "00333300",
};

/* Un PERSONNAGE de 16 × 16 : quatre tuiles de 8 × 8 d'un coup. */
Perso HEROS = {
  "000003333333000000",
  "00003222222300000",
  "0003222222230000",
  "0003211221123000",
  "0003211221123000",
  "0003222222230000",
  "0000322332230000",
  "0000033333300000",
  "0003333333330000",
  "0032222222223000",
  "0321222222221230",
  "0321222222221230",
  "0003222222230000",
  "0003322003223000",
  "0003220000223000",
  "0033330000333300",
};

/* --- carte "VILLAGE" --- */
void VILLAGE() {
  poser(9, 9, {
    "00000033",
    "00003322",
    "00332222",
    "33222222",
    "32222222",
    "33333333",
    "31111113",
    "31111113",
  });
  poser(10, 9, {
    "33000000",
    "22330000",
    "22223300",
    "22222333",
    "22222223",
    "33333333",
    "00111113",
    "31111113",
  });
  poser(9, 10, {
    "31111113",
    "31133113",
    "31133113",
    "31111113",
    "31111333",
    "31111311",
    "31111311",
    "33333311",
  });
  poser(10, 10, {
    "31111113",
    "31111113",
    "33311113",
    "11311113",
    "11311113",
    "11311113",
    "11311113",
    "11333333",
  });
  poser(6, 11, {
    "33333333",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
  });
  poser(7, 11, {
    "33333333",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
  });
  poser(8, 11, {
    "33333333",
    "21212121",
    "12121212",
```

CE QU’ON DOIT VOIR

Le titre « MON VILLAGE », la maison sur huit carrés de sol, un arbre de chaque côté (dans la palette de la forêt), le personnage devant la maison, et en bas « SOL 053 » et « VIDE 000 ».



l'écran, au bout de vingt images

```

"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(9, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(10, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(11, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(12, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(13, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
}
/* --- fin de la carte "VILLAGE" --- */

int main() {
/* Les couleurs : la palette 1 pour l'arbre (une forêt). */
couleurFond(1, 0, 22, 31, 14);
couleurFond(1, 1, 10, 24, 6);
couleurFond(1, 2, 4, 14, 2);
couleurFond(1, 3, 12, 6, 2);

texte(4, 3, "MON VILLAGE");

VILLAGE(); // la carte : SANS CET APPEL, RIEN NE S'AFFICHE

/* La tuile ARBRE, posée à côté de la maison, dans la palette 1. */
poser(6, 10, ARBRE);
teindre(6, 10, 1);
poser(13, 10, ARBRE);
teindre(13, 10, 1);

/* Le personnage HEROS, au pixel près : un lutin de 16 x 16. */
sprite16(0, 60, 72, HEROS);

/* Relire la carte : le numéro de la tuile à une case, 0 si elle est vide. */
texte(1, 15, "SOL");
nombre(6, 15, lire(9, 11));
texte(1, 16, "VIDE");
nombre(6, 16, lire(3, 11));

while (true) {
image();
}

return 0;
}

```

20. Les couleurs : palettes, variétés et thèmes

Une tuile porte des numéros ; la palette de sa case leur donne leurs couleurs — la même tuile peut être de trois couleurs à la fois.

La règle de la console : un carré de 8×8 — une tuile — n'a que **4 couleurs**. Pas une de plus, même sur Game Boy Color : chaque carré prend **une palette** de 4 couleurs. Le carré d'à côté peut en prendre une autre : c'est ainsi qu'un écran entier affiche bien plus de 4 couleurs.

Le jeu a 8 palettes, numérotées de 0 à 7, et la **0 est la normale** (les verts de la Game Boy). Une tuile ne contient pas de couleurs : elle contient des **numéros**, de 0 à 3. C'est la palette de la case où on la pose qui dit de quelle couleur est chaque numéro.

Le programme le montre : la même tuile `BRIQUE`, posée trois fois. `teindre(colonne, ligne, palette)` donne sa palette à une case : la palette 1 (désert), la palette 2 (océan), et la dernière n'a pas de `teindre` — elle reste dans la palette 0, la normale. **Une seule tuile, trois couleurs** : c'est tout l'intérêt des palettes.

`couleurFond(palette, numéro, rouge, vert, bleu)` choisit une couleur : chaque composante va de 0 à 31. Chaque palette va du plus **clair** (numéro 0) au plus **sombre** (numéro 3) — le jeu garde ainsi une version lisible en quatre nuances.

Dans l'atelier, sans écrire de code : les 8 palettes sont en haut de l'atelier des tuiles et de la carte. Clique une couleur, peins : ce que tu peins **prend sa palette**. «  Changer cette couleur » change une couleur **pour ce dessin seulement** : si d'autres tuiles (ou le texte) partagent sa palette, l'atelier lui en donne une copie dans une palette libre. «  **Variétés** » met dans une palette l'une des 11 palettes toutes prêtes (0 normale, campagne, bonbon, plage, forêt, glace, volcan, nuit, désert, océan, automne).

«  **Thèmes** » change d'un coup les 8 palettes du décor **et** les 8 des personnages, avec des couleurs **qui vont ensemble** : forêt, désert, glace, volcan, océan, hanté, bonbon — ou 0, la normale. Elles sont calculées : chaque palette part d'une teinte, et toutes descendent du clair au sombre par les mêmes quatre marches de lumière. **Ctrl+Z** revient en arrière.

Les personnages ont leurs 8 palettes à eux, de **3 couleurs + le transparent** (le numéro 0 ne s'affiche jamais : le décor se voit au travers). Un personnage de 16×16 est fait de 4 carrés : chacun peut avoir sa palette — jusqu'à 12 couleurs.

À toi : ajoute une quatrième brique en (10, 8) avec `teindre(10, 8, 1)` — elle prend les couleurs du désert. Puis, dans l'atelier, essaie «  Thèmes » et regarde les trois briques changer ensemble.

LE PROGRAMME — 1581 OCTETS

```
/* Une seule tuile : quatre NUMÉROS de couleur, de 0 (clair) à 3 (sombre). */
Tuile BRIQUE = {
    "33333333",
    "21112111",
    "21112111",
    "33333333",
    "11121112",
    "11121112",
    "33333333",
    "00000000",
};

int main() {
    /* La palette 1 : le désert – du plus clair au plus sombre. */
    couleurFond(1, 0, 31, 29, 22);
    couleurFond(1, 1, 28, 22, 10);
    couleurFond(1, 2, 20, 12, 4);
    couleurFond(1, 3, 8, 4, 2);
    /* La palette 2 : l'océan. */
    couleurFond(2, 0, 26, 31, 31);
    couleurFond(2, 1, 8, 24, 30);
    couleurFond(2, 2, 2, 12, 24);
    couleurFond(2, 3, 0, 3, 10);

    texte(3, 3, "UNE TUILE");
    texte(3, 4, "TROIS PALETTES");

    poser(4, 8, BRIQUE);
    teindre(4, 8, 1); // la case (4, 8) prend la palette 1 : le désert
    poser(6, 8, BRIQUE);
    teindre(6, 8, 2); // la même tuile, dans la palette 2 : l'océan
    poser(8, 8, BRIQUE); // pas de teindre : la palette 0, la normale

    while (true) {
        image();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Trois fois la même brique :
couleur sable (palette 1),
bleue (palette 2), et verte
(palette 0, la normale).

UNE TUILE
TROIS PALETTES



l'écran, au bout de vingt images

The screenshot shows the 'Brique' game editor interface. At the top, there are three search bars with the placeholder text 'chercher un dessin : nom, étiquette, ★'. Below these is a toolbar with a 'BRIQUE' icon and two buttons: '+ tuile 8 × 8' and '+ perso 16 × 16'. The main workspace displays a 4x4 grid of colored squares (nuances) representing the Game Boy palette. To the right of the grid is a button 'Passer en couleur'. Below the grid, there are several tool buttons: 'Peindre ✓', 'Gomme', 'Ligne', 'Rectangle', 'Cercle', 'Formes : contour', 'Remplir', 'Pipette', 'Sélectionner et déplacer', 'Miroir', 'Tourner', 'Copier le dessin', 'Coller ici', 'Dupliquer', and 'Effacer la tuile'. The bottom section features an 'Animation' section with a description and a list of animation options: 'Eau (vague)', 'Feu (flamme)', 'Herbe (vent)', 'Clignoter', and 'À la main'. Below this is a 'Police' section with a description and a dropdown menu to 'remplacer la lettre...'. The interface is dark-themed and includes various icons and labels for different elements.

21. Poser tes tuiles à la souris

Voir les carrés, tracer une zone — et le code s’écrit tout seul.

Jusqu’ici il fallait écrire `poser(6, 12, MUR);` à la main, case par case. Pour un sol de vingt cases, c’est vingt lignes, et une faute de frappe ne se voit pas.

Le **plan** ci-dessous montre les tuiles du programme telles qu’elles sont — des carrés, pas des numéros. On en choisit une, on **trace une zone** sur l’écran, et le programme reçoit ses `poser()`. La gomme remet du vide.

Le code écrit porte le **nom** de la tuile, jamais son numéro. C’est ce qui compte : un numéro devient faux le jour où l’on ajoute un dessin avant lui, alors qu’un nom reste juste. C’est le compilateur qui traduit, et c’est son travail.

L’outil ne possède que le bloc entre `// PLAN` et `// FIN DU PLAN`. Tout ce que tu écris ailleurs est à toi, et il n’y touche pas.

LE PROGRAMME — 1493 OCTETS

```
Tuile MUR = {
  "33333333",
  "32222223",
  "32122123",
  "32222223",
  "32221223",
  "32122223",
  "32222223",
  "33333333",
};

Tuile CAISSE = {
  "33333333",
  "31133113",
  "31133113",
  "33333333",
  "33333333",
  "31133113",
  "31133113",
  "33333333",
};

int main() {
  texte(4, 1, "TRACE UNE ZONE");

  // PLAN
  poser(0, 16, MUR);
  poser(1, 16, MUR);
  poser(2, 16, MUR);
  poser(3, 16, MUR);
  poser(9, 12, CAISSE);
  poser(10, 12, CAISSE);
  // FIN DU PLAN

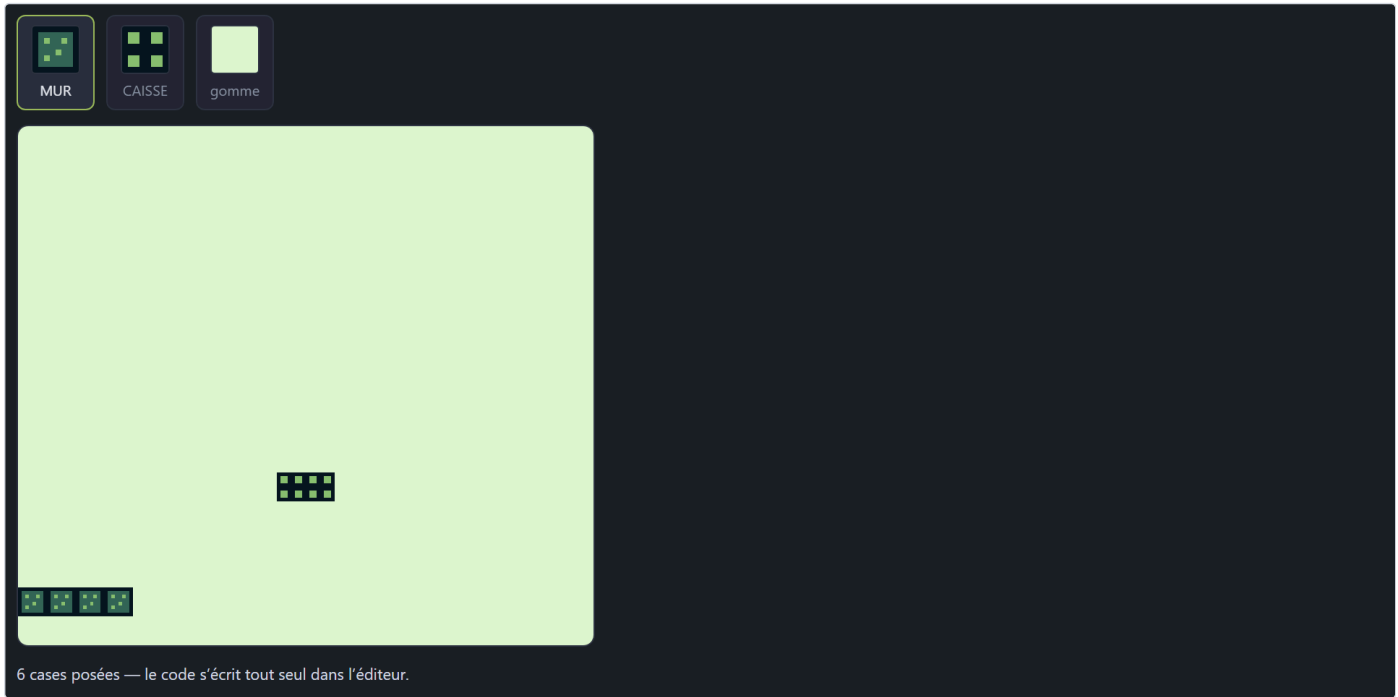
  while (true) {
    image();
  }

  return 0;
}
```

CE QU’ON DOIT VOIR



La même chose, à la souris



22. Poser une tuile

Une lettre à l'écran est une tuile comme une autre.

Les deux lignes affichent la même chose. `texte()` traduit des lettres en numéros de tuiles ; `poser(colonne, ligne, tuile)` pose le numéro directement.

La tuile 1 est le `A` de la police fournie, la 2 est un `B`. Il n'y a pas de magie : dessiner sa propre tuile, ce sera **ajouter un numéro à cette même suite**.

`lire(colonne, ligne)` fait l'inverse — il rend le numéro affiché à cet endroit. C'est ce qui servira pour les collisions.

LE PROGRAMME — 1401 OCTETS

```
int main() {
    texte(2, 4, "AVEC TEXTE ABC");

    texte(2, 8, "AVEC POSER");
    poser(14, 8, 1);
    poser(15, 8, 2);
    poser(16, 8, 3);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le même « ABC » sur les deux lignes.

```
AVEC TEXTE ABC
AVEC POSER ABC
```

l'écran, au bout de vingt images

23. Quatre nuances, quatre signes

Une tuile s’écrit en huit rangées de huit caractères.

. clair, **-** moyen clair, **+** moyen sombre, **#** sombre. On peut aussi écrire **0123** — c’est la même chose.

Le nom **BRIQUE** devient un numéro de tuile : il s’écrit partout où l’on attend un numéro. Le compilateur grave les seize octets dans la cartouche et range la tuile à sa place au démarrage.

Dans la page, cette même tuile se dessine à la souris — l’atelier écrit exactement ces huit lignes dans le programme. Les deux chemins mènent au même endroit.

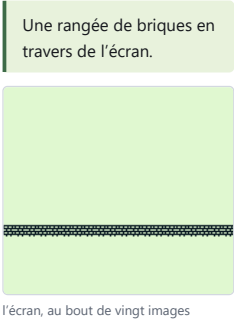
LE PROGRAMME — 1348 OCTETS

```
Tuile BRIQUE = {
    "#####",
    "#..#..#",
    "#####",
    "..#..#.",
    "#####",
    "#..#..#",
    "#####",
    "..#..#.",
};

int main() {
    for (uint8_t x = 0; x < 20; x++) poser(x, 12, BRIQUE);

    while (true) {
        image();
    }
}
```

CE QU’ON DOIT VOIR



La même chose, à la souris

Game Boy en C++

← l’atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

BRIQUE

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d’origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BRIQUE

tuile n° 44 — poser(x, y, BRIQUE)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

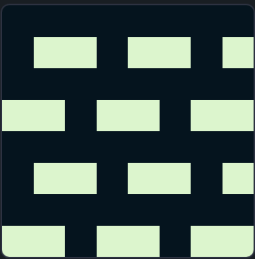
Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile



taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

BRIQUE ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

BRIQUE peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre...

24. Une rangée, puis un mur

Quatre-vingts appels en quatre lignes.

Écrire les quatre-vingts `poser()` à la main, ce sont quatre-vingts occasions de se tromper d'une case — et le jour où l'écran change de taille, il faut tout relire. Deux boucles pour les bords horizontaux, deux pour les verticaux. Les coins sont posés deux fois : cela ne coûte rien, et l'écrire autrement demanderait quatre conditions.

LE PROGRAMME — 1477 OCTETS

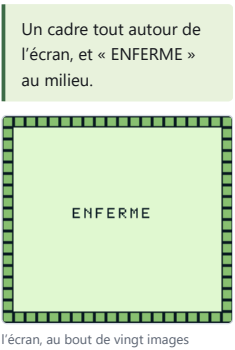
```
Tuile MUR = {
    "#####",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#####",
};

int main() {
    for (uint8_t x = 0; x < 20; x++) { poser(x, 0, MUR); poser(x, 17, MUR); }
    for (uint8_t y = 0; y < 18; y++) { poser(0, y, MUR); poser(19, y, MUR); }

    texte(6, 8, "ENFERME");

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR



La même chose, à la souris

Game Boy en C++

← l'atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★



MUR

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

MUR

tuile n° 44 — poser(x, y, MU

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

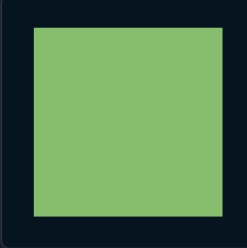
Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile



taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

MUR ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

MUR peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

25. Un damier

Le reste d’une division, et le motif apparaît.

`(x + y) % 2` vaut 0 une case sur deux, en quinconce — c’est la définition même d’un damier, et elle tient en trois caractères.

`a ? b : c` choisit en une expression. On aurait pu écrire un `if` sur quatre lignes ; ici la ligne dit « pose ceci ou cela », et c’est ce qu’on veut lire.

LE PROGRAMME — 1427 OCTETS

```
Tuile CLAIR = { ".....", ".....", ".....", ".....",
               ".....", ".....", ".....", "....." };
Tuile SOMBRE = { "#####", "#####", "#####", "#####",
                "#####", "#####", "#####", "#####" };

int main() {
    for (uint8_t y = 0; y < 18; y++) {
        for (uint8_t x = 0; x < 20; x++) {
            poser(x, y, (x + y) % 2 == 0 ? CLAIR : SOMBRE);
        }
    }

    while (true) {
        image();
    }
}
```

CE QU’ON DOIT VOIR



La même chose, à la souris

Game Boy en C++

← l'atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

CLAIR

SOMBRE

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d’origine, et rien de plus. Passer en couleur

nuance 3

Nuances

CLAIR

tuile n° 44 — poser(x, y, CLAIR

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

CLAIR ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

CLAIR peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre...

26. Un lutin, hors de la grille

Le fond est une grille de 8 en 8 ; un lutin ne l'est pas.

`sprite(numero, x, y, tuile)` pose un lutin **en pixels**, et non en cases : il peut se trouver à cheval entre deux tuiles du fond. C'est ce qui rend un mouvement doux possible.

Le premier argument est le **numéro du lutin**, de 0 à 39. C'est lui qu'on rappelle pour le déplacer, ou qu'on donne à `cache(n)` pour l'ôter.

LE PROGRAMME — 1332 OCTETS

```
Tuile HEROS = {
  "..###..",
  ".#-##-#",
  "#####",
  "#.####.",
  "#####",
  "..#...#",
  "#...#.",
  "#####",
};

int main() {
  sprite(0, 76, 68, HEROS);
  texte(3, 14, "IL FLOTTE");

  while (true) {
    image();
  }
}
```

CE QU'ON DOIT VOIR

Un personnage au milieu,
entre deux cases de la
grille.



l'écran, au bout de vingt images

chercher un dessin : nom, étiquette. ★

Game Boy en C++

← l'atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

HEROS

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

HEROS

tuile n° 44 — poser(x, y, HERO

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

HEROS ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

HEROS peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

27. Le front, ou pourquoi un appui compte trois fois

Agir au moment précis où le bouton s'enfonce.

Un jeu tourne à **soixante images par seconde**. Un appui dure au moins cinq images : lu au niveau, il compte cinq fois. Sur un menu, on traverse trois lignes sans rien voir.

La réponse tient en une variable : l'état du bouton à l'image **précédente**. On agit quand il vaut 1 maintenant et valait 0 avant — ce qu'on appelle agir **au front**.

Le programme montre les deux compteurs côte à côte. Tiens A : celui du haut s'emballe, celui du bas monte d'un seul cran.

LE PROGRAMME — 1501 OCTETS

```
int main() {
    uint8_t aAvant = 0;
    uint8_t niveau = 0;
    uint8_t front = 0;

    texte(1, 4, "NIVEAU");
    texte(1, 8, "FRONT");

    while (true) {
        image();

        uint8_t a = bouton(A);

        if (a && niveau < 9) niveau++;
        if (a && !aAvant && front < 9) front++;

        aAvant = a;

        poser(10, 4, 27 + niveau);
        poser(10, 8, 27 + front);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Un appui sur A : le compteur « NIVEAU » saute, « FRONT » avance d'un.

N I V E A U 0

F R O N T 0

l'écran, au bout de vingt images

28. Un lutin, au pixel près

Ce qui sépare un jeu d'action d'un jeu de cases.

Le décor vit sur une grille de huit pixels. Un personnage, non : s'il ne pouvait s'arrêter que sur des multiples de huit, il ne sauterait pas, il **se téléporterait**.

`sprite(numero, x, y, tuile)` pose un carré de huit sur huit **où l'on veut**. Le matériel en a quarante, numérotés de 0 à 39.

Il n'y a rien à faire pour l'effacer : on le repose ailleurs à chaque image, et il a bougé.

LE PROGRAMME — 1554 OCTETS

```
Tuile BALLE = {
    "00333300",
    "03222230",
    "32222223",
    "32222223",
    "32222223",
    "32222223",
    "03222230",
    "00333300",
};

int main() {
    uint8_t x = 20;
    uint8_t y = 60;

    texte(3, 2, "CROIX POUR BOUGER");

    while (true) {
        image();

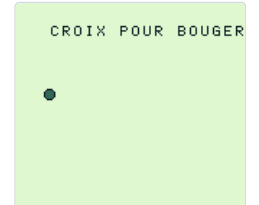
        if (bouton(DROITE) && x < 150) x += 2;
        if (bouton(GAUCHE) && x > 2) x -= 2;
        if (bouton(BAS) && y < 135) y += 2;
        if (bouton(HAUT) && y > 2) y -= 2;

        sprite(0, x, y, BALLE);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Une boule qu'on déplace
à la croix, sans à-coups.



l'écran, au bout de vingt images

29. Un objet qui tombe

La pesanteur, en seizièmes de pixel.

Un objet qui tombe ne descend pas à vitesse constante : il **accélère**. Toute la physique tient en deux additions — la pesanteur s'ajoute à la vitesse, la vitesse s'ajoute à la position — et c'est de les faire **dans cet ordre**, à chaque image, qui fabrique la courbe d'une chute.

Reste un ennui : la console **ne connaît pas la virgule**. Au premier instant, la balle avance de deux seizièmes de pixel — arrondi, cela fait zéro, et elle ne partirait jamais.

On garde donc la position en **deux morceaux** : les pixels dans `y`, et les seizièmes de pixel dans `frac`. Seul `y` est montré ; `frac` est la mémoire de ce qui n'est pas encore un pixel entier, et qui le deviendra.

`frac >> 4` est une division par seize, `frac & 15` en est le reste. Le processeur ne sait pas diviser ; décaler de quatre bits, en revanche, lui coûte un seul tour d'horloge.

La chute dure **trois quarts de seconde**. Une démonstration qui ne se joue qu'une fois ne se voit pas : on compte donc les images passées au sol, et **au bout de quarante la balle est renvoyée en haut**. Elle tombe en boucle, et l'on peut la regarder tomber autant qu'on veut.

La relance remet **les trois** : la hauteur, les seizièmes et la vitesse. Le sol vient de les mettre à zéro une ligne plus haut — mais un départ qui compte sur le ménage fait par quelqu'un d'autre n'est pas un départ, et le jour où l'on relancera d'ailleurs la balle partira du ciel avec la vitesse qu'elle avait en s'écrasant.

`G` vaut 2, et ce nombre est le seul réglage : à 1 la balle flotte comme sur la Lune, à 6 elle tombe comme une enclume. C'est là, et nulle part ailleurs, que se décide ce qu'on ressent.

Et 9,81 dans tout ça ? Elle est dans `G`. Le processeur n'a ni virgule ni mètres : `G` se compte en seizièmes de pixel par image, par image, soit 450 pixels par seconde carrée à soixante images par seconde. Décidez qu'un mètre fait 46 pixels, et ces 450 pixels deviennent **9,81 m/s²** — l'écran mesure alors 3,10 m de haut, et la chute de 2,60 m dure les trois quarts de seconde que la vraie physique annonce. La pesanteur est la bonne ; c'est l'unité qui a changé.

LE PROGRAMME — 1514 OCTETS

```
Tuile BALLE = {
    "00333300",
    "03222230",
    "32222223",
    "32222223",
    "32222223",
    "32222223",
    "03222230",
    "00333300",
};

const uint8_t CIEL = 8; // le haut de la balle quand on la lâche
const uint8_t SOL = 128; // le haut de la balle quand elle touche terre
const uint8_t G = 2; // la pesanteur, en seizièmes de pixel par image
const uint8_t ATTENTE = 40; // les images passées au sol avant de recommencer

int main() {
    uint8_t y = CIEL; // les pixels – ce que l'écran montre
    uint8_t frac = 0; // les seizièmes – ce qu'il ne sait pas montrer
    uint8_t vy = 0; // la vitesse, en seizièmes de pixel par image
    uint8_t pose = 0; // depuis combien d'images elle est posée

    texte(4, 2, "CHUTE LIBRE");
    texte(2, 4, "ET CA RECOMMENCE");
    texte(0, 17, "#####");

    while (true) {
        image();

        vy = vy + G;

        frac = frac + vy;
        y = y + (frac >> 4);
        frac = frac & 15;

        if (y >= SOL) {
            y = SOL;
            frac = 0;
            vy = 0;
            pose = pose + 1;
        }

        if (pose > ATTENTE) {
            y = CIEL;
            frac = 0;
            vy = 0;
            pose = 0;
        }

        sprite(0, 76, y, BALLE);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

La balle part lentement, prend de la vitesse, s'arrête net sur le sol — puis remonte d'un coup et recommence.

CHUTE LIBRE
ET CA RECOMMENCE



l'écran, au bout de vingt images

30. Un personnage de seize

Huit pixels de côté, c'est petit pour un héros.

Une **Tuile** fait huit pixels de côté. C'est la taille d'une case du décor, et c'est bien pour un mur ou une pièce — mais un personnage y tient mal. Un **Perso** en fait **seize** **sur seize** : seize rangées de seize signes, écrites comme celles d'une tuile.

Le matériel, lui, ne connaît que des carrés de huit. Un **Perso** occupe donc QUATRE numéros de tuile, dans cet ordre : haut-gauche, haut-droite, bas-gauche, bas-droite. Si **HEROS** vaut 44, il tient les numéros 44, 45, 46 et 47 — et le dessin suivant du programme commence à 48.

`sprite16(n, x, y, HEROS)` pose les **quatre lutins en un seul appel**, en carré, au pixel près. Le prix est dit franchement : il coûte quatre lutins des quarante, là où un `sprite()` n'en coûte qu'un. Une troupe de dix personnages de seize, c'est déjà tout le matériel. `cacher16(n)` les ôte tous les quatre.

Sur le fond, il n'y a pas de `poser16()` — une case du décor fait huit pixels, et le personnage en couvre quatre. On pose donc ses quarts un par un, et l'ordre est celui des numéros : `poser(10, 8, HEROS)`; `poser(11, 8, HEROS + 1)`; `poser(10, 9, HEROS + 2)`; `poser(11, 9, HEROS + 3)`. C'est ce que fait le programme ci-dessous, en plus du lutin.

Dans l'atelier, « + **perso 16 x 16** » en ajoute un, et la grille passe à seize sur seize. Le reste ne change pas : on peint, et le programme se réécrit.

LE PROGRAMME — 1679 OCTETS

```
Perso HEROS = {
    "...####.....",
    ".#####.....",
    "#####.....",
    "#+#+-#++#.....",
    "#+#+-#++#.....",
    "#+#+-#++#.....",
    "#####.....",
    "#####.....",
    "#++++++#.....",
    "#++++++#.....",
    "#++++++#.....",
    "#++++++#.....",
    "#++++++#.....",
    "#++++++#.....",
    "#++++++#.....",
    "#.#. #. #. #. #.",
    ".....",
    ".....",
    ".....",
    ".....",
    ".....",
    ".....",
    ".....",
};

uint8_t x = 60;

int main() {
    texte(1, 1, "GAUCHE DROITE");

    /* Le même personnage, POSÉ sur le fond : ses quatre quarts, un par case. */
    poser(10, 8, HEROS);
    poser(11, 8, HEROS + 1);
    poser(10, 9, HEROS + 2);
    poser(11, 9, HEROS + 3);

    while (true) {
        image();

        if (bouton(DROITE) && x < 144) x++;
        if (bouton(GAUCHE) && x > 8) x--;

        /* En lutin : quatre lutins posés en carré, en un seul appel. */
        sprite16(0, x, 100, HEROS);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Un héros posé sur le décor, et le même en lutin plus bas — GAUCHE et DROITE le déplacent au pixel près.

GAUCHE DROITE



l'écran, au bout de vingt images

chercher un dessin : nom, étiquette, ★

Game Boy en C++

← l'atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★



HEROS

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

HEROS

tuiles n° 44 à 47 — sprite16(n, x, y, HERO

– zoom

11×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer le personnage



taille réelle — 16 × 16 pixels

étiquettes

décor mur...

Animation

Un personnage s'anime dans La carte : Le joueur (la marche, et ses états — attente, course, attaque, blessure...) ou Les acteurs (« alterne avec »). Fais-lui d'abord ses images : « Dupliquer » HEROS, puis change la copie. Ici, ce sont les tuiles de 8 × 8 qui s'animent (l'eau, le feu, l'herbe).

31. Déplacer un lutin

Une position, quatre boutons — et surtout pas de front.

Ici l'on veut que ça bouge **tant que le doigt reste appuyé**. Pas de front, donc : le front sert à ce qui doit arriver une fois, pas à ce qui doit durer.

Savoir lequel des deux on veut est la moitié du travail sur cette machine.

LE PROGRAMME — 1412 OCTETS

```
Tuile HEROS = {
    "..####.", ".#-##-#.", "#####", "#.####.",
    "#####", "..#..#..", ".#...#.", "##...##",
};

int main() {
    uint8_t x = 76;
    uint8_t y = 68;

    while (true) {
        if (bouton(GAUCHE)) x--;
        if (bouton(DROITE)) x++;
        if (bouton(HAUT)) y--;
        if (bouton(BAS)) y++;

        sprite(0, x, y, HEROS);
        image();
    }
}
```

CE QU'ON DOIT VOIR

La croix déplace le personnage, pixel par pixel.



l'écran, au bout de vingt images

32. Le garder dans l'écran

Un octet ne prévient pas quand il repasse par zéro.

Sans le `x > GAUCHE_MAX`, arriver à 0 puis décrémenter donne **255** : le personnage réapparaît à droite. C'est le bogue le plus fréquent de cette machine, et il ne fait jamais planter — il téléporte, silencieusement.

Tester **avant** de bouger, et non corriger après. `if (x < 8) x = 8;` marche aussi, mais laisse la variable fausse pendant une ligne — et cette ligne finit toujours par en attirer d'autres.

LE PROGRAMME — 1420 OCTETS

```
Tuile HEROS = {
    ".####.", ".#-##-#.", "#####", "#.###.#",
    "#####", "..#..#..", ".#...#.", "##...##",
};

const uint8_t GAUCHE_MAX = 8;
const uint8_t DROITE_MAX = 152;

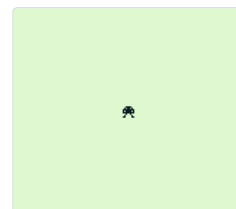
int main() {
    uint8_t x = 76;

    while (true) {
        if (bouton(GAUCHE) && x > GAUCHE_MAX) x--;
        if (bouton(DROITE) && x < DROITE_MAX) x++;

        sprite(0, x, 68, HEROS);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le personnage s'arrête
aux bords au lieu de
réapparaître en face.



l'écran, au bout de vingt images

33. Une vitesse qui monte

Accélérer, c'est une variable de plus.

Un objet qui accélère, c'est une **vitesse** qui monte. Tout ce qui suit — le saut, la pesanteur — n'est que cela, avec un signe en plus.

Elle ne montera pourtant **jamais jusqu'à 5** : plus le caillou va vite, plus tôt il touche le bas, et la chute suivante repart à 1. Le plafond écrit dans le code n'est pas celui qu'on observe — c'est le genre d'écart qu'on ne voit qu'en faisant tourner.

Pourquoi `depuis` **et non** `images() % 20 == 0`. Les deux paraissent équivalents, et ils ne le sont pas. Tester un multiple suppose que la boucle tourne **exactement une fois par image** ; dès que le corps s'alourdit, un tour franchit deux images d'un coup, le multiple est enjambé, et l'accélération n'arrive jamais.

Un compteur incrémenté à chaque tour et comparé avec `>=` ne peut pas rater son échéance. C'est la forme à prendre par défaut.

LE PROGRAMME — 1416 OCTETS

```
Tuile CAILLOU = {
    "..###..", ".#####", "#####", "#####",
    "#####", "#####", ".#####", ".#####",
};

int main() {
    uint8_t y = 0;
    uint8_t vitesse = 1;
    uint8_t depuis = 0;

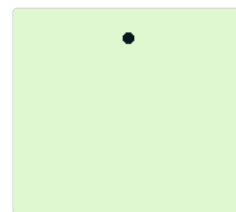
    while (true) {
        y += vitesse;
        if (y > 130) { y = 0; vitesse = 1; depuis = 0; }

        depuis++;
        if (depuis >= 20) { depuis = 0; if (vitesse < 5) vitesse++; }

        sprite(0, 76, y, CAILLOU);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un caillou qui tombe de plus en plus vite, et recommence.



l'écran, au bout de vingt images

34. Sauter, sans pesanteur

Compter les images qui restent à monter, plutôt qu'une vitesse signée.

Le front sert enfin : sans lui, garder **A** enfoncé relancerait le saut à chaque image, et le personnage collerait au plafond.

montee compte les images qui restent à monter. C'est plus simple qu'une vraie vitesse signée — et sur une machine **sans nombres négatifs**, plus simple veut dire plus juste.

La leçon « La pesanteur et le saut » reprend la même idée avec une vraie accélération.

LE PROGRAMME — 1462 OCTETS

```
Tuile HEROS = {
    "..####.", ".#-##-#.", "#####", "#.###.#",
    "#####", "..#.#..", ".#...#.", "##....##",
};

const uint8_t SOL = 120;

int main() {
    uint8_t y = SOL;
    uint8_t montee = 0;
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);

        /* On ne saute qu'au FRONT, et seulement depuis le sol. */
        if (appui && !avant && y == SOL) montee = 16;
        avant = appui;

        if (montee > 0) { y -= 2; montee--; }
        else if (y < SOL) y += 2;

        sprite(0, 76, y, HEROS);
        image();
    }
}
```

CE QU'ON DOIT VOIR

A fait sauter, et le personnage retombe au sol.



l'écran, au bout de vingt images

35. defiler(), et l'octet qui boucle

La carte fait 256 pixels — la même taille qu'un octet.

`defiler(x, y)` fait glisser tout le fond. La carte fait **256 pixels de côté** et revient toute seule à zéro.

`glisse` est un octet : il boucle exactement au bon moment, **sans qu'on ait rien à écrire**. C'est une coïncidence heureuse du matériel, et l'un des rares endroits où le débordement d'un octet est ce qu'on veut.

LE PROGRAMME — 1436 OCTETS

```
Tuile MOTIF = {
    "#-----", "-#-----", "--#-----", "---#-----",
    "----#---", "-----#--", "-----#-", "-----#",
};

int main() {
    for (uint8_t y = 0; y < 18; y++)
        for (uint8_t x = 0; x < 20; x++)
            poser(x, y, MOTIF);

    uint8_t glisse = 0;

    while (true) {
        if (bouton(DROITE)) glisse++;
        if (bouton(GAUCHE)) glisse--;
        defiler(glisse, 0);
        image();
    }
}
```

CE QU'ON DOIT VOIR

DROITE et GAUCHE font
glisser le motif sans fin.



l'écran, au bout de vingt images

36. Le son

Quatre voix, dont trois qu'on peut jouer.

La console a une puce sonore depuis toujours, et rien ne lui parlait. `note(voix, hauteur, duree, volume)` joue une note sur la voix **1** ou **2** — deux signaux carrés — et `bruit(duree, volume)` frappe sur la quatrième, celle du bruit : une explosion, un pas, un tir.

Les hauteurs portent des noms : `D04`, `RED4`, `MI4` ... de `D02` à `SI6`, cinq octaves. Un nom se relit ; un numéro de fréquence, non. La table est calculée à la compilation et gravée dans la cartouche — 120 octets pour tout le clavier.

La **durée** se compte en 256^e de seconde, de 1 à 64. Une durée de `0` veut dire « sans fin » : la note tient jusqu'à `silence(voix)`. Le **volume** va de 0 à 15.

Deux voix, c'est une mélodie et son accompagnement. Ici la voix 2 double la voix 1 une octave plus bas — `MELODIE[pas] - 12`, puisqu'une octave fait douze demi-tons.

LE PROGRAMME — 1583 OCTETS

```
const uint8_t MELODIE[] = { D04, MI4, SOL4, D05, SOL4, MI4, RE4, SOL4 };

uint8_t pas = 0;
uint8_t attente = 0;

int main() {
    texte(3, 5, "A POUR UN BRUIT");

    while (true) {
        image();

        attente++;

        if (attente > 24) {
            attente = 0;

            note(1, MELODIE[pas], 20, 12);
            note(2, MELODIE[pas] - 12, 20, 6);

            pas++;
            if (pas >= sizeof(MELODIE)) pas = 0;
        }

        if (bouton(A)) bruit(6, 12);
        if (bouton(START)) { silence(1); silence(2); }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Une mélodie qui tourne en boucle ; A frappe un bruit sec.

A POUR UN BRUIT

l'écran, au bout de vingt images

37. Écrire une musique

Une mélodie qui joue toute seule, pendant que le jeu tourne.

Un `Air` s'écrit comme une tuile se dessine : une suite de pas, dans le programme. Chaque pas est une note et son volume — `"D04 12"` —, ou l'un des deux pas qui n'en sont pas : `"--"` fait taire la voix, `"=="` laisse la note d'avant continuer.

C'est `"=="` qui fait les notes longues. Sans lui, une blanche s'écrit en jouant la même note à chaque pas, et l'oreille entendrait quatre coups au lieu d'une note tenue.

`jouer(voix, AIR, vitesse)` le lance. La **vitesse** est le nombre d'images que dure un pas : huit images font un peu moins de huit pas par seconde. Un quatrième argument, `1`, le fait recommencer sans fin.

Ensuite, **le programme n'a plus rien à tenir**. L'air avance tout seul, soixante fois par seconde, dans l'interruption de la console — même si le jeu, lui, est en retard. Un tempo qui ralentirait avec la charge du jeu ne serait pas un tempo.

La **partition ci-dessous** écrit ces lignes à la souris, et le bouton « Écouter » la fait entendre sans même compiler. Les deux voix chantantes sont la 1 et la 2 ; la quatrième, celle du bruit, se frappe avec `bruit(6, 10)`.

LE PROGRAMME — 1857 OCTETS

```
Air THEME = {
    "D04 12", "==",      "MI4 12", "==",      "SOL4 12", "==",      "D05 13", "==",
    "SI4 11", "==",      "SOL4 11", "==",      "MI4 11", "==",      "--",      "--",
};

Air BASSE = {
    "D03 8",  "==",      "==",      "==",      "SOL3 8",  "==",      "==",      "==",
    "MI3 8",  "==",      "==",      "==",      "--",      "--",      "--",      "--",
};

uint8_t joue = 0;

int main() {
    texte(4, 5, "LA MUSIQUE");
    texte(1, 8, "A  ARRETER OU JOUER");

    jouer(1, THEME, 8, 1);
    jouer(2, BASSE, 8, 1);
    joue = 1;

    while (true) {
        image();

        if (bouton(A) && joue) {
            silence(1);
            silence(2);
            joue = 0;
        }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Rien de neuf à l'écran — mais deux voix qui jouent, et « A » qui les arrête.

LA MUSIQUE

A ARRETER OU JOUER

l'écran, au bout de vingt images

La même chose, à la souris

THEMEBASSE+ airMIDIWAV

THEME▶ Écouter🎵 note– silence= tenir⏮⏭– pas+ pas8 images par pas



`jouer(1, THEME, 8);` — le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas.

Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

38. Une note, au front

Sans front, une note repart soixante fois par seconde.

`note(voix, hauteur, duree, volume)` : la voix est 1 ou 2, la durée se compte en images, le volume va de 0 à 15.

Les hauteurs s'écrivent `D04`, `RE4`, `MI4`... sur plusieurs octaves.

Le front est ici **indispensable** : sans lui, la note repartirait à chaque image et l'on n'entendrait qu'un grésillement continu.

LE PROGRAMME — 1403 OCTETS

```
int main() {
    texte(3, 8, "APPUIE SUR A");
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        if (appui && !avant) note(1, D04, 20, 12);
        avant = appui;
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un seul « bip » par appui, même en tenant la touche.

APPUIE SUR A

l'écran, au bout de vingt images

39. Une petite mélodie à la main

Une table gravée, et un pas toutes les vingt images.

`const uint8_t AIR[]` est **gravé dans la cartouche** : il ne prend aucun octet de mémoire de travail, et il ne peut pas changer. C'est exactement ce qu'on veut d'une mélodie.

`sizeof(AIR)` vaut 6, connu à la compilation. **Ajouter une note à la table suffit** — la boucle suit, sans qu'on touche à un nombre.

Ici le programme mène la mélodie et doit y penser à chaque tour. Le tutoriel suivant montre l'autre façon.

LE PROGRAMME — 1432 OCTETS

```
const uint8_t AIR[] = { D04, MI4, SOL4, D05, SOL4, MI4 };

int main() {
    uint8_t pas = 0;
    uint8_t depuis = 0;

    texte(2, 6, "NOTE");

    while (true) {
        depuis++;
        if (depuis >= 20) {
            depuis = 0;
            note(1, AIR[pas], 18, 12);
            nombre(9, 6, pas, 1);
            pas++;
            if (pas >= sizeof(AIR)) pas = 0;
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le numéro du pas monte de 0 à 5, puis recommence.

NOTE

l'écran, au bout de vingt images

40. Un bruit

La quatrième voix ne joue pas de notes.

`bruit(duree, volume)` frappe sur la voix du bruit — un pas, un tir, une porte qui claque.

Un troisième argument règle le **grain**, du sifflement aigu au choc sourd. C'est la seule voix qui n'ait pas de hauteur : il n'y a rien à accorder.

LE PROGRAMME — 1386 OCTETS

```
int main() {
    texte(4, 8, "A POUR TIRER");
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        if (appui && !avant) bruit(6, 10);
        avant = appui;
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un « tchac » sec à chaque appui sur A.

A POUR TIRER

l'écran, au bout de vingt images

41. Une partition qui avance seule

`jouer()` la lance, et le jeu fait autre chose pendant ce temps.

C'est toute la différence avec la mélodie à la main : là-bas le programme menait chaque note ; ici `jouer()` lance la partition et **elle avance toute seule**.

Un pas par élément. `"=="` tient la note d'avant, `"--"` fait un silence. Le troisième argument de `jouer()` est le nombre d'images par pas — le tempo.

Dans la page, cette partition s'écrit **à la souris**, sur une portée. L'atelier produit exactement ces lignes.

LE PROGRAMME — 1670 OCTETS

```
Air FANFARE = {
  "D04 12", "==", "MI4 12", "==", "SOL4 12", "==", "D05 12", "==", "==", "--",
};

int main() {
  jouer(1, FANFARE, 8);
  texte(4, 8, "CA JOUE");

  while (true) {
    image();
  }
}
```

CE QU'ON DOIT VOIR

Une fanfare qui monte, sans que le programme s'en occupe.



l'écran, au bout de vingt images

La même chose, à la souris

FANFARE

+ air

MIDI

WAV

FANFARE

▶ Écouter

🎵 note

– silence

= tenir

▼

▲

– pas

+ pas

8

images par pas

`jouer(1, FANFARE, 8);` — le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas. Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

42. Une musique qui reprend au bout

`airFini()` dit quand la partition est arrivée à sa fin.

`airFini(voix)` rend 1 quand la voix a fini sa partition. On relance, et l'on compte les tours — c'est la boucle musicale la plus simple qui soit.

`silence(1)` couperait la voix net, si l'on voulait arrêter au milieu.

LE PROGRAMME — 1755 OCTETS

```
Air BOUCLE = {
    "D04 12", "=", "MI4 12", "=", "SOL4 12", "=", "MI4 12", "=",
};

int main() {
    jouer(1, BOUCLE, 10);
    uint8_t tours = 0;

    /* Écrit AVANT la boucle : sans cela, l'écran reste vide une seconde et
    demie, et l'on croit que le programme n'a pas démarré. */
    texte(2, 10, "TOURS");
    nombre(9, 10, tours);

    while (true) {
        if (airFini(1)) {
            jouer(1, BOUCLE, 10);
            tours++;
            nombre(9, 10, tours);
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le compteur de tours monte à chaque reprise.



l'écran, au bout de vingt images

La même chose, à la souris

BOUCLE

+ air

MIDI

WAV

BOUCLE

▶ Écouter

♪ note

– silence

= tenir

▼

▲

– pas

+ pas

8

images par pas

The interface displays a piano roll with a vertical axis for pitch (labeled D05, D04, vol) and a horizontal axis for time. Notes are represented by horizontal bars. A volume slider is visible at the bottom left of the piano roll area.

`jouer(1, BOUCLE, 8);` — le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas. Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

43. Un tableau, et une boucle « for »

Ranger beaucoup de choses.

`uint8_t cases[20];` réserve vingt octets en mémoire de travail — de quoi tenir un puits, une carte, un inventaire. Ils vivent hors de la page rapide, qui ne contient que les variables.

Un tableau `const` est différent : il est **gravé dans la cartouche**. Il ne prend pas un octet de mémoire, et ne peut plus changer — le compilateur refuse d'y écrire. C'est ce qu'on veut pour une table de formes, un niveau, une courbe.

Le `for` de C++ tient sur une ligne ce que le `while` demandait en quatre, et **la variable du compteur n'existe que dans la boucle**. Deux boucles voisines peuvent toutes deux nommer leur compteur `i` sans se marcher dessus : c'est la portée du langage, et le compilateur la tient vraiment.

LE PROGRAMME — 1526 OCTETS

```
const uint8_t TABLE[] = { 1, 1, 2, 3, 5, 8, 3, 1, 4, 5 };

uint8_t cases[20];

int main() {
    texte(3, 3, "UN TABLEAU");
    texte(3, 11, "ET UNE TABLE");

    for (uint8_t i = 0; i < 20; i++) {
        cases[i] = i % 10;
    }

    for (uint8_t i = 0; i < 20; i++) {
        poser(i, 6, 27 + cases[i]);
    }

    for (uint8_t i = 0; i < 10; i++) {
        poser(i * 2, 14, 27 + TABLE[i]);
    }

    while (true) {
        image();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Les chiffres 0 à 9 deux fois, puis la table gravée, un chiffre sur deux.

```
UN TABLEAU
01234567890123456789

ET UNE TABLE
1 1 2 3 5 8 3 1 4 5
```

l'écran, au bout de vingt images

44. Le panneau, et les nuances

Un score qui reste en place pendant que le décor glisse.

Le décor défile ; le score, lui, ne doit pas bouger. Sans une seconde couche, il faudrait le redessiner à chaque image à la position que le défilement lui a donnée — et le moindre décalage se verrait.

Le **panneau** est cette couche : le matériel l'appelle la fenêtre, il la pose par-dessus le décor, et **elle ne défile pas**. `panneau(x, y)` la place, en pixels, et l'allume. On y écrit avec `textePanneau()` et `poserPanneau()`, exactement comme sur le fond.

Il n'y a que **deux cartes de fond** dans la console : le décor prend la première, le panneau la seconde. C'est un choix, et il vaut mieux le savoir que le découvrir.

`paletteFond(n0, n1, n2, n3)` choisit les quatre nuances, de la plus claire à la plus sombre — chaque nombre va de 0 à 3. Elles étaient posées une fois pour toutes à l'allumage ; ici on s'en sert pour assombrir le fond par paliers.

LE PROGRAMME — 1553 OCTETS

```
Tuile SOL = {
    "33333333",
    "32222223",
    "32122123",
    "32222223",
    "32221223",
    "32122223",
    "32222223",
    "33333333",
};

uint8_t d = 0;
uint8_t attente = 0;
uint8_t noirceur = 0;

int main() {
    for (uint8_t x = 0; x < 32; x++) {
        poser(x, 12, SOL);
        poser(x, 13, SOL);
    }

    textePanneau(1, 0, "SCORE 00");
    panneau(0, 128);

    while (true) {
        image();

        d++;
        defiler(d, 0);

        attente++;

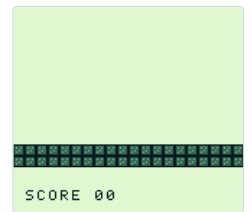
        if (attente > 40) {
            attente = 0;
            noirceur++;
            if (noirceur > 3) noirceur = 0;

            // le fond s'assombrit, puis revient
            paletteFond(noirceur, noirceur + 1, 2, 3);
        }
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Le sol glisse ; « SCORE 00
» reste collé en bas, et les
nuances changent.



l'écran, au bout de vingt images

45. Remplir un tableau, puis le dessiner

Un tableau déclaré hors des fonctions est **VIDE** au démarrage.

Un **index est un octet : 256 cases au plus**. Ce n'est pas une limite arbitraire — c'est ce que la machine sait adresser d'un seul registre.

Un tableau déclaré hors des fonctions vit en mémoire de travail, et il **ne contient rien** au démarrage. Le remplir est le travail de la première boucle ; l'oublier donne un décor entièrement plat, sans qu'aucune erreur ne le dise.

LE PROGRAMME — 1434 OCTETS

```
uint8_t hauteurs[20];

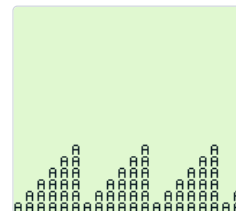
int main() {
    for (uint8_t i = 0; i < 20; i++) hauteurs[i] = i % 6;

    for (uint8_t i = 0; i < 20; i++)
        for (uint8_t h = 0; h <= hauteurs[i]; h++)
            poser(i, 17 - h, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Des colonnes en dents de scie, montant de 1 à 6 puis recommençant.



l'écran, au bout de vingt images

46. Une table gravée

Ce qui ne change jamais n'a rien à faire en mémoire.

`const uint8_t T[]` est **gravé dans la cartouche**, comme une tuile. Le compilateur l'annonce : ces octets sortent du décompte de la mémoire de travail.

Ôter le `const` la met en mémoire — et la rend modifiable, et la rend vide au démarrage. Les deux formes ont leur usage ; la différence se voit dans le compte rendu de compilation, et nulle part ailleurs.

LE PROGRAMME — 1362 OCTETS

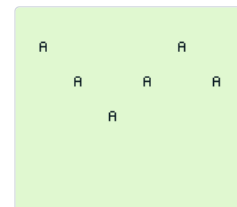
```
const uint8_t COLONNES[] = { 2, 5, 8, 11, 14, 17 };
const uint8_t LIGNES[]   = { 3, 6, 9, 6, 3, 6 };

int main() {
    for (uint8_t i = 0; i < sizeof(COLONNES); i++)
        poser(COLONNES[i], LIGNES[i], 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Six tuiles posées en zigzag, aux positions de la table.



l'écran, au bout de vingt images

47. Une grille

Deux index, et un calcul qu'on n'écrit pas.

`carte[1][c]` est rangé à plat, ligne après ligne, et le compilateur calcule `1 × LARGEUR + c`. C'est le calcul que tout le monde écrit à la main — et que tout le monde finit par écrire de travers le jour où la largeur change.

Le compilateur **exige les deux index** : `carte[3]` désignerait une rangée entière, et non un octet. Il le dit plutôt que de deviner.

LE PROGRAMME — 1594 OCTETS

```
const uint8_t LARGEUR = 10;
const uint8_t HAUTEUR = 8;

uint8_t carte[HAUTEUR][LARGEUR];

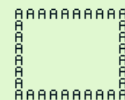
int main() {
    for (uint8_t l = 0; l < HAUTEUR; l++)
        for (uint8_t c = 0; c < LARGEUR; c++)
            carte[l][c] = (l == 0 || l == HAUTEUR - 1 || c == 0 || c == LARGEUR - 1) ? 1 : 0;

    for (uint8_t l = 0; l < HAUTEUR; l++)
        for (uint8_t c = 0; c < LARGEUR; c++)
            if (carte[l][c]) poser(c + 5, l + 5, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un rectangle creux, de dix sur huit, posé au milieu.



l'écran, au bout de vingt images

48. Chercher dans un tableau

Sortir dès qu'on a trouvé, et réserver une valeur pour « pas trouvé ».

`break` sort de la boucle ; `continue` passerait au tour suivant.

`ou = 255` sert de « pas trouvé ». Sur un octet il n'y a pas de valeur spéciale, alors **on en réserve une** — et l'on choisit celle qui ne peut pas être un index valide, puisqu'un tableau fait 256 cases au plus.

LE PROGRAMME — 1428 OCTETS

```
uint8_t valeurs[8];

int main() {
    for (uint8_t i = 0; i < 8; i++) valeurs[i] = i * 3;

    uint8_t cherche = 12;
    uint8_t ou = 255;

    for (uint8_t i = 0; i < 8; i++) {
        if (valeurs[i] == cherche) { ou = i; break; }
    }

    texte(2, 6, "TROUVE EN");
    nombre(13, 6, ou);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« TROUVE EN 004 » — 12
est bien la cinquième
valeur.

TROUVE EN 004

l'écran, au bout de vingt images

49. Un texte qui porte un nom

Le même message à deux endroits, écrit une fois.

`const char NOM[] = "..."` est gravé dans la cartouche, comme une table. Il s'écrit ensuite **partout où `texte()` attend des guillemets**.

Le même message répété trois fois entre guillemets, ce sont trois occasions de le changer à deux endroits. Ici il n'y en a qu'un.

LE PROGRAMME — 1304 OCTETS

```
const char TITRE[] = "GAME OVER";

int main() {
    texte(5, 6, TITRE);
    texte(5, 12, TITRE);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« GAME OVER » deux fois,
écrit une seule fois dans
le programme.

GAME OVER

GAME OVER

l'écran, au bout de vingt images

50. Une fonction qui prend des arguments

Écrire une fois ce qu'on fera dix fois.

Voici ce que le C++ apporte de plus visible. Une fonction prend des **arguments**, et rend une **valeur** : `uint8_t distance(uint8_t a, uint8_t b)`. On l'écrit une fois, on l'appelle partout, et l'endroit où l'on écrit `barre(2, 6, 5)` dit exactement ce qui va se passer.

Une fonction qui ne rend rien s'écrit `void`. Une fonction qui annonce rendre un `uint8_t` **doit** le faire : si aucun `return` ne rend de valeur, le compilateur refuse le fichier, plutôt que de laisser la fonction rendre ce qui traînait dans le processeur.

Un prix à payer, et il est dit franchement : **une fonction ne peut pas s'appeler elle-même**. Les arguments vivent à une place fixe, réservée une fois pour toutes — faute d'une pile praticable sur cette machine. Un appel imbriqué écraserait les arguments de l'appel en cours ; le compilateur nomme le cycle et s'arrête là.

LE PROGRAMME — 1475 OCTETS

```
Tuile BLOC = {
    "33333333",
    "30000003",
    "30222203",
    "30222203",
    "30222203",
    "30222203",
    "30000003",
    "33333333",
};

// Une fonction qui rend une valeur.
uint8_t distance(uint8_t a, uint8_t b) {
    if (a > b) return a - b;
    return b - a;
}

// Une fonction qui n'en rend aucune : « void ».
void barre(uint8_t colonne, uint8_t ligne, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, BLOC);
    }
}

int main() {
    texte(2, 2, "DES ARGUMENTS");

    barre(2, 6, 5);
    barre(2, 8, 12);
    barre(2, 10, distance(3, 20));

    while (true) {
        image();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Trois barres de longueurs différentes : 5, 12, puis 17 cases.

DES ARGUMENTS

```

#####
#####
#####
```

l'écran, au bout de vingt images

51. Découper son programme

Le même travail, écrit pour être relu dans trois jours.

Tout ce que fait ce programme tiendrait dans `main()`. Ce serait plus court à taper — et illisible dès qu'il grandit. Une boucle de jeu de deux cents lignes, on ne la relit pas : on la contourne, on lui ajoute une rustine à côté, et le désordre s'installe.

Une fonction est d'abord un nom. `dessinerLeSol()` dit ce qu'elle fait ; le lecteur n'a pas besoin d'ouvrir la boucle qui pose vingt tuiles pour le savoir. Une fonction sans argument ni retour reste utile — c'est même là qu'elle sert le plus.

Un nombre écrit en clair au milieu du code ne dit rien. `poser(colonne, 14, HERBE)` : pourquoi 14 ? `const uint8_t LIGNE_DU_SOL = 14;` répond, et le jour où le sol descend, il n'y a qu'un seul endroit à changer. Une `const` ne coûte **aucun octet** : sa valeur est connue à la compilation.

`return;` **au milieu d'une fonction** évite d'imbriquer trois `if`. `fairePasserLeTemps()` s'arrête tout de suite quand ce n'est pas encore l'heure, et la suite se lit à plat.

Un programme peut tenir sur plusieurs fichiers. `#include "dessins.cpp"` verse un autre fichier à cet endroit, avant toute compilation — et une faute y est signalée **avec le nom de son vrai fichier**, pas avec la ligne du texte collé. C'est ainsi qu'est écrit `exemples/decoupe/` : cinq fichiers, une seule cartouche, au dernier octet près.

Dans l'atelier, les fichiers sont les **onglets sous le titre** : « + fichier » en ajoute un, et il suffit de l'inclure. L'exemple « Un programme sur CINQ fichiers » les ouvre tous les cinq d'un coup, pour voir à quoi ça ressemble.

```

/*
 * Le même décor, écrit pour être RELU.
 *
 * Tout ce qui suit tiendrait dans main(). Ce serait plus court à taper, et
 * illisible trois jours plus tard.
 */

const uint8_t LIGNE_DU_SOL = 14;
const uint8_t LIGNE_DU_CIEL = 3;
const uint8_t COLONNES = 20;

Tuile HERBE = {
    "01100110",
    "11111111",
    "22222222",
    "22322322",
    "22222222",
    "23223223",
    "22222222",
    "33333333",
};

Tuile NUAGE = {
    "00000000",
    "00011100",
    "00111110",
    "01111111",
    "11111111",
    "01111110",
    "00000000",
    "00000000",
};

uint8_t nuage = 3;
uint8_t attente = 0;

/* Chaque fonction porte un nom qui DIT ce qu'elle fait. */
void ecrireLeTitre() {
    texte(5, 1, "MON DECOR");
}

void dessinerLeSol() {
    for (uint8_t colonne = 0; colonne < COLONNES; colonne++) {
        poser(colonne, LIGNE_DU_SOL, HERBE);
    }
}

void effacerLeCiel() {
    for (uint8_t colonne = 0; colonne < COLONNES; colonne++) {
        poser(colonne, LIGNE_DU_CIEL, 0);
    }
}

void dessinerLeCiel() {
    effacerLeCiel();
    poser(nuage, LIGNE_DU_CIEL, NUAGE);
}

void fairePasserLeTemps() {
    attente++;
    if (attente < 20) return;

    attente = 0;
    nuage++;
    if (nuage >= COLONNES) nuage = 0;
    dessinerLeCiel();
}

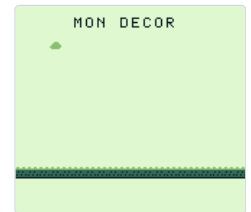
int main() {
    ecrireLeTitre();
    dessinerLeSol();
    dessinerLeCiel();

    while (true) {
        image();
        fairePasserLeTemps();
    }

    return 0;
}

```

Un titre, un sol d'herbe
sur toute la largeur, et un
nuage qui traverse le ciel.



l'écran, au bout de vingt images

La même chose, à la souris

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom. étiquette. ★

Game Boy en C++

← l'atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

HERBE

NUAGE

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

HERBE

tuile n° 44 — poser(x, y, HERB

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

HERBE ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

HERBE peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

52. Nommer un geste qu'on refait

Une fonction avec un argument, et trois appels.

Les fonctions prennent des arguments, et c'est là tout l'intérêt. Sans argument, il faudrait trois fonctions, ou une variable globale à poser avant chaque appel.

`void` dit qu'elle ne rend rien. Le compilateur le vérifie : une fonction qui annonce rendre un `uint8_t` sans jamais faire `return` est **refusée, avec sa ligne**.

LE PROGRAMME — 1370 OCTETS

```
Tuile BRIQUE = {
    "#####", "#..#..#", "#####", "..#..#",
    "#####", "#..#..#", "#####", "..#..#",
};

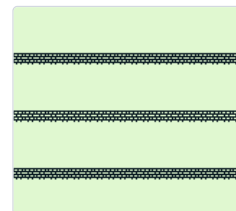
void rangee(uint8_t ligne) {
    for (uint8_t x = 0; x < 20; x++) poser(x, ligne, BRIQUE);
}

int main() {
    rangee(4);
    rangee(9);
    rangee(14);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois rangées de briques,
écrites par un seul bout
de code.



l'écran, au bout de vingt images

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

Game Boy en C++

← l'atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

BRIQUE

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BRIQUE

tuile n° 44 — poser(x, y, BRIQUE)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

BRIQUE ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

BRIQUE peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

53. Une fonction qui rend une valeur

Ce qui entre, ce qui sort — et ce qu'on ne réécrit pas deux fois.

Les deux appels rendent 70. Sans la fonction, il faudrait écrire le `if` deux fois — et le jour où l'on veut borner l'écart à 60, le corriger aux deux endroits.

Le résultat s'utilise dans un calcul : `nombre(10, 6, distance(a, b) * 2)` marche, comme n'importe quelle expression.

LE PROGRAMME — 1365 OCTETS

```
uint8_t distance(uint8_t a, uint8_t b) {
    if (a > b) return a - b;
    return b - a;
}

int main() {
    texte(2, 6, "ECART");
    nombre(10, 6, distance(30, 100));
    nombre(10, 8, distance(100, 30));

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

070 deux fois — l'ordre des arguments ne change rien.

```
ECART    070
          070
```

l'écran, au bout de vingt images

54. Un argument par défaut

Ce qui n'est pas écrit prend la valeur annoncée.

L'argument par défaut est rempli à l'**appel**, et non dans la fonction — comme en C++. Cela lui permet de dépendre de l'endroit d'où l'on appelle.

Une règle : un argument sans valeur par défaut **ne peut pas suivre** un argument qui en a une. Sinon, à `barre(4, 20)`, on ne pourrait plus deviner lequel manque. Le compilateur le refuse plutôt que de choisir à notre place.

LE PROGRAMME — 1378 OCTETS

```
void barre(uint8_t ligne, uint8_t combien, uint8_t tuile = 1) {
    for (uint8_t x = 0; x < combien; x++) poser(x, ligne, tuile);
}

int main() {
    barre(4, 20);
    barre(8, 10);
    barre(12, 5, 2);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois barres de longueurs différentes ; la dernière d'une autre tuile.

```
AAAAAAAAAAAAAAAAAAAAA
AAAAAAAAA
BBBBB
```

l'écran, au bout de vingt images

55. Pourquoi une fonction ne s'appelle pas elle-même

Le refus le plus important du projet — et comment dérouler la boucle.

Écrire `return n + somme(n - 1);` est **refusé**, avec le cycle nommé : « `somme` finit par s'appeler elle-même (somme → somme) ».

Les arguments vivent à une adresse fixe, réservée une fois pour toutes — c'est ce que fait tout compilateur C d'une machine huit bits, faute d'une pile praticable. Un appel imbriqué écraserait les arguments de l'appel en cours, et la fonction reprendrait son travail avec les valeurs de l'autre.

C'est un bogue silencieux, et indébogable. Le compilateur le nomme avant qu'il n'arrive. Voici la même chose, déroulée avec `while`.

LE PROGRAMME — 1358 OCTETS

```
uint8_t somme(uint8_t n) {
    uint8_t total = 0;
    while (n > 0) { total += n; n--; }
    return total;
}

int main() {
    texte(2, 6, "SOMME DE 1 A 5");
    nombre(8, 8, somme(5));

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

015 — la somme de 1 à 5, calculée sans récursion.

SOMME DE 1 A 5
015

l'écran, au bout de vingt images

56. #include, et les onglets de la page

Le fichier voisin est versé ici, avant toute compilation.

`#include "dessins.cpp"` **verse le fichier voisin à cet endroit** avant que le compilateur ne voie quoi que ce soit. Il n'y a donc ni éditeur de liens, ni ordre de compilation à connaître.

Dans la page, les fichiers voisins sont des **onglets** : `#include` ne voit pas la différence. Et quand une faute tombe dans un fichier inclus, le message désigne **son** fichier et **sa** ligne — pas la ligne du texte assemblé, que personne n'a jamais vue.

Le programme ci-dessous tient en un fichier ; la leçon « Découper son programme » en montre un sur cinq.

LE PROGRAMME — 1316 OCTETS

```
uint8_t doubler(uint8_t n) { return n * 2; }

int main() {
    texte(2, 6, "DOUBLE DE 21");
    nombre(8, 8, doubler(21));

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« DOUBLE DE 21 », et 042
en dessous — la fonction
a été appelée.

DOUBLE DE 21
042

l'écran, au bout de vingt images

57. Une « struct » : ce qui va ensemble

Une troupe d'ennemis, et non six tableaux parallèles.

Un ennemi a une position, une vitesse, une santé. Sans `struct`, cela fait trois tableaux qu'il faut penser à garder alignés — et le jour où l'on en trie un sans trier les autres, les vies changent de propriétaire.

Une `struct` range ces champs ensemble. `Etoile etoiles[4];` réserve quatre enregistrements ; `etoiles[i].x` en désigne un champ. Le compilateur calcule le décalage lui-même, et `sizeof(Etoile)` dit ce qu'un enregistrement pèse.

Le numéro d'un lutin peut être **calculé** : `sprite(i, ...)` dans une boucle affiche toute la troupe, chacun avec ses valeurs à lui. C'est ce qui rend la `struct` utile plutôt que décorative.

Une `struct` peut en contenir une autre : `struct Ennemi { Point ou; uint8_t vie; };`, et l'on écrit `troupe[i].ou.x`. Le compilateur additionne les décalages lui-même.

Ce qu'elle ne sait pas faire : s'affecter d'un bloc, se passer en argument, se rendre par une fonction. On nomme le champ qu'on veut lire ou écrire — et le compilateur le dit plutôt que de traduire à peu près.

LE PROGRAMME — 1690 OCTETS

```
Tuile CORPS = {
    "00333300",
    "03222230",
    "32222223",
    "32122123",
    "32222223",
    "32211223",
    "03222230",
    "00333300",
};

struct Etoile {
    uint8_t x;
    uint8_t y;
    uint8_t vitesse;
};

const uint8_t COMBIEN = 4;

Etoile etoiles[COMBIEN];

void placer() {
    for (uint8_t i = 0; i < COMBIEN; i++) {
        etoiles[i].x = 20 + i * 30;
        etoiles[i].y = 32 + i * 24;
        etoiles[i].vitesse = 1 + i;
    }
}

void avancerToutes() {
    for (uint8_t i = 0; i < COMBIEN; i++) {
        etoiles[i].x += etoiles[i].vitesse;

        if (etoiles[i].x > 152) etoiles[i].x = 0;

        sprite(i, etoiles[i].x, etoiles[i].y, CORPS);
    }
}

int main() {
    texte(2, 1, "UNE TROUPE");

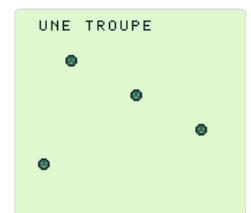
    placer();

    while (true) {
        image();
        avancerToutes();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Quatre boules qui traversent l'écran, chacune à sa vitesse.



l'écran, au bout de vingt images

58. Le décor qui défile

Un monde plus large que l'écran.

La carte de fond fait **32 cases sur 32**, soit 256 pixels de côté ; l'écran n'en montre que 20 sur 18. Le reste attend hors-champ.

`defiler(x, y)` fait glisser la fenêtre. Au-delà de 255, le compte revient tout seul à zéro : le décor est un **ruban sans fin**.

C'est ce qui permet un niveau aussi long qu'on veut : quand la caméra franchit une case, on redessine la colonne qui va entrer par la droite, à la place de celle qui vient de sortir par la gauche.

LE PROGRAMME — 1463 OCTETS

```
Tuile MUR = {
    "33333333",
    "32222223",
    "32122123",
    "32222223",
    "32221223",
    "32122223",
    "32222223",
    "33333333",
};

int main() {
    uint8_t d = 0;

    texte(2, 2, "DROITE POUR AVANCER");

    for (uint8_t x = 0; x < 32; x++) {
        poser(x, 14, MUR);
        poser(x, 15, MUR);
    }

    while (true) {
        image();

        if (bouton(DROITE)) d++;
        if (bouton(GAUCHE)) d--;

        defiler(d, 0);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Le sol glisse sous l'écran
quand on pousse la croix.



l'écran, au bout de vingt images

59. La caméra : c'est le monde qui bouge

Le héros ne se déplace pas — comme dans Mario.

Dans un jeu qui défile, le personnage **ne bouge pas à l'écran**. Il reste au milieu, et c'est le décor qui passe derrière lui. Mario n'avance pas : il piétine au centre pendant qu'un monde de plusieurs écrans glisse vers la gauche.

Il y a donc **deux espaces** : l'écran, qui fait 160 pixels de large, et le monde, qui fait ce qu'on veut. `defiler(camera, 0)` dit simplement **quel morceau du monde on regarde**.

Et voici pourquoi cela tient en si peu de lignes : **les lutins ne défilent pas**. `defiler()` ne déplace que le fond ; un `sprite()` est posé sur l'écran, à la place où on l'écrit. Le héros au milieu, c'est une ligne, toujours la même : `sprite(0, 76, 112, HEROS);`

Une seule variable suffit pour les deux rôles. `camera` est la fenêtre **et** la position du héros dans le monde : il est en `camera + 76`. Le jour où un ennemi arrive, lui vit dans le monde, et c'est à lui de se convertir — `sprite(1, xEnnemi - camera, 112, ENNEMI)`.

La preuve est à l'écran, et elle est gratuite : **même la consigne s'en va**. Elle est écrite dans le décor, donc elle voyage avec lui. Le bonhomme, lui, ne bouge pas d'un pixel.

LE PROGRAMME — 1677 OCTETS

```
Tuile HEROS = {
    "00333300",
    "03222230",
    "32122123",
    "32222223",
    "03222230",
    "03333330",
    "00300300",
    "03300330",
};

Tuile SOL = {
    "33333333",
    "32323232",
    "23232323",
    "32323232",
    "23232323",
    "32323232",
    "23232323",
    "32323232",
};

Tuile ARBRE = {
    "00033000",
    "00333300",
    "03333330",
    "33333333",
    "03333330",
    "00033000",
    "00033000",
    "00033000",
};

int main() {
    uint8_t camera = 0;

    texte(1, 2, "IL NE BOUGE PAS");
    texte(1, 4, "CEST LE MONDE QUI");
    texte(1, 5, "PASSE DERRIERE LUI");

    for (uint8_t c = 0; c < 32; c++) {
        poser(c, 15, SOL);
        poser(c, 16, SOL);
        poser(c, 17, SOL);
    }

    poser(3, 14, ARBRE);
    poser(12, 14, ARBRE);
    poser(25, 14, ARBRE);

    while (true) {
        image();

        if (bouton(DROITE)) camera = camera + 1;
        if (bouton(GAUCHE)) camera = camera - 1;

        defiler(camera, 0);
        sprite(0, 76, 112, HEROS);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Le bonhomme reste planté au milieu ; les arbres, le sol et même la consigne passent derrière lui.

IL NE BOUGE PAS
CEST LE MONDE QUI
PASSE DERRIERE LUI



l'écran, au bout de vingt images

60. Trois variables qui parlent de la même chose

Une `struct`, plutôt que `bossX`, `bossY`, `bossVie`.

Sans `struct`, on écrirait `bossX`, `bossY`, `bossVie` — et le jour où il faut un second ennemi, on recopie trois variables et l'on en oublie une.

Une `struct` fait **au plus 255 octets**, et chaque champ est un octet. C'est un enregistrement, pas un objet : ni héritage, ni constructeur.

La leçon « Une struct : ce qui va ensemble » en fait un usage complet ; celle-ci n'en montre que la forme.

LE PROGRAMME — 1336 OCTETS

```
struct Ennemi {
    uint8_t x;
    uint8_t y;
    uint8_t vie;
};

Ennemi boss;

int main() {
    boss.x = 40;
    boss.y = 60;
    boss.vie = 3;

    texte(2, 4, "X");    nombre(6, 4, boss.x);
    texte(2, 6, "VIE");  nombre(6, 6, boss.vie, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« X 040 » et « VIE 3 » : les deux champs de la même struct, relus.

```
X    040
VIE  3
```

l'écran, au bout de vingt images

61. Un tableau de « struct »

Trois ennemis, c'est UNE ligne de plus — pas neuf variables.

`troupe[i].vie` : l'index est calculé à l'exécution, le décalage du champ est connu à la compilation. **Le compilateur fait le calcul d'adresse** ; nous, nous écrivons ce que nous voulons dire.

C'est ici que la `struct` prend son sens. Passer de un à trois ennemis coûte une ligne ; sans elle, il aurait fallu six variables de plus et six lignes à tenir d'accord.

LE PROGRAMME — 1494 OCTETS

```
struct Ennemi {
    uint8_t x, y, vie;
};

Ennemi troupe[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) {
        troupe[i].x = 20 + i * 40;
        troupe[i].y = 60;
        troupe[i].vie = 3;
    }

    troupe[1].vie--;

    for (uint8_t i = 0; i < 3; i++) nombre(4 + i * 5, 8, troupe[i].vie, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« 3 2 3 » — seul celui du milieu a été touché.

3 2 3

l'écran, au bout de vingt images

62. Une méthode : la fonction qui connaît son objet

``troupe[i].avancer()`` plutôt que ``avancer(i)``.

Une méthode s'écrit dans la `struct`, et travaille sur l'objet devant le point. Dans son corps, les champs s'écrivent sous leur nom nu : `x++`, et non `this->x++`.

Ce qu'elle coûte, exactement : l'adresse de l'objet est rangée dans **deux octets réservés à cette méthode**, puis c'est le même `call` qu'ailleurs. Pas de table de fonctions, pas de pile d'objets — et le compilateur compte ces deux octets comme les autres.

Ce qui reste refusé, et pourquoi : `class` (le message propose `struct`), `virtual` (il faudrait une table de fonctions et un appel indirect), l'héritage (il déplacerait les décalages des champs), les constructeurs (du code implicite à la déclaration, alors que les globales prennent leur valeur avant `main`). Et une méthode **ne prend pas d'argument** : elle reçoit déjà son objet.

LE PROGRAMME — 1599 OCTETS

```
struct Ennemi {
    uint8_t x, y, vie;

    void avancer() { x++; }
    uint8_t vivant() { return vie > 0; }
};

Ennemi troupe[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) { troupe[i].x = 20 + i * 40; troupe[i].vie = 3; }
    troupe[1].vie = 0;

    while (true) {
        for (uint8_t i = 0; i < 3; i++)
            if (troupe[i].vivant()) troupe[i].avancer();

        for (uint8_t i = 0; i < 3; i++) nombre(3 + i * 6, 8, troupe[i].x);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Deux nombres qui montent, et celui du milieu immobile.

029 060 109

l'écran, au bout de vingt images

63. Les trois façons de faire la même chose

À la suite, par une fonction, par une méthode — et ce que chacune coûte.

À la suite, tout est dans `main` : `if (ennemiVie > 0) ennemiX++;`, avec deux variables globales. C'est le plus court, et pour **un** ennemi c'est le bon choix. Le jour où il en faut trois, il faut six variables et trois fois les mêmes lignes.

Par des fonctions : les données sont rangées dans un tableau de `struct`, le geste est nommé, et l'objet se désigne par son index — `avancer(i)`, avec `void` `avancer(uint8_t i) { if (troupe[i].vie > 0) troupe[i].x++; }`. Coût : un octet par argument.

Par des méthodes, ci-dessous : rigoureusement le même travail, mais le geste est écrit **là où vivent les données**, et l'objet ne se passe plus. Coût : deux octets par méthode.

Mesuré, sur ce programme : **1344 octets** à la suite, **1590** par fonctions, **1610** par méthodes. Aucune n'est meilleure dans l'absolu — la troisième rend en lisibilité ce qu'elle prend en octets, et c'est bien pour cela que le compilateur les compte et les annonce.

LE PROGRAMME — 1643 OCTETS

```
struct Ennemi {
    uint8_t x, vie;

    void avancer() { if (vie > 0) x++; }
    void blesser() { if (vie > 0) vie--; }
};

Ennemi troupe[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) { troupe[i].x = 20 + i * 40; troupe[i].vie = 3; }
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        for (uint8_t i = 0; i < 3; i++) {
            troupe[i].avancer();
            if (appui && !avant) troupe[i].blesser();
        }
        avant = appui;

        for (uint8_t i = 0; i < 3; i++) nombre(2 + i * 6, 8, troupe[i].x);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois nombres qui montent ; A les blesse tous les trois.

029 069 109

l'écran, au bout de vingt images

64. Une méthode qui en appelle une autre

Pourquoi chaque méthode a **SES** deux octets, et non deux partagés.

Dans `blesser()`, l'appel nu `vivant()` veut dire « **la mienne** » : l'objet en cours est passé tel quel, comme en C++.

Et voici pourquoi chaque méthode a **ses** deux octets plutôt que deux partagés par la `struct` : si `avancer()` appelait `autre.vivant()`, un emplacement commun serait écrasé, et `avancer()` reprendrait son travail sur le mauvais ennemi. Le partage aurait marché sur les exemples simples et échoué exactement là où c'est le plus dur à voir.

La règle du non-appel-à-soi-même vaut aussi ici : **une méthode ne peut pas s'appeler elle-même**, ni par un détour. Le compilateur nomme le cycle, méthode par méthode.

LE PROGRAMME — 1416 OCTETS

```
struct Ennemi {
    uint8_t x, vie;

    uint8_t vivant() { return vie > 0; }
    void blesser()   { if (vivant()) vie--; }
};

Ennemi troupe[2];

int main() {
    troupe[0].vie = 3;
    troupe[1].vie = 0;

    troupe[0].blesser();
    troupe[1].blesser();

    texte(2, 6, "VIVANT MORT");
    nombre(4, 8, troupe[0].vie, 1);
    nombre(9, 8, troupe[1].vie, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

« 2 » et « 0 » — le mort
n'est pas descendu à 255.

V I V A N T M O R T
2 0

l'écran, au bout de vingt images

65. Passer d'un écran à l'autre

Un titre, une partie, une fin.

Le squelette de n'importe quel jeu. Deux règles, et il n'y en a pas d'autres.

Une variable dit où l'on est. Un `enum` lui donne ses valeurs possibles, et leur donne des noms : `TITRE`, `JEU`, `FIN`. Le nom de l'`enum` devient un type d'un octet, et `Scene scene` se lit mieux que `uint8_t scene`.

Un seul endroit change d'écran. `allerA()` éteint, efface, redessine, rallume — toujours dans cet ordre. Si chaque bouton faisait son propre ménage, l'un finirait par oublier d'effacer, et deux écrans se mélangeraient.

Le `switch` aiguille sur la scène. Il fonctionne comme en C++, `break` **compris** : sans lui, l'exécution tombe dans le cas suivant. C'est un piège célèbre, mais c'est le sens du langage, et le trahir en douce serait pire.

On éteint l'écran pour effacer parce que repeindre toute la carte demande bien plus de temps qu'une image n'en contient. `effacer()` sans argument vide les 32×32 de la carte d'un coup — cette leçon écrivait autrefois ses deux boucles à la main, et n'effaçait alors que les 20×18 visibles : le décor qui défilait ramenait l'écran d'avant.

LE PROGRAMME — 1586 OCTETS

```
enum Scene { TITRE, JEU, FIN };

Scene scene = TITRE;

void allerA(Scene ou) {
    scene = ou;

    ecran(0);
    effacer();

    switch (scene) {
        case TITRE:
            texte(6, 5, "MON JEU");
            texte(4, 10, "A POUR JOUER");
            break;

        case JEU:
            texte(5, 4, "C EST PARTI");
            texte(4, 10, "A POUR FINIR");
            break;

        case FIN:
            texte(7, 5, "FINI");
            texte(3, 10, "A POUR REVENIR");
            break;
    }

    ecran(1);
}

int main() {
    uint8_t aAvant = 0;

    allerA(TITRE);

    while (true) {
        image();

        uint8_t a = bouton(A);

        if (a && !aAvant) {
            if (scene == TITRE) allerA(JEU);
            else if (scene == JEU) allerA(FIN);
            else allerA(TITRE);
        }

        aAvant = a;
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

« MON JEU », puis A change d'écran à chaque appui.

MON JEU

A POUR JOUER

l'écran, au bout de vingt images

66. La pesanteur et le saut

Tomber, et retomber.

Le processeur ne connaît **pas les nombres négatifs** : `0 - 1` vaut 255. Une vitesse qui doit monter *et* descendre ne peut donc pas s'écrire directement.

On la décale : `ZEROV` vaut 16, et la vitesse vit autour. **Moins de 16, on monte ; plus de 16, on tombe.** Sans ce décalage, un saut deviendrait une chute de 240 pixels par image.

`const uint8_t ZEROV = 16;` ne coûte rien : la valeur est connue à la compilation, et le compilateur l'écrit directement dans le code. Un `const` n'occupe pas d'octet de mémoire.

La pesanteur ajoute 1 à chaque image. Sauter, c'est poser une vitesse bien en dessous de 16 : elle remonte d'elle-même, le mouvement s'inverse, et la courbe est celle d'un vrai saut.

LE PROGRAMME — 1551 OCTETS

```
Tuile BALLE = {
    "00333300",
    "03222230",
    "32222223",
    "32222223",
    "32222223",
    "32222223",
    "03222230",
    "00333300",
};

const uint8_t ZEROV = 16;
const uint8_t SOL = 120;

int main() {
    uint8_t x = 76;
    uint8_t y = SOL;
    uint8_t vy = ZEROV;
    uint8_t auSol = 1;
    uint8_t aAvant = 0;

    texte(4, 2, "A POUR SAUTER");

    while (true) {
        image();

        uint8_t a = bouton(A);

        if (a && !aAvant && auSol) {
            vy = ZEROV - 7;
            auSol = 0;
        }

        aAvant = a;

        vy++;
        if (vy > ZEROV + 4) vy = ZEROV + 4;

        if (vy < ZEROV) y = y - ZEROV + vy;
        if (vy > ZEROV) y = y + vy - ZEROV;

        if (y >= SOL) {
            y = SOL;
            vy = ZEROV;
            auSol = 1;
        }

        sprite(0, x, y, BALLE);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

La boule saute, ralentit en l'air, et retombe sur le sol.

A POUR SAUTER



l'écran, au bout de vingt images

67. Le verre qui se brise

Quatre éclats, et trois variables.

La chute est celle de la leçon du même nom : les pixels dans `y`, les seizièmes dans `frac`, la pesanteur qui s'ajoute à la vitesse. Ce qui change ici, c'est **ce qui arrive au contact**.

Un objet qui se brise ne disparaît pas : il est **remplacé par ses morceaux**. Au choc, le verre part se cacher et quatre éclats prennent sa place, exactement à l'endroit où il s'est posé.

Cacher un lutin ne demande aucune fonction : on l'écrit à 160, hors des 144 lignes de l'écran. Il existe toujours — les quarante existent toujours — on choisit simplement de ne pas le voir. C'est pour cela que les cinq `sprite()` de la fin sont écrits une fois pour toutes, sans un seul `if` : ce sont les variables qui décident qui est visible.

On ne simule pas quatre éclats. Une image plus tôt ils étaient un seul objet : ils partent donc ensemble, à la même hauteur, et ne diffèrent que par ce qu'ils prennent de côté. Une seule hauteur `ey`, un seul compteur `t`, et les quatre positions se calculent — `76 - 3 * t`, `76 - t`, `76 + t`, `76 + 3 * t`. **Trois variables au lieu de douze**.

Leur envol est la vitesse décalée du saut : `ZEROV - 6` les projette vers le haut, la pesanteur les rattrape. Les deux éclats rapides retombent trois fois plus loin que les lents, sans qu'on ait rien à écrire pour ça — même temps de vol, vitesse triple.

`t` n'avance **que tant qu'ils volent**. Posés, il s'arrête, et les éclats restent exactement où ils sont tombés. C'est cette ligne-là, et aucune autre, qui fait la différence entre du verre brisé et quatre morceaux qui glissent à l'infini.

```

Tuile VERRE = {
    "00333300",
    "03211230",
    "32112223",
    "32122223",
    "32222223",
    "32222223",
    "03222230",
    "00333300",
};

Tuile ECLAT = {
    "00000000",
    "00000300",
    "00003130",
    "00031230",
    "00312300",
    "03123000",
    "03330000",
    "00000000",
};

const uint8_t CIEL = 8;
const uint8_t SOL = 128;
const uint8_t CACHE = 160; // hors de l'écran : un lutin écrit là ne se voit pas
const uint8_t G = 2;
const uint8_t ZEROV = 16; // la vitesse d'un éclat, décalée : 16 est l'immobilité
const uint8_t ATTENTE = 60;

int main() {
    uint8_t y = CIEL;
    uint8_t frac = 0;
    uint8_t vy = 0;

    uint8_t brise = 0;
    uint8_t t = 0; // images de vol - c'est lui qui écarte les éclats
    uint8_t ey = CACHE; // leur hauteur, la même pour les quatre
    uint8_t evy = ZEROV;
    uint8_t pose = 0;

    texte(0, 2, "IL TOMBE ET IL CASSE");
    texte(0, 17, "#####");

    while (true) {
        image();

        if (brise) {
            evy = evy + 1;
            if (evy < ZEROV) ey = ey - ZEROV + evy;
            if (evy > ZEROV) ey = ey + evy - ZEROV;

            if (ey >= SOL) {
                ey = SOL;
                evy = ZEROV;
            } else {
                t = t + 1;
            }

            pose = pose + 1;
            if (pose > ATTENTE) {
                brise = 0;
                pose = 0;
                t = 0;
                ey = CACHE;
                y = CIEL;
                frac = 0;
                vy = 0;
            }
        } else {
            vy = vy + G;
            frac = frac + vy;
            y = y + (frac >> 4);
            frac = frac & 15;

            if (y >= SOL) {
                brise = 1;
                y = CACHE;
                ey = SOL;
                evy = ZEROV - 6;
            }
        }

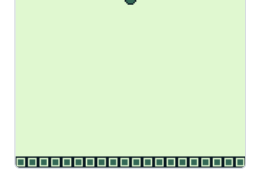
        sprite(0, 76, y, VERRE);
        sprite(1, 76 - 3 * t, ey, ECLAT);
        sprite(2, 76 - t, ey, ECLAT);
        sprite(3, 76 + t, ey, ECLAT);
        sprite(4, 76 + 3 * t, ey, ECLAT);
    }

    return 0;
}

```

Le verre tombe, éclate en quatre morceaux qui s'envolent et retombent éparpillés — puis tout recommence.

IL TOMBE ET IL CASSE



l'écran, au bout de vingt images

68. Un état de jeu

Quelques scènes, et un `switch` qui dit laquelle est en cours.

C'est presque toujours la forme d'un jeu : un `enum` de scènes, et un `switch`. Le reste — les ennemis, le décor — se branche dans le `case JEU`.

Le front est calculé **une fois**, en haut de la boucle, et servi à tous les cas. Le recalculer dans chaque `case` marcherait aussi — et un jour l'un des trois serait oublié.

Le score n'est écrit **que lorsqu'il change**, et l'étiquette une seule fois en entrant. Les réécrire à chaque image alourdit la boucle au point qu'elle déborde — et un `depuis` qui compte les tours avance alors trois fois moins vite que le temps réel.

LE PROGRAMME — 1562 OCTETS

```
enum Scene { TITRE, JEU, PERDU };

Scene ou = TITRE;
uint8_t score = 0;
uint8_t avant = 0;
uint8_t depuis = 0;

int main() {
    while (true) {
        uint8_t appui = bouton(START);
        uint8_t front = appui && !avant;
        avant = appui;

        switch (ou) {
            case TITRE:
                texte(4, 8, "APPUIE START");
                if (front) {
                    texte(4, 8, " ");
                    score = 0; depuis = 0; ou = JEU;
                    texte(2, 4, "SCORE"); // une fois, en entrant
                    nombre(9, 4, score);
                }
                break;

            case JEU:
                depuis++;
                if (depuis >= 30) { depuis = 0; score++; nombre(9, 4, score); }
                if (score >= 20) ou = PERDU;
                break;

            case PERDU:
                texte(5, 8, "PERDU");
                if (front) { texte(5, 8, " "); ou = TITRE; }
                break;
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

START lance la partie, le score monte, et à 20 c'est perdu.

APPUIE START

l'écran, au bout de vingt images

69. Le score, et le panneau

Une seconde couche, qui ne défile pas avec le décor.

Le **panneau** est une couche posée par-dessus le fond. Il ne défile pas : c'est exactement ce qu'on veut d'un score pendant que le décor glisse.

`panneau(x, y)` le place et l'allume, `cachePanneau()` l'ôte sans rien perdre de son contenu.

Deux détails qui se paient cher si on les oublie. Le compteur `depuis` plutôt qu'un multiple de `images()` : le corps de cette boucle écrit dans le panneau, il déborde d'une image, et le multiple serait enjambé — **le score resterait bloqué à zéro sans que rien ne le dise**. Et `nombrePanneau()` n'est appelé **que quand le score change** : le réécrire à chaque tour alourdit la boucle pour rien.

LE PROGRAMME — 1372 OCTETS

```
int main() {
    uint8_t score = 0;
    uint8_t depuis = 0;
    uint8_t glisse = 0;

    panneau(0, 0);
    textePanneau(1, 0, "SCORE");

    while (true) {
        depuis++;
        if (depuis >= 20) { depuis = 0; score++; nombrePanneau(8, 0, score); }

        glisse++;
        defiler(glisse, 0);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le score monte dans le panneau pendant que le fond glisse.

SCORE

l'écran, au bout de vingt images

70. Des ennemis qui vivent et qui meurent

Les méthodes, dans un vrai tour de boucle.

`hasard()` lit le compteur libre du matériel : un nombre imprévisible, sans table ni graine. `semer(n)` permet de choisir son point de départ quand on veut au contraire une suite reproductible.

`cache(n)` ôte le lutin de l'écran. Sans lui, un ennemi mort resterait affiché, immobile — ce qui se remarque tout de suite, heureusement.

LE PROGRAMME — 1786 OCTETS

```
Tuile ENNEMI = {
    "#####.", "#-#-#-#", "#####", "#.###.#",
    "#####", ".#.#.#.", "#..#.#", ".#...#.",
};

struct Monstre {
    uint8_t x, y, vie;

    uint8_t vivant() { return vie > 0; }
    void avancer()   { if (vivant()) x++; }
    void tuer()      { vie = 0; }
};

Monstre troupe[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) {
        troupe[i].x = 10 + i * 30;
        troupe[i].y = 40 + i * 24;
        troupe[i].vie = 1;
    }

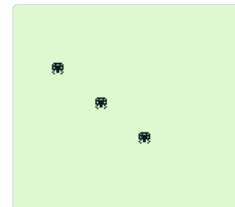
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        if (appui && !avant) troupe[hasard() % 3].tuer();
        avant = appui;

        for (uint8_t i = 0; i < 3; i++) {
            troupe[i].avancer();
            if (troupe[i].vivant()) sprite(i, troupe[i].x, troupe[i].y, ENNEMI);
            else cache(i);
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois monstres qui avancent ; A en fait disparaître un au hasard.



l'écran, au bout de vingt images

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

Game Boy en C++

← l'atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

ENNEMI

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

ENNEMI

tuile n° 44 — poser(x, y, ENNEMI)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

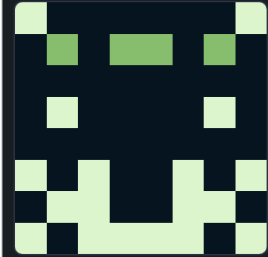
Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile



taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

ENNEMI ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

ENNEMI peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

71. Les collisions

Regarder **AVANT** de bouger, et non reculer après.

`lire(colonne, ligne)` rend le numéro de la tuile affichée. Comparer à `MUR` suffit : **le nom d'une tuile EST son numéro**.

La forme compte. On calcule la position **voulue**, on regarde ce qu'il y a, et l'on n'y va que si c'est libre. Bouger puis reculer marche aussi, mais laisse une image où le personnage est dans le mur — et cette image finit toujours par se voir.

LE PROGRAMME — 1669 OCTETS

```
Tuile MUR = {
    "#####", "#-----#", "#-----#", "#-----#",
    "#-----#", "#-----#", "#-----#", "#####",
};

int main() {
    for (uint8_t y = 0; y < 18; y++) { poser(0, y, MUR); poser(19, y, MUR); }
    for (uint8_t x = 0; x < 20; x++) { poser(x, 0, MUR); poser(x, 17, MUR); }
    poser(10, 8, MUR);

    uint8_t c = 5;
    uint8_t l = 8;

    while (true) {
        uint8_t vc = c;
        uint8_t vl = l;

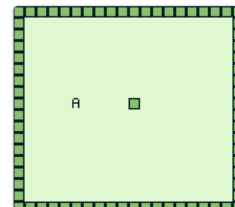
        if (bouton(DROITE)) vc++;
        if (bouton(GAUCHE)) vc--;
        if (bouton(BAS)) vl++;
        if (bouton(HAUT)) vl--;

        /* On regarde AVANT de bouger : la case visée est-elle libre ? */
        if (lire(vc, vl) != MUR) { c = vc; l = vl; }

        sprite(0, c * 8, l * 8, 1);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le curseur s'arrête net contre les murs et contre le bloc isolé.



l'écran, au bout de vingt images

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

Game Boy en C++

← l'atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

MUR

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

MUR

tuile n° 44 — poser(x, y, MUR

– zoom22×+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

MUR ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

MUR peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

72. Ralentir sans compter les images à la main

``retard()``` dit quand le tour de boucle a débordé.

`retard()` rend 1 si le tour précédent a duré **plus d'une image**. C'est la seule façon honnête de savoir qu'un jeu ne tient plus les soixante images par seconde : le compteur d'images de la page mesure le navigateur, pas la console.

Redessiner trois cent soixante tuiles à chaque tour est un bon moyen de faire monter ce compteur. **Un vrai jeu ne redessine que ce qui a changé** — et c'est exactement le débordement qui fait rater un `images() % 20 == 0`.

LE PROGRAMME — 1417 OCTETS

```
int main() {
    uint8_t sautees = 0;

    while (true) {
        /* Un travail volontairement lourd, pour voir la mesure bouger. La
           première ligne est laissée au compte rendu : la redessiner l'effacerait
           aussitôt, et l'on croirait que texte() ne marche pas. */
        for (uint8_t y = 2; y < 18; y++)
            for (uint8_t x = 0; x < 20; x++)
                poser(x, y, (x + y) % 4);

        if (retard()) sautees++;

        texte(1, 0, "RETARDS");
        nombre(10, 0, sautees);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le compteur de retards monte : la boucle ne tient pas l'image.



l'écran, au bout de vingt images

73. Passer à la salle suivante, sans bouton

Ce n'est pas un bouton qui change de salle : c'est le personnage qui sort du cadre.

La leçon « Passer d'un écran à l'autre » changeait de scène **sur un appui**. Ici, rien de tel : c'est la POSITION du héros qui décide — sortir par la droite fait apparaître la salle suivante, avec le héros collé au bord gauche.

La même règle que `allerA()` s'applique : `ecran(0)`, `effacer()`, redessiner, puis `ecran(1)` — toujours dans cet ordre, pour ne pas voir l'ancienne salle se mélanger à la nouvelle pendant qu'on l'efface.

Le héros ne s'arrête jamais à un bord comme dans « Le garder dans l'écran » : ici, dépasser le bord est justement ce qui déclenche le changement. Les deux techniques répondent à deux besoins opposés, avec le même test `x > DROITE_MAX`.

LE PROGRAMME — 1675 OCTETS

```
Tuile HEROS = {
    "..####.", ".#-##-#.", "#####", "#.####.#",
    "#####", "..#..#..", ".#...#.", "##...##",
};

const uint8_t GAUCHE_MAX = 8;
const uint8_t DROITE_MAX = 152;

uint8_t salle = 0;

void dessinerSalle() {
    ecran(0);
    effacer();

    if (salle == 0) {
        texte(6, 1, "SALLE A");
        poser(18, 8, 1);
    } else {
        texte(6, 1, "SALLE B");
        poser(1, 8, 1);
    }

    ecran(1);
}

int main() {
    uint8_t x = 76;
    uint8_t y = 68;

    dessinerSalle();

    while (true) {
        if (bouton(GAUCHE)) x--;
        if (bouton(DROITE)) x++;
        if (bouton(HAUT)) y--;
        if (bouton(BAS)) y++;

        if (x > DROITE_MAX && salle == 0) {
            salle = 1;
            x = GAUCHE_MAX;
            dessinerSalle();
        } else if (x < GAUCHE_MAX && salle == 1) {
            salle = 0;
            x = DROITE_MAX;
            dessinerSalle();
        }

        sprite(0, x, y, HEROS);
        image();
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

« SALLE A », puis « SALLE B » dès que le héros sort par la droite — il réapparaît collé au bord gauche.



l'écran, au bout de vingt images

74. Se souvenir d'une partie à l'autre

Un meilleur score qui survit à l'extinction.

La cartouche porte une pile et une petite mémoire. `sauver(numero, valeur)` y écrit, `sauvegarde(numero)` y relit — 256 cases, et elles sont encore là après avoir éteint la console.

Cette mémoire n'est pas ouverte en permanence : il faut la **déverrouiller** avant d'y toucher et la refermer aussitôt. Le compilateur pose le verrou lui-même à chaque accès. Ce n'est pas de la prudence excessive : laissée ouverte, une coupure de courant au mauvais moment la corrompt.

À la toute première partie, cette mémoire contient n'importe quoi. Un jeu y écrit donc **une marque à lui** et ne fait confiance au reste que s'il la retrouve. C'est ce que fait la première ligne de `main()`.

LE PROGRAMME — 1586 OCTETS

```
const uint8_t MARQUE = 42;

uint8_t meilleur = 0;
uint8_t score = 0;
uint8_t attente = 0;

int main() {
    // La cartouche a-t-elle déjà servi ?
    if (sauvegarde(0) != MARQUE) {
        sauver(0, MARQUE);
        sauver(1, 0);
    }

    meilleur = sauvegarde(1);

    texte(2, 4, "SCORE");
    texte(2, 8, "RECORD");

    while (true) {
        image();

        attente++;

        if (attente > 30) {
            attente = 0;

            if (score < 9) score++;

            if (score > meilleur) {
                meilleur = score;
                sauver(1, meilleur);
            }
        }

        poser(10, 4, 27 + score);
        poser(10, 8, 27 + meilleur);
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Le score monte, et le record le suit — puis lui survit.

SCORE 0

RECORD 0

l'écran, au bout de vingt images

75. La musique d'un jeu

Tout ensemble : un lutin qu'on déplace, une pièce à attraper, et un air qui ne s'arrête jamais.

Voici les dix niveaux réunis dans un seul programme : des tuiles dessinées, des lutins au pixel près, des boutons lus à chaque image, un score — et **deux airs**.

`jouer(1, MUSIQUE, 8, 1)` lance la musique de fond ; le quatrième argument, `1`, la fait **recommencer sans fin**. Ensuite le programme n'a plus rien à tenir : l'air avance tout seul dans l'interruption de la console, soixante fois par seconde, **même quand le jeu est en retard**.

Attraper la pièce lance `jouer(1, VICTOIRE, 5)` : la fanfare **prend la voix 1**, par-dessus la musique, exactement comme dans un vrai jeu. Elle, elle ne boucle pas.

`airFin(1)` dit quand la fanfare est arrivée au bout. C'est là qu'on relance la musique de fond — sans cela, la voix resterait muette après la victoire, et l'on chercherait longtemps pourquoi.

La **partition ci-dessous** écrit les deux airs à la souris, et « Écouter » les fait entendre sans compiler. Change une note : le programme se réécrit, la cartouche se refait, et la console joue ta version dans la seconde.

LE PROGRAMME — 2136 OCTETS

```
/* La musique tourne pendant que le jeu tourne. */

Air MUSIQUE = {
  "D04 10", "==", "SOL4 10", "==", "MI4 10", "==", "SOL4 10", "==",
  "FA4 10", "==", "LA4 10", "==", "SOL4 10", "==", "--", "--",
};

Air VICTOIRE = {
  "D05 14", "MI5 14", "SOL5 15", "==", "--",
};

Tuile HEROS = {
  "00333300",
  "03222330",
  "32122123",
  "32222223",
  "33222233",
  "03333330",
  "00300300",
  "03300330",
};

Tuile PIECE = {
  "00033000",
  "00311300",
  "03133130",
  "03133130",
  "03133130",
  "00311300",
  "00033000",
  "00000000",
};

uint8_t x = 40;
uint8_t piece = 60;
uint8_t score = 0;

int main() {
  texte(1, 1, "SCORE");
  jouer(1, MUSIQUE, 8, 1);

  while (true) {
    image();

    if (bouton(DROITE) && x < 152) x++;
    if (bouton(GAUCHE) && x > 8) x--;

    sprite(0, x, 80, HEROS);
    sprite(1, piece, 80, PIECE);

    /* Attrapée ? On avance le score, on joue la fanfare par-dessus la
       musique, et la pièce repart plus loin. */
    if (x + 6 > piece && piece + 6 > x) {
      if (score < 9) score++;
      poser(7, 1, 27 + score);
      jouer(1, VICTOIRE, 5);
      piece = piece + 40;
      if (piece > 150) piece = 20;
    }

    /* La fanfare finie, la musique reprend là où elle en était : au début. */
    if (airFin(1)) jouer(1, MUSIQUE, 8, 1);
  }

  return 0;
}
```

CE QU'ON DOIT VOIR

DROITE et GAUCHE déplacent le héros. La pièce attrapée fait monter le score et sonner une fanfare.

SCORE



l'écran, au bout de vingt images

La même chose, à la souris

MUSIQUE

VICTOIRE

+ air

MIDI

WAV

MUSIQUE

Écouter

note

- silence

= tenir

▼

▲

- pas

+ pas

8

images par pas

The interface displays a piano roll with a grid of 16 horizontal lanes. The bottom lane is labeled 'vol' and contains 16 green bars representing volume. The top lane is labeled 'MUSIQUE'. The grid shows several notes: a green note on the 10th lane at step 1, a green note on the 11th lane at step 4, a green note on the 12th lane at step 8, and a green note on the 13th lane at step 12. The notes are represented by green rectangles with stems.

`jouer(1, MUSIQUE, 8);`

— le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas.

Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

76. Le monde glisse, le héros ne bouge pas

La position dans le monde vaut « défilement + 76 ».

Le lutin ne bouge **jamais** : il est à 76 pixels, image après image. Ce qui change, c'est `defiler()`.

C'est ce qui distingue un écran fixe d'un monde. La position du personnage **dans le monde** vaut `monde + 76`, et c'est ce nombre-là qu'on compare au décor — pas celui du lutin, qui est une constante.

La leçon « La caméra : c'est le monde qui bouge » part du même principe et le pousse jusqu'au bord du niveau.

LE PROGRAMME — 1460 OCTETS

```
Tuile SOL = {
    "#####", "-----", "-----", "-----",
    "-----", "-----", "-----", "-----",
};

int main() {
    for (uint8_t x = 0; x < 20; x++)
        for (uint8_t y = 14; y < 18; y++)
            poser(x, y, SOL);

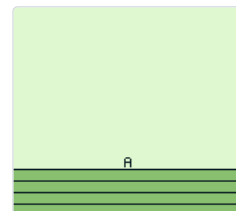
    uint8_t monde = 0;

    while (true) {
        if (bouton(DROITE)) monde++;
        if (bouton(GAUCHE)) monde--;

        defiler(monde, 0);
        sprite(0, 76, 104, 1);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le décor glisse ; le personnage reste rigoureusement au milieu.



l'écran, au bout de vingt images

La même chose, à la souris

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

Game Boy en C++

← l'atelier comp

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

SOL

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

SOL

tuile n° 44 — poser(x, y, SO

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

SOL ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

SOL peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

77. Un record qui survit à l'extinction

La cartouche a une pile, et de la mémoire qui reste.

`sauver(case, valeur)` et `sauvegarde(case)` écrivent et relisent la **mémoire sauvegardée de la cartouche** — celle qui survit à l'extinction.

Sur une vraie console, c'est une pile qui la tient. Dans la page, c'est le navigateur ; le programme, lui, ne voit aucune différence.

On ne sauve **que lorsque le record change**. Écrire à chaque image userait la mémoire d'une vraie cartouche, et n'apporterait rien.

LE PROGRAMME — 1466 OCTETS

```
int main() {
    uint8_t record = sauvegarde(0);
    uint8_t score = 0;
    uint8_t avant = 0;

    texte(1, 2, "RECORD");
    nombre(10, 2, record);

    while (true) {
        uint8_t appui = bouton(A);
        if (appui && !avant) score++;
        avant = appui;

        texte(1, 6, "SCORE");
        nombre(10, 6, score);

        if (score > record) {
            record = score;
            sauver(0, record);
            nombre(10, 2, record);
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Le record suit le score dès qu'il le dépasse.

RECORD	000
SCORE	000

l'écran, au bout de vingt images

78. Compter ce que coûte une boucle

Le compilateur ne cache pas ses prix.

Compiler donne son compte rendu : octets de programme, fonctions, variables, mémoire de travail. **Changer une ligne et recompiler montre ce qu'elle a coûté** — c'est la seule mesure qui ne mente pas.

Deux choses à savoir, mesurées et non supposées. `100 * x` et `x * 100` ne coûtent pas la même chose : la routine générale boucle sur son opérande de droite, alors le compilateur met le nombre connu à droite quand l'opérateur s'en moque.

Et `x / 16` devient quatre décalages, `x % 16` un `and` — **quand le diviseur est connu**. Sinon, c'est la routine générale, et elle se paie.

LE PROGRAMME — 1365 OCTETS

```
const uint8_t TABLE[] = { 3, 1, 4, 1, 5, 9, 2, 6 };

uint8_t total() {
    uint8_t somme = 0;
    for (uint8_t i = 0; i < sizeof(TABLE); i++) somme += TABLE[i];
    return somme;
}

int main() {
    texte(2, 6, "TOTAL");
    nombre(10, 6, total());

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

031 — la somme de la table gravée.

TOTAL 031

l'écran, au bout de vingt images

79. Le même jeu, deux fois

Une balle à la suite, trois balles par méthodes — le corps est le même.

À la suite, une balle demande quatre variables et huit lignes dans `main` : `if (versDroite) { x += 2; if (x > 150) versDroite = 0; }`, et ainsi de suite. Pour **une** balle, c'est parfait — 1427 octets.

Ci-dessous, trois balles **par méthodes**. Le corps de `avancer()` est **mot pour mot** celui de la version à la suite ; ce qui a changé, c'est qu'il est écrit une fois pour trois, et rangé avec les données sur lesquelles il travaille. 1956 octets, et quinze octets de mémoire.

La seule règle honnête : écrire à la suite tant que c'est un objet, nommer par une fonction quand le geste se répète, ranger dans une méthode quand les gestes et les données vont manifestement ensemble. Et ne pas choisir d'avance — recompiler, et lire le compte rendu.

LE PROGRAMME — 1989 OCTETS

```
Tuile BALLE = {
    ".....", ".....", ".....", ".....",
    ".....", ".....", ".....", ".....",
};

struct Balle {
    uint8_t x, y, versDroite, versBas, pas;

    void avancer() {
        if (versDroite) { x += pas; if (x > 150) versDroite = 0; }
        else { x -= pas; if (x < 10) versDroite = 1; }

        if (versBas) { y += pas; if (y > 130) versBas = 0; }
        else { y -= pas; if (y < 10) versBas = 1; }
    }
};

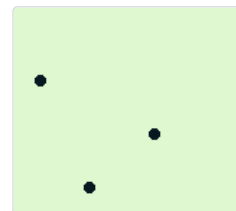
Balle balles[3];

int main() {
    for (uint8_t i = 0; i < 3; i++) {
        balles[i].x = 20 + i * 40;
        balles[i].y = 30 + i * 20;
        balles[i].versDroite = i % 2;
        balles[i].versBas = 1;
        balles[i].pas = 1 + i;
    }

    while (true) {
        for (uint8_t i = 0; i < 3; i++) {
            balles[i].avancer();
            sprite(i, balles[i].x, balles[i].y, BALLE);
        }
        image();
    }
}
```

CE QU'ON DOIT VOIR

Trois balles qui rebondissent, à trois vitesses différentes.



l'écran, au bout de vingt images

La même chose, à la souris

The screenshot displays the 'Atelier de la console' website interface. At the top, there's a navigation bar with a search icon and the text 'chercher un dessin : nom, étiquette, ★'. Below this, a list of search results is visible, each with the same search text. The main content area features a 'Game Boy' palette with four colors: black, light green, dark green, and dark blue. Below the palette, the text 'Game Boy — 4 nuances' is followed by a description: 'Les quatre nuances de la console d'origine, et rien de plus.' and a button 'Passer en couleur'. The 'BALLE' tile editor is the central focus, showing a 22x22 pixel grid. It includes a toolbar with various drawing tools: 'Peindre', 'Gomme', 'Ligne', 'Rectangle', 'Cercle', 'Formes : contour', 'Remplir', 'Pipette', 'Sélectionner et déplacer', 'Miroir', 'Tourner', 'Copier le dessin', 'Coller ici', 'Dupliquer', and 'Effacer la tuile'. The 'BALLE' tile is currently selected, and its properties are shown on the right: 'taille réelle — 8 x 8 pixels', 'étiquettes décor mur...', 'Animation' (with options: 'Eau (vague)', 'Feu (flamme)', 'Herbe (vent)', 'Clignoter', '+ À la main)'), and 'Police' (with a dropdown menu to 'remplacer la lettre...').

80. Ce que le compilateur refuse, et pourquoi

Le dernier tutorial n'apprend pas à écrire : il apprend à lire les refus.

C'est la règle qui gouverne le projet. **Un long traduit en douce sur un octet, c'est un jeu qui compte faux à partir de 256 et personne n'a été prévenu** ; un refus, c'est trois secondes perdues et une ligne à réécrire.

Cinq refus qu'on rencontre vraiment. `float vitesse = 1.5;` — la machine n'a pas de virgule. `class Ennemi { ... };` — le message propose `struct`, qui fait ce qu'on voulait, méthodes comprises. `carte[3] = 1;` sur une grille — il faut **deux** index, car `carte[3]` désigne une rangée entière. `uint8_t loin(Ennemi e)` — un argument est un octet ; une `struct` se passe par son nom, visible partout, ou par son index. `TABLE[0] = 9;` sur une table `const` — elle est gravée dans la cartouche.

Chacun porte **son numéro de ligne**, et dit quoi écrire à la place. C'est ce qui fait la différence entre un compilateur qui aide et un compilateur qui punit : les deux disent non, un seul dit pourquoi. Le programme ci-dessous est la version acceptée des cinq.

LE PROGRAMME — 1442 OCTETS

```
struct Ennemi { uint8_t x, y; };
const uint8_t TABLE[] = { 1, 2, 3 };
uint8_t carte[4][5];
Ennemi boss;

/* Pas la struct : son INDEX. Un argument est un octet. */
uint8_t loin(uint8_t cible) {
    if (boss.x > cible) return boss.x - cible;
    return cible - boss.x;
}

int main() {
    boss.x = 40;
    carte[2][3] = 1;          // deux index, pas un

    uint8_t vitesse = 2;      // un octet, pas un float

    texte(1, 4, "ECART");
    nombre(9, 4, loin(10));

    texte(1, 6, "GRILLE");
    nombre(9, 6, carte[2][3], 1);

    texte(1, 8, "TABLE");
    nombre(9, 8, TABLE[2], 1);

    texte(1, 10, "PAS");
    nombre(9, 10, vitesse, 1);

    while (true) {
        image();
    }
}
```

CE QU'ON DOIT VOIR

030, 1, 3, 2 — les cinq refus, écrits comme il faut.

ECART	030
GRILLE	1
TABLE	3
PAS	2

l'écran, au bout de vingt images

81. Ce qu'une ligne de C++ devient, en vrai

« `compte++` » n'existe pas pour la machine : elle ne connaît que lire, calculer, écrire.

Cliquer sur  **MODE MACHINE** juste après avoir compilé montre ce que la cartouche contient vraiment : des instructions du processeur, pas du C++. Ce mode se souvent d'où le compilateur a rangé chaque variable et chaque fonction — un « nom retrouvé » n'existe que pour une cartouche qui vient d'être compilée ici ; une cartouche venue d'ailleurs ne montre que des nombres.

`compte = 0` devient deux instructions : `xor a` (mettre le registre A à zéro) puis `ldh [main.compte], a` (l'écrire dans la case mémoire de compte). Il n'y a pas d'instruction « mettre une variable à zéro » sur ce processeur — seulement des registres et des adresses.

`compte++` n'est **pas** une seule instruction non plus : la machine doit LIRE la valeur (`ldh a, [main.compte]`), l'AUGMENTER dans un registre (`inc a`), puis la RÉÉCRIRE (`ldh [main.compte], a`). Trois instructions pour un `++`, parce que la puce ne calcule jamais directement en mémoire — tout passe par un registre.

Le `while (true)` du programme n'existe pas non plus dans la cartouche : c'est un simple retour en arrière, `jp`, qui saute à l'instruction d'avant plutôt que de continuer. La boucle est une manière humaine de LIRE le programme ; pour la machine, ce n'est qu'un saut.

LE PROGRAMME — 1293 OCTETS

```
int main() {
    uint8_t compte = 0;

    while (true) {
        compte++;
        nombre(8, 8, compte);
        image();
    }
}
```

CE QU'ON DOIT VOIR

Un nombre qui monte à l'écran ; dans MODE MACHINE, chaque ligne de ce programme devient plusieurs instructions.

017

l'écran, au bout de vingt images

82. Sinus, cosinus, une seule table

Pas de virgule flottante, pas de type négatif : une table gravée y suffit.

Il n'y a ni `sin()` ni `cos()` ici, et pas de type signé non plus — impossible d'écrire un nombre négatif directement. La solution tient dans **une table gravée** : chaque case vaut `40 + 40 * sin(angle)`, toujours entre 0 et 80, jamais en dessous de zéro.

`cos(angle) = sin(angle + 90°)` — et 90°, c'est un quart de tour. Avec trente-deux pas pour un tour complet, un quart de tour fait huit pas. Pas besoin d'une deuxième table : `cosinus` relit la MÊME, huit cases plus loin, avec `%` pour revenir au début quand ça déborde.

Le reste est déjà connu : `aAvant` pour agir **au front** — un seul basculement par appui, et non un par image —, et deux textes de même longueur, pour que rien ne traîne de l'un à l'autre.

LE PROGRAMME — 1562 OCTETS

```
Tuile BALLE = { "..####..", ".#---#.", "#-+--#-", "#-####-#", "#-####-#", "#-+--#-", ".#---#.", "..####.." };

const uint8_t SINUS[] = {
    40, 48, 55, 62, 68, 73, 77, 79,
    80, 79, 77, 73, 68, 62, 55, 48,
    40, 32, 25, 18, 12, 7, 3, 1,
    0, 1, 3, 7, 12, 18, 25, 32,
};

uint8_t x = 0;
uint8_t angle = 0;
uint8_t cosinus = 0;
uint8_t aAvant = 0;

int main() {
    while (true) {
        image();

        uint8_t a = bouton(A);
        if (a && !aAvant) cosinus = !cosinus;
        aAvant = a;

        if (cosinus) texte(1, 1, "A : COSINUS");
        else texte(1, 1, "A : SINUS  ");

        uint8_t indice = cosinus ? (angle + 8) % 32 : angle;
        uint8_t y = 28 + SINUS[indice];

        sprite(0, x, y, BALLE);

        x++;
        if (x > 152) x = 0;

        angle++;
        if (angle >= sizeof(SINUS)) angle = 0;
    }

    return 0;
}
```

CE QU'ON DOIT VOIR

Une balle qui ondule en traversant l'écran ; A bascule « SINUS » en « COSINUS », et elle saute d'un quart de tour.

A : SINUS



l'écran, au bout de vingt images